

药材的烘干问题

摘 要

本文把药材热风烘干抽象为圆柱体内热质耦合扩散与表面对流交换的初边值问题：由能量与水分守恒律导出一维径向变系数抛物方程组，中心取对称条件，表面取题给参数确定的 Robin 条件 $-k \partial_r T = h(T - T_{\text{air}})$ 与 $-D \partial_r C = h_m(C - C_{\text{air}})$ ，烘房状态由附件 1 逐点插值给出。四个问题共用同一套模型与程序，差别只在物性公式、时间范围与边界是否移动。数值上采用顶点型有限体积离散（ r 加权控制体、界面状态法计算非线性系数、Robin 通量直接施加于表面半控制体），配合稀疏雅可比自适应 BDF 积分，并在每个数据折点重启积分器；生产网格 $\Delta r = 0.0031 \text{ cm}$ （641 节点，与 0.1 cm 输出网格重合，无插值），容差 $\text{rtol} = 1 \times 10^{-9}$ 、 $\text{atol} = 1 \times 10^{-11}$ 。

问题一（预热平衡阶段，附录 2 物性）：1800 s 时中心温度 33.5753°C 、表面 36.7856°C （比烘房低 4.727 K ），中心水分浓度仍为 2.5500 kg/kg 、表面降至 1.5102 kg/kg ，体积平均失水 10.06% 。无量纲分析解释了“温度已穿透、水分仅在表层”的格局：热、质傅里叶数分别为 0.76 与 0.022 ，水分扩散长度仅 0.30 cm 。

问题二（全过程，附录 3 变物性）：3 h 时中心温度 49.8495°C 、中心水分 1.7662 kg/kg 、表面 1.0081 kg/kg ，平均失水 45.78% ；中心温度在 2.27 h 后与烘房相差不足 1 K ，此后扩散系数由含水率单独控制。

问题三：以“各处水分浓度低于 0.15 kg/kg ”即 $\max_r C(r, t) < 0.15$ 为判据、用事件求根精确定位，得烘干时间 57.48 h （2.40 天），落在题述“2–3 天”之内；此时表面水分已降至 0.0526 kg/kg ，中心成为控制点。

问题四（尺寸变化）：按干物质守恒在 Landau 物质坐标下求解收缩域问题（附件 2 的 $R(t)$ 用保单调 PCHIP 插值），烘干时长 51.09 h ；与同物性、半径固定 2 cm 的 129.85 h 相比缩短为 $1/2.54$ ，与扩散时间尺度之比 $(R_0/R_\infty)^2 = 2.79$ 相符，说明收缩通过缩短扩散路径主导了加速。

模型检验（校验器与求解器代码分离）：与常物性 Robin 圆柱贝塞尔级数解的最大误差为 1.0×10^{-5} （温度）与 1.1×10^{-5} （水分），观测阶 2.00；六级网格自收敛的观测阶为 2.05–2.32，烘干时间与最细网格之差 0.28 min （问题 3）、 0.03 min （问题 4）；离散水分与能量平衡是半离散系统的精确线性不变量（相对残差 $\sim 10^{-15}$ ）；极值原理与径向单调性全程成立；更严格容差、Radau 与限制最大步长的解相差不超过 4.2×10^{-9} 。独立编写的二维轴对称程序在端面绝热时与一维解相差 6.1×10^{-9} ，端部效应对烘干时间的影响仅 -0.004% ，支持一维化。灵敏度分析给出烘干时间对烘房温度平台、扩散前因子与收缩幅度的弹性为 -1.603 、 -0.893 、 -1.130 ，而对换热系数几乎不敏感（ -0.001 ）——烘干由内部扩散而非外部传热控制。模型评价一节进一步以湿球能量上限与平衡含水率驱动力量化了题给模型省略的蒸发潜热与吸附平衡两项机制的影响范围。

关键词：热质耦合扩散；对流边界条件；有限体积法；移动边界与 Landau 变换；验证与确认（V&V）；灵敏度分析

1 问题重述

1.1 问题背景

干燥是决定中药成品品质的关键工序之一，热风烘干是其中常见的方式。烘干过程主要包括**预热平衡**与**恒温干燥**两个阶段：通过调控烘房的温湿环境，使药材内部水分向表面迁移并被气流带走。工艺参数选取不当会导致干燥效率低、能耗高、成品质量不稳定；传统的试验优化模式成本高、周期长，因此需要借助数理分析与数值仿真获得干燥规律。

待干燥的药材形状近似为圆柱体，长 $L = 25\text{ cm}$ ，初始半径 $R_0 = 2\text{ cm}$ 。烘干开始时药材温度均匀为 $T_0 = 28^\circ\text{C}$ ，水分浓度（干基含水率）均匀为 $C_0 = 2.55\text{ kg/kg}$ 。烘房温度 $T_{\text{air}}(t)$ 与水分浓度 $C_{\text{air}}(t)$ 由附件 1 给出（0–14 400 s，每 60 s 一个样本，共 241 个）；药材半径 $R(t)$ 由附件 2 给出（0–72 h，每 1800 s 一个样本，共 145 个，由 2.000 cm 单调减至 1.198 cm）。药材的密度 ρ 、比热容 c_p 、热传导系数 k 与水分浓度扩散系数 D 由附录 2–4 的经验公式给出，其中 D 随含水率 C （以及问题 2–4 中的温度 T ）呈指数变化。

1.2 待解决的问题

问题 1（预热平衡阶段） 采用附录 2 的常物性参数（ $\rho = 820\text{ kg/m}^3$ 、 $c_p = 2600\text{ J/(kg K)}$ 、 $k = 0.36\text{ W/(m K)}$ 、 $h = 25\text{ W/(m}^2\text{ K)}$ 、 $h_m = 8 \times 10^{-7}\text{ m/s}$ 、 $D = 7 \times 10^{-9}\text{ e}^{-0.89/C}$ ），建立预热平衡阶段药材温度与水分浓度变化规律的模型，给出 $t \in \{100, 300, 600, 900, 1200, 1500, 1800\}\text{ s}$ 与 $r \in \{0, 0.5, 1, 1.5, 2\}\text{ cm}$ 处的结果（表 2、表 3），并把 1800 s 内每隔 1 s、每隔 0.1 cm 的完整结果写入 `result1.xlsx`。

问题 2（全过程模型） 烘干过程一般持续 2–3 天，预热平衡与恒温干燥阶段的参数有所不同；统一采用附录 3 的变物性经验公式建立整个烘干过程的模型，给出 3 h 内每隔 0.5 h 的结果（表 4、表 5），完整结果写入 `result2.xlsx`。

问题 3（烘干时长） 按照烘干要求，药材各处的水分浓度应低于 0.15 kg/kg ，确定烘干所需的时间（单位：h），给出每隔 6 h、每隔 0.5 cm 的水分浓度（表 6，含“烘干结束时间”行），完整结果（每 60 s、每 0.1 cm）写入 `result3.xlsx`。

问题 4（尺寸变化） 实际烘干中药材因水分流失发生尺寸变化；根据附件 2 给出的 $R(t)$ 与附录 4 的物性公式确定烘干时长，按表 7 的格式给出每隔 6 h 的水分浓度（含“药材表面”列），完整结果写入 `result4.xlsx`。

所有结果保留四位小数；结果文件的 A 列为时间（s），第 1 行为到药材中心的距离（cm）。本文四个结果文件均由附件 3 的模板生成，格式与模板逐格一致（见第 8 节与附录 C）。

1.3 题目中需要解释的三处不确定性

题目给出了全部物性与传递参数，但未写出控制方程与边界条件的具体形式，也未规定若干实现细节。本文对以下三处作出明确解释，并对每一处给出备选方案的定量对照（第 8.8 小节）：

- (a) **边界条件的形式**：把“对流换热系数 h ”与“对流传质系数 h_m ”按其量纲唯一地解释为表面 Robin 条件的系数，即 $-k \partial_r T = h (T_s - T_{\text{air}})$ 与 $-D \partial_r C = h_m (C_s - C_{\text{air}})$ 。这也使题给参数恰好构成一个封闭的定解问题（见假设 4）。
- (b) **附件 1 之外的烘房状态**：问题 3、4 的烘干持续数十小时，超出附件 1 的 4 h 覆盖范围。本文在样本之间线性插值，在 14 400 s 之后取恒温干燥阶段的平台值 $T_{\text{air}} = 49.999^\circ\text{C}$ 、 $C_{\text{air}} = 0.04999 \text{ kg/kg}$ （末 1 h 样本均值）。
- (c) **收缩域上方程的写法**：附件 2 的 $R(t)$ 只给出几何，未说明被边界“扫过”的物质如何处理。本文以干物质守恒的物质（Lagrangian）坐标为主模型，并给出实验室坐标（Eulerian）写法的对照结果。

2 问题分析

2.1 问题的共同结构

四个问题问的是同一个物理过程在不同参数体系与不同时间尺度上的表现，因此不应当分别建立四个模型，而应当建立一个带参数的初边值问题，用同一套离散与积分方法求解，四个问题只在下列三处不同：

问题	物性公式	时间范围与输出	计算域
1	附录 2（常物性 + $D(C)$ ）	0–1800 s，每 1 s	固定， $r \in [0, R_0]$
2	附录 3（ ρ, c_p, k 随 C ； $D(C, T)$ ）	0–3 h，每 1 s	固定
3	附录 3	0– t_{end} ，每 60 s	固定
4	附录 4	0– t_{end} ，每 60 s	收缩， $r \in [0, R(t)]$

2.2 关键困难与对策

- 三个量级的时间尺度分离**。以附录 2 的参数计，热扩散时间尺度 $R_0^2/\alpha = 2369 \text{ s}$ ，而水分扩散时间尺度 $R_0^2/D(C_0) = 22.5 \text{ h}$ ，两者相差 34 倍；温度与水分又通过 $D(C, T)$ 强耦合。这使方程组**刚性**：显式格式的**稳定步长**由温度方程决定（ $\Delta t \lesssim \Delta r^2/2\alpha$ ，在生产网格上约 $3 \times 10^{-3} \text{ s}$ ），而需要积分的总时长达 206929.7 s。对策是全隐式、变阶变步长的 BDF 方法（第 4.4 小节）。

2. **扩散系数跨越三到四个数量级。** $D \propto e^{-a/C}$ 在干燥后期急剧变小：以附录 3 计， D 由初始的 $1.35 \times 10^{-8} \text{ m}^2/\text{s}$ 降到烘干结束时表面的 $3.1 \times 10^{-12} \text{ m}^2/\text{s}$ ，比值 4320（图 4）。表面因此形成低扩散率的“硬壳”，传质 Biot 数 $h_m R_0/D$ 由 3.240 增至约 5131，过程由**内部扩散**而非外部对流控制——这一判断在灵敏度分析中被定量证实（弹性 -0.001 对 h 、 -0.893 对 D_0 ）。
3. **边界数据是折线与样条。** 附件 1 每 60 s 一个样本，线性插值后斜率在每个样本处跳变；附件 2 的 PCHIP 插值在每个节点处曲率跳变。多步法跨过折点时的误差不受容差控制（这一现象在初期实验中被观测到，见 MDR-0004 补充），对策是在**每个数据折点重启积分器**，段内强迫项光滑。
4. **“各处低于 0.15”是一个停止时刻问题。** 判据定义的时刻不落在输出网格上。对策是把 $g(t) = \max_i C_i(t) - 0.15$ 作为积分器的事件函数精确求根，并同时报告 60 s 网格上的值以与结果文件对照。
5. **问题 4 是移动边界问题。** 计算域随 $R(t)$ 收缩。若把附录方程视为实验室坐标下的方程（Eulerian），被边界扫过的物质离开计算域，水分总量不守恒；本文以干物质守恒的物质坐标（Lagrangian）为主模型，两者的差异作为解释不确定性的一部分定量给出（第 7.1 小节）。
6. **四位小数的精度要求。** 离散误差必须远小于 5×10^{-5} 。对策是把输出点取在网格节点上（ $\Delta r = 0.1/2^k \text{ cm}$ ），用收敛表确定 k ，并用解析级数解独立确认误差量级（第 8 节）。

2.3 技术路线

图 1 给出总体技术路线。所有计算在同一云端运行（run）内完成，每个阶段的输入输出散列、参数与环境写入清单，论文中的每一个数字都追溯到某个阶段的产物（附录 C）。

3 模型假设与符号说明

3.1 模型假设

下列假设分为两类：**题给假设**直接来自题目与附录，**附加假设**是本文为闭合模型而引入的。每条给出依据，并说明被违反时结果会向哪个方向偏移；其中可量化的偏差在第 8 节或第 9 节给出。

假设 1 (圆柱几何与一维径向化, 附加). 药材为半径 R 、长 L 的正圆柱，温度与水分浓度只依赖到轴心的距离 r 与时间 t ，即 $T = T(r, t)$ 、 $C = C(r, t)$ 。依据：长径比 $L/R_0 = 12.5$ ；在整个烘干时长内，水分的轴向傅里叶数上界为 $D_{\max} t_{\text{end}} / (L/2)^2 = 0.18 < 1$ ，端面影响不能扩散到中截面；热的轴向傅里叶数虽为 1.92，但药材温度在 3 h 内即与烘房接近一致（问题 2），此后不存在可被端面放大的轴向温差。验证：独立编写的二维轴对称程序

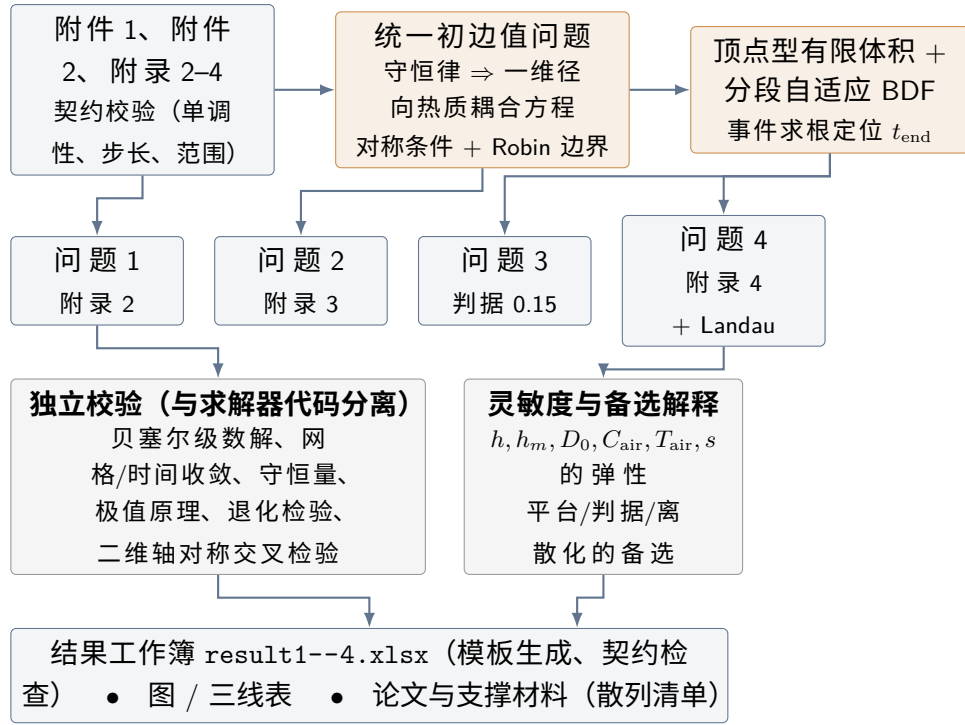


图 1 技术路线：由守恒律建立统一的一维径向热质耦合初边值问题，四个问题只改变物性、时间范围与边界是否移动；每一组解都经与求解器分离的独立校验，并给出灵敏度与备选解释。

给出端部效应对烘干时间的影响为 -0.004% (第 8.5 小节)，故一维化带来的偏差可忽略。

假设 2 (连续介质、各向同性与均匀初值，题给 + 附加). 药材是各向同性的连续介质，物性只通过局部的 C 与 T 依赖于位置；初始时刻 $T \equiv T_0 = 28^\circ\text{C}$ 、 $C \equiv C_0 = 2.55 \text{ kg/kg}$ (干基含水率)。依据：题目给出的经验公式均为 C 、 T 的显函数，未区分方向；题目给出均匀初值。后果：真实药材（纤维结构）的轴向扩散通常快于径向，忽略各向异性会略微高估烘干时间。

假设 3 (内部水分迁移服从 Fick 扩散，题给). 药材内部的水分通量（相对于固体骨架）为 $j = -\rho_s D(C, T) \nabla C$ ， ρ_s 为干物质表观密度；液态水与水蒸气的迁移合并为一个有效扩散系数 D 。依据：题目给出的正是“水分浓度扩散系数”的经验公式，其指数形式 $D = D_0 e^{-a/C} e^{-E/T_K}$ 是干燥文献中有效扩散率的常用拟合形式 [1-2]。后果：不区分毛细流动与蒸汽扩散，也不含总压梯度驱动的渗流 [3]。

假设 4 (表面为对流型 Robin 条件，传递系数恒定，题给 + 附加). 在表面 $r = R(t)$ 处

$$-k \partial_r T = h (T_s - T_{\text{air}}(t)), \quad -D \partial_r C = h_m (C_s - C_{\text{air}}(t)),$$

其中 $h = 25 \text{ W/(m}^2 \text{K)}$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 在四个问题中一律沿用（附录 3、4 未另给 h 、 h_m ）。依据： h 、 h_m 的量纲唯一地确定了上述形式； h 、 h_m 刻画烘房气流与表面之间的外部传递，不属于附录 3、4 所描述的药材内部物性 (MDR-0002)。后果： h_m 的取值误差对烘干时间的弹性为 -0.114 ， h 的为 -0.001 (表 10)，故该假设不是结果的敏感来源。

假设 5 (烘房状态空间均匀、时间上分段线性并在恒温阶段取平台, 附加). $T_{\text{air}}(t)$ 、 $C_{\text{air}}(t)$ 在药材表面处处相同; 在附件 1 的样本之间线性插值; $t > 14\,400\text{ s}$ 后取恒温干燥阶段的平台值 (末 1 h 样本均值) $T_{\text{air}} = 49.999^\circ\text{C}$ 、 $C_{\text{air}} = 0.04999\text{ kg/kg}$ 。依据: 附件 1 的温度在约 2 h 后围绕 50°C 波动 (相邻样本差 $\pm 0.3^\circ\text{C}$, 为测量噪声量级), “恒温干燥阶段”的物理含义是烘房保持设定值, 取末 1 h 均值是对设定值的无偏且抗单点噪声的估计 (MDR-0003)。备选: 取末样本值或名义值 $50^\circ\text{C}/0.05\text{ kg/kg}$ 的结果见表 11 (相差 -0.53% 以内)。

假设 6 (能量方程不含蒸发潜热源, 题给). 能量方程只含导热项, 表面只通过对流换热与烘房交换热量, 不扣除表面蒸发所需的潜热。依据: 题目给出的参数体系中没有潜热、蒸气压或平衡含水率, 表面传质律 $h_m(C_s - C_{\text{air}})$ 也不含湿球反馈; 强行加入潜热项会使该传质律给出的蒸发速率把表面温度压到 0°C 以下 (不适定, MDR-0008)。后果: 前十余小时药材温度被高估, 因而 D 被高估、烘干时间被低估。第 9 节以“潜热等效温降”作后验诊断 (峰值 54 K , 降到 1 K 以下需 16.1 h), 并以湿球能量上限给出界定性结果 ($+17.3\%$)。

假设 7 (问题 1–3 体积不变; 问题 4 均匀各向同性收缩且干物质守恒, 题给 + 附加). 问题 1–3 中 $R \equiv R_0$; 问题 4 中半径按附件 2 的 $R(t)$ 变化 (保单调三次 Hermite 插值 PCHIP[4], 72 h 之后保持末值 1.198 cm), 收缩是均匀的: 每个物质点的径向坐标与 $R(t)$ 成比例, 干物质总量守恒。依据: 题目在问题 1–3 未提尺寸变化, 在问题 4 直接给出 $R(t)$; 均匀收缩是干燥文献处理收缩的标准出发点 [5]。备选: 实验室坐标写法 (物质被边界扫出计算域) 的结果相差 3.06% (表 11)。

假设 8 (忽略辐射、收缩功与内部对流, 附加). 不计烘房壁面与药材之间的辐射换热、收缩过程的机械功与孔隙内的对流。依据: 题目只给出对流换热系数; 在 50°C 量级下, 线性化辐射换热系数 $h_r = 4\varepsilon\sigma T_K^3$ 只有 $h = 25\text{ W}/(\text{m}^2\text{ K})$ 的四分之一量级, 相当于把有效 h 提高不到三成; 而烘干时间对 h 的弹性仅为 -0.001 (表 10), 故影响可忽略。

假设 9 (烘干结束判据, 题给). 烘干结束时刻为 $t_{\text{end}} = \inf\{t > 0 : \max_{0 \leq r \leq R(t)} C(r, t) < C^*\}$, $C^* = 0.15\text{ kg/kg}$ 。依据: 题目要求“各处的水分浓度应低于 0.15 kg/kg ”。备选: 若按体积平均含水率判断, 烘干时间为 36.05 h (-37.28% , 表 11) ——判据的选择比任何参数扰动都更显著地改变答案。

3.2 符号说明

表 1 列出全文使用的主要符号; 其余符号在首次出现处定义。除特别说明外一律使用国际单位制, 含水率为干基 (kg 水/kg 干物质)。

表 1 全文使用的主要符号、含义、单位与取值来源

符号	含义	单位	取值/来源
t	时间	s	—
r	到药材中心（轴）的距离	m	$0 \leq r \leq R(t)$
$R(t)$	药材半径； $R_0 = R(0)$, R_∞ 为末值	m	附件 2； $R_0 = 0.02$ m
L	圆柱长度	m	0.25 m
ξ	无量纲径向坐标 $r/R(t)$ (Landau 坐标)	—	$\xi \in [0, 1]$
T	药材温度； $T_s = T(R, t)$ 为表面温度	°C	$T(r, 0) = 28$ °C
T_K	绝对温度 $T + 273.15$	K	用于 D 的公式
C	药材水分浓度（干基含水率）； $C_s = C(R, t)$	kg/kg	$C(r, 0) = 2.55$
$\bar{C}$	体积平均水分浓度	kg/kg	$\frac{2}{R^2} \int_0^R C r dr$
$T_{\text{air}}, C_{\text{air}}$	烘房温度与水分浓度	°C, kg/kg	附件 1 + 平台值
ρ, c_p, k	密度、比热容、热传导系数	kg/m ³ , J/(kg K), W/(m K)	附录 2-4
D	水分浓度扩散系数 $D_0 e^{-a/C} e^{-E/T_K}$	m ² /s	附录 2-4
α	热扩散率 $k/(\rho c_p)$	m ² /s	1.689×10^{-7} （附录 2）
h, h_m	对流换热系数、对流传质系数	W/(m ² K), m/s	$25, 8 \times 10^{-7}$
Bi, Bi_m	热、质 Biot 数 $hR_0/k, h_m R_0/D$	—	1.389, 3.240
Fo, Fo_m	热、质 Fourier 数 $\alpha t/R_0^2, Dt/R_0^2$	—	第 4.2 小节
C^*	烘干结束的水分浓度阈值	kg/kg	0.15
t_{end}	烘干结束时间	s（报告为 h）	求解结果
$n, \Delta\xi$	径向控制体个数、无量纲步长 $1/n$	—	节点数 641
Δr	径向网格步长 R_0/n	cm	0.0031
V_i	第 i 个控制体的无量纲测度	—	式 (8)
$\mathcal{F}_{i+1/2}$	界面 $i + \frac{1}{2}$ 上的无量纲水分通量	—	式 (9)
W	离散水分存量 $\sum_i V_i C_i$	kg/kg	式 (12)
p	观测收敛阶 $\log_2(e_k/e_{k+1})$	—	表 13
E_p	弹性 $(\Delta t_{\text{end}}/t_{\text{end}})/(\Delta p/p)$	—	表 10
L_v, ρ_{s0}	汽化潜热、干物质表观密度	J/kg, kg/m ³	第 9 节
$T_{\text{wb}}, C_{\text{eq}}$	烘房湿球温度、平衡含水率	°C, kg/kg	第 9 节

4 问题一的模型建立与求解

本节从守恒律出发建立四个问题共用的控制方程，给出离散格式及其守恒与极值性质，并给出问题 1（预热平衡阶段、附录 2 物性）的结果。

4.1 由守恒律到一维径向方程

水分。 以干物质骨架为参照系。记干物质表观密度为 ρ_s (kg 干物质每 m^3 湿物料)，干基含水率为 C ，则单位体积含水量为 $\rho_s C$ 。由假设 3，水分相对骨架的质量通量为 $j = -\rho_s D(C, T) \nabla C$ 。在任一固定物质区域 Ω 上作水分衡算并用散度定理，得 $\partial_t(\rho_s C) + \nabla \cdot j = 0$ ；在固定域上 ρ_s 为常数（假设 7），故（干燥过程中的热质迁移理论见文献 [6, 2]）

$$\frac{\partial C}{\partial t} = \nabla \cdot (D(C, T) \nabla C). \quad (1)$$

能量。 忽略蒸发潜热源与对流焓输运（假设 6、8），以 Fourier 定律 $\mathbf{q} = -k \nabla T$ 作能量衡算，得

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \nabla \cdot (k(C) \nabla T). \quad (2)$$

一维径向化。 由假设 1，场量只依赖 (r, t) ，圆柱坐标下 $\nabla \cdot (a \nabla u) = \frac{1}{r} \frac{\partial}{\partial r} (a r \frac{\partial u}{\partial r})$ ，于是得到本文的**控制方程**：对 $0 < r < R(t)$ 、 $t > 0$ ，

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(k(C) r \frac{\partial T}{\partial r} \right), \quad (3)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(D(C, T) r \frac{\partial C}{\partial r} \right), \quad (4)$$

配以中心对称条件、表面 Robin 条件（假设 4）与均匀初值（假设 2）：

$$\frac{\partial T}{\partial r} \Big|_{r=0} = \frac{\partial C}{\partial r} \Big|_{r=0} = 0, \quad (5)$$

$$-k \frac{\partial T}{\partial r} \Big|_{r=R} = h (T_s - T_{\text{air}}(t)), \quad -D \frac{\partial C}{\partial r} \Big|_{r=R} = h_m (C_s - C_{\text{air}}(t)), \quad (6)$$

$$T(r, 0) = T_0 = 28^\circ\text{C}, \quad C(r, 0) = C_0 = 2.55 \text{ kg/kg}. \quad (7)$$

量纲检查：(3) 两端为 W/m^3 ，(4) 两端为 $(\text{kg}/\text{kg})\text{s}^{-1}$ ，(6) 两端分别为 W/m^2 与 $(\text{kg}/\text{kg})\text{m s}^{-1}$ ，自洽。式 (3)–(7) 即四个问题共用的定解问题；问题 1–3 取 $R \equiv R_0$ ，问题 4 取 $R = R(t)$ （第 7.1 小节）。

问题 1 的特殊结构。 附录 2 的 ρ, c_p, k, h 为常数，仅 $D = 7 \times 10^{-9} e^{-0.89/C}$ 依赖 C 。于是(3)退化为**常系数线性方程**，与 C 无关；而(4)是拟线性方程，单向地由 C 自身决定。温度场因此有解析级数解，这一结构在第 8.1 小节被用作独立验证的基准。

4.2 无量纲数与两个时间尺度

以附录 2 的参数与初始状态计算：热扩散率 $\alpha = k/(\rho c_p) = 1.689 \times 10^{-7} \text{ m}^2/\text{s}$ ，初始扩散系数 $D(C_0) = 4.938 \times 10^{-9} \text{ m}^2/\text{s}$ ，

$$\text{Bi} = \frac{hR_0}{k} = 1.389, \quad \text{Bi}_m = \frac{h_m R_0}{D(C_0)} = 3.240, \quad \frac{\tau_{\text{热}}}{\tau_{\text{质}}} = \frac{R_0^2/\alpha}{R_0^2/D} = \frac{1}{34}.$$

$\text{Bi} \approx 1.4$ 说明内、外热阻相当，集总参数法 ($\text{Bi} \ll 1$) 不适用 [7]，必须解分布参数模型； $\text{Bi}_m > 3$ 说明传质的内阻已占优。两个时间尺度为 $\tau_{\text{热}} = R_0^2/\alpha = 2369 \text{ s}$ 与 $\tau_{\text{质}} = R_0^2/D = 22.5 \text{ h}$ ：在 1800 s 的预热阶段内，热 Fourier 数 $\text{Fo} = \alpha t/R_0^2 = 0.76$ 已是 $O(1)$ ，而质 Fourier 数只有 0.022，对应的扩散长度 $\sqrt{Dt} = 0.30 \text{ cm}$ 。这一预判与计算结果完全一致（图 2）：1800 s 时温度已渗透到中心，而水分的变化仍限制在表层约 0.5 cm 内。

4.3 有限体积离散

为了让同一套离散同时适用于固定域与收缩域，在无量纲坐标 $\xi = r/R(t) \in [0, 1]$ 上离散（固定域时 $R \equiv R_0$, $\xi = r/R_0$ ）。采用有限体积法 [8-9]，取**顶点型** (vertex-centred) 网格

$$\xi_i = i \Delta\xi, \quad i = 0, 1, \dots, n, \quad \Delta\xi = 1/n,$$

控制体为 $[\xi_{i-1/2}, \xi_{i+1/2}]$ ，其中 $\xi_{i\pm 1/2} = \xi_i \pm \Delta\xi/2$ ，并约定 $\xi_{-1/2} = 0$ 、 $\xi_{n+1/2} = 1$ ，即轴心与表面处为**半控制体**。控制体的径向测度（已除以 $2\pi R^2$ ）为

$$V_i = \int_{\xi_{i-1/2}}^{\xi_{i+1/2}} \xi \, d\xi = \frac{1}{2} (\xi_{i+1/2}^2 - \xi_{i-1/2}^2), \quad (8)$$

$$V_0 = \frac{\Delta\xi^2}{8}, \quad V_i = \xi_i \Delta\xi \quad (0 < i < n), \quad V_n = \frac{\Delta\xi}{2} - \frac{\Delta\xi^2}{8}.$$

把(4) 在控制体上积分并用散度定理，内部界面通量取二阶中心差分

$$\mathcal{F}_{i+1/2} = \xi_{i+1/2} D_{i+1/2} \frac{C_{i+1} - C_i}{\Delta\xi}, \quad i = 0, \dots, n-1, \quad (9)$$

轴心处由对称性取 $\mathcal{F}_{-1/2} = 0$ ，表面处**直接施加** Robin 通量 $\mathcal{F}_{n+1/2} = -R h_m (C_n - C_{\text{air}})$ （不引入虚拟节点，因而不损失精度也不破坏守恒）。于是得到半离散系统

$$R^2(t) V_i \frac{dC_i}{dt} = \mathcal{F}_{i+1/2} - \mathcal{F}_{i-1/2}, \quad \rho_i c_{p,i} R^2(t) V_i \frac{dT_i}{dt} = \mathcal{G}_{i+1/2} - \mathcal{G}_{i-1/2}, \quad (10)$$

其中 $\mathcal{G}$ 与 $\mathcal{F}$ 同构（把 D 换成 k 、 h_m 换成 h 、 C 换成 T ）， $\rho_i = \rho(C_i)$ 、 $c_{p,i} = c_p(C_i)$ 。

界面系数。 非线性系数在界面上取**界面状态法** $D_{i+1/2} = D\left(\frac{C_i + C_{i+1}}{2}, \frac{T_i + T_{i+1}}{2}\right)$ 、 $k_{i+1/2} = k\left(\frac{C_i + C_{i+1}}{2}\right)$ 。对 $D \propto e^{-a/C}$ 这类强非线性系数，界面状态法比系数的算术平均更接近 Kirchhoff 变换给出的精确界面通量（算术平均会系统性高估、调和平均会低估）。三种取法的对照见表 11：烘干时间之差不超过 0.11%，说明结果对这一离散化选择不敏感。

两个累积状态与离散守恒。 把边界损失本身作为两个附加的常微分方程状态并入系统：

$$\frac{dQ_m}{dt} = \frac{h_m(C_n - C_{\text{air}})}{R(t)}, \quad \frac{dQ_h}{dt} = \frac{h(T_n - T_{\text{air}})}{R(t)}, \quad Q_m(0) = Q_h(0) = 0. \quad (11)$$

未知量为 $\mathbf{y} = (T_0, \dots, T_n, C_0, \dots, C_n, Q_m, Q_h)^\top \in \mathbb{R}^N$, $N = 2(n+1) + 2 = 1284$ 。

命题 4.1 (离散水分守恒是精确的线性不变量). 记离散水分存量 $W(t) = \sum_{i=0}^n V_i C_i(t)$ 。则沿着半离散系统(10)–(11) 的任意解,

$$W(t) + Q_m(t) \equiv W(0) \quad \text{对一切 } t \text{ 成立.} \quad (12)$$

证明. 由(10) 求和, 内部通量成对相消 (telescoping) : $\frac{dW}{dt} = \sum_i \frac{\mathcal{F}_{i+1/2} - \mathcal{F}_{i-1/2}}{R^2} = \frac{\mathcal{F}_{n+1/2} - \mathcal{F}_{-1/2}}{R^2} = \frac{-R h_m(C_n - C_{\text{air}})}{R^2} = -\frac{dQ_m}{dt}$ 。 $\square$

W 即 (除以 $2\pi\rho_s R^2$ 后的) 单位长度含水量, 故(12) 就是“含水量的减少等于表面通量的时间积分”。由于它是**线性**不变量, BDF 与 Radau 都精确保持它 (直到舍入误差); 因此把它作为校验量, 检验的是离散格式本身而不是时间积分的精度——实测相对残差约 10^{-15} (表 8)。对附录 2 的常物性, 温度方程同样给出 $\rho c_p \sum_i V_i T_i + Q_h \equiv \text{常数}$ 。

命题 4.2 (离散极值原理). 设 $C_{\text{air}}(t) \leq C_0$ 。若 i^* 是某时刻 $\mathbf{C}$ 的最大分量所在的下标, 则 $dC_{i^*}/dt \leq 0$; 从而 $\max_i C_i(t) \leq \max\{C_0, \sup_s C_{\text{air}}(s)\}$, $\min_i C_i(t) \geq \min\{C_0, \inf_s C_{\text{air}}(s)\}$ 。温度分量同理。

证明. $D_{i+1/2} > 0$, $\xi_{i+1/2} \geq 0$ 。当 $0 < i^* < n$ 时 $C_{i^*\pm 1} \leq C_{i^*}$, 故(9) 给出 $\mathcal{F}_{i^*+1/2} \leq 0$ 且 $-\mathcal{F}_{i^*-1/2} \leq 0$, 右端非正; $i^* = 0$ 时只剩 $\mathcal{F}_{1/2} \leq 0$; $i^* = n$ 时边界项 $-R h_m(C_n - C_{\text{air}}) \leq 0$ (因 $C_n = \max_i C_i \geq C_0 \geq C_{\text{air}}$)。下界同理。 $\square$

命题 4.2 保证离散解不会出现非物理的负含水率或超过初值的“回潮”, 也保证了 D 的计算始终在其定义域内。校验器在全部输出时刻独立检查该性质 (表 8)。

精度。 在均匀网格上, (9) 是界面导数的二阶近似, V_i 取自控制体的精确积分, 故内部格式二阶; 轴心与表面的半控制体上 Robin 通量直接施加, 不引入额外误差。实测观测阶为 2.0–2.3 (表 13、表 14)。

4.4 时间积分、输出采样与算法

半离散系统刚性 ($\tau_{\text{热}}/\tau_{\text{质}} \sim 10^{-2}$, 且空间加密使刚性比再放大 n^2 倍), 必须用全隐式方法。本文采用 SciPy[10] 与 NumPy[11] 实现的变阶变步长 BDF[12–13], 并提供块三对角的雅可比稀疏结构 (每个节点只与自身及左右邻居的 T, C 耦合), 使每步的线性代数为 $O(N)$ 。三个实现要点:

1. **在数据折点重启。** $T_{\text{air}}, C_{\text{air}}$ 是折线、 R 是 PCHIP 样条，其导数在样本点处跳变。多步法跨过折点时的局部误差不受容差控制。因此把 $[0, t_{\text{终}}]$ 按附件 1 的全部样本时刻、切换到平台的时刻以及附件 2 的全部节点分段，逐段积分、段间重启（问题 3 共 241 段）。
2. **输出用稠密插值精确采样。** 所需时刻（每 1 s 或每 60 s）由积分器的稠密输出在段内插值得到，不使用最近邻。
3. **事件求根定位 t_{end} 。** 以 $g(t) = \max_i C_i(t) - C^*$ 为事件函数（连续、在干燥过程中单调下降穿零），由积分器在稠密输出上求根，得到不受输出网格分辨率限制的 t_{end} 。

Algorithm 1: 径向有限体积 + 分段自适应 BDF 求解（含烘干结束时刻）

输入: 物性公式集 $\mathcal{P}$ （附录 2/3/4）、烘房数据 $(T_{\text{air}}, C_{\text{air}})$ 、半径数据 $R(\cdot)$ 、网格数 n 、容差 $(\varepsilon_r, \varepsilon_a)$ 、输出时刻集 $\mathcal{T}$

输出: $\{T_i(t), C_i(t)\}_{t \in \mathcal{T}}$ 、烘干结束时刻 t_{end} 、校验量 Q_m, Q_h

- 1 构造网格 $\{\xi_i\}$ 与控制体测度 $\{V_i\}$ （式 (8)）； $\mathbf{y}(0) \leftarrow (T_0 \mathbf{1}, C_0 \mathbf{1}, 0, 0)$ ；
 - 2 $\mathcal{K} \leftarrow$ 附件 1 的样本时刻 $\cup$ 平台切换时刻 $\cup$ 附件 2 的 PCHIP 节点（升序去重）；
 - 3 构造雅可比稀疏模板（块三对角， $O(N)$ 个非零）；
 - 4 **foreach** 相邻折点区间 $[\tau_j, \tau_{j+1}] \subset [0, t_{\text{终}}]$ **do**
 - 5 在该区间上用 BDF 积分 $\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$ ，容差 $(\varepsilon_r, \varepsilon_a)$ ，稀疏雅可比；
 - 6 **if** $\mathcal{T} \cap [\tau_j, \tau_{j+1}] \neq \emptyset$ **then** 由稠密输出取出这些时刻的状态；
 - 7 **if** 尚未定位 t_{end} 且 $\max_i C_i(\tau_{j+1}) < C^*$ **then**
 - 8 在 $[\tau_j, \tau_{j+1}]$ 上对 $g(t) = \max_i C_i(t) - C^*$ 求根，得 t_{end} 与该时刻的剖面；
 - 9 $\mathbf{y}(\tau_{j+1})$ 作为下一段的初值（重启多步法）；
 - 10 **return** 采样结果、 t_{end} 、 Q_m, Q_h
-

复杂度与参数。 每个 BDF 步的代价为 $O(N)$ （带状 LU），总代价 $O(N \cdot N_{\text{步}})$ 。生产网格共 641 个节点， $\Delta r = 0.0031 \text{ cm} = 0.1/2^5 \text{ cm}$ ，使 0.1 cm 的输出点恰好落在节点上；容差 $\varepsilon_r = 1 \times 10^{-9}$ 、 $\varepsilon_a = 1 \times 10^{-11}$ 。问题 3 的完整求解共 22510 次右端项调用、3205 次 LU 分解，单次求解在 8 核云端实例上不足 30 s。

4.5 问题一的结果与分析

表 2 与表 3 给出题目要求格式的结果，图 2、图 3 给出对应的剖面与时程；每 1 s、每 0.1 cm 的完整结果见 result1.xlsx（时间 1 s–1800 s 每 1 s、位置 0–2 cm 每 0.1 cm，温度与水分浓度两张工作表）。

表 2 30 分钟内药材的温度 ($^{\circ}\text{C}$; 列为到药材中心的距离/cm)

时间/s	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 3 30 分钟内药材的水分浓度 (kg/kg ; 列为到药材中心的距离/cm)

时间/s	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

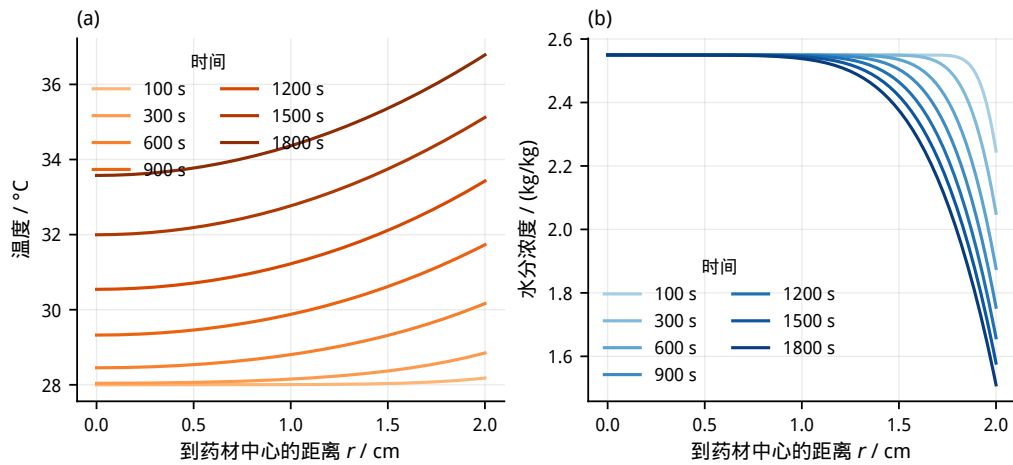


图 2 问题 1: 表 2、表 3 各时刻的温度 (a) 与水分浓度 (b) 径向剖面。1800 s 内热扩散已渗透到中心 (中心温度明显高于初值), 而水分的变化仍限制在表层约 0.5 cm, 与 $\text{Fo} = 0.76$ 、 $\text{Fo}_m = 0.022$ 的量级判断一致。

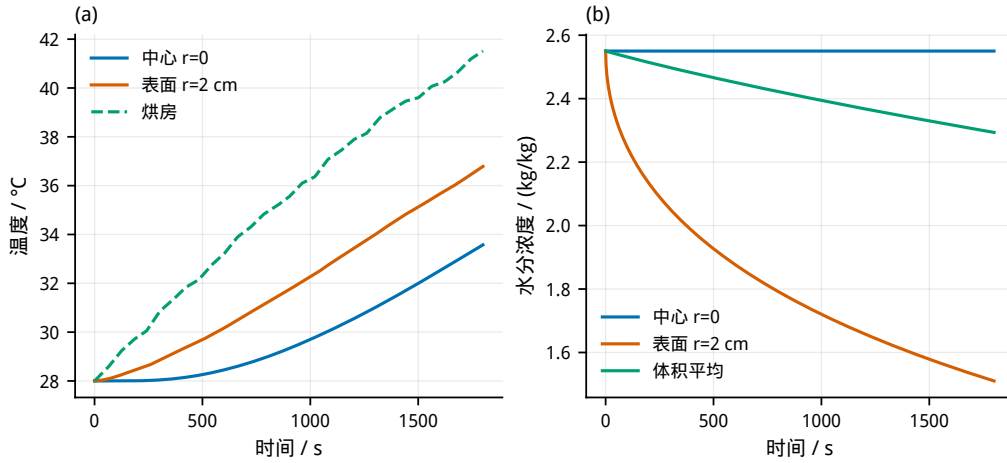


图 3 问题 1: 中心、表面与烘房温度 (a) 以及中心、表面、体积平均水分浓度 (b) 的历程。表面温度始终低于烘房温度 (1800 s 时相差 4.727 K)，中心又低于表面，与 $Bi = 1.389$ (内阻与外阻相当) 一致；中心水分浓度在全程保持初值，体积平均含水率的下降速率随表层变干而逐渐减慢。

结果分析。 (i) 1800 s 时中心温度 33.5753°C 、表面温度 36.7856°C ，同时刻烘房温度为 41.513°C ：表面比烘房低 4.727 K，中心又明显低于表面 (表 2)。截面上的不均匀度 (中心与表面之差) 与表面到烘房的温差是同一量级，这正是 $Bi = 1.389 \approx 1$ 的含义：内部导热热阻与外部对流热阻相当，二者共同决定预热速率，因而既不能用集总参数法、也不能假定表面温度等于烘房温度。(ii) 水分浓度在 $r \leq 1\text{ cm}$ 处四位小数下仍为初值 2.5500，表面降至 1.5102，体积平均为 2.2936，即 30 min 内失水 10.06%。表面附近出现陡峭的边界层：这正是 $Bi_m = 3.240 > 1$ 与 $Fo_m \ll 1$ 的表现。(iii) 该结构对后续问题有直接含义：干燥初期的失水几乎全部来自表层，表层含水率迅速下降使 $D \propto e^{-0.89/C}$ 显著减小，从而自发形成低扩散率的“硬壳”，把整个过程推向内部扩散控制。

5 问题二的模型建立与求解

5.1 模型的改变

问题 2 覆盖预热平衡与恒温干燥两个阶段，统一采用附录 3 的经验公式 (题目要求“相关经验公式统一采用附录 3 中的公式”)：

$$\begin{aligned} \rho &= 650 + 128C, & c_p &= 1450 + 2736 \frac{C}{C+1}, & k &= 0.21 + 0.38 \frac{C}{C+1}, \\ D &= 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T_K}, \end{aligned} \quad (13)$$

其中 $T_K = T + 273.15$ 为绝对温度。控制方程、边界条件与初值仍为(3)–(7)， h 、 h_m 沿用附录 2 的值 (假设 4)。与问题 1 相比有三点实质变化：

1. **双向耦合。** ρ, c_p, k 依赖 C ， D 依赖 C 与 T ，温度场与水分场**双向耦合**，不再有解析解。
2. **扩散系数的温度激活。** D 含 Arrhenius 因子 e^{-3850/T_K} ， $d \ln D / dT = E / T_K^2$ 。在同一含水率 C_0 下，温度由 28°C 升到恒温阶段的平台值使 D 由 5.642×10^{-9} 增大到

$1.35 \times 10^{-8} \text{ m}^2/\text{s}$, 即增大一倍以上。因此预热阶段虽然失水不多, 却通过升温为后续干燥“解锁”了扩散能力。

3. **与含水率的竞争。**因子 $e^{-0.45/C}$ 随 C 减小而减小。干燥初期升温占优 (D 增大), 后期失水占优 (D 急剧减小)。图 4 给出三组附录物性的 $D(C, T)$ 与热扩散率。

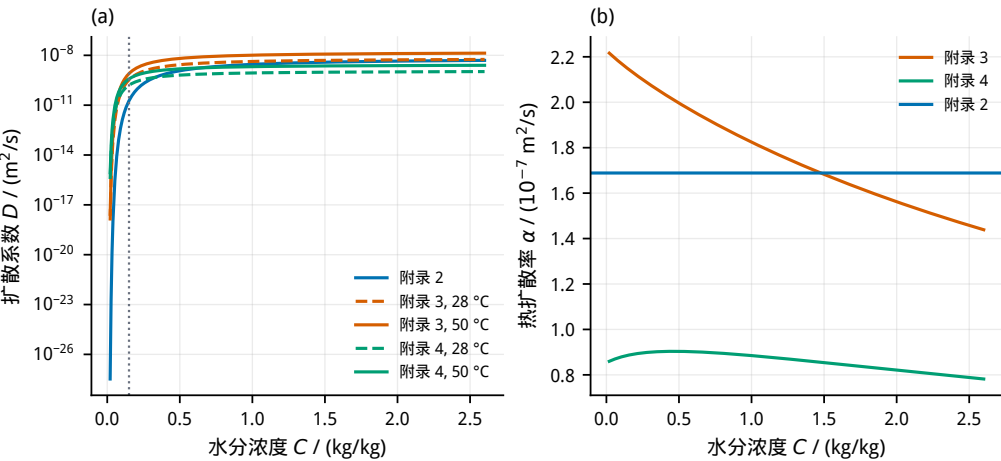


图 4 附录 2-4 的物性：水分浓度扩散系数随含水率与温度的变化 (a, 纵轴对数坐标) 与热扩散率 $\alpha = k/(\rho c_p)$ (b)。 D 在干燥过程中跨越三到四个数量级, 是刚性与“表面硬壳”的来源; 附录 4 的 D 在高含水率段比附录 3 小约 5.4 倍。

数值方法与第 4.3、4.4 小节完全相同, 只更换物性函数; 输出为 0-3 h 每 1 s (与表 3、表 4 的时刻一致, MDR-0005)。

5.2 结果与分析

表 4 3 小时内药材的温度 (°C; 列为到药材中心的距离/cm)

时间/h	0	0.5	1	1.5	2
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

结果分析。 (i) 3 h 时中心温度 49.8495°C 、表面 49.9664°C (表 4), 截面温差已进入小数点后第一位; 中心温度达到烘房温度 1 K 以内所需时间为 2.27 h, 与热扩散时间尺度 $\tau_{\text{热}}$ 及烘房自身的升温时间相当。此后可以认为药材是等温的, 这为后续把注意力集

表 5 3 小时内药材的水分浓度 (kg/kg; 列为到药材中心的距离/cm)

时间/h	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

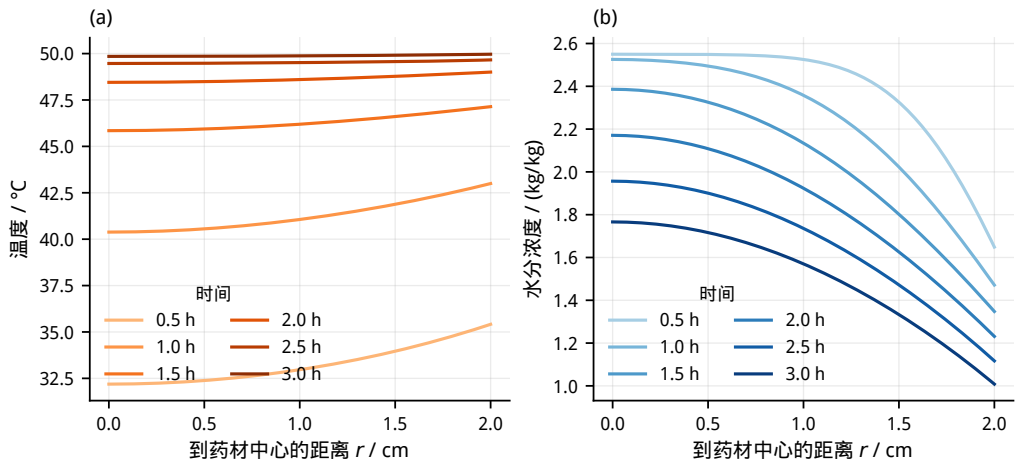


图 5 问题 2: 表 4、表 5 各时刻的温度 (a) 与水分浓度 (b) 径向剖面。温度剖面在 2 h 后几乎拉平 (表 4 各行内的差已进入小数点后第一位), 水分剖面则持续变陡。

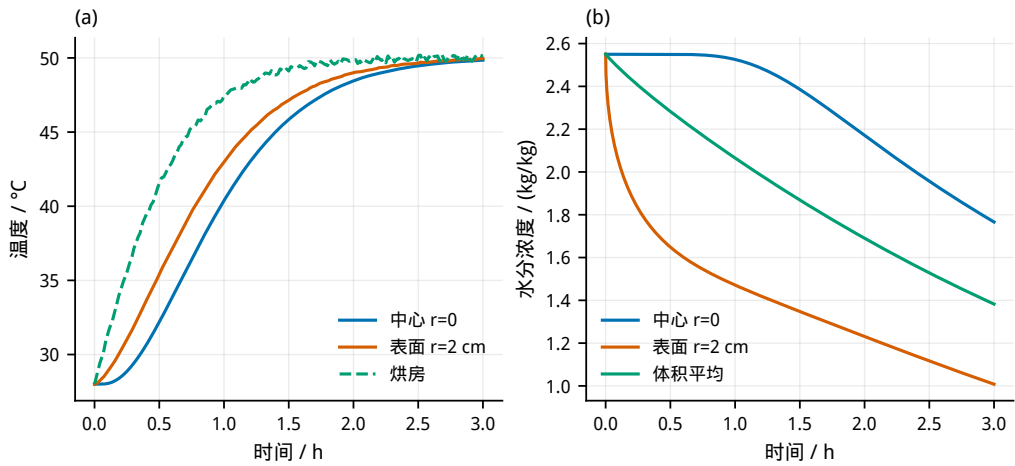


图 6 问题 2: 3 h 内温度 (a) 与水分浓度 (b) 的时程。中心温度在 2.27 h 后与烘房相差不足 1 K、2.69 h 后不足 0.5 K; 此后 D 的变化完全由含水率决定。

中在水分方程提供了依据。(ii) 3 h 时中心水分 1.7662、表面 1.0081、体积平均 1.3825 kg/kg, 累计失水 45.78%——远高于问题 1 的 10.06%, 因为温度升高把 D 整体抬高。(iii) 一个值得注意的现象: 3 h 时表面的扩散系数 $D = 1.027e - 08 \text{ m}^2/\text{s}$, 大于初始值 $5.642 \times 10^{-9} \text{ m}^2/\text{s}$, 尽管表面含水率已从 2.55 降到 1.0081。这说明在前 3 h 内温度激活仍压过含水率下降的影响; 转折发生在含水率降到约 0.5 kg/kg 之后 (图 4(a))。相应地, 传质 Biot 数由 3.240 降到 1.56, 随后在干燥后期又升到 10^3 量级。(iv) 表 3、表 4 的时刻均落在附件 1 的覆盖范围内, 因此问题 2 的结果不依赖假设 5 中的平台外推。

6 问题三的模型建立与求解

6.1 烘干结束时间的定义与求解

模型与问题 2 完全相同 (附录 3 物性、固定半径), 只是把积分区间延长到烘干结束。按假设 9,

$$t_{\text{end}} = \inf \left\{ t > 0 : \max_{0 \leq r \leq R_0} C(r, t) < C^* \right\}, \quad C^* = 0.15 \text{ kg/kg}. \quad (14)$$

由命题 4.2 与数值上验证的径向单调性 ($\partial_r C \leq 0$, 表 8), 最大值在中心取得, 故(14)等价于 $C(0, t_{\text{end}}) = C^*$; 但程序不假定这一点, 仍以 $\max_i C_i$ 作事件函数 (算法 1), 从而对任何剖面形状都成立。

为什么必须精确求根。 若只在 60 s 的输出网格上判断, 得到的是 t_{end} 向上取整到 60 s 的值: 事件求根给出 57.4805 h, 60 s 网格给出 57.4833 h, 与 result3.xlsx 的最后一行一致。

6.2 结果与分析

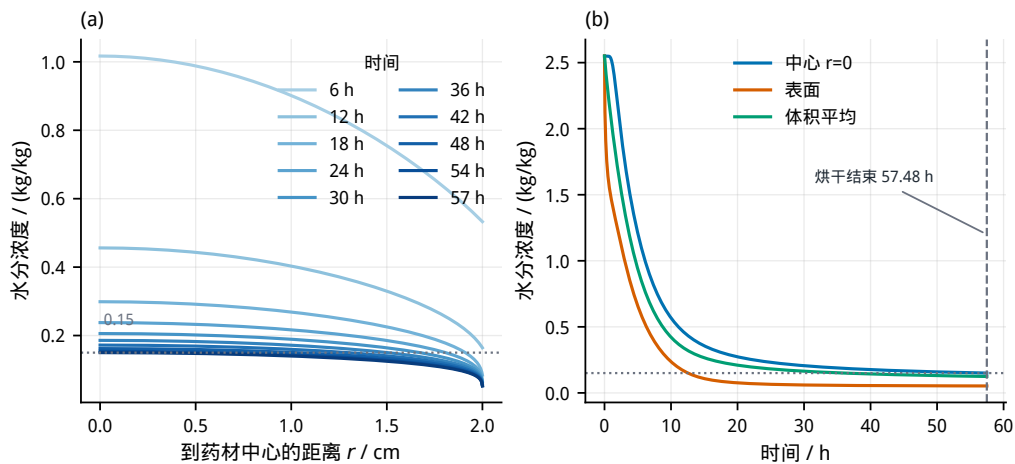


图 7 问题 3: 每 6 h 与烘干结束时刻的水分浓度剖面 (a) 以及中心、表面、体积平均水分浓度的时程 (b); 点线为判据 $C^* = 0.15 \text{ kg/kg}$, 虚线为烘干结束时刻 $t_{\text{end}} = 57.48 \text{ h}$ 。表面在十余小时内即降到判据以下 (表 6), 其后的绝大部分时间完全用于把中心的水分排出。

表 6 药材烘干过程的水分浓度 (kg/kg; 固定半径 2 cm, 附录 3 物性; 列为到药材中心的距离/cm)

时间/h	0	0.5	1	1.5	2
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1856	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0548
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1515	0.1436	0.1277	0.0530
烘干结束时间 (57.48 h)	0.1500	0.1477	0.1402	0.1249	0.0526

烘干结束时间为 $\max_r C(r, t) < 0.15$ 首次成立的精确时刻 (事件求根), 结果文件按 60 s 分辨率给出。

结果分析。 (i) **烘干所需时间为 $t_{\text{end}} = 57.48 \text{ h}$ (2.40 天)**, 落在题目所述“一般持续 2–3 天”的范围内——这是一个独立于数值细节的合理性旁证。(ii) 过程高度不均衡: 体积平均含水率在 3.42 h 内即损失一半, 而把**中心**降到 C^* 需要 57.48 h。烘干结束时表面含水率仅 0.0526、体积平均为 0.1243 kg/kg, 即为了满足“各处低于 0.15”, 大部分物料被过度干燥。若改用体积平均含水率作判据, 只需 36.05 h (–37.28%) ——判据的表述是本问题中最敏感的建模决策, 远超任何参数的 $\pm 20\%$ 扰动 (表 10、表 11)。(iii) 机理上, 长尾来自扩散系数的自反馈: 表面含水率降到 0.05 量级后 D 降至 $3.1 \times 10^{-12} \text{ m}^2/\text{s}$, 比初始值小 4320 倍, 传质 Biot 数升到约 5131。此时表面对流早已不是瓶颈 (对 h_m 的弹性仅 -0.114), **过程完全由内部扩散控制**。(iv) 因此工艺上缩短烘干时间的有效手段是提高烘房温度 (弹性 -1.603 , 因为 D 的 Arrhenius 因子) 或减小特征尺寸 (问题 4), 而不是加大风速 (提高 h 、 h_m)。

7 问题四的模型建立与求解

7.1 收缩域上的方程：物质坐标表述

问题 4 中计算域 $0 < r < R(t)$ 随附件 2 给出的 $R(t)$ 收缩 ($2.000 \text{ cm} \rightarrow 1.198 \text{ cm}$, 主要发生在前 12 h, 图 8(c))。附件 2 只给出几何, 未说明被边界“扫过”的物质如何处理, 因此必须补充一条物理原则。本文采用**干物质守恒** (假设 7): 收缩是均匀的, 每个物质点的径向坐标与 $R(t)$ 成比例, 于是 $\xi = r/R(t)$ 是**物质坐标** (Lagrangian 坐标, Landau 变换 [14–15])。

命题 7.1 (物质坐标下的方程). 在均匀收缩与干物质守恒的假设下, 以 $\xi = r/R(t)$ 为自

变量、 $\hat{C}(\xi, t) = C(R(t)\xi, t)$, 有

$$\left. \frac{\partial \hat{C}}{\partial t} \right|_{\xi} = \frac{1}{R^2(t)} \cdot \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(D \xi \frac{\partial \hat{C}}{\partial \xi} \right), \quad -\left. \frac{D}{R(t)} \frac{\partial \hat{C}}{\partial \xi} \right|_{\xi=1} = h_m (\hat{C}(1, t) - C_{\text{air}}), \quad (15)$$

温度方程同理（把 D 换成 $k/(\rho c_p)$ 并保留 ρc_p 因子）。方程中**不出现对流项**，收缩的全部作用体现为因子 $1/R^2(t)$ 。此外，以干基计的水分总量满足

$$\frac{d}{dt} \int_0^1 \hat{C} \xi d\xi = -\frac{h_m (\hat{C}(1, t) - C_{\text{air}})}{R(t)}, \quad (16)$$

即水分守恒。

证明要点. 干物质守恒与均匀收缩意味着每个物质环 $[\xi, \xi + d\xi]$ 包含的干物质 $\propto \xi d\xi$ 与时间无关；水分相对骨架的通量为 $-\rho_s D \partial_r C = -\rho_s D R^{-1} \partial_{\xi} \hat{C}$ ，其通过半径 $r = R\xi$ 处环面（面积 $\propto R\xi$ ）的总量 $\propto -\rho_s D \xi \partial_{\xi} \hat{C}$ 中 R 恰好相消。对物质环作衡算并除以其干物质质量（ $\propto R^2 \xi d\xi$ ）即得(15)；对 $\xi \in [0, 1]$ 积分并用对称条件即得(16)。□

备选：实验室坐标写法。 若把附录方程视为实验室坐标 (r, t) 下的方程并在 $0 < r < R(t)$ 上求解，同一变换会**额外**产生一个对流项：

$$\left. \frac{\partial \hat{C}}{\partial t} \right|_{\xi} = \frac{1}{R^2} \cdot \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(D \xi \frac{\partial \hat{C}}{\partial \xi} \right) + \xi \frac{\dot{R}}{R} \frac{\partial \hat{C}}{\partial \xi}. \quad (17)$$

此时水分总量的变化等于表面通量积分再加上 $C_s R \dot{R}$ 一项——被退缩的边界扫出计算域的水分被“丢弃”，总量不守恒（校验器测得这一伪损失占总失水的三成以上）。因此本文以(15)为主模型，把(17)的结果作为对照（表 11、表 16）。

实现。 第 4.3 小节的离散本来就写在 ξ 上，因此只需把常数 R_0^2 换成 $R^2(t)$ (Lagrangian) 或再加上(17)的对流项 (Eulerian, 界面用中心差分、表面用三点单侧差分)。 $R(t)$ 由附件 2 的样本作保单调 PCHIP 插值 [4]（保证 R 不出现非物理的回升），72 h 之后保持末值。物性改用附录 4：

$$\begin{aligned} \rho &= 760 + 90C, & c_p &= 1850 + 2150 \frac{C}{C+1}, & k &= 0.12 + 0.20 \frac{C}{C+1}, \\ D &= 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T_K}. \end{aligned} \quad (18)$$

退化检验。 令 $R \equiv R_0$ （但仍走移动边界的代码路径），两种写法都应退化为固定域格式。实测与固定域求解器之差为 9.4×10^{-14} ，为舍入误差量级，说明移动边界的实现没有引入额外误差。

7.2 结果与分析

结果分析。 (i) 考虑尺寸变化后的烘干时长为 $t_{\text{end}} = 51.09 \text{ h}$ (51.0893 h, 2.13 天)，此时半径已收缩到 1.200 cm；表 6 中 $r = 1.5, 2 \text{ cm}$ 两列在所有时刻均超出当前半径，故

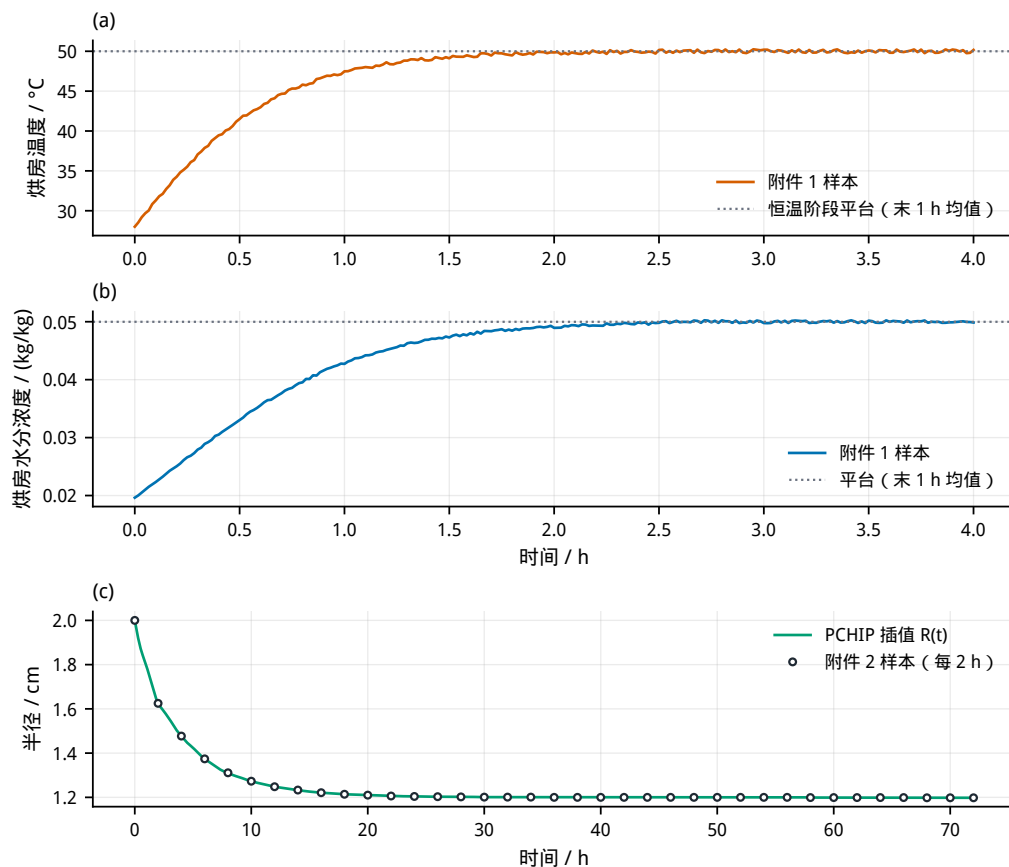


图 8 边界数据：附件 1 的烘房温度 (a) 与水分浓度 (b) 及恒温干燥阶段的平台值 (末 1 h 样本均值，假设 5)；附件 2 的半径样本与保单调 PCHIP 插值 $R(t)$ (c，为清晰起见每 2 h 标一个样本点)。

表 7 考虑尺寸变化的药材烘干过程水分浓度 (kg/kg；附录 4 物性；列为到药材中心的距离/cm，超出当前半径处为“-”)

时间/h	0	0.5	1	1.5	2	药材表面
6	1.7188	1.5370	1.0221	—	—	0.4205
12	0.7374	0.6544	0.4082	—	—	0.1669
18	0.4086	0.3686	0.2397	—	—	0.0892
24	0.2851	0.2610	0.1790	—	—	0.0673
30	0.2263	0.2094	0.1493	—	—	0.0595
36	0.1928	0.1796	0.1317	—	—	0.0560
42	0.1712	0.1604	0.1200	—	—	0.0541
48	0.1561	0.1468	0.1116	—	—	0.0530
烘干结束时间 (51.09 h)	0.1500	0.1413	0.1081	—	—	0.0526

固定物理位置的值由 $\xi = r/R(t)$ 网格上的三次样条取得；“药材表面”为 $\xi = 1$ 的节点值。

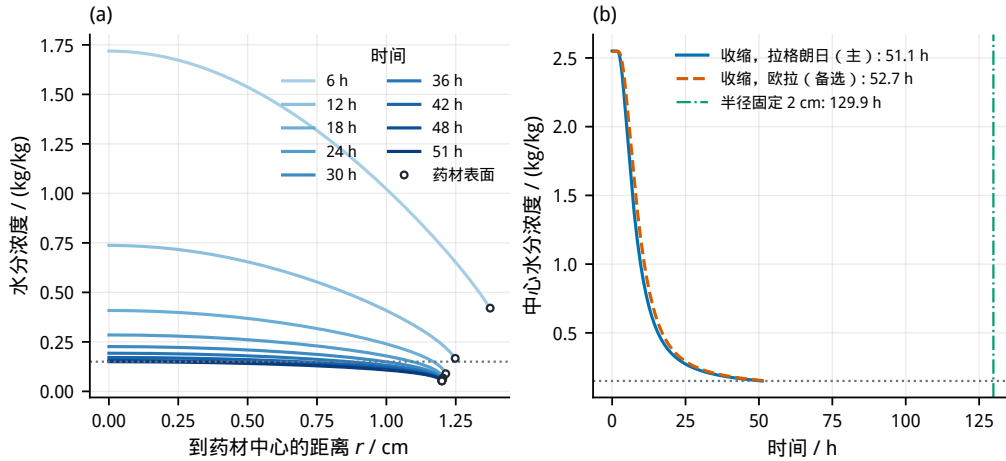


图 9 问题 4: 物理坐标下每 6 h 的水分浓度剖面与表面位置 (a, 空心圆为 $\xi = 1$ 的表面值, 超出当前半径处无定义); 中心水分浓度时程在主模型 (物质坐标)、实验室坐标备选与半径固定 2 cm 参照下的比较 (b)。

留空 (结果文件同样留空, 并另设“药材表面”列)。(ii) 收缩的作用可以定量解释。保持附录 4 的物性而把半径固定在 2 cm, 烘干需要 129.85 h, 是移动边界结果的 2.54 倍; 而扩散时间尺度 $\propto R^2$, 末半径对应的比值为 $(R_0/R_\infty)^2 = 2.79$ 。两个数字量级一致 (收缩需要时间完成, 故实际加速略小于上界), 说明**收缩通过缩短扩散路径主导了加速**, 末半径处的截面积只剩初始的 35.9%。(iii) 与问题 3 的 57.48 h 相比, 问题 4 更短 (51.09 h), 但这是两个效应叠加的结果: 附录 4 的扩散系数在高含水率段比附录 3 小约 5.4 倍 (使干燥变慢), 而收缩使之变快且占优。因此**不能**把问题 3 与问题 4 的时间差简单归因于“尺寸变化”, 必须用同物性对照 (第 (ii) 点) 才能分离两种效应——这是本文把固定半径参照一并计算的原因。(iv) 实验室坐标备选写法给出 52.65 h, 比主模型长 3.06% (表 16, 附录 A)。差异方向可以解释: 该写法丢弃了被退缩边界扫出的水分, 却同时丢弃了这部分物质所占的体积, 净效果是早期剖面偏湿 (图 9(b))。两者之差 3% 量级给出了“收缩如何处理”这一解释不确定性的规模。

8 模型检验

本节按验证与确认 (verification & validation) 的标准框架 [16-17] 检验前四节的结果: **验证**回答“方程解对了吗”, **确认**回答“解对了方程吗”。所有检验由**与求解器代码分离**的校验器完成——它只读取求解结果数组, 自行重算平衡量与解析解, 不调用求解器的任何内部函数 (附录 D 的 `pipelines/a/verify.py` 与 `pipelines/a/solver.py`)。表 8 汇总四个问题的校验结论, 下面逐项说明。

8.1 与解析级数解的对照

取常物性、恒定环境的无限长圆柱 (问题 1 的温度方程正是这一情形), 则(3)–(6) 有分离变量的解析解: 记 $\theta = (u - u_\infty)/(u_0 - u_\infty)$ 、 $Fo = at/R_0^2$ 、 $Bi = \beta R_0/a'$ (a 为扩

表 8 各问题的独立校验：水分守恒相对残差、极值原理、沿径向单调性、综合结论

问题	守恒残差	极值原理	单调性	通过
问题 1	6.74e-15	是	是	是
问题 2	1.97e-15	是	是	是
问题 3	2.68e-15	是	是	是
问题 4	3.84e-15	是	是	是

散系数、 β 为传递系数)，则

$$\theta(\xi, Fo) = \sum_{m=1}^{\infty} \frac{2 Bi}{(\lambda_m^2 + Bi^2) J_0(\lambda_m)} J_0(\lambda_m \xi) e^{-\lambda_m^2 Fo}, \quad \lambda_m J_1(\lambda_m) = Bi J_0(\lambda_m), \quad (19)$$

特征值 λ_m 位于 J_0 的相邻零点之间，由 Brent 法求得（取 400 项，截断误差远小于比较量级）[18-19]。分别以 $Bi = 1.389$ （温度， $a = \alpha$ ）与 $Bi_m = 3.240$ （把 D 冻结为 $D(C_0)$ 的水分方程）作对照，结果见表 14（附录 A）与图 10(b)：误差随 Δr 干净地二阶下降，观测阶 2.00（温度）与 2.00（水分）；在生产网格上最大绝对误差为 $1.0 \times 10^{-5} ^\circ\text{C}$ 与 $1.1 \times 10^{-5} \text{ kg/kg}$ ，比四位小数的舍入尺度 5×10^{-5} 小约一个数量级。这同时确认了离散格式的实现（控制体测度、半控制体上的 Robin 通量、轴心对称条件）是正确的。

8.2 网格收敛与观测阶

取 $\Delta r = 0.1/2^k \text{ cm}$ ， $k = 0, \dots, 6$ （ $k = 5$ 为生产网格），以最细网格为参照，在各问题表格所要求的时刻与位置上计算最大绝对误差 e_k 与观测阶 $p_k = \log_2(e_k/e_{k+1})$ ，结果见表 13（附录 A）与图 10(a)。观测阶为 2.05–2.32（问题 3、4 在粗网格上略低于 2，是因为烘干时刻本身随网格移动，误差同时包含“同一时刻的解误差”与“时刻差异”；细化后恢复二阶）。生产网格（ $k = 5$ ）上的误差为：问题 1 温度 1.4×10^{-6} 、问题 1 水分 2.2×10^{-5} 、问题 2 温度 8.6×10^{-7} 、问题 2 水分 6.9×10^{-7} 、问题 3 水分 9.5×10^{-6} 、问题 4 水分 1.9×10^{-6} ，全部小于四位小数的舍入尺度，因此表格与结果文件中的每一位小数都有意义。

烘干时间本身的收敛单列于表 15（附录 A）：生产网格上的 t_{end} 与最细网格相差 0.28 min（问题 3）与 0.03 min（问题 4）。以观测阶 $p = 2$ 对两级最细网格作 Richardson 外推 [20]， $t^* = t_6 + (t_6 - t_5)/3$ ，得极限估计 57.4743 h 与 51.0885 h，相应地生产网格的离散误差不超过 0.37 min 与 0.05 min，即 t_{end} 的两位小数（以小时计）是可靠的。

8.3 时间积分的无关性

表 9 给出在生产网格上改变时间积分设置的结果：把容差收紧一个量级、改用隐式 Runge-Kutta 方法 Radau IIA[12]、以及把最大步长强制限制为 $\Delta t \leq 3600, 900, 300 \text{ s}$ 。水分浓度的最大差异不超过 4.2×10^{-9} ，烘干时间的差异不超过 0.01 s；把容差放松两

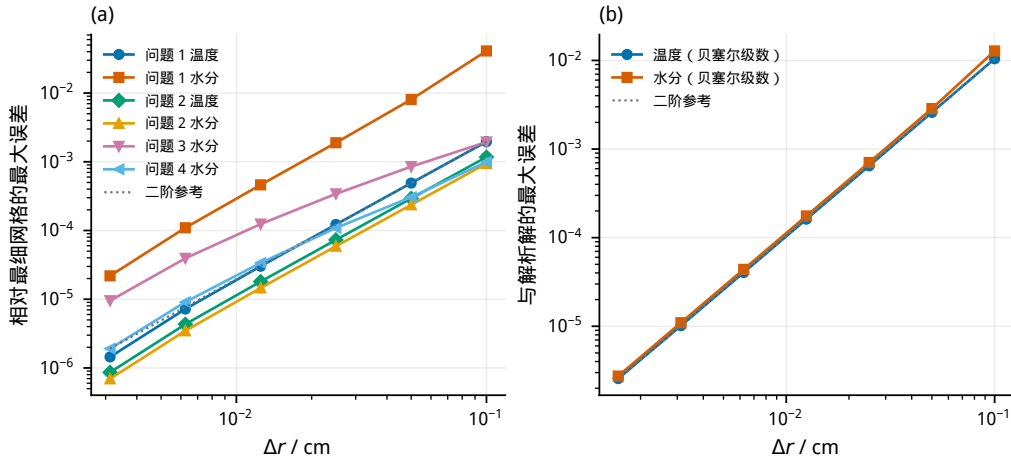


图 10 网格收敛：各问题在表格时刻与位置上相对最细网格 ($\Delta r = 0.1/2^6 \text{ cm}$) 的最大误差 (a) 与常物性算例相对贝塞尔级数解(19) 的最大误差 (b)；点线为二阶参考斜率。

个量级 ($\text{rtol} = 10^{-7}$, 表 9 第三行) 时差异仍远小于四位小数的舍入尺度。这说明结果与时间步长序列无关, 生产容差有充分余量。注: $\Delta t \leq 3600 \text{ s}$ 一行与生产设置逐位相同, 因为自适应 BDF 在本问题中从未取过超过 1 h 的步长, 该行证明的是“步长限制未被触发”。

表 9 时间积分无关性 (问题 3, 生产网格; 与生产设置 BDF, $\text{rtol}=1\text{e-}09$, $\text{atol}=1\text{e-}11$ 、步长不限的差异)

积分器、容差与最大步长	水分最大差	t_{end} 差/s	右端项调用次数
BDF, $\text{rtol}=1\text{e-}10$, $\text{atol}=1\text{e-}12$	3.61e-09	-0.01	32741
Radau, $\text{rtol}=1\text{e-}09$, $\text{atol}=1\text{e-}11$	4.24e-09	-0.01	34619
BDF, $\text{rtol}=1\text{e-}07$, $\text{atol}=1\text{e-}09$	2.55e-07	0.31	10523
BDF, $\text{rtol}=1\text{e-}09$, $\text{atol}=1\text{e-}11$, $\Delta t \leq 3600 \text{ s}$	0.00e+00	0.00	22528
BDF, $\text{rtol}=1\text{e-}09$, $\text{atol}=1\text{e-}11$, $\Delta t \leq 900 \text{ s}$	1.66e-09	-0.01	23048
BDF, $\text{rtol}=1\text{e-}09$, $\text{atol}=1\text{e-}11$, $\Delta t \leq 300 \text{ s}$	2.28e-09	-0.01	24915

$\Delta t \leq$ 行为限制最大步长的自适应 BDF (其余设置同生产); 步长限制改变的只是步序列, 结果差异应在容差量级。

8.4 守恒量与定性性质

离散守恒。 按命题 4.1, $W + Q_m$ 是半离散系统的**精确线性不变量**。校验器独立地由输出数组重算 $W(t) - W(0)$ 并与积分器内累积的 $-Q_m(t)$ 比较: 四个问题的相对残差均在 10^{-15} 量级 (表 8、图 11), 达到双精度舍入极限; 问题 1 的常物性温度方程另有能量不变量, 残差同量级。这一检验的价值在于它检验的是**离散格式** (控制体测度、界面通量的成对相消、边界通量的施加方式) 而非时间积分精度——任何一处不一致都会立即表现为 $O(1)$ 的残差。

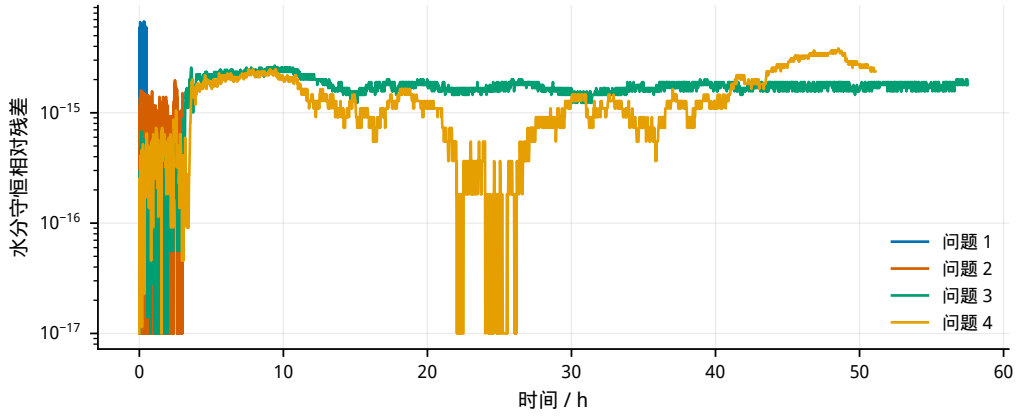


图 11 离散水分存量的变化与积分器内累积的表面通量之差，按最终失水量归一：四个问题的相对残差全程保持在 10^{-15} 量级（双精度舍入），确认离散格式严格守恒。

极值原理与单调性。 校验器在全部输出时刻检查三项（命题 4.2）：(i) T 与 C 分别落在由初值和环境值张成的包络

$$[\min(T_0, \min T_{\text{air}}), \max(T_0, \max T_{\text{air}})], \quad [\min(C_0, \min C_{\text{air}}), C_0]$$

之内；(ii) C 沿 r 单调不增；(iii) $\max_r C$ 在中心取得。三项在四个问题中全部成立（表 8），因此不存在非物理的负含水率、超调或内部极值。

判据的一致性。 校验器独立检查：在 $t < t_{\text{end}}$ 的全部输出行上 $\max_i C_i \geq C^*$ ，在 $t \geq t_{\text{end}}$ 的全部行上 $\max_i C_i < C^*$ ，且 t_{end} 处的剖面最大值等于 C^* （误差 $< 10^{-12}$ ）。这确认事件求根定位的确实是判据首次成立的时刻。

退化检验。 问题 4 的移动边界代码在 $R \equiv R_0$ 时应退化为固定域格式。两种写法与固定域求解器之差为 9.4×10^{-14} ，为舍入量级。

8.5 独立实现的交叉检验与一维化的端部效应

为检验实现本身以及假设 1，本文另行编写了一个二维轴对称 (r, z) 有限体积程序，直接在物理坐标下离散，不共享一维求解器的任何代码，轴向网格 $\Delta z = 0.50 \text{ cm}$ ：

1. **代码独立性：** 端面取绝热/不透条件时，二维解在中截面上应与一维解完全一致。实测最大差异为 6.1×10^{-9} （水分）与 $5.8 \times 10^{-8} \text{ }^\circ\text{C}$ （温度），为积分容差量级——两套独立编写的程序给出同一答案。
2. **端部效应：** 端面同样施加 Robin 条件时，中截面的水分浓度与一维解相差 2.3×10^{-5} ，中心点在烘干时刻的差异为 2.4×10^{-6} ，烘干时间由 58.50 h 变为 58.50 h，相对变化 -0.004% 。

端部效应之小与第 4.2 小节的量级判断一致：水分的轴向 Fourier 数上界仅 0.18，端面的影响来不及扩散到中截面。图 12 显示轴向水分分布只在端面附近的薄层内偏离中截面值。因此假设 1（一维径向化）被定量确认。

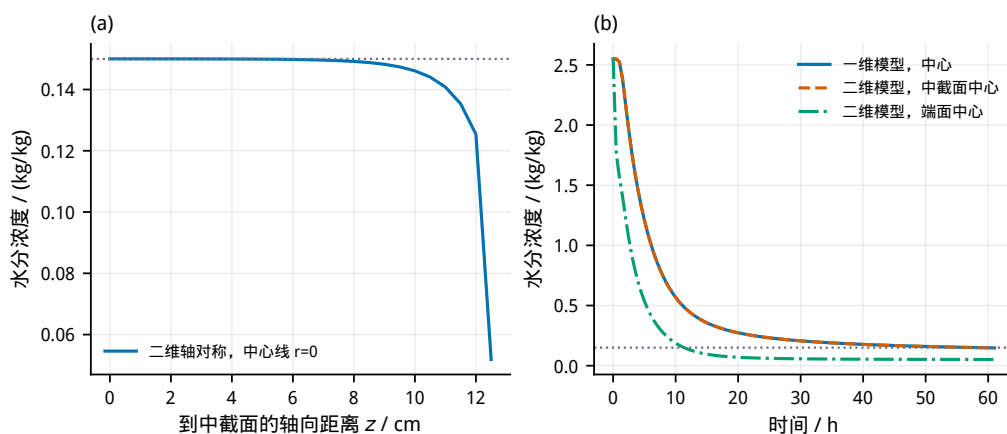


图 12 端部效应：一维烘干结束时刻二维模型中心线（ $r = 0$ ）上沿轴向的水分浓度分布（a）；一维解与二维解在中截面与端面处的中心水分时程（b）。端面附近的额外干燥只影响端部薄层，不影响中截面，烘干时间的相对变化仅 -0.004% 。

8.6 结果文件与表格的一致性检查

四个结果工作簿由附件 3 的模板生成（不重建工作簿结构），并由自动契约逐项检查：工作表名称、首行与首列与模板逐格一致，行数与时间步长符合题目要求，数值区无非数字单元、小数位不超过四位，问题 4 中超出当前半径的单元为空且“药材表面”列完整。全部契约检查通过（附录 C）。论文表格与结果文件由同一份求解结果生成，因此逐位一致：result3.xlsx 共 3449 个时间行（末行 57.4833 h）、result4.xlsx 共 3066 行。

8.7 灵敏度与不确定性

对六个可能存在不确定性的量分别取 $\times 0.8, 0.9, 1.1, 1.2$ 重新求解，以烘干时间 t_{end} 为指标，用 $\pm 10\%$ 的中心差分估计弹性 $E_p = (\Delta t_{\text{end}}/t_{\text{end}})/(\Delta p/p)$ （表 10、图 13）：

1. **烘房温度平台**的弹性最大（ -1.603 ）： D 的 Arrhenius 因子把温度的相对变化放大了 E/T_K 倍（一个数量级以上）。表 10 给出平台值 $\times 1.1$ 与 $\times 1.2$ 时的烘干时间，缩短幅度相当可观；但实际工艺中提高温度受药材有效成分热稳定性的限制（第 9 节）。
2. **扩散系数前因子** D_0 的弹性为 -0.893 （问题 3）与 -0.889 （问题 4），接近 -1 ：这正是扩散控制过程的特征（ $t \propto 1/D$ ）。 D_0 是经验拟合量，其不确定性直接、几乎等比例地传递到烘干时间，是本模型最主要的参数不确定性来源。
3. **收缩幅度** s 的弹性为 -1.130 ，与 $t \propto R^2$ 的标度一致，再次确认收缩通过缩短扩散路径起作用。

表 10 参数 $\pm 10\%$ 、 $\pm 20\%$ 时的烘干时间 (h) 与弹性 $E = (\Delta t_{\text{end}}/t_{\text{end}})/(\Delta p/p)$

参数	基准	$\times 0.8$	$\times 0.9$	$\times 1.1$	$\times 1.2$	弹性
h	57.48	57.50	57.49	57.47	57.47	-0.001
h_m	57.48	59.18	58.22	56.90	56.44	-0.114
D_0 (问题 3)	57.48	70.25	63.14	52.87	49.05	-0.893
C_{air}	57.48	57.31	57.38	57.60	57.74	0.019
T_{air} 平台	57.48	80.60	67.72	49.29	42.71	-1.603
收缩幅度 s (问题 4)	51.09	63.48	57.07	45.53	40.40	-1.130
D_0 (问题 4)	51.09	62.35	56.09	47.01	43.62	-0.889

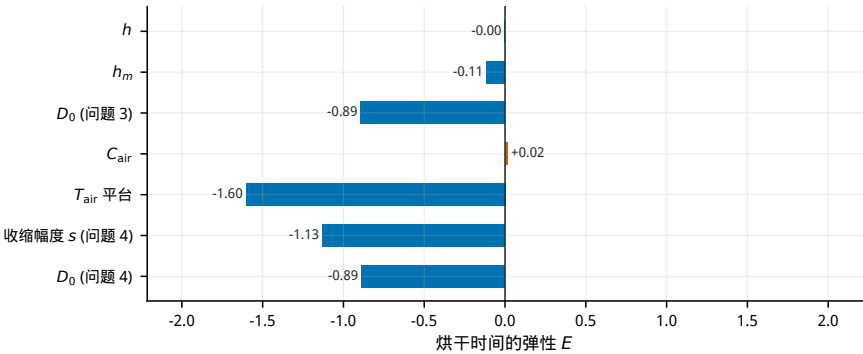


图 13 烘干时间对各参数的弹性 ($\pm 10\%$ 中心差分)。负值表示参数增大使烘干变快。烘房温度平台、扩散系数前因子与收缩幅度是主导因素，对流换热系数几乎无影响。

4. **传质系数** h_m 的弹性只有 -0.114 , **换热系数** h 几乎为零 (-0.001), **烘房湿度** C_{air} 为 0.019 。结论: 在题给参数下, 过程由内部扩散控制, 强化外部对流 (加大风速) 对缩短烘干时间几乎无效。

不确定性预算。 把上述结果整理为 t_{end} 的误差来源: 数值离散 (网格 ≤ 0.37 min, 时间积分 ≤ 0.01 s, 一维化 -0.004%) 合计远小于 0.1% ; 解释性选择 (平台取法 -0.53% 、界面平均 0.11%) 不超过 1% ; 参数不确定性 (若 D_0 有 $\pm 20\%$ 的拟合误差) 使 t_{end} 在表 10 的 $\times 0.8$ 与 $\times 1.2$ 两行之间变动, 约为两成; 而判据的表述 (“各处” vs “平均”) 可带来 -37.28% 的改变。因此本文报告的 t_{end} 在题给模型内部是精确的, 其现实不确定性主要来自经验公式的拟合精度与工艺判据的定义, 而非数值方法。

8.8 备选解释的对照

第 4 节以来所作的每一处解释都给出了备选结果 (表 11): 恒温阶段平台值的三种取法相差不超过 -0.53% ; 界面系数三种取法相差不超过 0.11% ; 问题 4 的实验室坐标写法相差 3.06% ; 半径固定的对照相差 154.17% (用于分离收缩与物性两种效应); 判据改用体积平均相差 -37.28% 。可见除判据本身外, 所有解释性选择都只带来百分之几以内的变化, 主结果是稳健的。

表 11 备选解释与离散化选择下的烘干时间 (相对主结果的变化)

备选	t_{end}/h	变化/%
平台取末样本 (MDR-0003 方案 2)	57.18	-0.53
平台取 $50\text{ }^{\circ}\text{C}$ 、 0.05 (方案 3)	57.48	-0.00
界面系数算术平均	57.46	-0.03
界面系数调和平均	57.54	0.11
问题 4 欧拉写法 (含对流项)	52.65	3.06
问题 4 物性、半径固定 2 cm	129.85	154.17
以平均含水率 < 0.15 为判据	36.05	-37.28

9 模型评价与推广

9.1 模型的优点

1. **一个模型覆盖四个问题。**从守恒律出发建立的(3)–(7) 是唯一的定解问题, 四个问题只更换物性、时间范围与边界是否移动; 所有结果由同一套程序、同一次云端运行产生。
2. **离散格式具有可证明的结构性质。**有限体积离散使水分 (及常物性下的能量) 平衡成为半离散系统的**精确线性不变量** (命题 4.1), 并满足离散极值原理 (命题 4.2)。这

两条不是事后的数值观察，而是格式的性质，保证了长时间积分不会累积质量误差、不会产生非物理的负含水率。

3. **精度可控且被独立确认。**输出点与网格节点重合（无插值误差），收敛阶、解析解误差与 Richardson 外推共同保证四位小数有意义；校验器与求解器代码分离，二维轴对称程序是完全独立的第二实现。题目未规定之处（边界条件形式、平台外推、收缩域写法、判据分辨率、界面平均）都给出了备选结果与差异幅度（表 11），使结论的稳健性可检验而不是靠断言。

9.2 题给模型省略的两项物理机制及其影响范围

题给参数体系把表面传质写成 $-D\partial_r C = h_m(C_s - C_{\text{air}})$ ，即把药材的干基含水率与烘房空气的“水分浓度”放在同一量纲下比较，并且能量方程不含蒸发潜热。主结果必须、也确实严格按题给参数计算；本小节在题给模型之外量化这两项被省略的机制能把答案推到什么范围。全部计算在问题 3 的设定下进行（固定半径、附录 3 物性），不改变第 4-7 节的任何结果。

(a) 蒸发潜热的后验一致性诊断。由问题 3 的解可以反算题给传质律隐含的蒸发质量通量 $j_w = \rho_{s0} h_m(C_s - C_{\text{air}})$ ，其中干物质表观密度 $\rho_{s0} = \rho(C_0)/(1 + C_0) = 275 \text{ kg/m}^3$ 。把其潜热负荷折算为“等效表面温降” $\Delta T_{\text{lat}} = L_v(T_s) j_w / h$ （即若要在能量上支付这一蒸发速率，表面需要比烘房低多少度）：初始时 $\Delta T_{\text{lat}} = 54 \text{ K}$ ，3 h 时 20.1 K，12 h 时 2.4 K，降到 1 K 以下需要 16.1 h（图 14(a)）。全过程蒸发的水量为 0.838 kg 每米长度，对应潜热 2.01 MJ/m，而题给能量边界条件全程供入的净对流热量只有 84.1 kJ/m（等于显热储量的增加），二者相差 24 倍。**结论：**题给模型在最初十余小时显著高估药材温度（真实过程中表面接近湿球温度），因而高估 D 、低估烘干时间；16.1 h 之后潜热的等效温降已不足 1 K，模型自愈。直接把潜热项加入能量边界条件是不可行的——题给传质律没有湿球反馈（蒸发速率不随表面降温而减小），会把表面温度压到 0°C 以下，问题不适定。

(b) 能量限制（恒速干燥段）的界定分析。把烘房的“水分浓度”按湿空气含湿量（kg 水/kg 干空气）解读（饱和蒸气压用 Magnus 公式 [21]，湿球温度由绝热饱和方程求根 [22]），则初始状态（ 28°C 与附件 1 的首个含湿量样本）对应相对湿度 82%、湿球温度 25.5°C ；恒温阶段（ 49.999°C 、 0.04999 kg/kg ）对应相对湿度 61%、湿球温度 41.7°C （湿球温差 8.3 K），数值上合理。经典干燥理论 [23, 2] 指出，恒速段的蒸发通量不能超过对流供热所能维持的湿球极限

$$j_{\text{max}}(t) = \frac{h [T_{\text{air}}(t) - T_{\text{wb}}(t)]}{L_v}, \quad (20)$$

恒温阶段 $j_{\text{max}} = 0.312 \text{ kg}/(\text{m}^2 \text{ h})$ ，而题给模型的初始蒸发通量是该上限的 22 倍。把表面损失律改为 $\min\{h_m(C_s - C_{\text{air}}), j_{\text{max}}/\rho_{s0}\}$ （这保持了命题 4.1 的守恒不变量，且当表

面变干后自动退化为题给模型)，得烘干时间 67.40 h，比主结果长 17.3%；上限在 17.0 h 后不再起作用。把上限取 $\times 0.5$ 与 $\times 2$ ，结果分别为 85.19 h 与 59.96 h（表 12）。另一方面，若全程按湿球极限蒸发，得到纯能量下界 21.4 h（-62.7%）——主结果远高于该下界，再次说明题给设定下的烘干时长由内部扩散控制而非能量供给控制。

(c) 平衡含水率驱动力。物理上传质的驱动力应为 $C_s - C_{eq}$ ， C_{eq} 是药材在烘房温湿度下的平衡含水率（由等温吸附曲线给出，题目未提供）。以常数 $C_{eq} \in \{0.075, 0.100, 0.125\}$ 替换 C_{air} ，烘干时间分别为 58.45、61.31、69.82 h（+1.7%、+6.7%、+21.5%）。当 $C_{eq} \rightarrow C^* = 0.15$ 时烘干时间发散：**终点判据必须与烘房湿度相匹配**，否则物理上不可达——这是本模型给出的最有工艺价值的结论之一。值得注意的是，这一单调性并非普遍成立：驱动力减小使表面更湿，而 $D \propto e^{-a/C}$ 使湿表面的扩散系数大得多，两种效应竞争；在附录 2 的律（ $a = 0.89$ ）下较大的 C_{eq} 反而可能略快（表面结壳减轻，case hardening），附录 3（ $a = 0.45$ ）下则单调变慢。

表 12 模型评价扩展（MDR-0008，问题 3 设定）：能量限制与平衡含水率驱动力下的烘干时间及相对主结果的变化

情形	参数	t_{end}/h	变化/%
题给模型（主结果）	—	57.48	0.0
蒸发通量受能量限制（湿球极限 $\times 0.5$ ）	$j_{max} = 0.156 \text{ kg}/(\text{m}^2 \cdot \text{h})$	85.19	48.2
蒸发通量受能量限制（湿球极限 $\times 1$ ）	$j_{max} = 0.312 \text{ kg}/(\text{m}^2 \cdot \text{h})$	67.40	17.3
蒸发通量受能量限制（湿球极限 $\times 2$ ）	$j_{max} = 0.623 \text{ kg}/(\text{m}^2 \cdot \text{h})$	59.96	4.3
驱动力取 $C_s - C_{eq}$ （平衡含水率）	$C_{eq} = 0.075 \text{ kg}/\text{kg}$	58.45	1.7
驱动力取 $C_s - C_{eq}$ （平衡含水率）	$C_{eq} = 0.1 \text{ kg}/\text{kg}$	61.31	6.7
驱动力取 $C_s - C_{eq}$ （平衡含水率）	$C_{eq} = 0.125 \text{ kg}/\text{kg}$	69.82	21.5
纯能量下界（全程按湿球极限蒸发）	$j_{max} = 0.312 \text{ kg}/(\text{m}^2 \cdot \text{h})$	21.42	-62.7

湿球极限 $j_{max} = h(T_{air} - T_{wb})/L_v$ 随烘房状态变化，表中给出恒温阶段的值。

9.3 局限

1. **表面条件的物理内涵。**主模型按题给参数把烘房含湿量与药材干基含水率直接相减，未使用等温吸附曲线；第 9.2 小节的 (c) 给出了这一简化的影响区间（+1.7% 至 +21.5%）。
2. **蒸发潜热未耦合。**见 (a)、(b)；界定性结果表明真实烘干时间很可能比主结果长 17.3% 量级（表 12）。注意情形 (b) 只限制质量通量而未改变能量方程（温度仍按题给模型演化），因此是**界定分析**而非完整的耦合模型。
3. **收缩的各向同性与已知性。**问题 4 直接使用给定的 $R(t)$ ，未建立含水率 $\rightarrow$ 体积的本构关系，也未考虑轴向收缩与开裂；均匀收缩假设在出现表面硬壳时会失真。

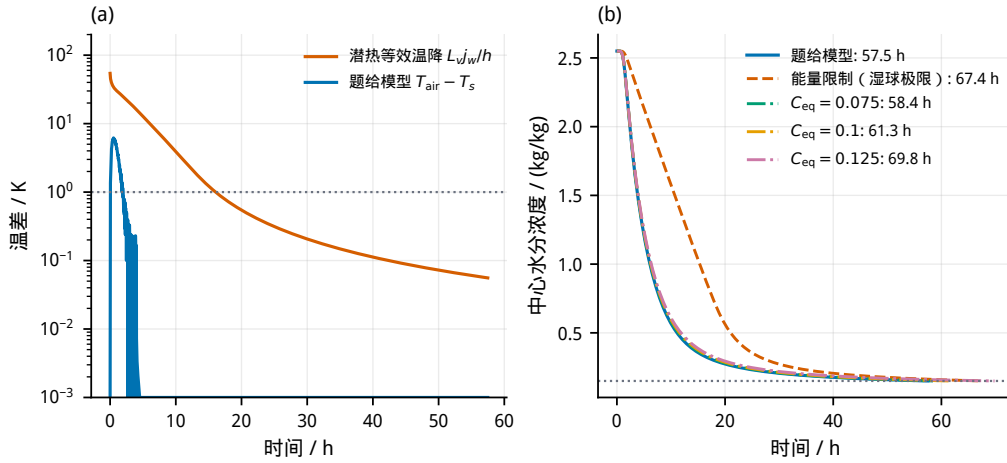


图 14 模型评价扩展：(a) 题给模型蒸发率所对应的潜热等效温降 L_{vjw}/h 与模型实际的表面温差 $T_{air} - T_s$ （纵轴对数坐标，点线为 1 K）——前十余小时前者远大于后者，说明题给能量方程未支付蒸发潜热；(b) 题给模型、能量限制情形与三档平衡含水率驱动力下的中心水分浓度时程，点线为判据 0.15 kg/kg。图 (a) 中模型温差在 2–4 h 间因烘房温度样本的 $\pm 0.3^\circ\text{C}$ 噪声而在零附近改变符号，负值在对数坐标下被截断，故呈现振荡带。

4. **物性与传递系数的常数化。** h 、 h_m 全程恒定（收缩使表面积减小、气流条件改变时它们会变化）； D 的经验公式在 $C \rightarrow 0$ 时的外推缺乏数据支撑，而正是这一区间决定了烘干末段的长尾。
5. **一维与均匀性。** 端部效应已量化为 -0.004% （可忽略），但各向异性、非均质与批次内个体差异未建模。
6. **烘房条件的外推。** 14 400 s 之后的烘房状态由平台值外推（假设 5），虽经三种取法对照（ $\leq -0.53\%$ ），但若实际工艺存在升温/降温程序，结果会改变——由弹性 -1.603 可直接估算。

9.4 推广与工艺含义

1. **模型的推广。** 本文的离散框架（Landau 坐标 + 有限体积 + 分段自适应 BDF + 事件求根）可直接推广到：球形与板状药材（只需更换控制体测度中的几何权 ξ^{d-1} ）；由含水率驱动的自适应收缩（把 $R(t)$ 换成由 $\bar{C}$ 决定的本构关系，方程形式不变）；含潜热耦合与吸附平衡的完整模型（第 9.2 小节）；以及以能耗或品质为目标的**烘房温湿度轨迹优化**——把 $T_{air}(t)$ 作为控制变量、以 t_{end} 与热敏成分降解量为目标，本文的求解器即可作为状态方程反复调用。
2. **工艺含义。** 灵敏度结果给出明确的优先级：(i) 提高烘房温度最有效（弹性 -1.603 ），但受热敏成分降解的约束，宜采用先低温排湿、后升温的分段程序；(ii) **减小特征尺寸最可靠**——切片或减小直径使 $t_{end} \propto R^2$ 下降（问题 4 中收缩本身就把烘干时间压缩到 $1/2.54$ ）；(iii) 加大风速（提高 h 、 h_m ）几乎无效（弹性 -0.001 、 -0.114 ），反而增加能耗；(iv) 降低烘房湿度的收益也很小（0.019），但必须保证 C_{eq} 显著低于 C^* ，

否则终点不可达；(v) 判据的定义值得与工艺方商榷：按“各处低于 0.15”需 57.48 h，而此时体积平均含水率已低至 0.1243 kg/kg，大量物料被过度干燥，能耗与品质均受损。

AI 工具使用声明

本研究在问题分析、方案比较、程序编写与调试、云端实验组织、数值核验、图表绘制以及论文文字组织等环节使用了生成式 AI 工具（Anthropic Claude Code，模型 Claude Fable 5.1）；所有数据处理、计算与排版在 Modal 云平台执行。使用过程与人工核验方式详见支撑材料中的《AI 工具使用详情.pdf》。

参考文献

- [1] ZOGZAS N P, MAROULIS Z B, MARINOS-KOURIS D. Moisture diffusivity data compilation in foodstuffs [J/OL]. *Drying Technology*, 1996, 14(10): 2225-2253. DOI: 10.1080/07373939608917205.
- [2] MUJUMDAR A S. *Handbook of industrial drying*[M]. 4th ed. Boca Raton, FL: CRC Press, 2014.
- [3] WHITAKER S. Simultaneous heat, mass, and momentum transfer in porous media: a theory of drying [J/OL]. *Advances in Heat Transfer*, 1977, 13: 119-203. DOI: 10.1016/S0065-2717(08)70223-5.
- [4] FRITSCH F N, CARLSON R E. Monotone piecewise cubic interpolation[J/OL]. *SIAM Journal on Numerical Analysis*, 1980, 17(2): 238-246. DOI: 10.1137/0717021.
- [5] RATTI C. Shrinkage during drying of foodstuffs[J/OL]. *Journal of Food Engineering*, 1994, 23(1): 91-105. DOI: 10.1016/0260-8774(94)90125-2.
- [6] LUIKOV A V. *Heat and mass transfer in capillary-porous bodies*[M]. Oxford: Pergamon Press, 1966.
- [7] INCROPERA F P, DEWITT D P, BERGMAN T L, et al. *Fundamentals of heat and mass transfer*[M]. 6th ed. Hoboken, NJ: John Wiley & Sons, 2007.
- [8] PATANKAR S V. *Numerical heat transfer and fluid flow*[M]. Washington, D.C.: Hemisphere Publishing Corporation, 1980.
- [9] QUARTERONI A, SACCO R, SALERI F. *Numerical mathematics*[M]. 2nd ed. Berlin: Springer, 2007.
- [10] VIRTANEN P, GOMMERS R, OLIPHANT T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J/OL]. *Nature Methods*, 2020, 17(3): 261-272. DOI: 10.1038/s41592-019-0686-2.
- [11] HARRIS C R, MILLMAN K J, VAN DER WALT S J, et al. Array programming with NumPy[J/OL]. *Nature*, 2020, 585(7825): 357-362. DOI: 10.1038/s41586-020-2649-2.
- [12] HAIRER E, WANNER G. *Solving ordinary differential equations II: Stiff and differential-algebraic problems* [M]. 2nd ed. Berlin: Springer, 1996.
- [13] SHAMPINE L F, REICHEL T M W. The MATLAB ODE suite[J/OL]. *SIAM Journal on Scientific Computing*, 1997, 18(1): 1-22. DOI: 10.1137/S1064827594276424.
- [14] LANDAU H G. Heat conduction in a melting solid[J/OL]. *Quarterly of Applied Mathematics*, 1950, 8(1): 81-94. DOI: 10.1090/qam/33441.
- [15] CRANK J. *Free and moving boundary problems*[M]. Oxford: Clarendon Press, 1984.
- [16] ROACHE P J. *Verification and validation in computational science and engineering*[M]. Albuquerque, NM: Hermosa Publishers, 1998.
- [17] OBERKAMPF W L, ROY C J. *Verification and validation in scientific computing*[M]. Cambridge: Cambridge University Press, 2010.
- [18] CARSLAW H S, JAEGER J C. *Conduction of heat in solids*[M]. 2nd ed. Oxford: Clarendon Press, 1959.
- [19] CRANK J. *The mathematics of diffusion*[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [20] RICHARDSON L F, GAUNT J A. The deferred approach to the limit[J/OL]. *Philosophical Transactions of the Royal Society A*, 1927, 226: 299-361. DOI: 10.1098/rsta.1927.0008.
- [21] ALDUCHOV O A, ESKRIDGE R E. Improved Magnus form approximation of saturation vapor pressure [J/OL]. *Journal of Applied Meteorology*, 1996, 35(4): 601-609. DOI: 10.1175/1520-0450(1996)035<0601:IMFAOS>2.0.CO;2.
- [22] ASHRAE. *ASHRAE handbook—fundamentals, chapter 1: Psychrometrics*[M]. Atlanta, GA: American Society of Heating, Refrigerating and Air-Conditioning Engineers, 2017.

- [23] SHERWOOD T K. The drying of solids—I[J/OL]. Industrial & Engineering Chemistry, 1929, 21(1): 12-16.
DOI: 10.1021/ie50229a004.

A 补充表格

本附录收录正文引用的大型表格：网格收敛表（表 13）、与解析级数解的对照表（表 14）、烘干时间的网格收敛表（表 15）以及问题 4 备选写法的结果表（表 16）。

表 13 网格收敛：以 $\Delta r = 0.1/2^6$ cm 为参照的最大绝对误差与观测阶数

量	k	$\Delta r/\text{cm}$	最大误差	观测阶
问题 1 温度	0	0.1000	1.96e-03	2.00
问题 1 温度	1	0.0500	4.90e-04	2.00
问题 1 温度	2	0.0250	1.22e-04	2.02
问题 1 温度	3	0.0125	3.02e-05	2.07
问题 1 温度	4	0.0063	7.20e-06	2.31
问题 1 温度	5	0.0031	1.45e-06	—
问题 1 水分	0	0.1000	4.10e-02	2.35
问题 1 水分	1	0.0500	8.05e-03	2.09
问题 1 水分	2	0.0250	1.89e-03	2.03
问题 1 水分	3	0.0125	4.61e-04	2.07
问题 1 水分	4	0.0063	1.10e-04	2.32
问题 1 水分	5	0.0031	2.19e-05	—
问题 2 温度	0	0.1000	1.17e-03	2.00
问题 2 温度	1	0.0500	2.93e-04	2.00
问题 2 温度	2	0.0250	7.30e-05	2.02
问题 2 温度	3	0.0125	1.81e-05	2.06
问题 2 温度	4	0.0063	4.32e-06	2.32
问题 2 温度	5	0.0031	8.64e-07	—
问题 2 水分	0	0.1000	9.45e-04	2.00
问题 2 水分	1	0.0500	2.37e-04	2.00
问题 2 水分	2	0.0250	5.90e-05	2.02
问题 2 水分	3	0.0125	1.46e-05	2.07
问题 2 水分	4	0.0063	3.47e-06	2.32
问题 2 水分	5	0.0031	6.95e-07	—
问题 3 水分	0	0.1000	1.94e-03	1.19
问题 3 水分	1	0.0500	8.50e-04	1.32
问题 3 水分	2	0.0250	3.41e-04	1.46
问题 3 水分	3	0.0125	1.24e-04	1.66
问题 3 水分	4	0.0063	3.94e-05	2.05
问题 3 水分	5	0.0031	9.51e-06	—
问题 4 水分	0	0.1000	9.96e-04	1.71
问题 4 水分	1	0.0500	3.06e-04	1.49
问题 4 水分	2	0.0250	1.09e-04	1.69
问题 4 水分	3	0.0125	3.38e-05	1.89
问题 4 水分	4	0.0063	9.09e-06	2.25
问题 4 水分	5	0.0031	1.91e-06	—

表 14 与常物性 Robin 圆柱贝塞尔级数解的最大绝对误差及观测阶数

量	k	$\Delta r/\text{cm}$	最大误差	观测阶
温度 (Bi=1.389)	0	0.1000	1.05e-02	2.01
温度 (Bi=1.389)	1	0.0500	2.59e-03	2.00
温度 (Bi=1.389)	2	0.0250	6.47e-04	2.00
温度 (Bi=1.389)	3	0.0125	1.62e-04	2.00
温度 (Bi=1.389)	4	0.0063	4.05e-05	2.00
温度 (Bi=1.389)	5	0.0031	1.01e-05	1.98
温度 (Bi=1.389)	6	0.0016	2.57e-06	–
水分 (Bi _m =3.240)	0	0.1000	1.28e-02	2.16
水分 (Bi _m =3.240)	1	0.0500	2.86e-03	2.02
水分 (Bi _m =3.240)	2	0.0250	7.05e-04	2.00
水分 (Bi _m =3.240)	3	0.0125	1.76e-04	2.00
水分 (Bi _m =3.240)	4	0.0063	4.39e-05	2.00
水分 (Bi _m =3.240)	5	0.0031	1.10e-05	2.00
水分 (Bi _m =3.240)	6	0.0016	2.74e-06	–

表 15 烘干时间随网格加密的收敛 (与最细网格之差, 单位 min)

问题	k	$\Delta r/\text{cm}$	t_{end}/h	与最细网格之差/min
问题 3	0	0.1000	58.5042	61.702
问题 3	1	0.0500	57.9133	26.249
问题 3	2	0.0250	57.6483	10.350
问题 3	3	0.0125	57.5377	3.711
问题 3	4	0.0063	57.4952	1.166
问题 3	5	0.0031	57.4805	0.279
问题 3	6	0.0016	57.4758	–
问题 4	0	0.1000	51.3337	14.702
问题 4	1	0.0500	51.1852	5.791
问题 4	2	0.0250	51.1224	2.021
问题 4	3	0.0125	51.0990	0.620
问题 4	4	0.0063	51.0914	0.165
问题 4	5	0.0031	51.0893	0.035
问题 4	6	0.0016	51.0887	–

表 16 备选写法 (eulerian, MDR-0006) 下的问题 4 水分浓度 (kg/kg)

时间/h	0	0.5	1	1.5	2	药材表面
6	1.9518	1.7824	1.2569	–	–	0.5669
12	0.8734	0.7763	0.4859	–	–	0.2144
18	0.4593	0.4130	0.2652	–	–	0.1020
24	0.3072	0.2805	0.1902	–	–	0.0713
30	0.2380	0.2198	0.1555	–	–	0.0610
36	0.2000	0.1861	0.1356	–	–	0.0567
42	0.1761	0.1647	0.1226	–	–	0.0545
48	0.1596	0.1499	0.1135	–	–	0.0532
烘干结束时间 (52.65 h)	0.1500	0.1413	0.1081	–	–	0.0526

B 支撑材料文件清单

下表列出支撑材料压缩包内的全部文件（相对路径）及其用途。压缩包内另含 MANIFEST.json，记录每个文件的 SHA-256。

文件	说明
AI_USAGE.md	AI 工具使用记录（源文件）
AI 工具使用详情.pdf	AI 工具使用详情
MANIFEST.json	支撑材料成员清单与 SHA-256
Makefile	本地快捷命令（只提交云端任务）
README.md	仓库说明与复现入口
REPRODUCE.md	复现说明
UNLICENSE	公有领域声明（Unlicense）
docs/ARCHITECTURE.md	工程规范、建模标准、决策记录
docs/CONTRIBUTING.md	工程规范、建模标准、决策记录
docs/DATA_NOTES.md	工程规范、建模标准、决策记录
docs/ENGINEERING_STANDARD.md	工程规范、建模标准、决策记录
docs/MODELING_STANDARD.md	工程规范、建模标准、决策记录

文件	说明
docs/RESULTS.md	工程规范、建模标准、决策记录
docs/RUNBOOK.md	工程规范、建模标准、决策记录
docs/adr/0001-cloud-only-execution.md	工程规范、建模标准、决策记录
docs/adr/0002-stage-run-manifest-model.md	工程规范、建模标准、决策记录
docs/adr/0003-independent-repos-vendored-runtime.md	工程规范、建模标准、决策记录
docs/mdr/0000-template.md	工程规范、建模标准、决策记录
docs/mdr/0001-governing-equations-and-radial-reduction.md	工程规范、建模标准、决策记录
docs/mdr/0002-transfer-coefficients-for-q2-q4.md	工程规范、建模标准、决策记录
docs/mdr/0003-chamber-conditions-interpolation-and-plateau.md	工程规范、建模标准、决策记录
docs/mdr/0004-numerical-scheme.md	工程规范、建模标准、决策记录
docs/mdr/0005-drying-end-criterion-and-output-ranges.md	工程规范、建模标准、决策记录
docs/mdr/0006-moving-boundary-formulation-q4.md	工程规范、建模标准、决策记录
docs/mdr/0007-verification-and-sensitivity-plan.md	工程规范、建模标准、决策记录
docs/mdr/0008-latent-heat-and-equilibrium-moisture-extension.md	工程规范、建模标准、决策记录
docs/mdr/0009-derived-paper-quantities.md	工程规范、建模标准、决策记录
docs/mdr/README.md	工程规范、建模标准、决策记录
docs/problem_brief.md	工程规范、建模标准、决策记录

文件	说明
<code>forge/__init__.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/config.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/context.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/contracts.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/events.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/hashing.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/manifest.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/modal_app.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/packaging.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/pdfqa.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/plotting.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/runner.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/tex.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/xlsx.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>manuscript/ai_usage.tex</code>	论文 LaTeX 源码
<code>manuscript/main.tex</code>	论文 LaTeX 源码
<code>manuscript/references.bib</code>	论文 LaTeX 源码
<code>manuscript/sections/00_abstract.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/01_restatement.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/02_analysis.tex</code>	论文 LaTeX 源码

文件	说明
manuscript/sections/03_assumptions.tex	论文 LaTeX 源码
manuscript/sections/04_models.tex	论文 LaTeX 源码
manuscript/sections/08_validation.tex	论文 LaTeX 源码
manuscript/sections/09_evaluation.tex	论文 LaTeX 源码
manuscript/sections/10_ai_statement.tex	论文 LaTeX 源码
manuscript/sections/ai_usage_body.tex	论文 LaTeX 源码
manuscript/sections/appendix_repro.tex	论文 LaTeX 源码
pipelines/__init__.py	本题建模与求解程序
pipelines/a/__init__.py	本题建模与求解程序
pipelines/a/common.py	本题建模与求解程序
pipelines/a/contracts.py	本题建模与求解程序
pipelines/a/derived.py	本题建模与求解程序
pipelines/a/extension.py	本题建模与求解程序
pipelines/a/figures.py	本题建模与求解程序
pipelines/a/outputs.py	本题建模与求解程序
pipelines/a/physics.py	本题建模与求解程序
pipelines/a/science.py	本题建模与求解程序
pipelines/a/solver.py	本题建模与求解程序
pipelines/a/stages.py	本题建模与求解程序
pipelines/a/verify.py	本题建模与求解程序
pipelines/common/__init__.py	通用阶段：数据接入、门禁、论文、质量检查、打包发布
pipelines/common/stages.py	通用阶段：数据接入、门禁、论文、质量检查、打包发布
pipelines/common/validation.py	通用阶段：数据接入、门禁、论文、质量检查、打包发布
pyproject.toml	项目元数据与静态检查配置
runs/20260912-104036-184d092/convergence/convergence.json	
runs/20260912-104036-184d092/convergence/events.jsonl	结构化运行日志
runs/20260912-104036-184d092/convergence/manifest.json	阶段清单(输入输出散列、环境、参数)

文件	说明
runs/20260912-104036-184d092/convergence/numbers.json	
runs/20260912-104036-184d092/convergence/raw_t_dry.json	
runs/20260912-104036-184d092/derived/derived.json	
runs/20260912-104036-184d092/derived/events.jsonl	结构化运行日志
runs/20260912-104036-184d092/derived/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-104036-184d092/derived/numbers.json	
runs/20260912-104036-184d092/extension/diagnostic.npz	
runs/20260912-104036-184d092/extension/events.jsonl	结构化运行日志
runs/20260912-104036-184d092/extension/extension.json	
runs/20260912-104036-184d092/extension/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-104036-184d092/extension/numbers.json	
runs/20260912-104036-184d092/extension/variants.npz	
runs/20260912-104036-184d092/extension/verification_extension.json	
runs/20260912-104036-184d092/figures/events.jsonl	论文图(矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figure_index.json	论文图(矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/fig_chamber.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/fig_chamber.png	论文图(矢量 PDF 与预览 PNG)

文件	说明
runs/20260912-104036-184d092/figures/figures/f ig_conservation.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_conservation.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_convergence.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_convergence.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_endeffect.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_endeffect.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_extension.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_extension.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_properties.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_properties.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_q1_history.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_q1_history.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_q1_profiles.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_q1_profiles.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_q2_history.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_q2_history.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-104036-184d092/figures/figures/f ig_q2_profiles.pdf	论文图 (矢量 PDF 与预览 PNG)

文件	说明
runs/20260912-104036-184d092/figures/figures/fig_q2_profiles.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-104036-184d092/figures/figures/fig_q3.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-104036-184d092/figures/figures/fig_q3.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-104036-184d092/figures/figures/fig_q4.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-104036-184d092/figures/figures/fig_q4.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-104036-184d092/figures/figures/fig_sensitivity.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-104036-184d092/figures/figures/fig_sensitivity.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-104036-184d092/figures/manifest.json	论文图（矢量 PDF 与预览 PNG）
runs/20260912-104036-184d092/ingest/data/附件 1_Sheet1.parquet	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-104036-184d092/ingest/data/附件 2_Sheet1.parquet	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-104036-184d092/ingest/events.jsonl	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-104036-184d092/ingest/inventory.json	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-104036-184d092/ingest/manifest.json	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-104036-184d092/ingest/problem_text.txt	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-104036-184d092/lint/events.jsonl	静态检查报告
runs/20260912-104036-184d092/lint/lint_report.json	静态检查报告
runs/20260912-104036-184d092/lint/manifest.json	静态检查报告
runs/20260912-104036-184d092/lint/mypy.txt	静态检查报告

文件	说明
runs/20260912-104036-184d092/lint/ruff_check.txt	静态检查报告
runs/20260912-104036-184d092/lint/ruff_format.txt	静态检查报告
runs/20260912-104036-184d092/q1/events.jsonl	结构化运行日志
runs/20260912-104036-184d092/q1/history.json	
runs/20260912-104036-184d092/q1/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-104036-184d092/q1/numbers.json	
runs/20260912-104036-184d092/q1/profiles_moist.parquet	
runs/20260912-104036-184d092/q1/profiles_temp.parquet	
runs/20260912-104036-184d092/q1/snapshots.npz	
runs/20260912-104036-184d092/q1/summary.json	
runs/20260912-104036-184d092/q1/tables.json	
runs/20260912-104036-184d092/q1/verification_q1.json	
runs/20260912-104036-184d092/q2/events.jsonl	
runs/20260912-104036-184d092/q2/history.json	
runs/20260912-104036-184d092/q2/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-104036-184d092/q2/numbers.json	
runs/20260912-104036-184d092/q2/profiles_moist.parquet	
runs/20260912-104036-184d092/q2/profiles_temp.parquet	
runs/20260912-104036-184d092/q2/snapshots.npz	
runs/20260912-104036-184d092/q2/summary.json	
runs/20260912-104036-184d092/q2/tables.json	
runs/20260912-104036-184d092/q2/verification_q2.json	
runs/20260912-104036-184d092/q3/dry_profile.npz	

文件	说明
runs/20260912-104036-184d092/q3/drying_q3.json	结构化运行日志
runs/20260912-104036-184d092/q3/events.jsonl	
runs/20260912-104036-184d092/q3/history.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-104036-184d092/q3/manifest.json	
runs/20260912-104036-184d092/q3/numbers.json	
runs/20260912-104036-184d092/q3/profiles_moist.parquet	
runs/20260912-104036-184d092/q3/profiles_temp.parquet	
runs/20260912-104036-184d092/q3/snapshots.npz	
runs/20260912-104036-184d092/q3/summary.json	
runs/20260912-104036-184d092/q3/tables.json	
runs/20260912-104036-184d092/q3/verification_q3.json	
runs/20260912-104036-184d092/q4/dry_profile.npz	
runs/20260912-104036-184d092/q4/drying_q4.json	
runs/20260912-104036-184d092/q4/drying_q4_alt.json	
runs/20260912-104036-184d092/q4/drying_q4_fixed.json	结构化运行日志
runs/20260912-104036-184d092/q4/events.jsonl	
runs/20260912-104036-184d092/q4/history.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-104036-184d092/q4/manifest.json	
runs/20260912-104036-184d092/q4/numbers.json	
runs/20260912-104036-184d092/q4/profiles_moist.parquet	
runs/20260912-104036-184d092/q4/profiles_moist_alt.parquet	
runs/20260912-104036-184d092/q4/profiles_temp.parquet	
runs/20260912-104036-184d092/q4/snapshots.npz	

文件	说明
runs/20260912-104036-184d092/q4/summary.json	
runs/20260912-104036-184d092/q4/tables.json	
runs/20260912-104036-184d092/q4/tables_alt.json	
runs/20260912-104036-184d092/q4/verification_q4.json	
runs/20260912-104036-184d092/qa/events.jsonl	论文与结果文件质量门禁报告
runs/20260912-104036-184d092/qa/manifest.json	论文与结果文件质量门禁报告
runs/20260912-104036-184d092/qa/qa_report.json	论文与结果文件质量门禁报告
runs/20260912-104036-184d092/results/events.jsonl	结果工作簿与求解输出
runs/20260912-104036-184d092/results/manifest.json	结果工作簿与求解输出
runs/20260912-104036-184d092/results/result1.xlsx	结果工作簿与求解输出
runs/20260912-104036-184d092/results/result2.xlsx	结果工作簿与求解输出
runs/20260912-104036-184d092/results/result3.xlsx	结果工作簿与求解输出
runs/20260912-104036-184d092/results/result4.xlsx	结果工作簿与求解输出
runs/20260912-104036-184d092/results/results_report.json	结果工作簿与求解输出
runs/20260912-104036-184d092/sensitivity/events.jsonl	结构化运行日志
runs/20260912-104036-184d092/sensitivity/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-104036-184d092/sensitivity/numbers.json	
runs/20260912-104036-184d092/sensitivity/sensitivity.json	

文件	说明
runs/20260912-104036-184d092/tables/events.jsonl	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/manifest.json	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _alternatives.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _alternatives.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _analytic.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _analytic.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _convergence.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _convergence.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _convergence_drytime.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _convergence_drytime.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _extension.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _extension.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _q1_moist.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _q1_moist.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _q1_temp.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _q1_temp.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab _q2_moist.csv	论文表(LaTeX 片段与 CSV)

文件	说明
runs/20260912-104036-184d092/tables/tables/tab_q2_moist.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_q2_temp.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_q2_temp.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_q3_moist.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_q3_moist.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_q4_moist.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_q4_moist.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_q4_moist_alt.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_q4_moist_alt.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_sensitivity.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_sensitivity.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_time_independence.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_time_independence.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_verification.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/tables/tables/tab_verification.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-104036-184d092/test/coverage.json	测试报告与覆盖率
runs/20260912-104036-184d092/test/events.jsonl	测试报告与覆盖率
runs/20260912-104036-184d092/test/junit.xml	测试报告与覆盖率

文件	说明
runs/20260912-104036-184d092/test/manifest.json	测试报告与覆盖率
runs/20260912-104036-184d092/test/pytest.txt	测试报告与覆盖率
runs/20260912-104036-184d092/test/test_report.json	测试报告与覆盖率
runs/20260912-104036-184d092/validate/data_profile.json	数据契约校验报告与数据画像
runs/20260912-104036-184d092/validate/events.jsonl	数据契约校验报告与数据画像
runs/20260912-104036-184d092/validate/manifest.json	数据契约校验报告与数据画像
runs/20260912-104036-184d092/validate/validation_report.json	数据契约校验报告与数据画像
runs/20260912-104036-184d092/verification/analytic.json	
runs/20260912-104036-184d092/verification/axisym_snapshots.npz	
runs/20260912-104036-184d092/verification/events.jsonl	结构化运行日志
runs/20260912-104036-184d092/verification/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-104036-184d092/verification/numbers.json	
runs/20260912-104036-184d092/verification/verification_report.json	
tests/__init__.py	单元 / 性质 / 集成测试
tests/conftest.py	单元 / 性质 / 集成测试
tests/integration/__init__.py	单元 / 性质 / 集成测试
tests/integration/test_ingest_validate.py	单元 / 性质 / 集成测试
tests/unit/__init__.py	单元 / 性质 / 集成测试
tests/unit/test_config.py	单元 / 性质 / 集成测试
tests/unit/test_contracts.py	单元 / 性质 / 集成测试
tests/unit/test_derived.py	单元 / 性质 / 集成测试
tests/unit/test_drying.py	单元 / 性质 / 集成测试

文件	说明
tests/unit/test_extension.py	单元 / 性质 / 集成测试
tests/unit/test_hashing.py	单元 / 性质 / 集成测试
tests/unit/test_manifest.py	单元 / 性质 / 集成测试
tests/unit/test_packaging_tex.py	单元 / 性质 / 集成测试
tests/unit/test_runner.py	单元 / 性质 / 集成测试
tests/unit/test_xlsx.py	单元 / 性质 / 集成测试
tools/__init__.py	本地控制面：提交云端任务、 下载、校验散列
tools/cli.py	本地控制面：提交云端任务、 下载、校验散列

C 复现记录

论文中的全部数值、表格与图形均由同一次云端运行（run 20260912-104036-184 d092，代码版本 133907f7e0094708390d1c38a4364b428ecbe78f，构建日期 2026-09-12）生成。各阶段状态与输出摘要如下。

阶段	状态	用时/s	输出摘要 (SHA-256 前 12 位)
convergence	completed	39.0	e7c97c4bb47b
derived	completed	0.0	ff7b3a16d642
extension	completed	13.0	0fb634d88917
figures	completed	22.0	9e12255aa52a
ingest	completed	1.0	8a1cfe99b7db
lint	completed	21.0	ed8b1eef0b56
paper	running	0.0	
q1	completed	2.0	ade62f528fb3
q2	completed	9.0	96d113bd82d9
q3	completed	13.0	9883d8006f55
q4	completed	97.0	af58b9c29cfa
qa	completed	3.0	de321a8f5541
results	completed	11.0	a16209cb604a
sensitivity	completed	52.0	433ff4353625
tables	completed	1.0	4e22ebca15e0
test	completed	17.0	89c56adc82b8

阶段	状态	用时/s	输出摘要 (SHA-256 前 12 位)
validate	completed	0.0	7386c362f110
verification	completed	36.0	746753d0aa5c

复现步骤

本文的全部结果由一次云端运行产生。安装 Modal 客户端并完成登录后，在仓库根目录依次执行：

1. `python3tools/cli.pyprovision`——验证云端工具链（T_EX Live、中文字体、科学计算栈）。
2. `python3tools/cli.pyrunningest,validate--new-run`——读入题目 PDF 与附件、记录其 SHA-256，并执行输入数据契约（时间列步长与单调性、半径单调不增、数值范围、无缺失）。
3. `python3tools/cli.pyrunq1,q2,q3,q4--sizemedium`——四个问题的求解与各自的独立校验。
4. `python3tools/cli.pyrunconvergence,verification,sensitivity,extension,derived`——收敛研究、解析解与二维交叉检验、灵敏度与备选解释、模型评价扩展、论文引用的派生量。
5. `python3tools/cli.pyrunresults,figures,tables`——由附件 3 的模板生成 `result1.xlsx-result4.xlsx`，生成矢量图与三线表。
6. `python3tools/cli.pyrunlint,test,paper,qa,package,release`——静态检查、单元与性质测试、论文排版、质量门禁、支撑材料打包与发布。

可追溯性

每个阶段目录下的 `manifest.json` 记录：阶段状态与用时、代码树摘要与 Git 提交、配置摘要、依赖阶段的输出摘要、参数、全部输入与输出文件的 SHA-256、度量指标、运行环境（Python 版本、平台、CPU 数、依赖包版本）。论文中的每一个数字都由某个阶段以数值宏的形式登记（`numbers.json`），或来自 `tables` 阶段生成的三线表；`.tex` 源文件中不存在手工键入的结果数字。随机性（若有）只经统一的随机种子接口设置。因此在相同的代码版本与输入下，重复运行得到逐位相同的结果与相同的支撑材料散列。

质量门禁

发布前必须全部通过：**lint** (ruffcheck、ruffformat--check、mypy)；**test** (全部单元与性质测试通过，附覆盖率报告)；**qa** (摘要 1 页、正文含参考文献不超过 30 页、无未定义引用、无 $\text{T}_\text{E}\text{X}$ 错误、字体全部嵌入、A4 页面、无过宽行、数值宏无冲突、含《AI 工具使用详情》、无身份信息、四个结果工作簿全部通过模板契约)；**package** (支撑材料成员与论文附录清单完全一致)。上表列出本次运行各阶段的状态与输出摘要。

D 完整程序

以下按文件收录建模、求解、检验、绘图、排版与打包的全部源程序，文件名与附录 B 一致。运行方式见附录 C。

Makefile

```
1 # Thin wrappers over tools/cli.py. Nothing here computes locally.
2 PY ?= python3
3 CLI := $(PY) tools/cli.py
4 STAGE ?= ingest
5 SIZE ?= small
6 RUN ?=
7 RUNFLAG := $(if $(RUN),--run-id $(RUN),)
8 FORCEFLAG := $(if $(FORCE),--force,)
9
10 .PHONY: fmt help provision run pipeline exec status logs runs download release lint test paper qa package smoke
11
12 help:
13     @echo "make provision          # verify cloud toolchain"
14     @echo "make run STAGE=q1 SIZE=medium # run one stage (RUN=<id> to continue a run, FORCE=1 to rerun)"
15     @echo "make pipeline                # ingest,validate,lint,test in one container"
16     @echo "make status | logs STAGE=q1 | runs | download STAGE=results"
17     @echo "make paper | qa | package | release VERSION=v1.0.0"
18
19 provision:
20     $(CLI) provision
21
22 run:
23     $(CLI) run $(STAGE) $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
24
25 pipeline:
26     $(CLI) run ingest,validate,lint,test $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
27
28 exec:
29     $(CLI) exec --code '$(CODE)' $(RUNFLAG)
30
31 status:
32     $(CLI) status $(RUNFLAG)
33
34 logs:
35     $(CLI) logs $(STAGE) $(RUNFLAG)
36
37 runs:
38     $(CLI) runs
39
40 download:
41     $(CLI) download $(RUNFLAG) $(if $(STAGE),--stage $(STAGE),)
```

```

42
43 lint:
44     $(CLI) run lint $(RUNFLAG) $(FORCEFLAG)
45
46 test:
47     $(CLI) run test $(RUNFLAG) $(FORCEFLAG)
48
49 paper:
50     $(CLI) run paper $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
51
52 qa:
53     $(CLI) run qa $(RUNFLAG) $(FORCEFLAG)
54
55 package:
56     $(CLI) run package $(RUNFLAG) $(FORCEFLAG)
57
58 release:
59     $(CLI) run release $(RUNFLAG) $(FORCEFLAG) && $(CLI) release --version $(VERSION) $(RUNFLAG) --force
60
61 smoke:
62     $(CLI) run ingest,validate,lint,test,paper,qa,package,release --new-run --param require_results=false --param
        ↪ allow_missing_results=true
63
64 fmt:
65     $(CLI) fmt $(RUNFLAG)

```

configs/default.toml

```

1 # Forge 运行时配置（默认档）。Stage 代码通过 ctx.cfg("a.b.c") 读取；
2 # `--profile <name>` 会叠加 configs/<name>.toml；`--param config_overrides='{...}'` 可临时覆盖。
3
4 [run]
5 seed = 20260912
6
7 [gates]
8 lint_required = ["ruff_check", "ruff_format", "mypy"]
9 mypy_paths = ["forge"]
10
11 [paper]
12 main = "main.tex"
13 # 需在同一 run 中已完成的 stage；其 tables/*.tex 与 figures/*.pdf/png 会被并入排版
14 sources = ["figures", "tables"]
15 max_body_pages = 30
16 max_overfull_pt = 20.0
17 code_globs = [
18     "forge/**/*.py", "pipelines/**/*.py", "tools/**/*.py", "tests/**/*.py",
19     "configs/*.toml", "configs/*.json", "Makefile", "pyproject.toml",
20 ]
21
22 [qa]
23 required_stages = ["lint", "test"]
24 results_stage = "results"
25 require_results = true
26 identity_denylist = [] # 硬门禁（正则）：出现即失败
27 identity_watchlist = ["大学", "学院", "赛区", "队号", "指导教师", "[\\w.-]+@[\\w.-]+\\.\\w+"] # 软提示：写入报告供复核
28
29 [package]
30 stages = ["ingest", "validate", "q1", "q2", "q3", "q4", "convergence", "verification", "sensitivity", "extension", "
        ↪ derived", "results", "figures", "tables", "lint", "test", "qa"]
31 code_globs = [
32     "forge/**/*.py", "pipelines/**/*.py", "tools/**/*.py", "tests/**/*.py", "configs/**",

```

```

33 "manuscript/**/*.tex", "manuscript/**/*.bib", "docs/**/*.md", "Makefile", "pyproject.toml",
34 "README.md", "UNLICENSE", "AI_USAGE.md",
35 ]
36 max_bytes = 20971520
37
38 [release]
39 paper_name = "论文.pdf"
40
41 # ---- Problem A: herb drying (see docs/mdr/) ----
42 [a.grid]
43 refine = 5 # production grid: dr = 0.1 / 2refine cm (nodes coincide with the 0.1 cm output grid)
44 convergence_levels = 7 # k = 0 ... levels-1 in the convergence study
45
46 [a.time]
47 method = "BDF"
48 rtol = 1e-9
49 atol = 1e-11
50
51 [a.chamber]
52 plateau = "mean_last_hour" # constant-temperature stage after 14400 s (MDR-0003): mean_last_hour | last_sample |
    ↪ nominal
53
54 [a.scheme]
55 face = "midpoint" # interface coefficient: midpoint | arithmetic | harmonic (MDR-0004)
56 formulation = "lagrangian" # moving boundary: lagrangian (dry matter conserved) | eulerian (MDR-0006)
57
58 [a.drying]
59 horizon_s = 432000 # first integration horizon for the drying-end search (5 days), doubled if needed
60 dt_out_s = 60
61
62 [a.verifcation]
63 axisym_nz = 25 # axial intervals of the independent 2-D model (dz = 0.5 cm)
64 axisym_refine = 0 # radial refinement of the 2-D model (dr = 0.1 cm)
65
66 [a.sensitivity]
67 factors = [0.8, 0.9, 1.1, 1.2]
68 workers = 8
69
70 [a.extension] # model-evaluation extension (MDR-0008), problem-3 setting
71 cap_factors = [0.5, 1.0, 2.0] # multiples of the wet-bulb energy limit  $h(T_{\text{air}} - T_{\text{wb}})/L_v$  on the evaporation flux
72 c_eq = [0.075, 0.10, 0.125] # equilibrium moisture replacing  $C_{\text{air}}$  in the driving force  $C_s - C_{\text{eq}}$ 

```

configs/project.json

```

1 {
2   "letter": "A",
3   "package": "a",
4   "slug": "herb-drying",
5   "title": "药材的烘干问题",
6   "app": "cumcm2026-a-herb-drying",
7   "volume": "cumcm2026-a-artifacts",
8   "problem_pdf": "A题.pdf",
9   "attachments_dir": "附件",
10  "templates_dir": "附件/附件3",
11  "result_files": [
12    "result1.xlsx",
13    "result2.xlsx",
14    "result3.xlsx",
15    "result4.xlsx"
16  ],
17  "github_repo": "cumcm2026-a-herb-drying"

```

```
18 }
```

forge/__init__.py

```
1 """Forge — cloud-only research engineering runtime (CUMCM 2026).
2
3 Every computation (data parsing, modelling, solving, tests, figures, LaTeX, PDF QA,
4 packaging) runs inside Modal. The local machine only edits text, drives git,
5 submits jobs, downloads artifacts and verifies transport hashes.
6 """
7
8 __version__ = "1.0.0"
```

forge/config.py

```
1 """TOML configuration with profile overlays and dotted access."""
2
3 from __future__ import annotations
4
5 import json
6 import tomllib
7 from pathlib import Path
8 from typing import Any
9
10 from .hashing import sha256_json
11
12 _MISSING = object()
13
14
15 def deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
16     out = dict(base)
17     for key, value in override.items():
18         if isinstance(value, dict) and isinstance(out.get(key), dict):
19             out[key] = deep_merge(out[key], value)
20         else:
21             out[key] = value
22     return out
23
24
25 class Config:
26     """Read-only dotted-access view over a merged configuration mapping."""
27
28     def __init__(self, data: dict[str, Any]) -> None:
29         self._data = json.loads(json.dumps(data))
30
31     def get(self, key: str, default: Any = None) -> Any:
32         cursor: Any = self._data
33         for part in key.split("."):
34             if not isinstance(cursor, dict) or part not in cursor:
35                 return default
36             cursor = cursor[part]
37         return cursor
38
39     def __getitem__(self, key: str) -> Any:
40         value = self.get(key, _MISSING)
41         if value is _MISSING:
42             raise KeyError(key)
43         return value
```

```

44
45     def section(self, key: str) -> dict[str, Any]:
46         value = self.get(key, {})
47         return dict(value) if isinstance(value, dict) else {}
48
49     def as_dict(self) -> dict[str, Any]:
50         return json.loads(json.dumps(self._data))
51
52     @property
53     def digest(self) -> str:
54         return sha256_json(self._data)
55
56
57 def load_project(repo: Path) -> dict[str, Any]:
58     return json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
59
60
61 def load_config(repo: Path, profile: str | None = None, overrides: dict[str, Any] | None = None) -> Config:
62     data = tomlib.loads((repo / "configs" / "default.toml").read_text(encoding="utf-8"))
63     if profile:
64         data = deep_merge(data, tomlib.loads((repo / "configs" / f"{profile}.toml").read_text(encoding="utf-8")))
65     if overrides:
66         data = deep_merge(data, overrides)
67     return Config(data)

```

forge/context.py

```

1  """Stage execution context: paths, config, params, logging, seeds and paper numbers."""
2
3  from __future__ import annotations
4
5  import json
6  import random
7  import re
8  import zlib
9  from pathlib import Path
10 from typing import Any
11
12 from .config import Config, load_config, load_project
13 from .events import EventLogger
14
15 _NUMBER_KEY = re.compile(r"^[A-Za-z]+$")
16
17
18 class StageContext:
19     """Everything a stage needs. Stages must write only under ``stage_dir``."""
20
21     def __init__(self, run_id: str, stage: str, params: dict[str, Any], repo: Path, vol: Path) -> None:
22         self.run_id = run_id
23         self.stage = stage
24         self.params = dict(params)
25         self.repo = repo
26         self.vol = vol
27         self.project = load_project(repo)
28         overrides = self.params.get("config_overrides") or None
29         self.config: Config = load_config(repo, self.params.get("profile"), overrides)
30         self.run_dir = vol / "runs" / run_id
31         self.stage_dir = self.run_dir / stage
32         self.stage_dir.mkdir(parents=True, exist_ok=True)
33         self.log = EventLogger(self.stage_dir / "events.jsonl", run_id, stage)
34         self._numbers: dict[str, str] = {}

```

```

35
36 # ---- raw inputs (immutable, at repo root, exactly as delivered by the organisers) ----
37 @property
38 def problem_pdf(self) -> Path:
39     return self.repo / self.project["problem_pdf"]
40
41 @property
42 def attachments(self) -> Path:
43     return self.repo / self.project["attachments_dir"]
44
45 @property
46 def templates(self) -> Path:
47     return self.repo / self.project["templates_dir"]
48
49 # ---- outputs ----
50 def out(self, *parts: str) -> Path:
51     path = self.stage_dir.joinpath(*parts)
52     path.parent.mkdir(parents=True, exist_ok=True)
53     return path
54
55 def dep(self, stage: str) -> Path:
56     """Directory of a completed dependency stage in the same run (raises otherwise)."""
57     directory = self.run_dir / stage
58     manifest = directory / "manifest.json"
59     if not manifest.exists():
60         raise RuntimeError(f"dependency stage '{stage}' has not run in run {self.run_id}")
61     status = json.loads(manifest.read_text(encoding="utf-8")).get("status")
62     if status != "completed":
63         raise RuntimeError(f"dependency stage '{stage}' has status {status!r} in run {self.run_id}")
64     return directory
65
66 def has_stage(self, stage: str) -> bool:
67     manifest = self.run_dir / stage / "manifest.json"
68     return manifest.exists() and json.loads(manifest.read_text(encoding="utf-8")).get("status") == "completed"
69
70 def dep_data(self, stage: str, name: str) -> Any:
71     return json.loads((self.dep(stage) / name).read_text(encoding="utf-8"))
72
73 # ---- parameters / configuration ----
74 def param(self, key: str, default: Any = None) -> Any:
75     return self.params.get(key, default)
76
77 def cfg(self, key: str, default: Any = None) -> Any:
78     return self.config.get(key, default)
79
80 # ---- determinism ----
81 def seed(self, salt: str = "") -> int:
82     base = int(self.cfg("run.seed", 0))
83     return (base + zlib.crc32(salt.encode("utf-8"))) % (2**31 - 1)
84
85 def seed_everything(self, salt: str = "") -> int:
86     value = self.seed(salt)
87     random.seed(value)
88     try:
89         import numpy as np
90
91         np.random.seed(value) # noqa: NPY002 - legacy global RNG used by third-party libraries
92     except ImportError: # pragma: no cover
93         pass
94     self.log.info("seed", salt=salt, value=value)
95     return value
96
97 # ---- numbers that flow into the paper as \val<Key> macros ----

```

```

98     def number(self, key: str, value: Any, fmt: str | None = None) -> str:
99         if not _NUMBER_KEY.match(key):
100             raise ValueError(f"paper number keys must be letters only (TeX macro names); got {key!r}")
101         if fmt is not None:
102             text = format(value, fmt)
103         elif isinstance(value, bool):
104             text = str(value)
105         elif isinstance(value, int):
106             text = str(value)
107         elif isinstance(value, float):
108             text = f"{value:.4f}"
109         else:
110             text = str(value)
111         self._numbers[key] = text
112         return text
113
114     def flush_numbers(self) -> None:
115         if not self._numbers:
116             return
117         path = self.stage_dir / "numbers.json"
118         merged: dict[str, str] = {}
119         if path.exists():
120             merged.update(json.loads(path.read_text(encoding="utf-8")))
121         merged.update(self._numbers)
122         path.write_text(json.dumps(merged, ensure_ascii=False, indent=2, sort_keys=True), encoding="utf-8")
123
124     # ---- json helpers ----
125     def write_json(self, name: str, obj: Any) -> Path:
126         path = self.out(name)
127         path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str), encoding="utf-8")
128         return path
129
130     def read_json(self, path: Path) -> Any:
131         return json.loads(path.read_text(encoding="utf-8"))

```

forge/contracts.py

```

1  """Lightweight DataFrame contracts (schema + range + monotonicity + custom checks)."""
2
3  from __future__ import annotations
4
5  import re
6  from collections.abc import Callable
7  from dataclasses import dataclass
8  from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13
14 @dataclass(frozen=True)
15 class Column:
16     name: str
17     kind: str = "any" # int | float | number | str | datetime | any
18     nullable: bool = False
19     min: float | None = None
20     max: float | None = None
21     regex: str | None = None
22     unique: bool = False
23     monotonic: str | None = None # increasing | non_decreasing | decreasing | non_increasing
24     allowed: tuple[Any, ...] | None = None

```

```

25     step: float | None = None # constant difference between consecutive values
26
27
28 @dataclass(frozen=True)
29 class FrameContract:
30     name: str
31     columns: tuple[Column, ...]
32     min_rows: int = 1
33     max_rows: int | None = None
34     extra_columns: str = "forbid" # forbid | allow
35     checks: tuple[Callable[[pd.DataFrame], list[str]], ...] = ()
36
37
38 def normalise_columns(df: pd.DataFrame) -> pd.DataFrame:
39     out = df.copy()
40     out.columns = pd.Index([str(c).strip() for c in out.columns])
41     return out
42
43
44 def validate_frame(df: pd.DataFrame, contract: FrameContract) -> dict[str, Any]:
45     df = normalise_columns(df)
46     errors: list[str] = []
47     warnings: list[str] = []
48     expected = [c.name for c in contract.columns]
49     missing = [c for c in expected if c not in df.columns]
50     extra = [c for c in df.columns if c not in expected]
51     if missing:
52         errors.append(f"missing columns: {missing}")
53     if extra and contract.extra_columns == "forbid":
54         errors.append(f"unexpected columns: {extra}")
55     n = len(df)
56     if n < contract.min_rows:
57         errors.append(f"rows {n} < min_rows {contract.min_rows}")
58     if contract.max_rows is not None and n > contract.max_rows:
59         errors.append(f"rows {n} > max_rows {contract.max_rows}")
60
61     for col in contract.columns:
62         if col.name not in df.columns:
63             continue
64         series = df[col.name]
65         nulls = int(series.isna().sum())
66         if nulls and not col.nullable:
67             errors.append(f"{col.name}: {nulls} null values")
68         values = series.dropna()
69         if col.kind in {"int", "float", "number"}:
70             numeric = pd.to_numeric(values, errors="coerce")
71             bad = int(numeric.isna().sum())
72             if bad:
73                 errors.append(f"{col.name}: {bad} non-numeric values")
74             numeric = numeric.dropna().astype(float)
75             if col.kind == "int" and len(numeric) and not np.all(np.mod(numeric.to_numpy(), 1) == 0):
76                 errors.append(f"{col.name}: non-integer values present")
77             if col.min is not None and len(numeric) and float(numeric.min()) < col.min:
78                 errors.append(f"{col.name}: min {numeric.min()} < {col.min}")
79             if col.max is not None and len(numeric) and float(numeric.max()) > col.max:
80                 errors.append(f"{col.name}: max {numeric.max()} > {col.max}")
81             if col.monotonic and len(numeric) > 1:
82                 diff = np.diff(numeric.to_numpy())
83                 ok = {
84                     "increasing": bool((diff > 0).all()),
85                     "non_decreasing": bool((diff >= 0).all()),
86                     "decreasing": bool((diff < 0).all()),
87                     "non_increasing": bool((diff <= 0).all()),

```

```

88         }[col.monotonic]
89         if not ok:
90             errors.append(f"{col.name}: not {col.monotonic}")
91         if col.step is not None and len(numeric) > 1:
92             diff = np.diff(numeric.to_numpy())
93             if not np.allclose(diff, col.step):
94                 errors.append(f"{col.name}: step is not constant {col.step} (min {diff.min()}, max {diff.max()})")
95     elif col.kind == "str":
96         if col.regex:
97             pattern = re.compile(col.regex)
98             bad_values = [str(v) for v in values.astype(str) if not pattern.fullmatch(str(v))]
99             if bad_values:
100                 errors.append(f"{col.name}: {len(bad_values)} values violate /{col.regex}/, e.g. {bad_values[:3]}")
101     elif col.kind == "datetime":
102         parsed = pd.to_datetime(values, errors="coerce")
103         bad = int(parsed.isna().sum())
104         if bad:
105             errors.append(f"{col.name}: {bad} unparsable datetimes")
106     if col.unique and values.duplicated().any():
107         errors.append(f"{col.name}: {int(values.duplicated().sum())} duplicated values")
108     if col.allowed is not None:
109         unexpected = sorted(set(values.unique()) - set(col.allowed), key=str)
110         if unexpected:
111             errors.append(f"{col.name}: values outside allowed set: {unexpected[:5]}")
112
113     for check in contract.checks:
114         try:
115             errors.extend(check(df))
116         except Exception as exc:
117             errors.append(f"check {getattr(check, '__name__', check)!r} raised {exc!r}")
118
119     return {
120         "contract": contract.name,
121         "ok": not errors,
122         "rows": n,
123         "columns": list(df.columns),
124         "errors": errors,
125         "warnings": warnings,
126     }

```

forge/events.py

```

1  """Structured JSONL event logging with a human-readable mirror on stdout."""
2
3  from __future__ import annotations
4
5  import json
6  import time
7  from datetime import UTC, datetime
8  from pathlib import Path
9  from typing import Any
10
11
12  def _short(value: Any, limit: int = 200) -> str:
13      text = value if isinstance(value, str) else json.dumps(value, ensure_ascii=False, default=str)
14      return text if len(text) <= limit else text[: limit - 1] + "..."
15
16
17  class EventLogger:
18      """Append-only event stream: one JSON object per line, mirrored to stdout."""
19

```

```

20 def __init__(self, path: Path, run_id: str, stage: str) -> None:
21     path.parent.mkdir(parents=True, exist_ok=True)
22     self._fh = path.open("a", encoding="utf-8")
23     self.path = path
24     self.run_id = run_id
25     self.stage = stage
26     self._t0 = time.monotonic()
27
28 def emit(self, event: str, level: str = "info", **fields: Any) -> None:
29     record = {
30         "ts": datetime.now(UTC).isoformat(timespec="milliseconds"),
31         "elapsed_s": round(time.monotonic() - self._t0, 3),
32         "run_id": self.run_id,
33         "stage": self.stage,
34         "level": level,
35         "event": event,
36         **fields,
37     }
38     self._fh.write(json.dumps(record, ensure_ascii=False, default=str) + "\n")
39     self._fh.flush()
40     extra = " ".join(f"{k}={_short(v)}" for k, v in fields.items())
41     print(f"[{record['elapsed_s']:9.3f}s] {level.upper():5s} {self.stage}:{event} {extra}", flush=True)
42
43 def info(self, event: str, **fields: Any) -> None:
44     self.emit(event, "info", **fields)
45
46 def warn(self, event: str, **fields: Any) -> None:
47     self.emit(event, "warn", **fields)
48
49 def error(self, event: str, **fields: Any) -> None:
50     self.emit(event, "error", **fields)
51
52 def metric(self, name: str, value: Any, **fields: Any) -> None:
53     self.emit("metric", "info", name=name, value=value, **fields)
54
55 def close(self) -> None:
56     if not self._fh.closed:
57         self._fh.close()

```

forge/hashing.py

```

1 """Content hashing helpers (SHA-256 everywhere; JSON hashed in canonical form)."""
2
3 from __future__ import annotations
4
5 import hashlib
6 import json
7 from collections.abc import Iterable
8 from pathlib import Path
9 from typing import Any
10
11 CHUNK = 1 << 20
12
13
14 def sha256_file(path: Path) -> str:
15     h = hashlib.sha256()
16     with path.open("rb") as fh:
17         for chunk in iter(lambda: fh.read(CHUNK), b""):
18             h.update(chunk)
19     return h.hexdigest()
20

```

```

21
22 def sha256_bytes(data: bytes) -> str:
23     return hashlib.sha256(data).hexdigest()
24
25
26 def sha256_json(obj: Any) -> str:
27     payload = json.dumps(obj, sort_keys=True, ensure_ascii=False, separators=(",", ":"), default=str)
28     return sha256_bytes(payload.encode("utf-8"))
29
30
31 def hash_tree(root: Path, ignore: Iterable[str] = ()) -> dict[str, dict[str, Any]]:
32     """Return ``{relative_posix_path: {"sha256": ..., "bytes": ...}}`` for every file under ``root``.
33
34     Any path component contained in ``ignore`` excludes the file (directory names or file names).
35     The result is sorted by path so that its canonical JSON digest is stable.
36     """
37     ignored = set(ignore)
38     out: dict[str, dict[str, Any]] = {}
39     if not root.exists():
40         return out
41     for path in sorted(root.rglob("*")):
42         if not path.is_file():
43             continue
44         rel = path.relative_to(root)
45         if ignored & set(rel.parts):
46             continue
47         out[rel.as_posix()] = {"sha256": sha256_file(path), "bytes": path.stat().st_size}
48     return out
49
50
51 def tree_digest(tree: dict[str, dict[str, Any]]) -> str:
52     return sha256_json({k: v["sha256"] for k, v in sorted(tree.items())})

```

forge/manifest.py

```

1  """Run/stage manifests: the unit of lineage, reproducibility and audit."""
2
3  from __future__ import annotations
4
5  import importlib.metadata
6  import json
7  import os
8  import platform
9  import sys
10 import traceback
11 from dataclasses import asdict, dataclass, field, fields
12 from datetime import UTC, datetime
13 from pathlib import Path
14 from typing import Any
15
16 from .hashing import hash_tree, tree_digest
17
18 TRACKED_PACKAGES = (
19     "numpy",
20     "scipy",
21     "pandas",
22     "pyarrow",
23     "openpyxl",
24     "matplotlib",
25     "scikit-learn",
26     "statsmodels",

```

```

27     "ortools",
28     "highspy",
29     "numba",
30     "cvxpy",
31     "networkx",
32     "pymupdf",
33     "pytest",
34     "hypothesis",
35     "ruff",
36     "mypy",
37 )
38 RECORD_FILES = ("manifest.json", "events.jsonl") # run records, excluded from the outputs digest
39
40
41 def utc_now() -> str:
42     return datetime.now(UTC).isoformat(timespec="seconds")
43
44
45 def runtime_info() -> dict[str, Any]:
46     packages: dict[str, str] = {}
47     for name in TRACKED_PACKAGES:
48         try:
49             packages[name] = importlib.metadata.version(name)
50         except importlib.metadata.PackageNotFoundError:
51             continue
52     return {
53         "python": sys.version.split()[0],
54         "platform": platform.platform(),
55         "cpu_count": os.cpu_count(),
56         "modal_task_id": os.getenv("MODAL_TASK_ID"),
57         "modal_is_remote": os.getenv("MODAL_IS_REMOTE") == "1" or os.getenv("MODAL_TASK_ID") is not None,
58         "packages": packages,
59         "env": {k: os.environ.get(k) for k in ("OMP_NUM_THREADS", "OPENBLAS_NUM_THREADS", "PYTHONHASHSEED")},
60     }
61
62
63 @dataclass
64 class Manifest:
65     schema: str = "forge.manifest/1"
66     run_id: str = ""
67     stage: str = ""
68     status: str = "running" # running | completed | failed
69     started_at: str = field(default_factory=utc_now)
70     finished_at: str | None = None
71     duration_s: float | None = None
72     code_ref: dict[str, Any] = field(default_factory=dict)
73     params: dict[str, Any] = field(default_factory=dict)
74     config_digest: str = ""
75     deps: dict[str, str] = field(default_factory=dict)
76     inputs: dict[str, dict[str, Any]] = field(default_factory=dict)
77     outputs: dict[str, dict[str, Any]] = field(default_factory=dict)
78     outputs_digest: str = ""
79     metrics: dict[str, Any] = field(default_factory=dict)
80     runtime: dict[str, Any] = field(default_factory=dict)
81     error: dict[str, Any] | None = None
82
83 @classmethod
84 def start(
85     cls,
86     run_id: str,
87     stage: str,
88     params: dict[str, Any],
89     code_ref: dict[str, Any],

```

```

90     config_digest: str,
91     deps: dict[str, str],
92     inputs: dict[str, dict[str, Any]],
93 ) -> Manifest:
94     manifest = cls(
95         run_id=run_id,
96         stage=stage,
97         params=params,
98         code_ref=code_ref,
99         config_digest=config_digest,
100         deps=deps,
101         inputs=inputs,
102     )
103     manifest.runtime = runtime_info()
104     return manifest
105
106 def complete(self, stage_dir: Path, metrics: dict[str, Any]) -> None:
107     self.outputs = hash_tree(stage_dir, ignore=RECORD_FILES)
108     self.outputs_digest = tree_digest(self.outputs)
109     self.metrics = json.loads(json.dumps(metrics, default=str))
110     self.status = "completed"
111     self._finish()
112
113 def fail(self, exc: BaseException) -> None:
114     self.error = {"type": type(exc).__name__, "message": str(exc), "traceback": traceback.format_exc()}
115     self.status = "failed"
116     self._finish()
117
118 def _finish(self) -> None:
119     self.finished_at = utc_now()
120     started = datetime.fromisoformat(self.started_at)
121     self.duration_s = round((datetime.fromisoformat(self.finished_at) - started).total_seconds(), 3)
122
123 def write(self, path: Path) -> None:
124     path.parent.mkdir(parents=True, exist_ok=True)
125     path.write_text(json.dumps(asdict(self), ensure_ascii=False, indent=2, default=str), encoding="utf-8")
126
127 @classmethod
128 def read(cls, path: Path) -> Manifest:
129     raw = json.loads(path.read_text(encoding="utf-8"))
130     known = {f.name for f in fields(cls)}
131     return cls(**{k: v for k, v in raw.items() if k in known})
132
133 def verify_outputs(self, stage_dir: Path) -> list[str]:
134     """Return a list of mismatches between the manifest and the files on disk (empty == verified)."""
135     problems: list[str] = []
136     current = hash_tree(stage_dir, ignore=RECORD_FILES)
137     for rel, meta in self.outputs.items():
138         if rel not in current:
139             problems.append(f"missing: {rel}")
140         elif current[rel]["sha256"] != meta["sha256"]:
141             problems.append(f"sha256 mismatch: {rel}")
142     for rel in current:
143         if rel not in self.outputs:
144             problems.append(f"unexpected file: {rel}")
145     return problems

```

forge/modal_app.py

```
1 """Modal application — the only compute surface of this repository.
2
3 Local side (tools/cli.py) only submits:
4     modal run forge/modal_app.py --stage ingest --run-id <id> --params '{...}' --size small
5 Inside the container the repository is mounted read-only at /repo and the persistent
6 volume at /vol; stages write exclusively to /vol/runs/<run_id>/<stage>/.
7 """
8
9 from __future__ import annotations
10
11 import contextlib
12 import io
13 import json
14 import os
15 import sys
16 import traceback
17 from pathlib import Path
18
19 import modal
20
21 _HERE = Path(__file__).resolve()
22 REPO = _HERE.parents[1]
23
24
25 def _load_project() -> dict:
26     for candidate in (REPO / "configs" / "project.json", Path("/repo/configs/project.json")):
27         if candidate.exists():
28             return json.loads(candidate.read_text(encoding="utf-8"))
29     raise FileNotFoundError("configs/project.json not found next to forge/modal_app.py")
30
31
32 PROJECT = _load_project()
33 app = modal.App(PROJECT["app"])
34 volume = modal.Volume.from_name(PROJECT["volume"], create_if_missing=True)
35
36 APT_PACKAGES = [
37     "texlive-xetex",
38     "texlive-lang-chinese",
39     "texlive-latex-base",
40     "texlive-latex-recommended",
41     "texlive-latex-extra",
42     "texlive-science",
43     "texlive-pictures",
44     "texlive-bibtex-extra",
45     "texlive-fonts-recommended",
46     "fonts-noto-cjk",
47     "fonts-noto-core",
48     "fonts-droid-fallback",
49     "fonts-wqy-microhei",
50     "fontconfig",
51     "poppler-utils",
52     "zip",
53     "unzip",
54 ]
55
56 PIP_PACKAGES = [
57     "numpy==2.2.6",
58     "scipy==1.15.3",
59     "pandas==2.2.3",
60     "pyarrow==20.0.0",
61     "openpyxl==3.1.5",
62     "matplotlib==3.10.3",
```

```

62     "scikit-learn==1.6.1",
63     "statsmodels==0.14.4",
64     "ortools==9.12.4544",
65     "highspy==1.10.0",
66     "numba==0.61.2",
67     "cvxpy==1.6.5",
68     "networkx==3.4.2",
69     "pydantic==2.11.4",
70     "pymupdf==1.26.7",
71     "pytest==8.3.5",
72     "pytest-cov==6.1.1",
73     "hypothesis==6.131.9",
74     "ruff==0.11.10",
75     "mypy==1.15.0",
76     "pandas-stubs==2.2.3.250308",
77     "types-openpyxl==3.1.5.20250602",
78 ]
79 SKIP_PARTS = {
80     ".git",
81     "artifacts",
82     "releases",
83     ".forge",
84     "__pycache__",
85     ".mypy_cache",
86     ".ruff_cache",
87     ".pytest_cache",
88     ".hypothesis",
89     ".DS_Store",
90 }
91
92
93 def _ignore(path: Path) -> bool:
94     return bool(SKIP_PARTS & set(path.parts)) or path.suffix in {".pyc", ".pyo"}
95
96
97 image = (
98     modal.Image.debian_slim(python_version="3.11")
99     .apt_install(*APT_PACKAGES)
100     .pip_install(*PIP_PACKAGES)
101     .env(
102         {
103             "OMP_NUM_THREADS": "1",
104             "OPENBLAS_NUM_THREADS": "1",
105             "MKL_NUM_THREADS": "1",
106             "MPLBACKEND": "Agg",
107             "MPLCONFIGDIR": "/tmp/mpl",
108             "PYTHONHASHSEED": "0",
109             "PYTHONUNBUFFERED": "1",
110             "PYTHONDONTWRITEBYTECODE": "1",
111             "PYTHONPATH": "/repo",
112             "FORGE_REPO": "/repo",
113             "FORGE_VOL": "/vol",
114             "MODAL_IS_REMOTE": "1",
115             "TEXMFVAR": "/tmp/texmf-var",
116             "TEXMFCONFIG": "/tmp/texmf-config",
117             "HOME": "/root",
118         }
119     )
120     .add_local_dir(REPO, "/repo", ignore=_ignore)
121 )
122
123 VOLUME_MOUNT = {"vol": volume}
124

```

```

125
126 def _bootstrap() -> None:
127     if "/repo" not in sys.path:
128         sys.path.insert(0, "/repo")
129     os.chdir("/repo")
130     Path("/tmp/mpl").mkdir(exist_ok=True)
131
132
133 def _run(stage: str, run_id: str, params_json: str, force: bool) -> dict:
134     _bootstrap()
135     volume.reload()
136     from forge.runner import execute
137
138     params = json.loads(params_json or "{}")
139     return execute(
140         stage_name=stage,
141         run_id=run_id,
142         params=params,
143         force=force,
144         repo=Path("/repo"),
145         vol=Path("/vol"),
146         commit=volume.commit,
147     )
148
149
150 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=2, memory=4096, timeout=2 * 3600)
151 def run_small(stage: str, run_id: str, params: str, force: bool) -> dict:
152     return _run(stage, run_id, params, force)
153
154
155 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=8, memory=16384, timeout=6 * 3600)
156 def run_medium(stage: str, run_id: str, params: str, force: bool) -> dict:
157     return _run(stage, run_id, params, force)
158
159
160 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=32, memory=65536, timeout=12 * 3600)
161 def run_large(stage: str, run_id: str, params: str, force: bool) -> dict:
162     return _run(stage, run_id, params, force)
163
164
165 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=4, memory=8192, timeout=3600)
166 def exec_python(code: str, run_id: str) -> str:
167     """Run an ad-hoc Python snippet in the cloud (exploration, status, debugging). Never local."""
168     _bootstrap()
169     volume.reload()
170     buffer = io.StringIO()
171     scope = {
172         "__name__": "__forge_exec__",
173         "RUN_DIR": Path("/vol/runs") / run_id,
174         "VOL": Path("/vol"),
175         "REPO": Path("/repo"),
176         "RUNS": Path("/vol/runs"),
177     }
178     with contextlib.redirect_stdout(buffer), contextlib.redirect_stderr(buffer):
179         try:
180             exec(compile(code, "<forge-exec>", "exec"), scope) # noqa: S102 - explicit cloud REPL
181         except BaseException:
182             traceback.print_exc(file=buffer)
183     volume.commit()
184     return buffer.getvalue()
185
186
187 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=2, memory=4096, timeout=600)

```

```

188 def provision() -> dict:
189     """Prove the toolchain: interpreter, packages, TeX engine, fonts, solver availability."""
190     import importlib.metadata
191     import subprocess
192
193     _bootstrap()
194     info: dict = {"modal_task_id": os.getenv("MODAL_TASK_ID"), "cpu_count": os.cpu_count()}
195     info["python"] = sys.version.split()[0]
196     info["packages"] = {p.split("=")[0]: importlib.metadata.version(p.split("=")[0]) for p in PIP_PACKAGES}
197     info["xelatex"] = subprocess.run(["xelatex", "--version"], capture_output=True, text=True).stdout.splitlines()[0]
198     info["bibtex"] = subprocess.run(["bibtex", "--version"], capture_output=True, text=True).stdout.splitlines()[0]
199     info["fc_cjk"] = subprocess.run(["fc-match", "Noto Serif CJK SC"], capture_output=True, text=True).stdout.strip()
200     info["kpse"] = {
201         name: bool(subprocess.run(["kpsewhich", name], capture_output=True, text=True).stdout.strip())
202         for name in (
203             "ctexart.cls",
204             "gbt7714.sty",
205             "siunitx.sty",
206             "algorithm2e.sty",
207             "booktabs.sty",
208             "tikz.sty",
209             "listings.sty",
210             "cleveref.sty",
211             "tcolorbox.sty",
212             "pgfplots.sty",
213         )
214     }
215     from ortools.sat.python import cp_model
216
217     model = cp_model.CpModel()
218     x = model.NewIntVar(0, 10, "x")
219     model.Add(x >= 3)
220     model.Minimize(x)
221     solver = cp_model.CpSolver()
222     info["cpsat"] = {"status": solver.StatusName(solver.Solve(model)), "x": solver.Value(x)}
223     import highspy
224
225     info["highs"] = highspy.Highs().version()
226     from forge.plotting import cjk_font_family
227
228     info["figure_cjk_font"] = cjk_font_family()
229     return info
230
231
232 @app.local_entrypoint()
233 def main(
234     stage: str = "ingest",
235     run_id: str = "",
236     params: str = "{}",
237     force: bool = False,
238     size: str = "small",
239     spawn: bool = False,
240 ) -> None:
241     """Submit one stage (or ``all`` with params.stages) to the cloud and print its result."""
242     if stage == "provision":
243         print("PROVISION " + json.dumps(provision.remote(), ensure_ascii=False, default=str))
244         return
245     if stage == "exec":
246         payload = json.loads(params or "{}")
247         print(exec_python.remote(payload.get("code", ""), run_id or "adhoc"))
248         return
249     fn = {"small": run_small, "medium": run_medium, "large": run_large}[size]
250     if spawn:

```

```

251         call = fn.spawn(stage, run_id, params, force)
252         print(f"FUNCTION_CALL_ID {call.object_id}")
253         return
254     result = fn.remote(stage, run_id, params, force)
255     print("STAGE_RESULT " + json.dumps(result, ensure_ascii=False, default=str))

```

forge/packaging.py

```

1  """Deterministic packaging of support materials (stable member order and timestamps)."""
2
3  from __future__ import annotations
4
5  import zipfile
6  from collections.abc import Iterable
7  from pathlib import Path
8  from typing import Any
9
10 from .hashing import sha256_file
11
12 FIXED_TIME = (2026, 1, 1, 0, 0, 0)
13 SKIP_PARTS = {
14     ".git",
15     "__pycache__",
16     ".DS_Store",
17     ".mypy_cache",
18     ".ruff_cache",
19     ".pytest_cache",
20     ".hypothesis",
21     "artifacts",
22     "releases",
23     ".forge",
24     "build",
25 }
26
27
28 def code_members(repo: Path, globs: Iterable[str]) -> list[tuple[str, Path]]:
29     found: dict[str, Path] = {}
30     for pattern in globs:
31         for path in sorted(repo.glob(pattern)):
32             if not path.is_file():
33                 continue
34             rel = path.relative_to(repo)
35             if SKIP_PARTS & set(rel.parts) or path.suffix == ".pyc":
36                 continue
37             found[rel.as_posix()] = path
38     return sorted(found.items())
39
40
41 def run_members(run_dir: Path, stages: Iterable[str], *, prefix: str = "runs") -> list[tuple[str, Path]]:
42     members: list[tuple[str, Path]] = []
43     for stage in stages:
44         directory = run_dir / stage
45         if not directory.is_dir():
46             continue
47         for path in sorted(directory.rglob("*")):
48             rel = path.relative_to(directory)
49             if path.is_file() and not (SKIP_PARTS & set(rel.parts)):
50                 members.append((f"{prefix}/{run_dir.name}/{stage}/{rel.as_posix()}", path))
51     return members
52
53

```

```

54 def build_zip(members: Iterable[tuple[str, Path]], out: Path, *, comment: str = "") -> dict[str, Any]:
55     ordered = sorted(dict(members).items())
56     out.parent.mkdir(parents=True, exist_ok=True)
57     total = 0
58     with zipfile.ZipFile(out, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9) as zf:
59         if comment:
60             zf.comment = comment.encode("utf-8")
61         for name, source in ordered:
62             info = zipfile.ZipInfo(name, date_time=FIXED_TIME)
63             info.compress_type = zipfile.ZIP_DEFLATED
64             info.external_attr = 0o644 << 16
65             data = source.read_bytes()
66             total += len(data)
67             zf.writestr(info, data)
68     with zipfile.ZipFile(out) as zf:
69         bad = zf.testzip()
70         count = len(zf.namelist())
71     if bad:
72         raise RuntimeError(f"zip member failed CRC check: {bad}")
73     return {
74         "path": str(out),
75         "members": count,
76         "uncompressed_bytes": total,
77         "bytes": out.stat().st_size,
78         "sha256": sha256_file(out),
79     }

```

forge/pdfqa.py

```

1  """PDF inspection for quality gates (PyMuPDF)."""
2
3  from __future__ import annotations
4
5  import re
6  from collections.abc import Iterable
7  from pathlib import Path
8  from typing import Any
9
10 A4_PT = (595.276, 841.89)
11
12
13 def inspect_pdf(path: Path, *, with_text: bool = True) -> dict[str, Any]:
14     import fitz  # PyMuPDF
15
16     doc = fitz.open(path)
17     pages: list[dict[str, Any]] = []
18     texts: list[str] = []
19     fonts: dict[int, dict[str, Any]] = {}
20     unembedded: list[str] = []
21     for index, page in enumerate(doc, start=1):
22         rect = page.rect
23         pages.append(
24             {
25                 "page": index,
26                 "width_pt": round(rect.width, 2),
27                 "height_pt": round(rect.height, 2),
28                 "a4": abs(rect.width - A4_PT[0]) < 3 and abs(rect.height - A4_PT[1]) < 3,
29             }
30         )
31     if with_text:
32         texts.append(page.get_text("text"))

```

```

33     for font in page.get_fonts(full=True):
34         xref, ftype, basefont = font[0], font[2], font[3]
35         if xref in fonts:
36             continue
37         embedded = ftype == "Type3"
38         if not embedded and xref:
39             try:
40                 embedded = len(doc.extract_font(xref)[3]) > 0
41             except Exception:
42                 embedded = False
43         fonts[xref] = {"name": basefont, "type": ftype, "embedded": bool(embedded)}
44         if not embedded:
45             unembedded.append(basefont)
46     return {
47         "pages": len(doc),
48         "page_boxes": pages,
49         "all_a4": all(p["a4"] for p in pages),
50         "fonts": list(fonts.values()),
51         "unembedded_fonts": sorted(set(unembedded)),
52         "texts": texts,
53         "metadata": doc.metadata,
54     }
55
56
57 def scan_strings(texts: Iterable[str], patterns: Iterable[str], *, context: int = 40) -> list[dict[str, Any]]:
58     compiled = [(p, re.compile(p)) for p in patterns]
59     hits: list[dict[str, Any]] = []
60     for page_no, text in enumerate(texts, start=1):
61         for raw, pattern in compiled:
62             for match in pattern.finditer(text):
63                 start, end = max(0, match.start() - context), min(len(text), match.end() + context)
64                 hits.append({"page": page_no, "pattern": raw, "context": text[start:end].replace("\n", " ")})
65     return hits

```

forge/plotting.py

```

1  """House style for paper figures: vector PDF output, CJK-capable fonts, accessible palette."""
2
3  from __future__ import annotations
4
5  import logging
6  import warnings
7  from pathlib import Path
8  from typing import Any
9
10 OKABE_ITO = ["#0072B2", "#D55E00", "#009E73", "#E69F00", "#CC79A7", "#56B4E9", "#F0E442", "#000000"]
11 # (family name as matplotlib registers it, font file). TTC files expose only their first face,
12 # which is why the Noto CJK collection is addressed by its JP face name.
13 _CJK_CANDIDATES = (
14     ("WenQuanYi Micro Hei", "/usr/share/fonts/truetype/wqy/wqy-microhei.ttc"),
15     ("Noto Sans CJK JP", "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"),
16     ("Droid Sans Fallback", "/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf"),
17 )
18 _PROBE_TEXT = "温度 T 水分 C 药材 频段 关键词 abc 0.25"
19 _STATE: dict[str, Any] = {}
20
21
22 class _Catcher(logging.Handler):
23     def __init__(self) -> None:
24         super().__init__()
25         self.records: list[str] = []

```

```

26
27     def emit(self, record: logging.LogRecord) -> None:
28         self.records.append(record.getMessage())
29
30
31 def _probe(family: str) -> bool:
32     """True when ``family`` (with DejaVu Sans as Latin fallback) renders the probe text into PDF cleanly."""
33     from matplotlib import pyplot as plt
34
35     catcher = _Catcher()
36     loggers = [
37         logging.getLogger(n) for n in ("matplotlib.font_manager", "matplotlib.backends.backend_pdf", "matplotlib")
38     ]
39     for lg in loggers:
40         lg.addHandler(catcher)
41     try:
42         with warnings.catch_warnings(record=True) as caught:
43             warnings.simplefilter("always")
44             fig, ax = plt.subplots(figsize=(2.5, 1))
45             ax.text(0.05, 0.5, _PROBE_TEXT, fontfamily=[family, "DejaVu Sans"])
46             ax.set_axis_off()
47             probe = Path("/tmp") / f"forge_font_probe_{family.replace(' ', '_')}.pdf"
48             fig.savefig(probe, format="pdf")
49             plt.close(fig)
50             messages = [str(w.message) for w in caught] + catcher.records
51             bad = [m for m in messages if "missing from font" in m or "not found" in m.lower()]
52             return not bad
53     except Exception:
54         return False
55     finally:
56         for lg in loggers:
57             lg.removeHandler(catcher)
58
59
60 def cjk_font_family() -> str:
61     """Pick a CJK font that renders into PDF without missing glyphs (cached per process)."""
62     if "family" in _STATE:
63         return str(_STATE["family"])
64     import matplotlib
65
66     matplotlib.use("Agg")
67     from matplotlib import font_manager
68
69     chosen = "DejaVu Sans"
70     for family, path in _CJK_CANDIDATES:
71         if not Path(path).exists():
72             continue
73         try:
74             font_manager.fontManager.addfont(path)
75         except Exception:
76             continue
77         if _probe(family):
78             chosen = family
79             break
80     _STATE["family"] = chosen
81     return chosen
82
83
84 def setup(font_size: float = 9.0) -> str:
85     import matplotlib
86
87     matplotlib.use("Agg")
88     from cycler import cycler

```

```

89     from matplotlib import pyplot as plt
90
91     family = cjk_font_family()
92     plt.rcParams.update(
93         {
94             "font.family": [family, "DejaVu Sans"],
95             "font.size": font_size,
96             "axes.titlesize": font_size + 1,
97             "axes.labelsize": font_size,
98             "legend.fontsize": font_size - 1,
99             "xtick.labelsize": font_size - 1,
100            "ytick.labelsize": font_size - 1,
101            "axes.prop_cycle": cycler(color=OKABE_ITO),
102            "axes.spines.top": False,
103            "axes.spines.right": False,
104            "axes.grid": True,
105            "grid.alpha": 0.25,
106            "grid.linewidth": 0.6,
107            "lines.linewidth": 1.4,
108            "figure.dpi": 150,
109            "savefig.dpi": 300,
110            "pdf.fonttype": 42,
111            "ps.fonttype": 42,
112            "axes.unicode_minus": False,
113            "figure.constrained_layout.use": True,
114        }
115    )
116    return family
117
118
119 def new_figure(width_in: float = 6.3, height_in: float = 3.4, **kwargs: Any) -> tuple[Any, Any]:
120     from matplotlib import pyplot as plt
121
122     setup()
123     return plt.subplots(figsize=(width_in, height_in), **kwargs)
124
125
126 def save(fig: Any, pdf_path: Path, *, png_preview: bool = True, dpi: int = 200) -> dict[str, Any]:
127     from matplotlib import pyplot as plt
128
129     pdf_path.parent.mkdir(parents=True, exist_ok=True)
130     fig.savefig(pdf_path, format="pdf")
131     out = {"pdf": str(pdf_path), "bytes": pdf_path.stat().st_size}
132     if png_preview:
133         png = pdf_path.with_suffix(".png")
134         fig.savefig(png, format="png", dpi=dpi)
135         out["png"] = str(png)
136     plt.close(fig)
137     return out

```

forge/runner.py

```

1  """Stage registry and executor. A stage is a function ``fn(ctx) -> metrics``.
2
3  Guarantees: one manifest per (run, stage); dependencies must be completed in the same run
4  (or copied from ``from_run``); a completed stage is skipped unless ``force``; failures are
5  recorded with a traceback and re-raised; the volume is committed after every stage.
6  """
7
8  from __future__ import annotations
9

```

```

10 import importlib
11 import json
12 import shutil
13 import time
14 from collections.abc import Callable, Iterable
15 from dataclasses import dataclass
16 from pathlib import Path
17 from typing import Any
18
19 from .context import StageContext
20 from .hashing import hash_tree, tree_digest
21 from .manifest import Manifest
22
23 StageFn = Callable[[StageContext], dict[str, Any] | None]
24 CODE_IGNORE = (
25     ".git",
26     "artifacts",
27     "releases",
28     "__pycache__",
29     ".mypy_cache",
30     ".ruff_cache",
31     ".pytest_cache",
32     ".hypothesis",
33     ".DS_Store",
34     ".forge",
35 )
36
37
38 @dataclass(frozen=True)
39 class StageSpec:
40     name: str
41     fn: StageFn
42     deps: tuple[str, ...]
43     description: str
44
45
46 STAGES: dict[str, StageSpec] = {}
47
48
49 def stage(name: str, deps: Iterable[str] = (), description: str = "") -> Callable[[StageFn], StageFn]:
50     def decorator(fn: StageFn) -> StageFn:
51         if name in STAGES:
52             raise ValueError(f"duplicate stage registration: {name}")
53         doc = (fn.__doc__ or "").strip().splitlines()
54         STAGES[name] = StageSpec(name, fn, tuple(deps), description or (doc[0] if doc else ""))
55         return fn
56
57     return decorator
58
59
60 def discover(repo: Path) -> None:
61     project = json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
62     importlib.import_module("pipelines.common.stages")
63     importlib.import_module(f"pipelines.{project['package']}.stages")
64
65
66 def list_stages(repo: Path) -> list[dict[str, Any]]:
67     discover(repo)
68     return [{"name": s.name, "deps": list(s.deps), "description": s.description} for s in STAGES.values()]
69
70
71 def _raw_input_prefixes(repo: Path) -> tuple[str, ...]:
72     project = json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))

```

```

73     return (project["problem_pdf"], project["attachments_dir"].rstrip("/") + "/" )
74
75
76 def execute(
77     stage_name: str,
78     run_id: str,
79     params: dict[str, Any],
80     force: bool,
81     repo: Path,
82     vol: Path,
83     commit: Callable[[], None] | None = None,
84 ) -> dict[str, Any]:
85     discover(repo)
86     if stage_name == "all":
87         results = [execute(s, run_id, params, force, repo, vol, commit) for s in params.get("stages", [])]
88         return {
89             "stage": "all",
90             "run_id": run_id,
91             "results": results,
92             "status": "completed" if all(r.get("status") in {"completed", "skipped"}) for r in results) else "failed",
93         }
94     if stage_name not in STAGES:
95         raise KeyError(f"unknown stage {stage_name!r}; known stages: {sorted(STAGES)}")
96     spec = STAGES[stage_name]
97     run_dir = vol / "runs" / run_id
98     stage_dir = run_dir / stage_name
99     manifest_path = stage_dir / "manifest.json"
100
101     if manifest_path.exists() and not force:
102         previous = Manifest.read(manifest_path)
103         if previous.status == "completed":
104             return {
105                 "stage": stage_name,
106                 "run_id": run_id,
107                 "status": "skipped",
108                 "reason": "already completed in this run (use --force to rerun)",
109                 "outputs_digest": previous.outputs_digest,
110             }
111
112     deps_digest: dict[str, str] = {}
113     for dep in spec.deps:
114         dep_manifest = run_dir / dep / "manifest.json"
115         if not dep_manifest.exists() and params.get("from_run"):
116             source = vol / "runs" / str(params["from_run"]) / dep
117             if (source / "manifest.json").exists():
118                 shutil.copytree(source, run_dir / dep, dirs_exist_ok=True)
119             if not dep_manifest.exists():
120                 raise RuntimeError(
121                     f"stage '{stage_name}' requires '{dep}', which has not run in run {run_id} "
122                     f"(run it first, or pass from_run=<run_id> to reuse its outputs)"
123                 )
124         dep_state = Manifest.read(dep_manifest)
125         if dep_state.status != "completed":
126             raise RuntimeError(f"dependency '{dep}' has status {dep_state.status!r} in run {run_id}")
127         deps_digest[dep] = dep_state.outputs_digest
128
129     if stage_dir.exists():
130         shutil.rmtree(stage_dir)
131     ctx = StageContext(run_id, stage_name, params, repo, vol)
132     code_tree = hash_tree(repo, ignore=CODE_IGNORE)
133     prefixes = _raw_input_prefixes(repo)
134     inputs = {k: v for k, v in code_tree.items() if k.startswith(prefixes)}
135     code_ref = {**dict(params.get("code_ref") or {}), "code_digest": tree_digest(code_tree), "files": len(code_tree)}

```



```

29
30 def summarize_log(log: str) -> dict[str, Any]:
31     overfull = [float(x) for x in _OVERFULL.findall(log)]
32     missing = _MISSING.findall(log)
33     pages = _PAGES.search(log)
34     return {
35         "undefined_references": sorted(set(_UNDEF_REF.findall(log))),
36         "undefined_citations": sorted(set(_UNDEF_CIT.findall(log))),
37         "overfull_count": len(overfull),
38         "overfull_max_pt": max(overfull) if overfull else 0.0,
39         "missing_characters": len(missing),
40         "missing_character_samples": sorted(set(missing))[:10],
41         "errors": _ERROR.findall(log)[:10],
42         "pages_from_log": int(pages.group(1)) if pages else None,
43     }
44
45
46 def aux_page(aux: Path, label: str) -> int | None:
47     """Page number recorded for ``\label{label}`` in the .aux file (hyperref-compatible)."""
48     if not aux.exists():
49         return None
50     pattern = re.compile(r"\newlabel\{" + re.escape(label) + r"\}\{[^}]*\}\{(\d+)\}")
51     match = pattern.search(aux.read_text(encoding="utf-8", errors="replace"))
52     return int(match.group(1)) if match else None
53
54
55 def compile_tex(
56     workdir: Path,
57     main: str,
58     jobname: str,
59     *,
60     passes: int = 3,
61     bibtex: bool = True,
62     timeout: int = 1200,
63 ) -> dict[str, Any]:
64     env = tex_env()
65     runs: list[dict[str, Any]] = []
66     log_path = workdir / f"{jobname}.log"
67
68     def xelatex(tag: str) -> None:
69         proc = subprocess.run(
70             ["xelatex", "-interaction=nonstopmode", "-halt-on-error", "-file-line-error", f"-jobname={jobname}", main],
71             cwd=workdir,
72             capture_output=True,
73             text=True,
74             env=env,
75             timeout=timeout,
76             check=False,
77         )
78         runs.append({"pass": tag, "rc": proc.returncode})
79         if proc.returncode != 0:
80             log = log_path.read_text(encoding="utf-8", errors="replace") if log_path.exists() else proc.stdout
81             errors = _ERROR.findall(log)
82             raise RuntimeError(
83                 f"xelatex failed on pass {tag} for {main}: "
84                 + " | ".join(errors[:5])
85                 + "\n--- log tail ---\n"
86                 + log[-4000:]
87             )
88
89     xelatex("1")
90     aux = workdir / f"{jobname}.aux"
91     if bibtex and aux.exists():

```

```

92     aux_text = aux.read_text(encoding="utf-8", errors="replace")
93     if "\\citation" in aux_text and "\\bibdata" in aux_text:
94         proc = subprocess.run(
95             ["bibtex", jobname], cwd=workdir, capture_output=True, text=True, env=env, timeout=timeout, check=False
96         )
97         runs.append({"pass": "bibtex", "rc": proc.returncode, "tail": proc.stdout[-1500:]})
98     for i in range(2, passes + 1):
99         xelatex(str(i))
100     log_text = log_path.read_text(encoding="utf-8", errors="replace")
101     return {
102         "pdf": workdir / f"{jobname}.pdf",
103         "log": log_path,
104         "aux": aux,
105         "runs": runs,
106         "summary": summarize_log(log_text),
107     }
108
109
110 def tex_escape(text: str) -> str:
111     replacements = {
112         "\\": r"\textbackslash{}",
113         "&": r"\&",
114         "%": r"\%",
115         "$": r"\$",
116         "#": r"\#",
117         "_": r"\_",
118         "{": r"\{",
119         "}": r"\}",
120         "~": r"\textasciitilde{}",
121         "^": r"\textasciicircum{}",
122     }
123     return "".join(replacements.get(ch, ch) for ch in text)
124
125
126 def tex_path(path: str) -> str:
127     """Typeset a file path so that it can break across lines (hyperref/xurl); falls back to \texttt."""
128     if any(ch in path for ch in "{}%#\\") or not path.isascii():
129         return f"\\texttt{{{tex_escape(path)}}}" # CJK names need the CJK mono font, not the url font
130     return f"\\nolinkurl{{{path}}}"

```

forge/xlsx.py

```

1  """Workbook I/O: reading attachments, filling result templates, checking result contracts."""
2
3  from __future__ import annotations
4
5  import math
6  from collections.abc import Iterable, Sequence
7  from dataclasses import dataclass
8  from pathlib import Path
9  from typing import Any
10
11  import openpyxl
12  import pandas as pd
13
14  BIG_SHEET_CELLS = 300_000
15
16
17  def read_sheets(path: Path, header: int | None = 0) -> dict[str, pd.DataFrame]:
18     with pd.ExcelFile(path) as xf:
19         return {str(name): xf.parse(name, header=header) for name in xf.sheet_names}

```

```

20
21
22 def sheet_headers(path: Path) -> dict[str, list[Any]]:
23     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
24     out: dict[str, list[Any]] = {}
25     for ws in wb.worksheets:
26         row = list(next(ws.iter_rows(min_row=1, max_row=1, values_only=True), ()))
27         while row and row[-1] is None:
28             row.pop()
29         out[str(ws.title)] = row
30     wb.close()
31     return out
32
33
34 def _clean(value: Any, decimals: int | None) -> Any:
35     if hasattr(value, "item") and not isinstance(value, str | bytes):
36         try:
37             value = value.item()
38         except (ValueError, TypeError):
39             pass
40     if isinstance(value, float):
41         if math.isnan(value) or math.isinf(value):
42             return None
43         return round(value, decimals) if decimals is not None else value
44     return value
45
46
47 def write_result(
48     template: Path,
49     out: Path,
50     sheets: dict[str, tuple[Sequence[Any], Iterable[Sequence[Any]]]],
51     *,
52     decimals: int | None = 4,
53     float_format: str = "0.0000",
54 ) -> dict[str, Any]:
55     """Fill a competition result template.
56
57     ``sheets`` maps sheet name -> (header_row, rows). The template's A1 text is always kept;
58     header cells after A1 are taken from ``header_row[1:]``. Sheets not mentioned keep their
59     template content. Floats are rounded to ``decimals`` and formatted with ``float_format``.
60     Very large sheets fall back to openpyxl's write-only mode (no per-cell number format).
61
62     """
63     template_wb = openpyxl.load_workbook(template)
64     names = list(template_wb.sheetnames)
65     unknown = [n for n in sheets if n not in names]
66     if unknown:
67         raise KeyError(f"sheets {unknown} not in template {template.name} (has {names})")
68     a1 = {ws.title: ws.cell(row=1, column=1).value for ws in template_wb.worksheets}
69     materialised = {name: (list(header), [list(r) for r in rows]) for name, (header, rows) in sheets.items()}
70     cells = sum(max(len(h), 1) * (len(rows) + 1) for h, rows in materialised.values())
71     stats: dict[str, Any] = {}
72     out.parent.mkdir(parents=True, exist_ok=True)
73
74     if cells > BIG_SHEET_CELLS:
75         wb = openpyxl.Workbook(write_only=True)
76         for name in names:
77             ws = wb.create_sheet(name)
78             if name in materialised:
79                 header, rows = materialised[name]
80                 full_header = [a1[name], *header[1:]]
81                 ws.append(full_header)
82                 for row in rows:
83                     ws.append([_clean(v, decimals) for v in row])

```

```

83         stats[name] = {"rows": len(rows), "cols": len(full_header)}
84     else:
85         for tpl_row in template_wb[name].iter_rows(values_only=True):
86             ws.append(list(tpl_row))
87     wb.save(out)
88 else:
89     wb = template_wb
90     for name, (header, rows) in materialised.items():
91         index = wb.sheetnames.index(name)
92         wb.remove(wb[name])
93         ws = wb.create_sheet(name, index) # fresh sheet: no stale template cells or dimensions
94         full_header = [a1[name], *header[1:]]
95         for j, value in enumerate(full_header, start=1):
96             ws.cell(row=1, column=j, value=_clean(value, None))
97         for i, row in enumerate(rows, start=2):
98             for j, value in enumerate(row, start=1):
99                 cleaned = _clean(value, decimals)
100                 cell = ws.cell(row=i, column=j, value=cleaned)
101                 if isinstance(cleaned, float) and j > 1:
102                     cell.number_format = float_format
103         stats[name] = {"rows": len(rows), "cols": len(full_header)}
104     wb.save(out)
105 return {"path": str(out), "sheets": stats, "write_only": cells > BIG_SHEET_CELLS, "cells": cells}
106
107
108 @dataclass(frozen=True)
109 class SheetContract:
110     name: str
111     min_rows: int = 1 # data rows (excluding the header)
112     max_rows: int | None = None
113     header_len: int | None = None # exact number of header cells, if known
114     numeric_from_col: int | None = None # 0-based; every data cell from this column must be numeric
115     allow_blank: bool = False # blank numeric cells permitted (e.g. moving-boundary tables)
116     decimals: int | None = 4 # numeric cells must carry at most this many decimals
117     text_columns: tuple[int, ...] = () # 0-based columns that must be non-empty text
118
119
120 @dataclass(frozen=True)
121 class WorkbookContract:
122     file: str
123     template: str
124     sheets: tuple[SheetContract, ...]
125
126
127 def check_workbook(path: Path, contract: WorkbookContract, template: Path) -> dict[str, Any]:
128     errors: list[str] = []
129     if not path.exists():
130         return {"file": contract.file, "ok": False, "errors": ["file missing"], "sheets": {}}
131     try:
132         tpl_headers = sheet_headers(template)
133     except Exception as exc:
134         tpl_headers = {}
135         errors.append(f"template unreadable: {exc!r}")
136     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
137     report: dict[str, Any] = {}
138     names = wb.sheetnames
139     if tpl_headers and names != list(tpl_headers):
140         errors.append(f"sheet names {names} differ from template {list(tpl_headers)}")
141     for sc in contract.sheets:
142         if sc.name not in names:
143             errors.append(f"sheet {sc.name!r} missing")
144             continue
145         ws = wb[sc.name]

```

```

146 rows = ws.iter_rows(values_only=True)
147 first = next(rows, None)
148 header: list[Any] = list(first) if first is not None else []
149 while header and header[-1] is None:
150     header.pop()
151 tpl = tpl_headers.get(sc.name)
152 if tpl and header and header[0] != tpl[0]:
153     errors.append(f"{sc.name}: A1 {header[0]!r} differs from template {tpl[0]!r}")
154 if sc.header_len is not None and len(header) != sc.header_len:
155     errors.append(f"{sc.name}: header has {len(header)} cells, expected {sc.header_len}")
156 n = 0
157 bad_numeric = blank = bad_decimals = bad_text = 0
158 for row in rows:
159     values = list(row)
160     if all(v is None or (isinstance(v, str) and not v.strip()) for v in values):
161         continue
162     n += 1
163     if sc.numeric_from_col is not None:
164         for v in values[sc.numeric_from_col : len(header) or None]:
165             if v is None or (isinstance(v, str) and not v.strip()):
166                 blank += 1
167             elif not isinstance(v, int | float) or isinstance(v, bool):
168                 bad_numeric += 1
169             elif sc.decimals is not None and isinstance(v, float) and abs(round(v, sc.decimals) - v) > 1e-12:
170                 bad_decimals += 1
171         for j in sc.text_columns:
172             v = values[j] if j < len(values) else None
173             if v is None or not str(v).strip():
174                 bad_text += 1
175 if n < sc.min_rows:
176     errors.append(f"{sc.name}: {n} data rows < {sc.min_rows}")
177 if sc.max_rows is not None and n > sc.max_rows:
178     errors.append(f"{sc.name}: {n} data rows > {sc.max_rows}")
179 if bad_numeric:
180     errors.append(f"{sc.name}: {bad_numeric} non-numeric cells in numeric region")
181 if blank and not sc.allow_blank:
182     errors.append(f"{sc.name}: {blank} blank cells in numeric region")
183 if bad_decimals:
184     errors.append(f"{sc.name}: {bad_decimals} cells with more than {sc.decimals} decimals")
185 if bad_text:
186     errors.append(f"{sc.name}: {bad_text} empty cells in required text columns")
187 report[sc.name] = {
188     "rows": n,
189     "header": [str(h) for h in header[:6]] + (["..."] if len(header) > 6 else []),
190     "blank": blank,
191 }
192 wb.close()
193 return {"file": contract.file, "ok": not errors, "errors": errors, "sheets": report}

```

pipelines/__init__.py

pipelines/a/__init__.py

pipelines/a/common.py

```
1  """Shared helpers for the Problem-A science stages: loading inputs, building specs, storing solutions."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import numpy as np
8  import pandas as pd
9
10 from forge.context import StageContext
11 from pipelines.a.physics import (
12     APPENDIX2,
13     APPENDIX3,
14     APPENDIX4,
15     R0,
16     Chamber,
17     FluxCap,
18     Properties,
19     Radius,
20 )
21 from pipelines.a.solver import ProblemSpec, Solution, sample_physical, subgrid_indices
22
23 R_OUT_CM = np.round(np.arange(0, 21) * 0.1, 1) # 0.0 ... 2.0 cm
24 R_TABLE_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
25 COLS = [f"{r:.1f}" for r in R_OUT_CM]
26 PROPS = {"q1": APPENDIX2, "q2": APPENDIX3, "q3": APPENDIX3, "q4": APPENDIX4}
27
28
29 def load_chamber(ctx: StageContext, plateau: str | None = None) -> Chamber:
30     df = pd.read_parquet(ctx.dep("ingest") / "data" / "附件1__Sheet1.parquet")
31     rule = plateau or str(ctx.cfg("a.chamber.plateau", "mean_last_hour"))
32     return Chamber.from_series(
33         df["时间"].to_numpy(float), df["温度"].to_numpy(float), df["水分浓度"].to_numpy(float), rule
34     )
35
36
37 def load_radius(ctx: StageContext, shrink_scale: float = 1.0) -> Radius:
38     df = pd.read_parquet(ctx.dep("ingest") / "data" / "附件2__Sheet1.parquet")
39     return Radius.from_series(df["时间"].to_numpy(float), df["半径"].to_numpy(float), shrink_scale)
40
41
42 def production_n(ctx: StageContext) -> int:
43     return 20 * 2 ** int(ctx.cfg("a.grid.refine", 3))
44
45
46 def time_options(ctx: StageContext) -> dict[str, Any]:
47     return {
48         "method": str(ctx.cfg("a.time.method", "BDF")),
49         "rtol": float(ctx.cfg("a.time.rtol", 1e-7)),
50         "atol": float(ctx.cfg("a.time.atol", 1e-9)),
51     }
52
53
54 def build_spec(
55     ctx: StageContext,
```

```

56     problem: str,
57     *,
58     props: Properties | None = None,
59     chamber: Chamber | None = None,
60     radius: Radius | None = None,
61     formulation: str | None = None,
62     face_scheme: str | None = None,
63 ) -> ProblemSpec:
64     """Default specification of problem q1...q4 with optional overrides (sensitivity / alternatives)."""
65     if radius is None:
66         radius = load_radius(ctx) if problem == "q4" else Radius.constant(R0)
67     return ProblemSpec(
68         props=props or PROPS[problem],
69         chamber=chamber or load_chamber(ctx),
70         radius=radius,
71         formulation=formulation or str(ctx.cfg("a.scheme.formulation", "lagrangian")),
72         face_scheme=face_scheme or str(ctx.cfg("a.scheme.face", "midpoint")),
73     )
74
75
76 def profiles_frame(sol: Solution, field: str) -> pd.DataFrame:
77     """Field values on the 0.1 cm output grid (fixed radius: coincident nodes) as a DataFrame with a 't' column."""
78     values = getattr(sol, field)
79     idx = subgrid_indices(len(sol.xi) - 1)
80     frame = pd.DataFrame(values[:, idx], columns=COLS)
81     frame.insert(0, "t", sol.t)
82     return frame
83
84
85 def physical_frame(sol: Solution, field: str) -> pd.DataFrame:
86     """Field values at fixed physical radii 0...2 cm (NaN beyond R(t)) plus the surface value, shrinking domain."""
87     values = getattr(sol, field)
88     rows = np.full((len(sol.t), len(R_OUT_CM)), np.nan)
89     for i in range(len(sol.t)):
90         rows[i] = sample_physical(values[i], sol.xi, float(sol.radius[i]), R_OUT_CM / 100.0)
91     frame = pd.DataFrame(rows, columns=COLS)
92     frame.insert(0, "t", sol.t)
93     frame["surface"] = values[:, -1]
94     frame["radius_cm"] = sol.radius * 100.0
95     return frame
96
97
98 def table_cells(frame: pd.DataFrame, times: np.ndarray, radii_cm: np.ndarray = R_TABLE_CM) -> list[list[float]]:
99     """Rows [t, v(r1), ...] of a profiles frame at the requested instants (must exist exactly in 't')."""
100     out: list[list[float]] = []
101     lookup = {float(t): i for i, t in enumerate(frame["t"].to_numpy(float))}
102     for t in times:
103         i = lookup[float(t)]
104         out.append([float(t)] + [float(frame.at[i, f"{r:.1f}"]) for r in radii_cm])
105     return out
106
107
108 def snapshot_arrays(sol: Solution, times: np.ndarray) -> dict[str, np.ndarray]:
109     lookup = {float(t): i for i, t in enumerate(sol.t)}
110     idx = [lookup[float(t)] for t in times if float(t) in lookup]
111     return {
112         "t": sol.t[idx],
113         "xi": sol.xi,
114         "temp": sol.temp[idx],
115         "moist": sol.moist[idx],
116         "radius": sol.radius[idx],
117     }
118

```

```

119
120 def mean_moisture(sol: Solution) -> np.ndarray:
121     """Volume-averaged (dry-matter-weighted for the Lagrangian case) moisture history: 2 * sum vol_i C_i."""
122     from pipelines.a.verify import control_volumes
123
124     return 2.0 * (sol.moist @ control_volumes(sol.xi))
125
126
127 def spec_to_recipe(spec: ProblemSpec) -> dict[str, Any]:
128     """Picklable description of a spec (workers rebuild the interpolants themselves)."""
129     from dataclasses import asdict
130
131     ch = spec.chamber
132     rd = spec.radius
133     return {
134         "props": asdict(spec.props),
135         "chamber": {
136             "t": ch.t.tolist(),
137             "temp": ch.temp.tolist(),
138             "hum": ch.hum.tolist(),
139             "temp_plateau": ch.temp_plateau,
140             "hum_plateau": ch.hum_plateau,
141             "temp_scale": ch.temp_scale,
142             "hum_scale": ch.hum_scale,
143         },
144         "radius": {"t": rd.t.tolist(), "r": rd.r.tolist(), "shrink_scale": rd.shrink_scale},
145         "formulation": spec.formulation,
146         "face_scheme": spec.face_scheme,
147         "t_init": spec.t_init,
148         "c_init": spec.c_init,
149         "flux_cap": None
150         if spec.flux_cap is None
151         else {
152             "t": spec.flux_cap.t.tolist(),
153             "cap": spec.flux_cap.cap.tolist(),
154             "plateau": spec.flux_cap.plateau,
155             "rho_s": spec.flux_cap.rho_s,
156             "scale": spec.flux_cap.scale,
157         },
158     }
159
160
161 def spec_from_recipe(recipe: dict[str, Any]) -> ProblemSpec:
162     ch = recipe["chamber"]
163     rd = recipe["radius"]
164     fc = recipe.get("flux_cap")
165     return ProblemSpec(
166         props=Properties(**recipe["props"]),
167         chamber=Chamber(
168             np.asarray(ch["t"], float),
169             np.asarray(ch["temp"], float),
170             np.asarray(ch["hum"], float),
171             float(ch["temp_plateau"]),
172             float(ch["hum_plateau"]),
173             float(ch["temp_scale"]),
174             float(ch["hum_scale"]),
175         ),
176         radius=Radius(np.asarray(rd["t"], float), np.asarray(rd["r"], float), float(rd["shrink_scale"])),
177         formulation=str(recipe["formulation"]),
178         face_scheme=str(recipe["face_scheme"]),
179         t_init=float(recipe["t_init"]),
180         c_init=float(recipe["c_init"]),
181         flux_cap=None

```

```

182         if fc is None
183         else FluxCap(
184             np.asarray(fc["t"], float),
185             np.asarray(fc["cap"], float),
186             float(fc["plateau"]),
187             float(fc["rho_s"]),
188             float(fc["scale"]),
189         ),
190     )

```

pipelines/a/contracts.py

```

1  """Data contracts for Problem A (herb drying): inputs and result workbooks."""
2
3  from __future__ import annotations
4
5  from forge.contracts import Column, FrameContract
6  from forge.xlsx import SheetContract, WorkbookContract
7
8  INPUT_CONTRACTS: dict[str, FrameContract] = {
9      "附件1__Sheet1": FrameContract(
10         name="A.附件1.烘房温湿度",
11         columns=(
12             Column("时间", "int", min=0, unique=True, monotonic="increasing", step=60),
13             Column("温度", "float", min=0, max=120),
14             Column("水分浓度", "float", min=0, max=1),
15         ),
16         min_rows=200,
17     ),
18     "附件2__Sheet1": FrameContract(
19         name="A.附件2.药材半径",
20         columns=(
21             Column("时间", "int", min=0, unique=True, monotonic="increasing"),
22             Column("半径", "float", min=0.1, max=2.0, monotonic="non_increasing"),
23         ),
24         min_rows=100,
25     ),
26 }
27
28 _RADIAL = 22 # A1 label + 21 distances 0.0 ... 2.0 (step 0.1 cm)
29
30 RESULT_CONTRACTS: list[WorkbookContract] = [
31     WorkbookContract(
32         "result1.xlsx",
33         "result1.xlsx",
34         (
35             SheetContract("温度", min_rows=1800, max_rows=1801, header_len=_RADIAL, numeric_from_col=0),
36             SheetContract("水分浓度", min_rows=1800, max_rows=1801, header_len=_RADIAL, numeric_from_col=0),
37         ),
38     ),
39     WorkbookContract(
40         "result2.xlsx",
41         "result2.xlsx",
42         (
43             SheetContract("温度", min_rows=10800, header_len=_RADIAL, numeric_from_col=0),
44             SheetContract("水分浓度", min_rows=10800, header_len=_RADIAL, numeric_from_col=0),
45         ),
46     ),
47     WorkbookContract(
48         "result3.xlsx", "result3.xlsx", (SheetContract("Sheet1", min_rows=60, header_len=_RADIAL, numeric_from_col=0),)
49     ),

```

```

50     WorkbookContract(
51         "result4.xlsx", "result4.xlsx", (SheetContract("Sheet1", min_rows=60, numeric_from_col=0, allow_blank=True),)
52     ),
53 ]

```

pipelines/a/derived.py

```

1  """Derived quantities quoted in the paper: dimensionless groups, time scales, extrapolated errors, solver statistics.
2
3  Every number the manuscript quotes must come from a stage's ``ctx.number`` registration (docs/ENGINEERING_STANDARD.md,
4  MDR-0009). This stage computes the a-priori estimates used in the analysis and verification sections from the stored
5  outputs of the science stages only (no new simulations), so that the text of the paper carries no hand-typed arithmetic
6      ↪ .
7
8  """
9
10 from __future__ import annotations
11
12 from typing import Any
13
14 import numpy as np
15
16 from forge.context import StageContext
17 from forge.runner import stage
18 from pipelines.a.physics import APPENDIX3, APPENDIX4, C_INIT, C_TARGET, LENGTH, R0, thermal_diffusivity
19
20 HOUR = 3600.0
21 Q1_END_S = 1800.0
22
23 def fourier_number(diffusivity: float, t: float, length: float) -> float:
24     """Fo = D t / L^2."""
25     return float(diffusivity * t / length**2)
26
27 def richardson_error(values: list[float], order: float = 2.0) -> tuple[float, float]:
28     """Richardson extrapolation of a sequence computed on grids halved at every level (finest last).
29
30     Returns ``(limit, error)``: the limit estimated from the two finest levels and the absolute error of the
31     second-finest level (the production grid) with respect to that limit.
32     """
33     if len(values) < 2:
34         raise ValueError("need at least two levels")
35     fine, coarse = float(values[-1]), float(values[-2])
36     limit = fine + (fine - coarse) / (2.0**order - 1.0)
37     return limit, abs(coarse - limit)
38
39
40 def first_time_within(t: np.ndarray, a: np.ndarray, b: np.ndarray, tol: float) -> float | None:
41     """First instant after which |a - b| stays below ``tol`` for the rest of the record (None if it never does)."""
42     ok = np.abs(np.asarray(a, dtype=float) - np.asarray(b, dtype=float)) < tol
43     if not ok.any():
44         return None
45     bad = np.where(~ok)[0]
46     start = 0 if len(bad) == 0 else int(bad[-1]) + 1
47     return None if start >= len(t) else float(np.asarray(t, dtype=float)[start])
48
49
50 @stage(
51     "derived",
52     deps=("q1", "q2", "q3", "q4", "convergence"),

```

```

53     description="Dimensionless groups, time scales, extrapolated grid errors and solver statistics quoted in the paper"
54     ↵ ,
55 )
56 def derived(ctx: StageContext) -> dict[str, Any]:
57     q1 = ctx.dep_data("q1", "summary.json")
58     q2h = ctx.dep_data("q2", "history.json")
59     q3 = ctx.dep_data("q3", "summary.json")
60     q4 = ctx.dep_data("q4", "summary.json")
61     conv = ctx.dep_data("convergence", "convergence.json")
62
63     # problem 1 (appendix 2, initial state): time scales, Fourier numbers, diffusion length, stiffness ratio
64     alpha1 = float(q1["biot"]["alpha"])
65     d1 = float(q1["biot"]["D"])
66     t_heat = R0**2 / alpha1
67     t_mass = R0**2 / d1
68     fo_heat = fourier_number(alpha1, Q1_END_S, R0)
69     fo_mass = fourier_number(d1, Q1_END_S, R0)
70     diffusion_length = float(np.sqrt(d1 * Q1_END_S))
71
72     # problem 2: thermal equilibration of the centre with the chamber
73     t2 = np.asarray(q2h["t"], dtype=float)
74     gap_1k = first_time_within(t2, q2h["centre_temp"], q2h["air_temp"], 1.0)
75     gap_half = first_time_within(t2, q2h["centre_temp"], q2h["air_temp"], 0.5)
76
77     # problem 3 (appendix 3): axial Fourier numbers for the 1-D reduction, diffusivity range (stiffness)
78     t3 = float(q3["t_dry_s"])
79     t_plateau = float(q3["spec"]["chamber"]["temp_plateau"])
80     alpha3 = thermal_diffusivity(APPENDIX3, C_INIT)
81     d_max = float(APPENDIX3.diffusivity(np.array(C_INIT), np.array(t_plateau)))
82     d_target = float(APPENDIX3.diffusivity(np.array(C_TARGET), np.array(t_plateau)))
83     d_surface_dry = float(APPENDIX3.diffusivity(np.array(q3["surface_moist_at_dry"]), np.array(t_plateau)))
84     bi_mass_dry = APPENDIX3.hm * R0 / d_surface_dry
85     d4_max = float(APPENDIX4.diffusivity(np.array(C_INIT), np.array(t_plateau)))
86     d4_target = float(APPENDIX4.diffusivity(np.array(C_TARGET), np.array(t_plateau)))
87     half_length = LENGTH / 2.0
88     fo_axial_heat = fourier_number(alpha3, t3, half_length)
89     fo_axial_mass = fourier_number(d_max, t3, half_length)
90
91     # drying-time discretisation error of the production grid (Richardson, order 2, two finest levels)
92     rows3 = conv["grid"]["q3_t_dry"]
93     rows4 = conv["grid"]["q4_t_dry"]
94     limit3, err3 = richardson_error([float(r["t_dry_h"]) for r in rows3])
95     limit4, err4 = richardson_error([float(r["t_dry_h"]) for r in rows4])
96
97     # problem 4: shrinkage geometry
98     r_end_cm = float(q4["spec"]["radius"]["r_end_cm"])
99     r0_cm = float(q4["spec"]["radius"]["r_start_cm"])
100     area_ratio = (r_end_cm / r0_cm) ** 2
101     fixed_ratio = float(q4["fixed_radius"]["t_dry_h"]) / float(q4["main"]["t_dry_h"])
102
103     n = int(q1["n"])
104     report = {
105         "aspect_ratio": LENGTH / R0,
106         "system_size": 2 * (n + 1) + 2,
107         "q1": {
108             "alpha": alpha1,
109             "D0": d1,
110             "t_heat_s": t_heat,
111             "t_mass_h": t_mass / HOUR,
112             "stiffness_ratio": alpha1 / d1,
113             "Fo_heat_1800s": fo_heat,
114             "Fo_mass_1800s": fo_mass,
115             "diffusion_length_cm": diffusion_length * 100.0,

```

```

115     },
116     "q2": {"centre_within_1K_h": None if gap_1k is None else gap_1k / HOUR, "centre_within_0p5K_h": gap_half},
117     "q3": {
118         "alpha3_C0": alpha3,
119         "D_max": d_max,
120         "D_at_target": d_target,
121         "D_surface_at_dry": d_surface_dry,
122         "D_ratio_max_to_surface": d_max / d_surface_dry,
123         "Bi_mass_at_dry": bi_mass_dry,
124         "Fo_axial_heat": fo_axial_heat,
125         "Fo_axial_mass_upper": fo_axial_mass,
126         "richardson_limit_h": limit3,
127         "production_error_min": err3 * 60.0,
128         "stats": q3["stats"],
129     },
130     "q4": {
131         "richardson_limit_h": limit4,
132         "production_error_min": err4 * 60.0,
133         "r_end_cm": r_end_cm,
134         "area_ratio": area_ratio,
135         "inverse_radius_ratio_squared": 1.0 / area_ratio,
136         "fixed_to_moving_ratio": fixed_ratio,
137         "D4_max": d4_max,
138         "D4_at_target": d4_target,
139         "D_ratio_three_to_four_init": d_max / d4_max,
140         "stats": q4["stats"],
141     },
142 }
143 ctx.write_json("derived.json", report)
144
145 ctx.number("AspectRatio", LENGTH / R0, ".1f")
146 ctx.number("SystemSize", 2 * (n + 1) + 2)
147 ctx.number("QoneHeatTimeScaleS", t_heat, ".0f")
148 ctx.number("QoneMassTimeScaleH", t_mass / HOUR, ".1f")
149 ctx.number("QoneStiffnessRatio", alpha1 / d1, ".0f")
150 ctx.number("QoneFourierHeat", fo_heat, ".2f")
151 ctx.number("QoneFourierMass", fo_mass, ".3f")
152 ctx.number("QoneDiffusionLengthCm", diffusion_length * 100.0, ".2f")
153 if gap_1k is not None:
154     ctx.number("QtwoCentreWithinOneKelvinHours", gap_1k / HOUR, ".2f")
155 if gap_half is not None:
156     ctx.number("QtwoCentreWithinHalfKelvinHours", gap_half / HOUR, ".2f")
157 ctx.number("FoAxialHeat", fo_axial_heat, ".2f")
158 ctx.number("FoAxialMassUpper", fo_axial_mass, ".2f")
159 ctx.number("DmaxAppendixThree", d_max, ".2e")
160 ctx.number("DtargetAppendixThree", d_target, ".1e")
161 ctx.number("DsurfaceAtDryQthree", d_surface_dry, ".1e")
162 ctx.number("DratioQthree", d_max / d_surface_dry, ".0f")
163 ctx.number("BiotMassAtDryQthree", bi_mass_dry, ".0f")
164 ctx.number("DmaxAppendixFour", d4_max, ".2e")
165 ctx.number("DtargetAppendixFour", d4_target, ".1e")
166 ctx.number("DratioThreeToFourInit", d_max / d4_max, ".1f")
167 ctx.number("QthreeDryHoursExtrapolated", limit3, ".4f")
168 ctx.number("QthreeGridErrorMin", err3 * 60.0, ".2f")
169 ctx.number("QfourDryHoursExtrapolated", limit4, ".4f")
170 ctx.number("QfourGridErrorMin", err4 * 60.0, ".2f")
171 ctx.number("QthreeNfev", int(q3["stats"]["nfev"]))
172 ctx.number("QthreeNlu", int(q3["stats"]["nlu"]))
173 ctx.number("QthreeSegments", int(q3["stats"]["segments"]))
174 ctx.number("QfourNfev", int(q4["stats"]["nfev"]))
175 ctx.number("QfourRadiusFinalCm", r_end_cm, ".3f")
176 ctx.number("QfourAreaRatioPct", 100.0 * area_ratio, ".1f")
177 ctx.number("QfourRadiusRatioSquared", 1.0 / area_ratio, ".2f")

```

```

178     ctx.number("QfourFixedToMovingRatio", fixed_ratio, ".2f")
179     ctx.number("ChamberSamples", int(q3["spec"]["chamber"]["samples"]))
180     ctx.number("ChamberEndHours", float(q3["spec"]["chamber"]["t_end"]) / HOUR, ".0f")
181     ctx.number("RadiusSamples", int(q4["spec"]["radius"]["samples"]))
182     ctx.number("RadiusEndHours", float(q4["spec"]["radius"]["t_end"]) / HOUR, ".0f")
183     return {"numbers": 39, "fo_axial_heat": fo_axial_heat, "fo_axial_mass_upper": fo_axial_mass}

```

pipelines/a/extension.py

```

1  """Model-evaluation extension (MDR-0008): the physics the problem's stylised model leaves out, quantified.
2
3  * a-posteriori latent-heat diagnostic of the problem-3 solution: the latent-heat demand of the model's own
4    evaporation rate, expressed as the equivalent surface cooling  $L_v j_w / h$ , against the convective supply;
5  * psychrometric reading of the chamber state (relative humidity, wet-bulb temperature);
6  * energy-limited variant: the evaporation flux capped by the wet-bulb limit  $h (T_{air} - T_{wb})/L_v$  (the
7    constant-rate period of classical drying theory) and the pure energy lower bound on the drying time;
8  * equilibrium-moisture driving force:  $C_{air}$  replaced by a constant  $C_{eq}$  in the mass boundary condition.
9
10 The main results of problems 1-4 are untouched; everything here feeds the paper's model-evaluation section.
11 """
12
13 from __future__ import annotations
14
15 from dataclasses import replace
16 from typing import Any
17
18 import numpy as np
19
20 from forge.context import StageContext
21 from forge.runner import stage
22 from pipelines.a.common import build_spec, production_n, spec_to_recipe, time_options
23 from pipelines.a.physics import (
24     C_INIT,
25     R0,
26     FluxCap,
27     dry_solid_density,
28     latent_heat,
29     relative_humidity,
30     wet_bulb_temperature,
31 )
32 from pipelines.a.science import HOUR, _parallel
33 from pipelines.a.solver import ProblemSpec
34
35 ORDINALS = ["One", "Two", "Three", "Four", "Five", "Six"]
36
37
38 def slug(label: str) -> str:
39     """File-system and npz friendly variant key: 'cap:1.0' -> 'cap_1p0'."""
40     return label.replace(":", "_").replace(".", "p")
41
42
43 def _first_time_below(t: np.ndarray, values: np.ndarray, threshold: float) -> float | None:
44     idx = np.where(values < threshold)[0]
45     return None if len(idx) == 0 else float(t[idx[0]])
46
47
48 def latent_diagnostic(hist: dict[str, Any], spec: ProblemSpec, rho_s: float) -> dict[str, Any]:
49     """Latent-heat demand implied by the model's own evaporation rate versus its convective heat supply.
50
51      $j_w = \rho_s h_m (C_s - C_{air})$  is the evaporation mass flux behind the moisture boundary condition; the energy
52     equation of the problem ignores its latent heat  $L_v j_w$ . The equivalent surface cooling  $L_v j_w / h$  is the

```

```

53     temperature depression a coupled model would need to carry that load by convection alone.
54     """
55     t = np.asarray(hist["t"], dtype=float)
56     ts = np.asarray(hist["surface_temp"], dtype=float)
57     ta = np.asarray(hist["air_temp"], dtype=float)
58     cs = np.asarray(hist["surface_moist"], dtype=float)
59     ca = np.asarray(hist["air_hum"], dtype=float)
60     mean_c = np.asarray(hist["mean_moist"], dtype=float)
61     flux = np.asarray(spec.surface_moisture_loss(t, cs, ca), dtype=float) # (kg/kg) m/s
62     j_w = rho_s * flux # kg/(m^2 s)
63     q_lat = latent_heat(ts) * j_w # W/m^2
64     q_conv = spec.props.h * (ta - ts) # W/m^2 delivered by the model's thermal boundary condition
65     dt_lat = q_lat / spec.props.h # K
66     circ = 2.0 * np.pi * R0
67     e_lat = np.concatenate([[0.0], np.cumsum(0.5 * (q_lat[1:] + q_lat[:-1]) * np.diff(t))]) * circ # J/m
68     e_conv = np.concatenate([[0.0], np.cumsum(0.5 * (q_conv[1:] + q_conv[:-1]) * np.diff(t))]) * circ
69     water_removed = rho_s * np.pi * R0**2 * (mean_c[0] - mean_c) # kg per m length
70     at: dict[str, dict[str, float]] = {}
71     for label, hours in (("three_h", 3.0), ("twelve_h", 12.0), ("twenty_four_h", 24.0)):
72         i = int(np.argmax(np.abs(t - hours * HOUR)))
73         at[label] = {
74             "t_h": float(t[i] / HOUR),
75             "dt_lat_K": float(dt_lat[i]),
76             "q_lat_W_m2": float(q_lat[i]),
77             "q_conv_W_m2": float(q_conv[i]),
78             "e_lat_MJ_m": float(e_lat[i] / 1e6),
79             "e_conv_kJ_m": float(e_conv[i] / 1e3),
80         }
81     below_1k = _first_time_below(t, dt_lat, 1.0)
82     below_01k = _first_time_below(t, dt_lat, 0.1)
83     return {
84         "arrays": {
85             "t": t,
86             "dt_lat": dt_lat,
87             "model_dt": ta - ts,
88             "q_lat": q_lat,
89             "q_conv": q_conv,
90             "e_lat": e_lat,
91             "e_conv": e_conv,
92             "water_removed": water_removed,
93         },
94         "summary": {
95             "rho_s0": rho_s,
96             "j_w_initial": float(j_w[0]),
97             "dt_lat_peak_K": float(dt_lat.max()),
98             "dt_lat_peak_t_h": float(t[int(np.argmax(dt_lat))] / HOUR),
99             "dt_lat_below_1K_h": None if below_1k is None else below_1k / HOUR,
100             "dt_lat_below_0p1K_h": None if below_01k is None else below_01k / HOUR,
101             "at": at,
102             "e_lat_total_MJ_per_m": float(e_lat[-1] / 1e6),
103             "e_conv_total_kJ_per_m": float(e_conv[-1] / 1e3),
104             "latent_to_convective_ratio": float(e_lat[-1] / max(e_conv[-1], 1e-300)),
105             "water_removed_kg_per_m": float(water_removed[-1]),
106         },
107     }
108
109
110 def chamber_psyrometrics(spec: ProblemSpec, cap: FluxCap) -> dict[str, Any]:
111     ch = spec.chamber
112     t_plateau = ch.temp_plateau * ch.temp_scale
113     w_plateau = ch.hum_plateau * ch.hum_scale
114     wb_plateau = wet_bulb_temperature(t_plateau, w_plateau)
115     t0, w0 = float(ch.temp[0]), float(ch.hum_scale * ch.hum[0])

```

```

116     wb0 = wet_bulb_temperature(t0, w0)
117     return {
118         "assumption": "chamber moisture concentration read as the humidity ratio of the air (kg water per kg dry air)"
119         " at 101325 Pa; Magnus saturation pressure; ASHRAE adiabatic-saturation wet bulb",
120         "initial": {
121             "temp": t0,
122             "w": w0,
123             "rh": float(relative_humidity(t0, w0)),
124             "wet_bulb": wb0,
125             "wet_bulb_depression": t0 - wb0,
126             "flux_cap": float(cap.cap[0]),
127         },
128         "plateau": {
129             "temp": t_plateau,
130             "w": w_plateau,
131             "rh": float(relative_humidity(t_plateau, w_plateau)),
132             "wet_bulb": wb_plateau,
133             "wet_bulb_depression": t_plateau - wb_plateau,
134             "flux_cap": cap.plateau,
135             "latent_heat": float(latent_heat(wb_plateau)),
136         },
137     }
138
139
140 @stage(
141     "extension",
142     deps=("ingest", "q3"),
143     description="Model-evaluation extension: latent-heat consistency, wet-bulb energy limit, C_eq driving force",
144 )
145 def extension(ctx: StageContext) -> dict[str, Any]:
146     n = production_n(ctx)
147     opts = time_options(ctx)
148     horizon = float(ctx.cfg("a.drying.horizon_s", 432000))
149     cap_factors = [float(f) for f in ctx.cfg("a.extension.cap_factors", [0.5, 1.0, 2.0])]
150     c_eq_values = [float(c) for c in ctx.cfg("a.extension.c_eq", [0.075, 0.10, 0.125])]
151     base = build_spec(ctx, "q3")
152     chamber, props = base.chamber, base.props
153     rho_s0 = float(dry_solid_density(props, C_INIT))
154     q3 = ctx.dep_data("q3", "summary.json")
155     t3 = float(q3["t_dry_s"])
156     hist = ctx.dep_data("q3", "history.json")
157
158     # 1. a-posteriori latent-heat diagnostic of the problem-3 solution
159     diag = latent_diagnostic(hist, base, rho_s0)
160     np.savez_compressed(ctx.out("diagnostic.npz"), **diag["arrays"])
161     ctx.log.info("extension.diagnostic", **{k: v for k, v in diag["summary"].items() if not isinstance(v, dict)})
162
163     # 2. psychrometrics of the chamber and the wet-bulb energy limit on the evaporation flux
164     cap = FluxCap.wet_bulb(chamber, props.h, rho_s0)
165     psychro = chamber_psychrometrics(base, cap)
166     mean_end = float(q3["mean_moist_at_dry"])
167     t_bound = rho_s0 * R0 * (C_INIT - mean_end) / (2.0 * cap.plateau) # lateral surface only, plateau limit
168     ctx.log.info("extension.psychrometrics", **psychro["plateau"], energy_bound_h=t_bound / HOUR)
169
170     # 3. variants: energy-capped evaporation (x factors) and equilibrium-moisture driving force
171     jobs: list[dict[str, Any]] = []
172     for f in cap_factors:
173         spec = replace(base, flux_cap=replace(cap, scale=f))
174         jobs.append(
175             {
176                 "label": f"cap:{f}",
177                 "recipe": spec_to_recipe(spec),
178                 "n": n,

```

```

179         "opts": opts,
180         "mode": "dry",
181         "horizon": horizon,
182         "checks": True,
183     }
184 )
185 for c in c_eq_values:
186     spec = replace(base, chamber=replace(chamber, hum=np.full_like(chamber.hum, c), hum_plateau=c, hum_scale=1.0))
187     jobs.append(
188         {
189             "label": f"ceq:{c}",
190             "recipe": spec_to_recipe(spec),
191             "n": n,
192             "opts": opts,
193             "mode": "dry",
194             "horizon": horizon,
195             "checks": True,
196         }
197     )
198 results = {r["label"]: r for r in _parallel(ctx, jobs)}
199
200 variants: list[dict[str, Any]] = []
201 arrays: dict[str, np.ndarray] = {}
202 checks: dict[str, Any] = {}
203 for label, res in results.items():
204     kind, raw = label.split(":")
205     value = float(raw)
206     t_dry = float(res["t_dry"])
207     row: dict[str, Any] = {
208         "case": kind,
209         "value": value,
210         "label": label,
211         "slug": slug(label),
212         "t_dry_h": t_dry / HOUR,
213         "t_dry_grid_h": float(res["t_dry_grid"]) / HOUR,
214         "diff_pct": 100.0 * (t_dry / t3 - 1.0),
215         "checks_passed": bool(res["checks"]["passed"]),
216         "moisture_residual": float(res["checks"]["moisture_balance"]["max_relative_residual"]),
217         "nfev": int(res["stats"]["nfev"]),
218     }
219     tt = np.asarray(res["mean_moist_t"], dtype=float)
220     if kind == "cap":
221         scaled = replace(cap, scale=value)
222         cs = np.asarray(res["surface_moist"], dtype=float)
223         ca = np.asarray(res["air_hum"], dtype=float)
224         active = props.hm * (cs - ca) > np.asarray(scaled.value(tt), dtype=float) / rho_s0 * (1.0 + 1e-9)
225         row["cap_release_h"] = float(tt[np.where(active)[0][-1]] / HOUR) if active.any() else 0.0
226         row["cap_plateau_kg_m2_s"] = cap_plateau * value
227         row["cap_plateau_kg_m2_h"] = cap_plateau * value * HOUR
228     checks[label] = res["checks"]
229     variants.append(row)
230     for key in ("mean_moist", "centre_moist", "surface_moist"):
231         arrays[f"{slug(label)}_{key}"] = np.asarray(res[key], dtype=float)
232     arrays[f"{slug(label)}_t"] = tt
233 np.savez_compressed(ctx.out("variants.npz"), **arrays)
234 checks["passed"] = bool(all(v["checks_passed"] for v in variants))
235 ctx.write_json("verification_extension.json", checks)
236 report = {
237     "base_t_dry_h": t3 / HOUR,
238     "rho_s0": rho_s0,
239     "psychrometrics": psychro,
240     "diagnostic": diag["summary"],
241     "energy_bound_h": t_bound / HOUR,

```

```

242     "cap_factors": cap_factors,
243     "c_eq": c_eq_values,
244     "variants": variants,
245     "n": n,
246 }
247 ctx.write_json("extension.json", report)
248
249 # 4. paper numbers
250 d = diag["summary"]
251 ctx.number("ExtDrySolidDensity", rho_s0, ".0f")
252 ctx.number("ExtRhInitPct", 100.0 * psychro["initial"]["rh"], ".0f")
253 ctx.number("ExtRhPlateauPct", 100.0 * psychro["plateau"]["rh"], ".0f")
254 ctx.number("ExtWetBulbInit", psychro["initial"]["wet_bulb"], ".1f")
255 ctx.number("ExtWetBulbPlateau", psychro["plateau"]["wet_bulb"], ".1f")
256 ctx.number("ExtWetBulbDepression", psychro["plateau"]["wet_bulb_depression"], ".1f")
257 ctx.number("ExtLatentHeatMJ", psychro["plateau"]["latent_heat"] / 1e6, ".3f")
258 ctx.number("ExtFluxCapPlateau", cap.plateau, ".2e")
259 ctx.number("ExtFluxCapPlateauKgPerHour", cap.plateau * HOUR, ".3f")
260 ctx.number("ExtFluxCapInit", float(cap.cap[0]), ".2e")
261 ctx.number("ExtModelFluxInit", d["j_w_initial"], ".2e")
262 ctx.number("ExtFluxRatioInit", d["j_w_initial"] / float(cap.cap[0]), ".0f")
263 ctx.number("ExtLatentDeltaTPeak", d["dt_lat_peak_K"], ".0f")
264 ctx.number("ExtLatentDeltaTThreeH", d["at"]["three_h"]["dt_lat_K"], ".1f")
265 ctx.number("ExtLatentDeltaTTwelveH", d["at"]["twelve_h"]["dt_lat_K"], ".1f")
266 ctx.number("ExtLatentDeltaTTwentyFourH", d["at"]["twenty_four_h"]["dt_lat_K"], ".2f")
267 if d["dt_lat_below_1K_h"] is not None:
268     ctx.number("ExtLatentBelowOneKHours", d["dt_lat_below_1K_h"], ".1f")
269 if d["dt_lat_below_0p1K_h"] is not None:
270     ctx.number("ExtLatentBelowTenthKHours", d["dt_lat_below_0p1K_h"], ".1f")
271 ctx.number("ExtLatentEnergyMJ", d["e_lat_total_MJ_per_m"], ".2f")
272 ctx.number("ExtConvEnergyKJ", d["e_conv_total_kJ_per_m"], ".1f")
273 ctx.number("ExtLatentToConvRatio", d["latent_to_convective_ratio"], ".0f")
274 ctx.number("ExtWaterRemovedKg", d["water_removed_kg_per_m"], ".3f")
275 ctx.number("ExtEnergyBoundHours", t_bound / HOUR, ".1f")
276 ctx.number("ExtEnergyBoundPct", 100.0 * (t_bound / t3 - 1.0), ".1f")
277 cap_rows = [v for v in variants if v["case"] == "cap"]
278 ceq_rows = [v for v in variants if v["case"] == "ceq"]
279 for i, row in enumerate(cap_rows):
280     tag = ORDINALS[i]
281     ctx.number(f"ExtCap{tag}Factor", row["value"], ".1f")
282     ctx.number(f"ExtCap{tag}Hours", row["t_dry_h"], ".2f")
283     ctx.number(f"ExtCap{tag}Pct", row["diff_pct"], ".1f")
284     ctx.number(f"ExtCap{tag}ReleaseHours", row["cap_release_h"], ".1f")
285     if row["value"] == 1.0:
286         ctx.number("ExtCappedHours", row["t_dry_h"], ".2f")
287         ctx.number("ExtCappedPct", row["diff_pct"], ".1f")
288         ctx.number("ExtCappedReleaseHours", row["cap_release_h"], ".1f")
289 for i, row in enumerate(ceq_rows):
290     tag = ORDINALS[i]
291     ctx.number(f"ExtCeq{tag}Value", row["value"], ".3f")
292     ctx.number(f"ExtCeq{tag}Hours", row["t_dry_h"], ".2f")
293     ctx.number(f"ExtCeq{tag}Pct", row["diff_pct"], ".1f")
294 return {
295     "jobs": len(jobs),
296     "checks_passed": checks["passed"],
297     "energy_bound_h": t_bound / HOUR,
298     "variants": {v["label"]: v["t_dry_h"] for v in variants},
299 }

```

pipelines/a/figures.py

```
1 """Paper figures (vector PDF + PNG preview) built only from stage outputs.
2
3 Style: house palette (Okabe-Ito, fixed order for categorical series), single-hue sequential ramps for
4 time-ordered profiles (light = early, dark = late), one measure per axis, thin marks, recessive grid.
5 """
6
7 from __future__ import annotations
8
9 import json
10 from pathlib import Path
11 from typing import Any
12
13 import numpy as np
14 import pandas as pd
15
16 from forge import plotting
17 from forge.context import StageContext
18 from forge.runner import stage
19 from pipelines.a.physics import APPENDIX2, APPENDIX3, APPENDIX4, C_TARGET, Radius
20
21 HOUR = 3600.0
22 INK = "#1F2933"
23 MUTED = "#6B7280"
24 CAT = plotting.OKABE_ITO
25
26
27 def _ramp(name: str, count: int) -> list[Any]:
28     from matplotlib import colormaps
29
30     cmap = colormaps[name]
31     return [cmap(x) for x in np.linspace(0.35, 0.95, count)]
32
33
34 def _load(path: Path) -> Any:
35     return json.loads(path.read_text(encoding="utf-8"))
36
37
38 def _finish(ctx: StageContext, fig: Any, name: str, index: list[dict[str, str]], purpose: str) -> None:
39     info = plotting.save(fig, ctx.out("figures", f"{name}.pdf"))
40     index.append({"file": f"{name}.pdf", "purpose": purpose, "bytes": str(info["bytes"])})
41     ctx.log.info("figure", name=name, bytes=info["bytes"])
42
43
44 def _profiles(ax: Any, snaps: Any, field: str, unit: str, cmap: str, label_fmt: Any) -> None:
45     t = snaps["t"]
46     colors = _ramp(cmap, len(t))
47     for i, ti in enumerate(t):
48         r_cm = snaps["xi"] * snaps["radius"][i] * 100.0
49         ax.plot(r_cm, snaps[field][i], color=colors[i], label=label_fmt(ti), lw=1.4)
50     ax.set_xlabel("到药材中心的距离 $r$ / cm")
51     ax.set_ylabel(unit)
52     ax.legend(frameon=False, ncol=2, title="时间", title_fontsize=8)
53
54
55 def _history(ax: Any, hist: dict[str, Any], keys: list[tuple[str, str]], unit: str, scale: float = 1.0) -> None:
56     t = np.asarray(hist["t"]) / scale
57     for j, (key, label) in enumerate(keys):
58         style = "--" if key.startswith("air") else "-"
59         ax.plot(t, hist[key], style, color=CAT[j], label=label, lw=1.4)
60     ax.set_ylabel(unit)
61     ax.legend(frameon=False)
```

```

62
63
64 @stage(
65     "figures",
66     deps=("ingest", "q1", "q2", "q3", "q4", "convergence", "verification", "sensitivity", "extension"),
67     description="Paper figures: data, properties, profiles/histories of q1-q4, convergence, verification, sensitivity",
68 )
69 def figures(ctx: StageContext) -> dict[str, Any]:
70     plotting.setup()
71     index: list[dict[str, str]] = []
72     data = ctx.dep("ingest") / "data"
73     a1 = pd.read_parquet(data / "附件1__Sheet1.parquet")
74     a2 = pd.read_parquet(data / "附件2__Sheet1.parquet")
75
76     # 1. chamber conditions and radius data -----
77     q3sum = _load(ctx.dep("q3") / "summary.json")
78     fig, axes = plotting.new_figure(6.3, 5.4, nrows=3, sharex=False)
79     t1 = a1["时间"].to_numpy(float) / HOUR
80     axes[0].plot(t1, a1["温度"], color=CAT[1], lw=1.2, label="附件 1 样本")
81     axes[0].axhline(
82         q3sum["spec"]["chamber"]["temp_plateau"], color=MUTED, ls=":", lw=1, label="恒温阶段平台 (末 1 h 均值)"
83     )
84     axes[0].set_ylabel("烘房温度 / °C")
85     axes[0].legend(frameon=False, loc="lower right")
86     axes[1].plot(t1, a1["水分浓度"], color=CAT[0], lw=1.2, label="附件 1 样本")
87     axes[1].axhline(q3sum["spec"]["chamber"]["hum_plateau"], color=MUTED, ls=":", lw=1, label="平台 (末 1 h 均值)")
88     axes[1].set_ylabel("烘房水分浓度 / (kg/kg)")
89     axes[1].set_xlabel("时间 / h")
90     axes[1].legend(frameon=False, loc="lower right")
91     t2 = a2["时间"].to_numpy(float) / HOUR
92     hist4 = _load(ctx.dep("q4") / "history.json")
93     radius = Radius.from_series(a2["时间"].to_numpy(float), a2["半径"].to_numpy(float))
94     t_fine = np.linspace(0.0, float(a2["时间"].max()), 2000)
95     axes[2].plot(t_fine / HOUR, np.asarray(radius.value(t_fine)) * 100.0, color=CAT[2], lw=1.2, label="PCHIP 插值 R(t)"
96         ↪ )
97     axes[2].plot(
98         t2[:4], a2["半径"].to_numpy(float)[:4], "o", ms=3, color=INK, mfc="white", label="附件 2 样本 (每 2 h)"
99     )
100     axes[2].set_ylabel("半径 / cm")
101     axes[2].set_xlabel("时间 / h")
102     axes[2].legend(frameon=False)
103     for ax, tag in zip(axes, "abc"):
104         ax.set_title(f"({tag})", loc="left", fontsize=9)
105     _finish(ctx, fig, "fig_chamber", index, "附件 1 烘房温湿度 (a,b) 与附件 2 半径及其 PCHIP 插值 (c) ; 边界条件数据")
106
107     # 2. property laws -----
108     fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
109     c = np.linspace(0.02, 2.6, 400)
110     axes[0].plot(c, APPENDIX2.diffusivity(c, np.full_like(c, 28.0)), color=CAT[0], label="附录 2")
111     for j, (props, name) in enumerate(((APPENDIX3, "附录 3"), (APPENDIX4, "附录 4"))):
112         for temp, ls in ((28.0, "--"), (50.0, "-")):
113             axes[0].plot(
114                 c, props.diffusivity(c, np.full_like(c, temp)), ls, color=CAT[j + 1], label=f"{name}, {temp:g} °C"
115             )
116     axes[0].set_yscale("log")
117     axes[0].set_xlabel("水分浓度 $C$ / (kg/kg)")
118     axes[0].set_ylabel("扩散系数 $D$ / (m$^2$/s)")
119     axes[0].axvline(C_TARGET, color=MUTED, ls=":", lw=1)
120     axes[0].legend(frameon=False, fontsize=7)
121     for j, (props, name) in enumerate(((APPENDIX3, "附录 3"), (APPENDIX4, "附录 4"))):
122         axes[1].plot(c, props.k(c) / (props.rho(c) * props.cp(c)) * 1e7, color=CAT[j + 1], label=name)
123     axes[1].axhline(APPENDIX2.k0 / (APPENDIX2.rho0 * APPENDIX2.cp0) * 1e7, color=CAT[0], label="附录 2")
124     axes[1].set_xlabel("水分浓度 $C$ / (kg/kg)")

```

```

124 axes[1].set_ylabel(r"热扩散率 $\alpha$ / ($10^{-7}$ m$^2$/s)")
125 axes[1].legend(frameon=False, fontsize=7)
126 for ax, tag in zip(axes, "ab"):
127     ax.set_title(f"({tag})", loc="left", fontsize=9)
128 _finish(
129     ctx,
130     fig,
131     "fig_properties",
132     index,
133     "三组附录物性：水分扩散系数随 C、T 的变化 (a, 对数坐标) 与热扩散率 (b)；说明干燥后期表面干层的刚性",
134 )
135
136 # 3-4. problem 1 -----
137 snaps = np.load(ctx.dep("q1") / "snapshots.npz")
138 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
139 _profiles(axes[0], snaps, "temp", "温度 / °C", "Oranges", lambda t: f"{t:.0f} s")
140 _profiles(axes[1], snaps, "moist", "水分浓度 / (kg/kg)", "Blues", lambda t: f"{t:.0f} s")
141 for ax, tag in zip(axes, "ab"):
142     ax.set_title(f"({tag})", loc="left", fontsize=9)
143 _finish(ctx, fig, "fig_q1_profiles", index, "问题 1: 表 1/2 各时刻的温度 (a) 与水分浓度 (b) 径向剖面")
144 hist = _load(ctx.dep("q1") / "history.json")
145 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
146 _history(
147     axes[0], hist, [("centre_temp", "中心 r=0"), ("surface_temp", "表面 r=2 cm"), ("air_temp", "烘房)], "温度 / °C"
148 )
149 _history(
150     axes[1],
151     hist,
152     [("centre_moist", "中心 r=0"), ("surface_moist", "表面 r=2 cm"), ("mean_moist", "体积平均)],
153     "水分浓度 / (kg/kg)",
154 )
155 for ax, tag in zip(axes, "ab"):
156     ax.set_xlabel("时间 / s")
157     ax.set_title(f"({tag})", loc="left", fontsize=9)
158 _finish(
159     ctx,
160     fig,
161     "fig_q1_history",
162     index,
163     "问题 1: 中心、表面与烘房温度 (a) 及中心、表面、平均水分浓度 (b) 随时间的变化",
164 )
165
166 # 5-6. problem 2 -----
167 snaps = np.load(ctx.dep("q2") / "snapshots.npz")
168 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
169 _profiles(axes[0], snaps, "temp", "温度 / °C", "Oranges", lambda t: f"{t / HOUR:.1f} h")
170 _profiles(axes[1], snaps, "moist", "水分浓度 / (kg/kg)", "Blues", lambda t: f"{t / HOUR:.1f} h")
171 for ax, tag in zip(axes, "ab"):
172     ax.set_title(f"({tag})", loc="left", fontsize=9)
173 _finish(ctx, fig, "fig_q2_profiles", index, "问题 2: 表 3/4 各时刻的温度 (a) 与水分浓度 (b) 径向剖面")
174 hist = _load(ctx.dep("q2") / "history.json")
175 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
176 _history(
177     axes[0],
178     hist,
179     [("centre_temp", "中心 r=0"), ("surface_temp", "表面 r=2 cm"), ("air_temp", "烘房)],
180     "温度 / °C",
181     HOUR,
182 )
183 _history(
184     axes[1],
185     hist,
186     [("centre_moist", "中心 r=0"), ("surface_moist", "表面 r=2 cm"), ("mean_moist", "体积平均)],

```

```

187     "水分浓度 / (kg/kg)",
188     HOUR,
189 )
190 for ax, tag in zip(axes, "ab"):
191     ax.set_xlabel("时间 / h")
192     ax.set_title(f"({tag})", loc="left", fontsize=9)
193 _finish(
194     ctx, fig, "fig_q2_history", index, "问题 2: 3 h 内温度 (a) 与水分浓度 (b) 的时程; 表面在预热阶段即接近烘房温度"
195 )
196
197 # 7. problem 3 -----
198 snaps = np.load(ctx.dep("q3") / "snapshots.npz")
199 hist = _load(ctx.dep("q3") / "history.json")
200 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
201 _profiles(axes[0], snaps, "moist", "水分浓度 / (kg/kg)", "Blues", lambda t: f"{t / HOUR:.0f} h")
202 axes[0].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
203 axes[0].annotate("0.15", xy=(0.02, C_TARGET), xytext=(0.02, C_TARGET + 0.08), color=MUTED, fontsize=7)
204 _history(
205     axes[1],
206     hist,
207     [{"centre_moist", "中心 r=0"}, {"surface_moist", "表面"}, {"mean_moist", "体积平均"}],
208     "水分浓度 / (kg/kg)",
209     HOUR,
210 )
211 axes[1].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
212 axes[1].axvline(q3sum["t_dry_h"], color=MUTED, ls="--", lw=1)
213 axes[1].annotate(
214     f"烘干结束 {q3sum['t_dry_h']:.2f} h",
215     xy=(q3sum["t_dry_h"], 1.2),
216     xytext=(q3sum["t_dry_h"] * 0.55, 1.6),
217     color=INK,
218     fontsize=7,
219     arrowprops={"arrowstyle": "-", "color": MUTED, "lw": 0.8},
220 )
221 axes[1].set_xlabel("时间 / h")
222 for ax, tag in zip(axes, "ab"):
223     ax.set_title(f"({tag})", loc="left", fontsize=9)
224 _finish(
225     ctx,
226     fig,
227     "fig_q3",
228     index,
229     "问题 3: 每 6 h 与烘干结束时刻的水分剖面 (a) 及中心、表面、平均水分浓度时程 (b), 虚线为 0.15 kg/kg 判据",
230 )
231
232 # 8. problem 4 -----
233 snaps = np.load(ctx.dep("q4") / "snapshots.npz")
234 q4sum = _load(ctx.dep("q4") / "summary.json")
235 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
236 _profiles(axes[0], snaps, "moist", "水分浓度 / (kg/kg)", "Blues", lambda t: f"{t / HOUR:.0f} h")
237 axes[0].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
238 surf_r = snaps["radius"] * 100.0
239 axes[0].plot(surf_r, snaps["moist"][:, -1], "o", ms=3, color=INK, mfc="white", label="药材表面", zorder=5)
240 axes[0].legend(frameon=False, ncol=2, fontsize=7, title="时间", title_fontsize=8)
241 t4 = np.asarray(hist4["t"]) / HOUR
242 axes[1].plot(
243     t4, hist4["centre_moist"], color=CAT[0], label=f"收缩, 拉格朗日 (主): {q4sum['main']['t_dry_h']:.1f} h"
244 )
245 alt_hist = pd.read_parquet(ctx.dep("q4") / "profiles_moist_alt.parquet")
246 axes[1].plot(
247     alt_hist["t"].to_numpy(float) / HOUR,
248     alt_hist["0.0"].to_numpy(float),
249     "--",

```

```

250     color=CAT[1],
251     label=f"收缩, 欧拉 (备选) : {q4sum['alternative']['t_dry_h']:.1f} h",
252 )
253 fixed_t = q4sum["fixed_radius"]["t_dry_h"]
254 axes[1].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
255 axes[1].axvline(fixed_t, color=CAT[2], ls="-.", lw=1, label=f"半径固定 2 cm: {fixed_t:.1f} h")
256 axes[1].set_xlabel("时间 / h")
257 axes[1].set_ylabel("中心水分浓度 / (kg/kg)")
258 axes[1].legend(frameon=False, fontsize=7)
259 for ax, tag in zip(axes, "ab"):
260     ax.set_title(f"({tag})", loc="left", fontsize=9)
261 _finish(
262     ctx,
263     fig,
264     "fig_q4",
265     index,
266     "问题 4: 物理坐标下每 6 h 的水分剖面与表面位置 (a) ; 中心水分时程: 主模型、欧拉备选、固定半径 (b) ",
267 )
268
269 # 9. convergence and analytic errors -----
270 conv = _load(ctx.dep("convergence") / "convergence.json")
271 ver = _load(ctx.dep("verification") / "verification_report.json")
272 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
273 keys = [
274     ("q1_temp", "问题 1 温度"),
275     ("q1_moist", "问题 1 水分"),
276     ("q2_temp", "问题 2 温度"),
277     ("q2_moist", "问题 2 水分"),
278     ("q3_moist", "问题 3 水分"),
279     ("q4_moist", "问题 4 水分"),
280 ]
281 markers = "osD~v<"
282 for j, (key, label) in enumerate(keys):
283     rows = conv["grid"][key]
284     axes[0].loglog(
285         [r["dr_cm"] for r in rows],
286         [r["max_abs_error"] for r in rows],
287         marker=markers[j],
288         ms=4,
289         color=CAT[j],
290         label=label,
291         lw=1.2,
292     )
293 dr = np.array([0.1 / 2**k for k in range(conv["levels"] - 1)])
294 e0 = conv["grid"]["q1_temp"][0]["max_abs_error"]
295 axes[0].loglog(dr, e0 * (dr / dr[0]) ** 2, color=MUTED, ls=":", lw=1, label="二阶参考")
296 axes[0].set_xlabel(r"$\Delta r$ / cm")
297 axes[0].set_ylabel("相对最细网格的最大误差")
298 axes[0].legend(frameon=False, fontsize=6.5)
299 for j, (field, label) in enumerate(((("temp", "温度 (贝塞尔级数)"), ("moist", "水分 (贝塞尔级数)"))):
300     rows = ver["analytic"][field]["levels"]
301     axes[1].loglog(
302         [r["dr_cm"] for r in rows],
303         [r["max_abs_error"] for r in rows],
304         marker=markers[j],
305         ms=4,
306         color=CAT[j],
307         label=label,
308         lw=1.2,
309     )
310 drv = np.array([r["dr_cm"] for r in ver["analytic"][field]["levels"]])
311 e0 = ver["analytic"][field]["levels"][0]["max_abs_error"]
312 axes[1].loglog(drv, e0 * (drv / drv[0]) ** 2, color=MUTED, ls=":", lw=1, label="二阶参考")

```

```

313 axes[1].set_xlabel(r"$\Delta r$ / cm")
314 axes[1].set_ylabel("与解析解的最大误差")
315 axes[1].legend(frameon=False, fontsize=6.5)
316 for ax, tag in zip(axes, "ab"):
317     ax.set_title(f"({tag})", loc="left", fontsize=9)
318 _finish(
319     ctx,
320     fig,
321     "fig_convergence",
322     index,
323     "网格收敛：各问题相对最细网格的误差（a）与常物性算例相对贝塞尔级数解的误差（b），虚线为二阶斜率",
324 )
325
326 # 10. conservation residual histories -----
327 fig, ax = plotting.new_figure(6.3, 2.6)
328 for j, prob in enumerate(("q1", "q2", "q3", "q4")):
329     h = _load(ctx.dep(prob) / "history.json")
330     t = np.asarray(h["t"])
331     w = np.asarray(h["mean_moist"]) / 2.0
332     rad = np.asarray(h["radius_cm"]) / 100.0
333     cum = np.asarray(h["cum_moist_loss"]) # int hm (C_s - C_air)/R dt accumulated inside the integrator
334     if np.allclose(rad, rad[0]): # d/dt (R^2 W) = -R hm (C_s - C_air)
335         lhs = rad[0] ** 2 * (w - w[0])
336         cum = -(rad[0] ** 2) * cum
337     else: # Lagrangian: dW/dt = -hm (C_s - C_air)/R
338         lhs = w - w[0]
339         cum = -cum
340     res = np.abs(lhs - cum) / max(abs(lhs[-1]), 1e-300)
341     ax.semilogy(t / HOUR, np.maximum(res, 1e-17), color=CAT[j], label=f"问题 {prob[1]}", lw=1.2)
342 ax.set_xlabel("时间 / h")
343 ax.set_ylabel("水分守恒相对残差")
344 ax.legend(frameon=False)
345 _finish(
346     ctx,
347     fig,
348     "fig_conservation",
349     index,
350     "离散水分总量变化与积分器内累积的表面通量之差（相对最终失水量）：四个问题均在机器精度量级",
351 )
352
353 # 11. sensitivity -----
354 sens = _load(ctx.dep("sensitivity") / "sensitivity.json")
355 names = {
356     "h": "$h$",
357     "hm": "$h_m$",
358     "D0": "$D_0$ (问题 3)",
359     "Cair": "$C_{\mathrm{air}}$",
360     "Tplateau": "$T_{\mathrm{air}}$ 平台",
361     "shrink": "收缩幅度 $s$ (问题 4)",
362     "D0q4": "$D_0$ (问题 4)",
363 }
364 rows = [r for r in sens["table"] if r.get("elasticity") is not None]
365 fig, ax = plotting.new_figure(6.3, 2.6)
366 labels = [names.get(r["parameter"], r["parameter"]) for r in rows]
367 vals = [r["elasticity"] for r in rows]
368 colors = [CAT[1] if v > 0 else CAT[0] for v in vals]
369 ax.barh(labels, vals, color=colors, height=0.55)
370 for y, v in enumerate(vals):
371     ax.text(
372         v + (0.02 if v >= 0 else -0.02),
373         y,
374         f"{v:+.2f}",
375         va="center",

```

```

376         ha="left" if v >= 0 else "right",
377         fontsize=7,
378         color=INK,
379     )
380     ax.axvline(0, color=INK, lw=0.8)
381     ax.set_xlabel("烘干时间的弹性 $E$" )
382     lim = max(abs(v) for v in vals) * 1.35 + 0.05
383     ax.set_xlim(-lim, lim)
384     ax.invert_yaxis()
385     _finish(ctx, fig, "fig_sensitivity", index, "烘干时间对各参数的弹性 ( $\pm 10\%$  中心差分)：正值表示参数增大使烘干变慢")
386
387     # 12. end effects (2-D axisymmetric) -----
388     ax2 = np.load(ctx.dep("verification") / "axisym_snapshots.npz")
389     ee = ver["end_effect_2d"]
390     fig, axes = plotting.new_figure(6.3, 2.7, ncols=2)
391     axes[0].plot(
392         ee["axial_profile_at_1d_dry"]["z_cm"],
393         ee["axial_profile_at_1d_dry"]["centre_line_moist"],
394         color=CAT[0],
395         label="二维轴对称, 中心线 r=0",
396     )
397     axes[0].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
398     axes[0].set_xlabel("到中截面的轴向距离 $z$ / cm")
399     axes[0].set_ylabel("水分浓度 / (kg/kg)")
400     axes[0].legend(frameon=False, fontsize=7)
401     t = ax2["t"] / HOUR
402     axes[1].plot(t, ax2["moist_1d"][:, 0], color=CAT[0], label="一维模型, 中心")
403     axes[1].plot(t, ax2["moist_mid"][:, 0], "--", color=CAT[1], label="二维模型, 中截面中心")
404     axes[1].plot(t, ax2["moist_end"][:, 0], "-.", color=CAT[2], label="二维模型, 端面中心")
405     axes[1].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
406     axes[1].set_xlabel("时间 / h")
407     axes[1].set_ylabel("水分浓度 / (kg/kg)")
408     axes[1].legend(frameon=False, fontsize=7)
409     for ax, tag in zip(axes, "ab"):
410         ax.set_title(f"({tag})", loc="left", fontsize=9)
411     _finish(
412         ctx,
413         fig,
414         "fig_endeffect",
415         index,
416         "端部效应：一维烘干结束时刻二维模型中心线沿轴向的水分 (a)；一维与二维（中截面、端面）中心水分时程 (b)",
417     )
418
419     # 13. model-evaluation extension (MDR-0008) -----
420     ext = _load(ctx.dep("extension") / "extension.json")
421     dg = np.load(ctx.dep("extension") / "diagnostic.npz")
422     va = np.load(ctx.dep("extension") / "variants.npz")
423     hist3 = _load(ctx.dep("q3") / "history.json")
424     fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
425     t = dg["t"] / HOUR
426     axes[0].semilogy(t, np.maximum(dg["dt_lat"], 1e-3), color=CAT[1], lw=1.4, label="潜热等效温降 $L_v j_w/h$")
427     axes[0].semilogy(
428         t, np.maximum(dg["model_dt"], 1e-3), color=CAT[0], lw=1.4, label="题给模型 $T_{\\mathrm{air}}-T_s$")
429     )
430     axes[0].axhline(1.0, color=MUTED, ls=":", lw=1)
431     axes[0].set_ylim(1e-3, 2e2)
432     axes[0].set_xlabel("时间 / h")
433     axes[0].set_ylabel("温差 / K")
434     axes[0].legend(frameon=False, fontsize=7)
435     axes[1].plot(
436         np.asarray(hist3["t"]) / HOUR,
437         hist3["centre_moist"],
438         color=CAT[0],

```

```

439         lw=1.4,
440         label=f"题给模型: {ext['base_t_dry_h']:.1f} h",
441     )
442     styles = {"cap": ("--", CAT[1]), "ceq": ("-. ", CAT[2])}
443     shown = [v for v in ext["variants"] if (v["case"] == "cap" and v["value"] == 1.0) or v["case"] == "ceq"]
444     ceq_colors = [CAT[2], CAT[3], CAT[4], CAT[5]]
445     k = 0
446     for v in shown:
447         ls, color = styles[v["case"]]
448         if v["case"] == "ceq":
449             color = ceq_colors[k % len(ceq_colors)]
450             k += 1
451             label = f"$C_{\{\mathrm{{eq}}\}}={v['value']:g}$: {v['t_dry_h']:.1f} h"
452         else:
453             label = f"能量限制 (湿球极限) : {v['t_dry_h']:.1f} h"
454         axes[1].plot(
455             va[f"{v['slug']}_t"] / HOUR, va[f"{v['slug']}_centre_moist"], ls, color=color, lw=1.3, label=label
456         )
457     axes[1].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
458     axes[1].set_xlabel("时间 / h")
459     axes[1].set_ylabel("中心水分浓度 / (kg/kg)")
460     axes[1].legend(frameon=False, fontsize=7)
461     for ax, tag in zip(axes, "ab"):
462         ax.set_title(f"({tag})", loc="left", fontsize=9)
463     _finish(
464         ctx,
465         fig,
466         "fig_extension",
467         index,
468         "模型评价扩展: 题给模型蒸发率的潜热等效温降与模型实际温差 (a) ; 能量限制与平衡含水率驱动力下的中心水分时程 (b) ",
469     )
470
471     ctx.write_json("figure_index.json", index)
472     return {"figures": len(index)}

```

pipelines/a/outputs.py

```

1  """Deliverable stages: result workbooks (organisers' templates) and booktabs tables (+ CSV twins)."""
2
3  from __future__ import annotations
4
5  import csv
6  import json
7  from pathlib import Path
8  from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13 from forge import xlsx
14 from forge.context import StageContext
15 from forge.runner import stage
16 from pipelines.a.common import COLS, R_OUT_CM, R_TABLE_CM
17
18 HOUR = 3600.0
19
20
21 def _rows(frame: pd.DataFrame, t_min: float, columns: list[str]) -> list[list[Any]]:
22     sel = frame[frame["t"] >= t_min]
23     times = sel["t"].to_numpy(float)
24     values = sel[columns].to_numpy(float)

```

```

25     out: list[list[Any]] = []
26     for t, row in zip(times, values):
27         out.append([int(t + 0.5), *[None if np.isnan(v) else float(v) for v in row]])
28     return out
29
30
31 @stage("results", deps=("q1", "q2", "q3", "q4"), description="Fill result1-4.xlsx from the organisers' templates")
32 def results(ctx: StageContext) -> dict[str, Any]:
33     header = ["", *[float(r) for r in R_OUT_CM]]
34     report: dict[str, Any] = {}
35     for prob, name in (("q1", "result1.xlsx"), ("q2", "result2.xlsx")):
36         d = ctx.dep(prob)
37         temp = pd.read_parquet(d / "profiles_temp.parquet")
38         moist = pd.read_parquet(d / "profiles_moist.parquet")
39         sheets = {"温度": (header, _rows(temp, 1.0, COLS)), "水分浓度": (header, _rows(moist, 1.0, COLS))}
40         report[name] = xlsx.write_result(ctx.templates / name, ctx.out(name), sheets)
41         ctx.log.info("results.written", file=name, **report[name]["sheets"])
42     moist3 = pd.read_parquet(ctx.dep("q3") / "profiles_moist.parquet")
43     report["result3.xlsx"] = xlsx.write_result(
44         ctx.templates / "result3.xlsx", ctx.out("result3.xlsx"), {"Sheet1": (header, _rows(moist3, 60.0, COLS))}
45     )
46     moist4 = pd.read_parquet(ctx.dep("q4") / "profiles_moist.parquet")
47     header4 = [*header, "药材表面"]
48     report["result4.xlsx"] = xlsx.write_result(
49         ctx.templates / "result4.xlsx",
50         ctx.out("result4.xlsx"),
51         {"Sheet1": (header4, _rows(moist4, 60.0, [*COLS, "surface"]))},
52     )
53     ctx.write_json("results_report.json", report)
54     return {name: info["sheets"] for name, info in report.items()}
55
56
57 # -----
58 # tables
59 # -----
60
61
62 def _fmt(v: Any, digits: int = 4) -> str:
63     if v is None or (isinstance(v, float) and np.isnan(v)):
64         return "--"
65     if isinstance(v, str):
66         return v
67     return f"{float(v):.{digits}f}"
68
69
70 def _write_table(
71     ctx: StageContext,
72     name: str,
73     caption: str,
74     label: str,
75     header: list[str],
76     rows: list[list[Any]],
77     *,
78     digits: int | list[int] = 4,
79     align: str | None = None,
80     note: str | None = None,
81 ) -> None:
82     ncol = len(header)
83     digs = [digits] * ncol if isinstance(digits, int) else digits
84     align = align or ("l" + "r" * (ncol - 1))
85     lines = [
86         "% generated by forge (tables stage) — do not edit",
87         "\\begin{table}[htbp]",

```

```

88         "\\centering",
89         f"\\caption{{{caption}}}",
90         f"\\label{{{label}}}",
91         f"\\begin{{tabular}}{{@{{}}{align}@{{}}}}",
92         "\\toprule",
93         " & ".join(header) + " \\\\",
94         "\\midrule",
95     ]
96     for row in rows:
97         lines.append(" & ".join(_fmt(v, digs[j]) for j, v in enumerate(row)) + " \\\\")
98     lines += ["\\bottomrule", "\\end{tabular}"]
99     if note:
100         lines.append(f"\\par\\smallskip\\footnotesize {note}")
101     lines.append("\\end{table}")
102     ctx.out("tables", f"{name}.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
103     with ctx.out("tables", f"{name}.csv").open("w", encoding="utf-8", newline="") as fh:
104         writer = csv.writer(fh)
105         writer.writerow(header)
106         for row in rows:
107             writer.writerow(_fmt(v, digs[j]) for j, v in enumerate(row))
108
109
110 def _load(path: Path) -> Any:
111     return json.loads(path.read_text(encoding="utf-8"))
112
113
114 @stage(
115     "tables",
116     deps=("q1", "q2", "q3", "q4", "convergence", "verification", "sensitivity", "extension"),
117     description="LaTeX (booktabs) tables of the paper with CSV twins",
118 )
119 def tables(ctx: StageContext) -> dict[str, Any]:
120     r_head = [f"{r:g}" for r in R_TABLE_CM]
121     q1 = _load(ctx.dep("q1") / "tables.json")
122     _write_table(
123         ctx,
124         "tab_q1_temp",
125         "30 分钟内药材的温度 (\\si{\\celsius}; 列为到药材中心的距离/cm) ",
126         "tab:q1-temp",
127         ["时间/s", *r_head],
128         [[int(r[0]), *r[1:]] for r in q1["temp"]],
129         digits=[0, 4, 4, 4, 4, 4],
130     )
131     _write_table(
132         ctx,
133         "tab_q1_moist",
134         "30 分钟内药材的水分浓度 (kg/kg; 列为到药材中心的距离/cm) ",
135         "tab:q1-moist",
136         ["时间/s", *r_head],
137         [[int(r[0]), *r[1:]] for r in q1["moist"]],
138         digits=[0, 4, 4, 4, 4, 4],
139     )
140     q2 = _load(ctx.dep("q2") / "tables.json")
141     _write_table(
142         ctx,
143         "tab_q2_temp",
144         "3 小时内药材的温度 (\\si{\\celsius}; 列为到药材中心的距离/cm) ",
145         "tab:q2-temp",
146         ["时间/h", *r_head],
147         [[r[0] / HOUR, *r[1:]] for r in q2["temp"]],
148         digits=[1, 4, 4, 4, 4, 4],
149     )
150     _write_table(

```

```

151     ctx,
152     "tab_q2_moist",
153     "3 小时内药材的水分浓度 (kg/kg; 列为到药材中心的距离/cm) ",
154     "tab:q2-moist",
155     ["时间/h", *r_head],
156     [[r[0] / HOUR, *r[1:]] for r in q2["moist"]],
157     digits=[1, 4, 4, 4, 4, 4],
158 )
159 q3 = _load(ctx.dep("q3") / "tables.json")["moist"]
160 rows3 = [[f"{r[0]:g}", *r[1:]] for r in q3[:-1]] + [[f"烘干结束时间 ({q3[-1][0]:.2f} h) ", *q3[-1][1:]]
161 _write_table(
162     ctx,
163     "tab_q3_moist",
164     "药材烘干过程的水分浓度 (kg/kg; 固定半径 2 cm, 附录 3 物性; 列为到药材中心的距离/cm) ",
165     "tab:q3-moist",
166     ["时间/h", *r_head],
167     rows3,
168     note="烘干结束时间为  $\max_r C(r,t) < 0.15$  首次成立的精确时刻 (事件求根), 结果文件按 60 s 分辨率给出。",
169 )
170 q4 = _load(ctx.dep("q4") / "tables.json")["moist"]
171 rows4 = [[f"{r[0]:g}", *r[1:]] for r in q4[:-1]] + [[f"烘干结束时间 ({q4[-1][0]:.2f} h) ", *q4[-1][1:]]
172 _write_table(
173     ctx,
174     "tab_q4_moist",
175     "考虑尺寸变化的药材烘干过程水分浓度 (kg/kg; 附录 4 物性; 列为到药材中心的距离/cm, 超出当前半径处为 "--) ",
176     "tab:q4-moist",
177     ["时间/h", *r_head, "药材表面"],
178     rows4,
179     note="固定物理位置的值由  $\xi=r/R(t)$  网格上的三次样条取得; "药材表面"为  $\xi=1$  的节点值。",
180 )
181 q4_alt = _load(ctx.dep("q4") / "tables_alt.json")
182 rows4a = [[f"{r[0]:g}", *r[1:]] for r in q4_alt["moist"][:-1]] + [
183     [f"烘干结束时间 ({q4_alt['moist'][-1][0]:.2f} h) ", *q4_alt["moist"][-1][1:]]
184 ]
185 _write_table(
186     ctx,
187     "tab_q4_moist_alt",
188     f"备选写法 ({q4_alt['formulation']}, MDR-0006) 下的问题 4 水分浓度 (kg/kg) ",
189     "tab:q4-moist-alt",
190     ["时间/h", *r_head, "药材表面"],
191     rows4a,
192 )
193
194 # convergence
195 conv = _load(ctx.dep("convergence") / "convergence.json")
196 rows_c: list[list[Any]] = []
197 keys = [
198     ("q1_temp", "问题 1 温度"),
199     ("q1_moist", "问题 1 水分"),
200     ("q2_temp", "问题 2 温度"),
201     ("q2_moist", "问题 2 水分"),
202     ("q3_moist", "问题 3 水分"),
203     ("q4_moist", "问题 4 水分"),
204 ]
205 for key, label in keys:
206     for row in conv["grid"][key]:
207         rows_c.append([label, row["k"], row["dr_cm"], row["max_abs_error"], row.get("observed_order")])
208 _write_table(
209     ctx,
210     "tab_convergence",
211     f"网格收敛: 以  $\Delta r = 0.1/2^{\{\text{conv['levels']} - 1\}}$  cm 为参照的最大绝对误差与观测阶数",
212     "tab:convergence",
213     ["量", "$k$", " $\Delta r$ /cm", "最大误差", "观测阶"],

```

```

214     [a, b, c, f"{d:.2e}", "--" if e is None else f"{e:.2f}"] for a, b, c, d, e in rows_c],
215     digits=[0, 0, 4, 0, 0],
216     align="llrrr",
217 )
218 rows_d = []
219 for prob, label in (("q3", "问题 3"), ("q4", "问题 4")):
220     for row in conv["grid"][f"{prob}_t_dry"]:
221         rows_d.append([label, row["k"], row["dr_cm"], row["t_dry_h"], row.get("diff_to_fineest_min")])
222 _write_table(
223     ctx,
224     "tab_convergence_drytime",
225     "烘干时间随网格加密的收敛 (与最细网格之差, 单位 min) ",
226     "tab:convergence-drytime",
227     ["问题", "$k$", "$\\Delta r$/cm", "$t_{\\mathrm{end}}$/h", "与最细网格之差/min"],
228     [a, b, c, f"{d:.4f}", "--" if e is None else f"{e:.3f}"] for a, b, c, d, e in rows_d],
229     digits=[0, 0, 4, 0, 0],
230     align="llrrr",
231 )
232 rows_t = []
233 for row in conv["time"].values():
234     opt = row["options"]
235     label = f"{opt['method']}, rtol={opt['rtol']:.0e}, atol={opt['atol']:.0e}"
236     if opt.get("max_step"):
237         label += f", $\\Delta t\\le{opt['max_step']:g}$ s"
238     rows_t.append([label, f"{row['max_abs_diff_moist']:.2e}", f"{row['t_dry_diff_s']:.2f}", row["nfev"]])
239 _write_table(
240     ctx,
241     "tab_time_independence",
242     "时间积分无关性 (问题 3, 生产网格; 与生产设置 ",
243     f"{ctx.cfg('a.time.method', 'BDF')}, rtol={float(ctx.cfg('a.time.rtol', 1e-9)):.0e}, ",
244     f"atol={float(ctx.cfg('a.time.atol', 1e-11)):.0e}、步长不限的差异) ",
245     "tab:time-independence",
246     ["积分器、容差与最大步长", "水分最大差", "$t_{\\mathrm{end}}$ 差/s", "右端项调用次数"],
247     rows_t,
248     digits=[0, 0, 0, 0],
249     align="lrrr",
250     note="$\\Delta t\\le$ 行为限制最大步长的自适应 BDF (其余设置同生产); ",
251     "步长限制改变的只是步序列, 结果差异应在容差量级。",
252 )
253
254 # verification summary
255 ver = _load(ctx.dep("verification") / "verification_report.json")
256 checks: list[list[Any]] = []
257 for prob in ("q1", "q2", "q3", "q4"):
258     v = _load(ctx.dep(prob) / f"verification_{prob}.json")
259     mb = v["moisture_balance"]
260     ex = v["extremum"]
261     checks.append(
262         [
263             f"问题 {prob[1]}",
264             f"{mb['max_relative_residual']:.2e}",
265             "是" if ex["moist_within_bounds"] and ex["temp_within_bounds"] else "否",
266             "是" if ex["moist_monotone_in_r"] else "否",
267             "是" if v["passed"] else "否",
268         ]
269     )
270 _write_table(
271     ctx,
272     "tab_verification",
273     "各问题的独立校验: 水分守恒相对残差、极值原理、沿径向单调性、综合结论",
274     "tab:verification",
275     ["问题", "守恒残差", "极值原理", "单调性", "通过"],
276     checks,

```

```

277     digits=[0, 0, 0, 0, 0],
278     align="lrccc",
279 )
280 rows_a = []
281 for field, label in (("temp", "温度 (Bi=%.3f) "), ("moist", "水分 (Bi$_m$=%.3f) ")):
282     a = ver["analytic"][field]
283     for row in a["levels"]:
284         rows_a.append(
285             [
286                 label % a["biot"],
287                 row["k"],
288                 row["dr_cm"],
289                 f"{row['max_abs_error']:.2e}",
290                 "--" if row.get("observed_order") is None else f"{row['observed_order']:.2f}",
291             ]
292         )
293 _write_table(
294     ctx,
295     "tab_analytic",
296     "与常物性 Robin 圆柱贝塞尔级数解的最大绝对误差及观测阶数",
297     "tab:analytic",
298     ["量", "$k$", "$\\Delta r$/cm", "最大误差", "观测阶"],
299     rows_a,
300     digits=[0, 0, 4, 0, 0],
301     align="llrrr",
302 )
303
304 # sensitivity
305 sens = _load(ctx.dep("sensitivity") / "sensitivity.json")
306 names = {
307     "h": "$h$",
308     "hm": "$h_m$",
309     "D0": "$D_0$ (问题 3) ",
310     "Cair": "$C_{\\mathrm{air}}$",
311     "Tplateau": "$T_{\\mathrm{air}}$ 平台",
312     "shrink": "收缩幅度 $s$ (问题 4) ",
313     "D0q4": "$D_0$ (问题 4) ",
314 }
315 rows_s = []
316 for row in sens["table"]:
317     rows_s.append(
318         [names.get(row["parameter"], row["parameter"]), row["base_h"]]
319         + [row.get(f"x{f}") for f in sens["factors"]]
320         + [row.get("elasticity")]
321     )
322 _write_table(
323     ctx,
324     "tab_sensitivity",
325     "参数  $\pm 10\%$ 、 $\pm 20\%$  时的烘干时间 (h) 与弹性  $E=(\\Delta t_{\\mathrm{end}}/t_{\\mathrm{end}})/(\\Delta p/p)$ ",
326     "tab:sensitivity",
327     ["参数", "基准"] + [f"$\\times{f:g}$" for f in sens["factors"]] + ["弹性"],
328     rows_s,
329     digits=[0, 2, 2, 2, 2, 2, 3],
330 )
331 alt_names = {
332     "plateau_last_sample": "平台取末样本 (MDR-0003 方案 2) ",
333     "plateau_nominal": "平台取 50  $^\\circ\\mathrm{C}$ 、0.05 (方案 3) ",
334     "face_arithmetic": "界面系数算术平均",
335     "face_harmonic": "界面系数调和平均",
336     "q4_eulerian": "问题 4 欧拉写法 (含对流项) ",
337     "q4_lagrangian": "问题 4 拉格朗日写法",
338     "q4_fixed_radius": "问题 4 物性、半径固定 2 cm",
339     "q3_mean_criterion": "以平均含水率  $<0.15$  为判据",

```

```

340 }
341 rows_alt = [[alt_names.get(r["case"], r["case"]), r["t_dry_h"], r["diff_pct"]] for r in sens["alternatives"]]
342 _write_table(
343     ctx,
344     "tab_alternatives",
345     "备选解释与离散化选择下的烘干时间（相对主结果的变化）",
346     "tab:alternatives",
347     ["备选", " $t_{\mathrm{end}}$ $/h", "变化/\%"],
348     rows_alt,
349     digits=[0, 2, 2],
350     align="lrr",
351 )
352 # model-evaluation extension (MDR-0008)
353 ext = _load(ctx.dep("extension") / "extension.json")
354 rows_e: list[list[Any]] = [{"题给模型（主结果）", "--", ext["base_t_dry_h"], 0.0}]
355 for row in ext["variants"]:
356     if row["case"] == "cap":
357         name = f"蒸发通量受能量限制（湿球极限  $\times \{row['value']:g\}$  $"
358         param = f" $j_{\mathrm{max}} = \{row['cap\_plateau\_kg\_m2\_h']:.3f\}$  kg/(m $^2 \cdot h$ )"
359     else:
360         name = "驱动力取  $C_s - C_{\mathrm{eq}}$  $（平衡含水率）"
361         param = f" $C_{\mathrm{eq}} = \{row['value']:g\}$  kg/kg"
362     rows_e.append([name, param, row["t_dry_h"], row["diff_pct"]])
363 rows_e.append(
364     [
365         "纯能量下界（全程按湿球极限蒸发）",
366         f" $j_{\mathrm{max}} = \{ext['psychrometrics']['plateau']['flux\_cap'] * HOUR:.3f\}$  kg/(m $^2 \cdot h$ )",
367         ext["energy_bound_h"],
368         100.0 * (ext["energy_bound_h"] / ext["base_t_dry_h"] - 1.0),
369     ]
370 )
371 _write_table(
372     ctx,
373     "tab_extension",
374     "模型评价扩展（MDR-0008，问题 3 设定）：能量限制与平衡含水率驱动力下的烘干时间及相对主结果的变化",
375     "tab:extension",
376     ["情形", "参数", " $t_{\mathrm{end}}$ $/h", "变化/\%"],
377     rows_e,
378     digits=[0, 0, 2, 1],
379     align="llrr",
380     note="湿球极限  $j_{\mathrm{max}} = h \cdot (T_{\mathrm{air}} - T_{\mathrm{wb}}) / L_v$  随烘房状态变化，表中给出恒温阶段的值。",
381 )
382 written = sorted(p.name for p in (ctx.stage_dir / "tables").glob("*.tex"))
383 return {"tables": len(written), "files": written}

```

pipelines/a/physics.py

```

1 """Physical laws and boundary data of the herb-drying problem (SI units; temperatures in degC).
2
3 Property laws follow the problem's appendices (MDR-0001/0002), the drying-chamber history and its
4 constant-temperature plateau follow MDR-0003, the shrinking radius follows MDR-0006.
5 """
6
7 from __future__ import annotations
8
9 from dataclasses import dataclass, replace
10 from typing import Any
11
12 import numpy as np
13 from scipy.interpolate import PchipInterpolator
14

```

```

15 R0 = 0.02 # initial radius, m
16 LENGTH = 0.25 # cylinder length, m
17 T_INIT = 28.0 # degC
18 C_INIT = 2.55 # kg/kg (dry basis)
19 C_TARGET = 0.15 # kg/kg, drying finished when every point is below this value
20 KELVIN = 273.15
21 C_FLOOR = 1e-6 # only used inside D(C, T) to avoid division by zero
22 T_DATA_END = 14400.0 # last chamber sample, s
23 PLATEAU_WINDOW = 3600.0 # averaging window for the constant-temperature plateau, s
24
25
26 @dataclass(frozen=True)
27 class Properties:
28     """rho = rho0 + rho1*C; cp = cp0 + cp1*C/(C+1); k = k0 + k1*C/(C+1); D = d0*exp(-da/C)*exp(-de/T_K)."""
29
30     name: str
31     rho0: float
32     rho1: float
33     cp0: float
34     cp1: float
35     k0: float
36     k1: float
37     d0: float
38     da: float
39     de: float
40     h: float = 25.0 # W/(m^2 K), appendix 2 (carried over to problems 2-4, MDR-0002)
41     hm: float = 8e-7 # m/s, appendix 2
42     d_const: float | None = None # constant-D override (analytic verification only)
43
44     def rho(self, c: np.ndarray) -> np.ndarray:
45         return self.rho0 + self.rho1 * c
46
47     def cp(self, c: np.ndarray) -> np.ndarray:
48         return self.cp0 + self.cp1 * c / (c + 1.0)
49
50     def k(self, c: np.ndarray) -> np.ndarray:
51         return self.k0 + self.k1 * c / (c + 1.0)
52
53     def diffusivity(self, c: np.ndarray, t_degc: np.ndarray) -> np.ndarray:
54         if self.d_const is not None:
55             return np.full_like(np.asarray(c, dtype=float), self.d_const)
56         cs = np.maximum(c, C_FLOOR)
57         out = self.d0 * np.exp(-self.da / cs)
58         if self.de:
59             out = out * np.exp(-self.de / (np.asarray(t_degc, dtype=float) + KELVIN))
60         return out
61
62     def scaled(self, **factors: float) -> Properties:
63         """Return a copy with the named fields multiplied by the given factors (sensitivity runs)."""
64         return replace(self, **{key: getattr(self, key) * value for key, value in factors.items()})
65
66     def describe(self) -> dict[str, Any]:
67         return {
68             "name": self.name,
69             "rho": f"{self.rho0} + {self.rho1} C",
70             "cp": f"{self.cp0} + {self.cp1} C/(C+1)",
71             "k": f"{self.k0} + {self.k1} C/(C+1)",
72             "D": f"{self.d0} exp(-{self.da}/C) exp(-{self.de}/T_K)" if self.d_const is None else f"{self.d_const}",
73             "h": self.h,
74             "hm": self.hm,
75         }
76
77

```

```

78 APPENDIX2 = Properties("appendix2", 820.0, 0.0, 2600.0, 0.0, 0.36, 0.0, 7e-9, 0.89, 0.0)
79 APPENDIX3 = Properties("appendix3", 650.0, 128.0, 1450.0, 2736.0, 0.21, 0.38, 2.4e-3, 0.45, 3850.0)
80 APPENDIX4 = Properties("appendix4", 760.0, 90.0, 1850.0, 2150.0, 0.12, 0.20, 4.2e-4, 0.30, 3850.0)
81 PROPERTIES = {"appendix2": APPENDIX2, "appendix3": APPENDIX3, "appendix4": APPENDIX4}
82
83
84 @dataclass(frozen=True)
85 class Chamber:
86     """Drying-chamber temperature and humidity: linear interpolation of attachment 1, constant plateau after it."""
87
88     t: np.ndarray
89     temp: np.ndarray
90     hum: np.ndarray
91     temp_plateau: float
92     hum_plateau: float
93     temp_scale: float = 1.0 # multiplies the plateau only (sensitivity)
94     hum_scale: float = 1.0 # multiplies the whole humidity history (sensitivity)
95
96     @classmethod
97     def from_series(cls, t: np.ndarray, temp: np.ndarray, hum: np.ndarray, plateau: str = "mean_last_hour") -> Chamber:
98         t = np.asarray(t, dtype=float)
99         temp = np.asarray(temp, dtype=float)
100         hum = np.asarray(hum, dtype=float)
101         if plateau == "mean_last_hour":
102             window = t >= t[-1] - PLATEAU_WINDOW
103             tp, hp = float(temp[window].mean()), float(hum[window].mean())
104         elif plateau == "last_sample":
105             tp, hp = float(temp[-1]), float(hum[-1])
106         elif plateau == "nominal":
107             tp, hp = 50.0, 0.05
108         else:
109             raise ValueError(f"unknown plateau rule {plateau!r}")
110         return cls(t, temp, hum, tp, hp)
111
112     @classmethod
113     def constant(cls, temp: float, hum: float) -> Chamber:
114         return cls(np.array([0.0, 1.0]), np.array([temp, temp]), np.array([hum, hum]), temp, hum)
115
116     def air_temperature(self, t: Any) -> Any:
117         return np.interp(t, self.t, self.temp, right=self.temp_plateau * self.temp_scale)
118
119     def air_humidity(self, t: Any) -> Any:
120         return self.hum_scale * np.interp(t, self.t, self.hum, right=self.hum_plateau)
121
122     def knots(self) -> np.ndarray:
123         """Instants where the forcing is not smooth (sample times incl. the switch to the plateau)."""
124         return self.t[1:] if len(self.t) > 2 else np.array([])
125
126     def describe(self) -> dict[str, Any]:
127         return {
128             "samples": len(self.t),
129             "t_end": float(self.t[-1]),
130             "temp_plateau": self.temp_plateau * self.temp_scale,
131             "hum_plateau": self.hum_plateau * self.hum_scale,
132             "temp_scale": self.temp_scale,
133             "hum_scale": self.hum_scale,
134         }
135
136
137 class Radius:
138     """Radius history R(t): monotone PCHIP through attachment 2 (m), held at its last value afterwards."""
139
140     def __init__(

```

```

141     self, t: np.ndarray, r: np.ndarray, shrink_scale: float = 1.0, *, force_interpolant: bool = False
142 ) -> None:
143     self.t = np.asarray(t, dtype=float)
144     self.r = np.asarray(r, dtype=float)
145     self.shrink_scale = shrink_scale
146     self.is_constant = bool(np.all(self.r == self.r[0])) and not force_interpolant
147     self._t0, self._t1 = float(self.t[0]), float(self.t[-1])
148     if not self.is_constant:
149         self._pchip = PchipInterpolator(self.t, self.r, extrapolate=False)
150         self._deriv = self._pchip.derivative()
151
152 @classmethod
153 def from_series(cls, t: np.ndarray, r_cm: np.ndarray, shrink_scale: float = 1.0) -> Radius:
154     r = R0 - shrink_scale * (R0 - np.asarray(r_cm, dtype=float) / 100.0)
155     return cls(np.asarray(t, dtype=float), r, shrink_scale)
156
157 @classmethod
158 def constant(cls, r: float = R0) -> Radius:
159     return cls(np.array([0.0, 1.0]), np.array([r, r]), 0.0)
160
161 def value(self, t: Any) -> Any:
162     if self.is_constant:
163         return np.full_like(np.asarray(t, dtype=float), self.r[0]) if np.ndim(t) else float(self.r[0])
164     return self._pchip(np.clip(t, self._t0, self._t1))
165
166 def knots(self) -> np.ndarray:
167     """PCHIP knots (curvature jumps) including the switch to the constant tail."""
168     return np.array([]) if self.is_constant else self.t[1:]
169
170 def rate(self, t: Any) -> Any:
171     if self.is_constant:
172         return np.zeros_like(np.asarray(t, dtype=float)) if np.ndim(t) else 0.0
173     tt = np.asarray(t, dtype=float)
174     inside = (tt > self._t0) & (tt < self._t1)
175     return np.where(inside, self._deriv(np.clip(tt, self._t0, self._t1)), 0.0)
176
177 def describe(self) -> dict[str, Any]:
178     return {
179         "samples": len(self.t),
180         "t_end": float(self.t[-1]),
181         "r_start_cm": float(self.r[0] * 100),
182         "r_end_cm": float(self.r[-1] * 100),
183         "shrink_scale": self.shrink_scale,
184     }
185
186
187 def thermal_diffusivity(props: Properties, c: float) -> float:
188     cc = np.asarray(c, dtype=float)
189     return float(props.k(cc) / (props.rho(cc) * props.cp(cc)))
190
191
192 def biot_numbers(props: Properties, c: float, t_deg: float, r: float = R0) -> dict[str, float]:
193     cc = np.asarray(c, dtype=float)
194     return {
195         "Bi_heat": float(props.h * r / props.k(cc)),
196         "Bi_mass": float(props.hm * r / props.diffusivity(cc, np.asarray(t_deg))),
197         "alpha": thermal_diffusivity(props, c),
198         "D": float(props.diffusivity(cc, np.asarray(t_deg))),
199     }
200
201
202 # -----
203 # Psychrometrics and the energy-limited evaporation cap (model-evaluation extension, MDR-0008)

```

```

204 # -----
205
206 P_ATM = 101325.0 # Pa
207 L_REF = 2.501e6 # J/kg, latent heat of vaporisation of water at 0 degC
208 L_SLOPE = 2361.0 # J/(kg K), linear decrease of the latent heat with temperature (0-100 degC)
209
210
211 def latent_heat(t_degC: Any) -> Any:
212     """Latent heat of vaporisation of water, J/kg (linear fit to the steam tables, 0-100 degC)."""
213     return L_REF - L_SLOPE * np.asarray(t_degC, dtype=float)
214
215
216 def saturation_pressure(t_degC: Any) -> Any:
217     """Saturation vapour pressure over liquid water, Pa (Magnus form, Alduchov and Eskridge 1996)."""
218     t = np.asarray(t_degC, dtype=float)
219     return 610.94 * np.exp(17.625 * t / (t + 243.04))
220
221
222 def humidity_ratio(p_vapour: Any, p_total: float = P_ATM) -> Any:
223     """Humidity ratio (kg water per kg dry air) of moist air with vapour partial pressure ``p_vapour``."""
224     pv = np.asarray(p_vapour, dtype=float)
225     return 0.622 * pv / (p_total - pv)
226
227
228 def vapour_pressure(w: Any, p_total: float = P_ATM) -> Any:
229     """Vapour partial pressure, Pa, of moist air with humidity ratio ``w``."""
230     ww = np.asarray(w, dtype=float)
231     return ww * p_total / (0.622 + ww)
232
233
234 def relative_humidity(t_degC: Any, w: Any) -> Any:
235     """Relative humidity (0-1) of air at dry-bulb temperature ``t_degC`` with humidity ratio ``w``."""
236     return vapour_pressure(w) / saturation_pressure(t_degC)
237
238
239 def wet_bulb_temperature(t_degC: float, w: float) -> float:
240     """Thermodynamic wet-bulb (adiabatic-saturation) temperature, degC.
241
242     ASHRAE Handbook - Fundamentals (2017), chapter 1, eq. (33), temperatures in degC and energies in kJ/kg:
243      $w = ((2501 - 2.326 t^*) w_s(t^*) - 1.006 (t - t^*)) / (2501 + 1.86 t - 4.186 t^*)$ . Saturated air returns t.
244     """
245     from scipy.optimize import brentq
246
247     t = float(t_degC)
248     target = float(w)
249
250     def residual(tw: float) -> float:
251         ws = float(humidity_ratio(saturation_pressure(tw)))
252         return ((2501.0 - 2.326 * tw) * ws - 1.006 * (t - tw)) / (2501.0 + 1.86 * t - 4.186 * tw) - target
253
254     if residual(t) <= 0.0:
255         return t
256     return float(brentq(residual, -40.0, t, xtol=1e-10, maxiter=200))
257
258
259 def dry_solid_density(props: Properties, c: Any) -> Any:
260     """Dry-matter density  $\rho_s = \rho(C)/(1 + C)$ , kg/m3 (bulk density per unit of dry-basis moisture)."""
261     cc = np.asarray(c, dtype=float)
262     return props.rho(cc) / (1.0 + cc)
263
264
265 @dataclass(frozen=True)
266 class FluxCap:

```

```

267     """Upper bound on the surface evaporation mass flux, kg/(m2 s), sampled on the chamber history.
268
269     ``value(t)`` interpolates linearly between the samples and holds ``plateau`` afterwards; ``rho_s`` converts
270     the bound into the (kg/kg) m/s units of the moisture boundary condition; ``scale`` multiplies the bound.
271     """
272
273     t: np.ndarray
274     cap: np.ndarray
275     plateau: float
276     rho_s: float
277     scale: float = 1.0
278
279     @classmethod
280     def wet_bulb(cls, chamber: Chamber, h: float, rho_s: float, scale: float = 1.0) -> FluxCap:
281         """Energy limit h (Tair - Twb)/Lv(Twb) of a surface held at the wet-bulb temperature."""
282
283         def bound(temp: float, hum: float) -> float:
284             tw = wet_bulb_temperature(temp, hum)
285             return float(h * (temp - tw) / latent_heat(tw))
286
287         caps = np.array([bound(float(a), float(b)) for a, b in zip(chamber.temp, chamber.hum_scale * chamber.hum)])
288         plateau = bound(chamber.temp_plateau * chamber.temp_scale, chamber.hum_plateau * chamber.hum_scale)
289         return cls(np.asarray(chamber.t, dtype=float), caps, plateau, rho_s, scale)
290
291     def value(self, t: Any) -> Any:
292         return self.scale * np.interp(t, self.t, self.cap, right=self.plateau)
293
294     def describe(self) -> dict[str, Any]:
295         return {
296             "cap_initial": float(self.cap[0]) * self.scale,
297             "cap_plateau": self.plateau * self.scale,
298             "rho_s": self.rho_s,
299             "scale": self.scale,
300         }

```

pipelines/a/science.py

```

1  """Science stages of Problem A: q1-q4 (solutions + independent checks), convergence, verification, sensitivity."""
2
3  from __future__ import annotations
4
5  import math
6  import re
7  from concurrent.futures import ProcessPoolExecutor
8  from dataclasses import replace
9  from typing import Any
10
11  import numpy as np
12
13  from forge.context import StageContext
14  from forge.runner import stage
15  from pipelines.a import verify
16  from pipelines.a.common import (
17      PROPS,
18      R_TABLE_CM,
19      build_spec,
20      load_chamber,
21      load_radius,
22      mean_moisture,
23      physical_frame,
24      production_n,

```

```

25     profiles_frame,
26     snapshot_arrays,
27     spec_from_recipe,
28     spec_to_recipe,
29     table_cells,
30     time_options,
31 )
32 from pipelines.a.physics import (
33     APPENDIX2,
34     C_INIT,
35     C_TARGET,
36     R0,
37     T_INIT,
38     Chamber,
39     Radius,
40     biot_numbers,
41 )
42 from pipelines.a.solver import ProblemSpec, Solution, sample_physical, solve, solve_until_dry, subgrid_indices
43
44 Q1_TABLE_T = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
45 Q2_TABLE_T = np.arange(1, 7) * 1800.0
46 HOUR = 3600.0
47
48
49 # -----
50 # helpers
51 # -----
52
53
54 def _store_solution(ctx: StageContext, sol: Solution, spec: ProblemSpec, snapshot_times: np.ndarray) -> None:
55     moving = not spec.radius.is_constant
56     for field in ("temp", "moist"):
57         frame = physical_frame(sol, field) if moving else profiles_frame(sol, field)
58         frame.to_parquet(ctx.out(f"profiles_{field}.parquet"), index=False)
59     np.savez_compressed(ctx.out("snapshots.npz"), **snapshot_arrays(sol, snapshot_times))
60     history = {
61         "t": sol.t.tolist(),
62         "air_temp": sol.air_temp.tolist(),
63         "air_hum": sol.air_hum.tolist(),
64         "radius_cm": (sol.radius * 100.0).tolist(),
65         "centre_temp": sol.temp[:, 0].tolist(),
66         "surface_temp": sol.temp[:, -1].tolist(),
67         "centre_moist": sol.moist[:, 0].tolist(),
68         "surface_moist": sol.moist[:, -1].tolist(),
69         "mean_moist": mean_moisture(sol).tolist(),
70         "surface_moist_flux": sol.surface_moisture_flux(spec.props).tolist(),
71         "cum_moist_loss": None if sol.cum_moist_loss is None else sol.cum_moist_loss.tolist(),
72     }
73     ctx.write_json("history.json", history)
74
75
76 def _checks(spec: ProblemSpec, sol: Solution) -> dict[str, Any]:
77     balance = verify.moisture_balance(
78         sol.t,
79         sol.xi,
80         sol.moist,
81         sol.radius,
82         sol.air_hum,
83         spec.props.hm,
84         cum_loss=sol.cum_moist_loss,
85         flux=spec.surface_moisture_loss(sol.t, sol.moist[:, -1], sol.air_hum),
86     )
87     extremum = verify.extremum_checks(sol.temp, sol.moist, sol.air_temp, sol.air_hum, spec.t_init, spec.c_init)

```

```

88     report: dict[str, Any] = {"moisture_balance": balance, "extremum": extremum}
89     if spec.props.rho1 == 0 and spec.props.cpl == 0 and spec.props.k1 == 0 and spec.radius.is_constant:
90         report["energy_balance"] = verify.energy_balance(
91             sol.t, sol.xi, sol.temp, float(sol.radius[0]), sol.air_temp, spec.props, cum_loss=sol.cum_heat_loss
92         )
93     if sol.t_dry is not None and sol.dry_profile is not None:
94         report["drying"] = verify.drying_checks(sol.t, sol.moist, sol.t_dry, sol.dry_profile)
95     passed = [
96         balance["max_relative_residual"] < 1e-4,
97         extremum["temp_within_bounds"],
98         extremum["moist_within_bounds"],
99         extremum["moist_monotone_in_r"],
100        extremum["moist_argmax_is_centre"],
101    ]
102    if "energy_balance" in report:
103        passed.append(report["energy_balance"]["max_relative_residual"] < 1e-4)
104    if "drying" in report:
105        d = report["drying"]
106        passed += [d["rows_before_all_wet"], d["rows_after_all_dry"], d["profile_max_error_at_t_dry"] < 1e-6]
107    report["passed"] = bool(all(passed))
108    return report
109
110
111 def _run_recipe(job: dict[str, Any]) -> dict[str, Any]:
112     """Worker: solve one recipe and return compact arrays (used by the parallel studies)."""
113     spec = spec_from_recipe(job["recipe"])
114     n = int(job["n"])
115     opts = dict(job["opts"])
116     out: dict[str, Any] = {"label": job["label"], "n": n, "opts": opts}
117     if job["mode"] == "dry":
118         sol = solve_until_dry(spec, n, float(job.get("dt_out", 60.0)), horizon=float(job["horizon"]), **opts)
119         idx = subgrid_indices(n)
120         snap_t = np.asarray(job.get("snapshot_times", []), dtype=float)
121         lookup = {float(t): i for i, t in enumerate(sol.t)}
122         rows = [lookup[float(t)] for t in snap_t if float(t) in lookup]
123         out.update(
124             {
125                 "t_dry": sol.t_dry,
126                 "t_dry_grid": sol.stats.get("t_dry_grid"),
127                 "snapshot_t": sol.t[rows].tolist(),
128                 "snapshot_moist": sol.moist[rows][:, idx].tolist(),
129                 "dry_profile_sub": None if sol.dry_profile is None else sol.dry_profile[idx].tolist(),
130                 "mean_moist_t": sol.t.tolist(),
131                 "mean_moist": mean_moisture(sol).tolist(),
132                 "centre_moist": sol.moist[:, 0].tolist(),
133                 "surface_moist": sol.moist[:, -1].tolist(),
134                 "air_hum": sol.air_hum.tolist(),
135                 "stats": sol.stats,
136             }
137         )
138         if job.get("checks"):
139             out["checks"] = _checks(spec, sol)
140     else:
141         t_out = np.asarray(job["t_out"], dtype=float)
142         sol = solve(spec, n, t_out, **opts)
143         idx = subgrid_indices(n)
144         out.update({"t": sol.t.tolist(), "temp": sol.temp[:, idx].tolist(), "moist": sol.moist[:, idx].tolist()})
145         out["stats"] = sol.stats
146     return out
147
148
149 def _parallel(ctx: StageContext, jobs: list[dict[str, Any]]) -> list[dict[str, Any]]:
150     workers = max(1, min(int(ctx.cfg("a.sensitivity.workers", 8)), len(jobs)))

```

```

151     ctx.log.info("parallel.start", jobs=len(jobs), workers=workers)
152     results: list[dict[str, Any]] = []
153     with ProcessPoolExecutor(max_workers=workers) as pool:
154         for i, res in enumerate(pool.map(_run_recipe, jobs)):
155             results.append(res)
156             ctx.log.info("parallel.progress", done=i + 1, total=len(jobs), label=res["label"])
157     return results
158
159
160 def _tag(text: str) -> str:
161     """Letters-only CamelCase key fragment for ctx.number (digits are spelled out)."""
162     out = text
163     for old, new in (("q1", "Qone"), ("q2", "Qtwo"), ("q3", "Qthree"), ("q4", "Qfour"), ("D0", "Dzero")):
164         out = out.replace(old, new)
165     parts = re.split(r"[^A-Za-z]+", out)
166     return "".join(p[:1].upper() + p[1:] for p in parts if p)
167
168
169 def _table_times_hours(t_dry: float, step_h: float = 6.0) -> np.ndarray:
170     last = math.floor(t_dry / HOUR / step_h)
171     return np.arange(1, last + 1) * step_h * HOUR
172
173
174 # -----
175 # q1: preheating stage, constant properties (appendix 2)
176 # -----
177
178
179 @stage("q1", deps=("ingest",), description="Problem 1: preheating stage (0-1800 s), constant properties + D(C)")
180 def q1(ctx: StageContext) -> dict[str, Any]:
181     spec = build_spec(ctx, "q1")
182     n = production_n(ctx)
183     opts = time_options(ctx)
184     t_out = np.arange(0.0, 1800.0 + 0.5, 1.0)
185     sol = solve(spec, n, t_out, **opts)
186     ctx.log.info("q1.solved", **sol.stats)
187     _store_solution(ctx, sol, spec, Q1_TABLE_T)
188     temp = profiles_frame(sol, "temp")
189     moist = profiles_frame(sol, "moist")
190     tables = {"temp": table_cells(temp, Q1_TABLE_T), "moist": table_cells(moist, Q1_TABLE_T)}
191     ctx.write_json("tables.json", tables)
192     checks = _checks(spec, sol)
193     ctx.write_json("verification_q1.json", checks)
194     bi = biot_numbers(APPENDIX2, C_INIT, T_INIT)
195     ctx.write_json("summary.json", {"spec": spec.describe(), "n": n, "stats": sol.stats, "biot": bi})
196     mean_c = mean_moisture(sol)
197     ctx.number("QoneCentreTempEnd", float(sol.temp[-1, 0]))
198     ctx.number("QoneSurfaceTempEnd", float(sol.temp[-1, -1]))
199     ctx.number("QoneCentreMoistEnd", float(sol.moist[-1, 0]))
200     ctx.number("QoneSurfaceMoistEnd", float(sol.moist[-1, -1]))
201     ctx.number("QoneMeanMoistEnd", float(mean_c[-1]))
202     ctx.number("QoneMoistLossPct", float(100.0 * (1.0 - mean_c[-1] / mean_c[0])), ".2f")
203     ctx.number("QoneBiotHeat", bi["Bi_heat"], ".3f")
204     ctx.number("QoneBiotMass", bi["Bi_mass"], ".3f")
205     ctx.number("QoneAlpha", bi["alpha"], ".3e")
206     ctx.number("QoneDinit", bi["D"], ".3e")
207     ctx.number("QoneAirTempEnd", float(sol.air_temp[-1]), ".3f")
208     ctx.number("QoneSurfaceTempLag", float(sol.air_temp[-1] - sol.temp[-1, -1]), ".3f")
209     ctx.number("GridNodes", n + 1)
210     ctx.number("GridDrCm", 2.0 / n, ".4f")
211     ctx.number("TimeRtol", opts["rtol"], ".0e")
212     ctx.number("TimeAtol", opts["atol"], ".0e")
213     return {"n": n, "checks_passed": checks["passed"], **{k: v for k, v in sol.stats.items() if k != "message"}}

```

```

214
215
216 # -----
217 # q2: whole process, variable properties (appendix 3), first 3 h at 1 s
218 # -----
219
220
221 @stage("q2", deps=("ingest",), description="Problem 2: whole-process model with appendix-3 properties, 0-3 h at 1 s")
222 def q2(ctx: StageContext) -> dict[str, Any]:
223     spec = build_spec(ctx, "q2")
224     n = production_n(ctx)
225     opts = time_options(ctx)
226     t_out = np.arange(0.0, 3.0 * HOUR + 0.5, 1.0)
227     sol = solve(spec, n, t_out, **opts)
228     ctx.log.info("q2.solved", **sol.stats)
229     _store_solution(ctx, sol, spec, Q2_TABLE_T)
230     temp = profiles_frame(sol, "temp")
231     moist = profiles_frame(sol, "moist")
232     tables = {"temp": table_cells(temp, Q2_TABLE_T), "moist": table_cells(moist, Q2_TABLE_T)}
233     ctx.write_json("tables.json", tables)
234     checks = _checks(spec, sol)
235     ctx.write_json("verification_q2.json", checks)
236     bi0 = biot_numbers(spec.props, C_INIT, T_INIT)
237     bi_end = biot_numbers(spec.props, float(sol.moist[-1, -1]), float(sol.temp[-1, -1]))
238     ctx.write_json(
239         "summary.json", {"spec": spec.describe(), "n": n, "stats": sol.stats, "biot_init": bi0, "biot_end": bi_end}
240     )
241     mean_c = mean_moisture(sol)
242     ctx.number("QtwoCentreTempThreeH", float(sol.temp[-1, 0]))
243     ctx.number("QtwoSurfaceTempThreeH", float(sol.temp[-1, -1]))
244     ctx.number("QtwoCentreMoistThreeH", float(sol.moist[-1, 0]))
245     ctx.number("QtwoSurfaceMoistThreeH", float(sol.moist[-1, -1]))
246     ctx.number("QtwoMeanMoistThreeH", float(mean_c[-1]))
247     ctx.number("QtwoMoistLossPct", float(100.0 * (1.0 - mean_c[-1] / mean_c[0])), ".2f")
248     ctx.number("QtwoDinit", bi0["D"], ".3e")
249     ctx.number("QtwoDsurfaceThreeH", bi_end["D"], ".3e")
250     ctx.number("QtwoBiotMassThreeH", bi_end["Bi_mass"], ".2f")
251     ctx.number("QtwoAlphaInit", bi0["alpha"], ".3e")
252     return {"n": n, "checks_passed": checks["passed"], **{k: v for k, v in sol.stats.items() if k != "message"}}
253
254
255 # -----
256 # q3: drying time (appendix 3, fixed radius)
257 # -----
258
259
260 def _drying_outputs(
261     ctx: StageContext, sol: Solution, spec: ProblemSpec, key: str, moving: bool
262 ) -> tuple[dict[str, Any], list[list[Any]]]:
263     assert sol.t_dry is not None and sol.dry_profile is not None
264     t_dry = sol.t_dry
265     rows_t = _table_times_hours(t_dry)
266     frame = physical_frame(sol, "moist") if moving else profiles_frame(sol, "moist")
267     table: list[list[Any]] = []
268     if moving:
269         lookup = {float(t): i for i, t in enumerate(sol.t)}
270         for t, row in zip(rows_t, table_cells(frame, rows_t)):
271             surface = float(frame.at[lookup[float(t)], "surface"])
272             table.append([t / HOUR, *[None if np.isnan(v) else float(v) for v in row[1:]], surface])
273         radius_dry = float(spec.radius.value(t_dry))
274         end_vals = sample_physical(sol.dry_profile, sol.xi, radius_dry, R_TABLE_CM / 100.0)
275         end_row = [t_dry / HOUR, *[None if np.isnan(v) else float(v) for v in end_vals], float(sol.dry_profile[-1])]
276     else:

```

```

277     table = [[t / HOUR, *row[1:]] for t, row in zip(rows_t, table_cells(frame, rows_t))]
278     idx = subgrid_indices(len(sol.xi) - 1)
279     prof = sol.dry_profile[idx]
280     end_row = [t_dry / HOUR, *[float(prof[round(r / 0.1)]) for r in R_TABLE_CM]]
281 table.append(end_row)
282 mean_c = mean_moisture(sol)
283 below = np.where(mean_c < C.TARGET)[0]
284 half = np.where(mean_c <= 0.5 * mean_c[0])[0]
285 summary = {
286     "t_dry_s": t_dry,
287     "t_dry_h": t_dry / HOUR,
288     "t_dry_grid_s": sol.stats.get("t_dry_grid"),
289     "t_dry_grid_h": float(sol.stats.get("t_dry_grid", 0.0)) / HOUR,
290     "t_mean_below_h": None if len(below) == 0 else float(sol.t[below[0]] / HOUR),
291     "t_half_loss_h": None if len(half) == 0 else float(sol.t[half[0]] / HOUR),
292     "surface_moist_at_dry": float(sol.dry_profile[-1]),
293     "mean_moist_at_dry": float(2.0 * (sol.dry_profile @ verify.control_volumes(sol.xi))),
294     "rows": int(len(sol.t) - 1),
295     "table_times_h": (rows_t / HOUR).tolist(),
296 }
297 ctx.write_json(f"drying_{key}.json", summary)
298 return summary, table
299
300
301 @stage("q3", deps=("ingest",), description="Problem 3: drying time until every point is below 0.15 kg/kg")
302 def q3(ctx: StageContext) -> dict[str, Any]:
303     spec = build_spec(ctx, "q3")
304     n = production_n(ctx)
305     opts = time_options(ctx)
306     sol = solve_until_dry(
307         spec, n, float(ctx.cfg("a.drying.dt_out_s", 60.0)), horizon=float(ctx.cfg("a.drying.horizon_s", 432000)), **
308         ↪ opts
309     )
310     assert sol.t_dry is not None
311     ctx.log.info("q3.solved", t_dry_h=sol.t_dry / HOUR, **sol.stats)
312     snaps = np.concatenate([_table_times_hours(sol.t_dry), [sol.t[-1]]])
313     _store_solution(ctx, sol, spec, snaps)
314     np.savez_compressed(ctx.out("dry_profile.npz"), xi=sol.xi, moist=sol.dry_profile, temp=sol.dry_temp, t=sol.t_dry)
315     summary, table = _drying_outputs(ctx, sol, spec, "q3", moving=False)
316     ctx.write_json("tables.json", {"moist": table})
317     checks = _checks(spec, sol)
318     ctx.write_json("verification_q3.json", checks)
319     ctx.write_json("summary.json", {"spec": spec.describe(), "n": n, "stats": sol.stats, **summary})
320     ctx.number("QthreeDryHours", summary["t_dry_h"], ".2f")
321     ctx.number("QthreeDryHoursFour", summary["t_dry_h"], ".4f")
322     ctx.number("QthreeDryHoursGrid", summary["t_dry_grid_h"], ".4f")
323     ctx.number("QthreeDrySeconds", summary["t_dry_s"], ".1f")
324     ctx.number("QthreeDryDays", summary["t_dry_h"] / 24.0, ".2f")
325     ctx.number("QthreeSurfaceMoistAtDry", summary["surface_moist_at_dry"])
326     ctx.number("QthreeMeanMoistAtDry", summary["mean_moist_at_dry"])
327     if summary["t_mean_below_h"] is not None:
328         ctx.number("QthreeMeanCriterionHours", summary["t_mean_below_h"], ".2f")
329     if summary["t_half_loss_h"] is not None:
330         ctx.number("QthreeHalfLossHours", summary["t_half_loss_h"], ".2f")
331     ctx.number("QthreeResultRows", summary["rows"])
332     ctx.number("QthreeAirTempPlateau", spec.chamber.temp_plateau, ".3f")
333     ctx.number("QthreeAirHumPlateau", spec.chamber.hum_plateau, ".5f")
334     return {"n": n, "t_dry_h": summary["t_dry_h"], "checks_passed": checks["passed"], "nfev": sol.stats["nfev"]}
335
336 # -----
337 # q4: shrinking radius (attachment 2), appendix-4 properties, Lagrangian main + Eulerian alternative
338 # -----

```

```

339
340
341 @stage("q4", deps=("ingest",), description="Problem 4: shrinking cylinder (attachment 2) with appendix-4 properties")
342 def q4(ctx: StageContext) -> dict[str, Any]:
343     n = production_n(ctx)
344     opts = time_options(ctx)
345     dt_out = float(ctx.cfg("a.drying.dt_out_s", 60.0))
346     horizon = float(ctx.cfg("a.drying.horizon_s", 432000))
347     main_form = str(ctx.cfg("a.scheme.formulation", "lagrangian"))
348     alt_form = "eulerian" if main_form == "lagrangian" else "lagrangian"
349     spec = build_spec(ctx, "q4", formulation=main_form)
350     sol = solve_until_dry(spec, n, dt_out, horizon=horizon, **opts)
351     assert sol.t_dry is not None and sol.dry_profile is not None
352     ctx.log.info("q4.solved", formulation=main_form, t_dry_h=sol.t_dry / HOUR, **sol.stats)
353     snaps = np.concatenate([_table_times_hours(sol.t_dry), [sol.t[-1]]])
354     _store_solution(ctx, sol, spec, snaps)
355     np.savez_compressed(
356         ctx.out("dry_profile.npz"),
357         xi=sol.xi,
358         moist=sol.dry_profile,
359         temp=sol.dry_temp,
360         t=sol.t_dry,
361         radius=float(spec.radius.value(sol.t_dry)),
362     )
363     summary, table = _drying_outputs(ctx, sol, spec, "q4", moving=True)
364     ctx.write_json("tables.json", {"moist": table})
365     checks = _checks(spec, sol)
366
367     # alternative formulation (MDR-0006) and the fixed-radius reference with the same properties
368     alt_spec = replace(spec, formulation=alt_form)
369     alt = solve_until_dry(alt_spec, n, dt_out, horizon=horizon, **opts)
370     assert alt.t_dry is not None
371     ctx.log.info("q4.alternative", formulation=alt_form, t_dry_h=alt.t_dry / HOUR, **alt.stats)
372     alt_summary, alt_table = _drying_outputs(ctx, alt, alt_spec, "q4_alt", moving=True)
373     physical_frame(alt, "moist").to_parquet(ctx.out("profiles_moist_alt.parquet"), index=False)
374     ctx.write_json("tables_alt.json", {"moist": alt_table, "formulation": alt_form})
375     alt_checks = _checks(alt_spec, alt)
376     fixed_spec = replace(spec, radius=Radius.constant(R0))
377     fixed = solve_until_dry(fixed_spec, n, dt_out, horizon=horizon, **opts)
378     assert fixed.t_dry is not None
379     fixed_summary, _ = _drying_outputs(ctx, fixed, fixed_spec, "q4_fixed", moving=False)
380
381     # degenerate check: both moving-boundary code paths with a (formally non-constant) constant radius
382     pseudo = Radius(np.array([0.0, 1.0, 2.0]), np.array([R0, R0, R0]), force_interpolant=True)
383     t_short = np.arange(0.0, 6.0 * HOUR + 0.5, 600.0)
384     ref = solve(fixed_spec, n, t_short, **opts)
385     degenerate: dict[str, Any] = {}
386     for form in ("lagrangian", "eulerian"):
387         test = solve(replace(spec, radius=pseudo, formulation=form), n, t_short, **opts)
388         degenerate[form] = {
389             "max_abs_diff_moist": float(np.max(np.abs(test.moist - ref.moist))),
390             "max_abs_diff_temp": float(np.max(np.abs(test.temp - ref.temp))),
391         }
392     degenerate["passed"] = bool(
393         all(v["max_abs_diff_moist"] < 1e-6 and v["max_abs_diff_temp"] < 1e-4 for v in degenerate.values())
394     )
395     checks["degenerate_constant_radius"] = degenerate
396     checks["passed"] = bool(checks["passed"] and degenerate["passed"])
397     checks["alternative_formulation"] = alt_checks
398     ctx.write_json("verification_q4.json", checks)
399
400     radius_dry = float(spec.radius.value(sol.t_dry)) * 100.0
401     ctx.write_json(

```

```

402     "summary.json",
403     {
404         "spec": spec.describe(),
405         "n": n,
406         "stats": sol.stats,
407         "main": summary,
408         "alternative": {"formulation": alt_form, **alt_summary},
409         "fixed_radius": fixed_summary,
410         "radius_at_dry_cm": radius_dry,
411     },
412 )
413 ctx.number("QfourDryHours", summary["t_dry_h"], ".2f")
414 ctx.number("QfourDryHoursFour", summary["t_dry_h"], ".4f")
415 ctx.number("QfourDryHoursGrid", summary["t_dry_grid_h"], ".4f")
416 ctx.number("QfourDrySeconds", summary["t_dry_s"], ".1f")
417 ctx.number("QfourDryDays", summary["t_dry_h"] / 24.0, ".2f")
418 ctx.number("QfourDryHoursAlt", alt_summary["t_dry_h"], ".2f")
419 ctx.number("QfourDryHoursFixed", fixed_summary["t_dry_h"], ".2f")
420 ctx.number("QfourAltDiffPct", 100.0 * (alt_summary["t_dry_h"] / summary["t_dry_h"] - 1.0), ".2f")
421 ctx.number("QfourFixedDiffPct", 100.0 * (fixed_summary["t_dry_h"] / summary["t_dry_h"] - 1.0), ".2f")
422 ctx.number("QfourRadiusAtDryCm", radius_dry, ".3f")
423 ctx.number("QfourSurfaceMoistAtDry", summary["surface_moist_at_dry"])
424 ctx.number("QfourMeanMoistAtDry", summary["mean_moist_at_dry"])
425 ctx.number("QfourResultRows", summary["rows"])
426 worst = max(v["max_abs_diff_moist"] for v in degenerate.values() if isinstance(v, dict))
427 ctx.number("QfourDegenerateMaxDiff", worst, ".1e")
428 return {
429     "n": n,
430     "t_dry_h": summary["t_dry_h"],
431     "t_dry_alt_h": alt_summary["t_dry_h"],
432     "t_dry_fixed_h": fixed_summary["t_dry_h"],
433     "checks_passed": checks["passed"],
434 }
435
436
437 # -----
438 # convergence: grid refinement, time tolerance, integrator cross-check
439 # -----
440
441
442 @stage(
443     "convergence",
444     deps=("ingest", "q3", "q4"),
445     description="Grid/time convergence of all four problems (observed orders, drying-time convergence)",
446 )
447 def convergence(ctx: StageContext) -> dict[str, Any]:
448     levels = int(ctx.cfg("a.grid.convergence_levels", 5))
449     refine = int(ctx.cfg("a.grid.refine", 3))
450     opts = time_options(ctx)
451     horizon = float(ctx.cfg("a.drying.horizon_s", 432000))
452     t3 = ctx.dep_data("q3", "summary.json")["t_dry_s"]
453     t4 = ctx.dep_data("q4", "summary.json")["main"]["t_dry_s"]
454     jobs: list[dict[str, Any]] = []
455     for k in range(levels):
456         n = 20 * 2**k
457         for prob, t_out in (("q1", Q1_TABLE_T), ("q2", Q2_TABLE_T)):
458             jobs.append(
459                 {
460                     "label": f"{prob}:k{k}",
461                     "recipe": spec_to_recipe(build_spec(ctx, prob)),
462                     "n": n,
463                     "opts": opts,
464                     "mode": "fixed",

```

```

465         "t_out": np.concatenate([[0.0], t_out]).tolist(),
466     }
467 )
468 for prob, t_dry in (("q3", t3), ("q4", t4)):
469     jobs.append(
470         {
471             "label": f"{prob}:k{k}",
472             "recipe": spec_to_recipe(build_spec(ctx, prob)),
473             "n": n,
474             "opts": opts,
475             "mode": "dry",
476             "horizon": horizon,
477             "snapshot_times": _table_times_hours(t_dry).tolist(),
478         }
479     )
480 # time-tolerance, integrator and maximum-step cross-checks on the production grid (problem 3, the longest run)
481 n_prod = 20 * 2**refine
482 time_cases = (
483     ("tol:strict", {"rtol": 1e-10, "atol": 1e-12}),
484     ("tol:radau", {"method": "Radau"}),
485     ("tol:loose", {"rtol": 1e-7, "atol": 1e-9}),
486     ("dt:3600", {"max_step": 3600.0}),
487     ("dt:900", {"max_step": 900.0}),
488     ("dt:300", {"max_step": 300.0}),
489 )
490 for label, extra in time_cases:
491     jobs.append(
492         {
493             "label": label,
494             "recipe": spec_to_recipe(build_spec(ctx, "q3")),
495             "n": n_prod,
496             "opts": {**opts, **extra},
497             "mode": "dry",
498             "horizon": horizon,
499             "snapshot_times": _table_times_hours(t3).tolist(),
500         }
501     )
502 results = {r["label"]: r for r in _parallel(ctx, jobs)}
503 report: dict[str, Any] = {"levels": levels, "production_refine": refine, "grid": {}, "time": {}}
504 for prob in ("q1", "q2"):
505     for field in ("temp", "moist"):
506         lv = [
507             {"k": k, "dr_cm": 0.1 / 2**k, "v": np.asarray(results[f"{prob}:k{k}"][field])[1:, :5]}
508             for k in range(levels)
509         ]
510         report["grid"][f"{prob}_{field}"] = verify.convergence_table(lv, "v")
511 for prob in ("q3", "q4"):
512     common = set.intersection(*(set(results[f"{prob}:k{k}"]["snapshot_t"]) for k in range(levels)))
513     lv = []
514     for k in range(levels):
515         res = results[f"{prob}:k{k}"]
516         keep = [i for i, t in enumerate(res["snapshot_t"]) if t in common]
517         lv.append({"k": k, "dr_cm": 0.1 / 2**k, "v": np.asarray(res["snapshot_moist"])[keep[:, :5])})
518     report["grid"][f"{prob}_moist"] = verify.convergence_table(lv, "v")
519     report["grid"][f"{prob}_moist_times_h"] = sorted(t / HOUR for t in common)
520     dry = [results[f"{prob}:k{k}"]["t_dry"] for k in range(levels)]
521     rows = [{"k": k, "dr_cm": 0.1 / 2**k, "t_dry_h": d / HOUR} for k, d in enumerate(dry)]
522     for i in range(levels - 1):
523         rows[i]["diff_to_fineest_min"] = (dry[i] - dry[-1]) / 60.0
524     report["grid"][f"{prob}_t_dry"] = rows
525 base = results[f"q3:k{refine}"]
526 for label, _extra in time_cases:
527     r = results[label]

```

```

528     common = set(r["snapshot_t"]) & set(base["snapshot_t"])
529     a = np.asarray([row for t, row in zip(r["snapshot_t"], r["snapshot_moist"]) if t in common])
530     b = np.asarray([row for t, row in zip(base["snapshot_t"], base["snapshot_moist"]) if t in common])
531     report["time"][label] = {
532         "t_dry_diff_s": float(r["t_dry"] - base["t_dry"]),
533         "max_abs_diff_moist": float(np.max(np.abs(a - b))),
534         "nfev": r["stats"]["nfev"],
535         "options": r["opts"],
536     }
537     report["production"] = {
538         "q1_temp_err": report["grid"]["q1_temp"][refine]["max_abs_error"] if refine < levels - 1 else None,
539         "q1_moist_err": report["grid"]["q1_moist"][refine]["max_abs_error"] if refine < levels - 1 else None,
540         "q2_temp_err": report["grid"]["q2_temp"][refine]["max_abs_error"] if refine < levels - 1 else None,
541         "q2_moist_err": report["grid"]["q2_moist"][refine]["max_abs_error"] if refine < levels - 1 else None,
542         "q3_moist_err": report["grid"]["q3_moist"][refine]["max_abs_error"] if refine < levels - 1 else None,
543         "q4_moist_err": report["grid"]["q4_moist"][refine]["max_abs_error"] if refine < levels - 1 else None,
544     }
545     ctx.write_json("convergence.json", report)
546     ctx.write_json("raw_t_dry.json", {k: v.get("t_dry") for k, v in results.items() if "t_dry" in v})
547     orders = {}
548     for key, rows in report["grid"].items():
549         if key.endswith(("_t_dry", "_times_h")):
550             continue
551         vals = [r["observed_order"] for r in rows if r.get("observed_order") is not None]
552         orders[key] = vals[-1] if vals else None
553         tag = _tag(key)
554         if vals:
555             ctx.number(f"ConvOrder{tag}", vals[-1], ".2f")
556         if refine < levels - 1:
557             ctx.number(f"ConvErr{tag}", rows[refine]["max_abs_error"], ".1e")
558     for prob in ("q3", "q4"):
559         rows = report["grid"][f"{prob}_t_dry"]
560         tag = "Qthree" if prob == "q3" else "Qfour"
561         ctx.number(f"ConvDryDiff{tag}Min", abs(rows[refine]["diff_to_fineest_min"]), ".2f")
562     ctx.number("TimeStrictDiffMoist", report["time"]["tol:strict"]["max_abs_diff_moist"], ".1e")
563     ctx.number("TimeRadauDiffMoist", report["time"]["tol:radau"]["max_abs_diff_moist"], ".1e")
564     ctx.number("TimeStrictDryDiffSec", abs(report["time"]["tol:strict"]["t_dry_diff_s"]), ".2f")
565     ctx.number("TimeRadauDryDiffSec", abs(report["time"]["tol:radau"]["t_dry_diff_s"]), ".2f")
566     ctx.number("TimeStepCoarseDiffMoist", report["time"]["dt:3600"]["max_abs_diff_moist"], ".1e")
567     ctx.number("TimeStepCoarseDryDiffSec", abs(report["time"]["dt:3600"]["t_dry_diff_s"]), ".2f")
568     ctx.number("TimeStepFineDiffMoist", report["time"]["dt:300"]["max_abs_diff_moist"], ".1e")
569     ctx.number("TimeStepFineDryDiffSec", abs(report["time"]["dt:300"]["t_dry_diff_s"]), ".2f")
570     return {"orders": orders, "jobs": len(jobs)}
571
572
573 # -----
574 # verification: analytic solution, independent 2-D model (cross-check and end effects)
575 # -----
576
577
578 def _analytic_case(ctx: StageContext, field: str, levels: int, opts: dict[str, Any]) -> dict[str, Any]:
579     """Constant-property Robin cylinder with constant ambient; solver vs Bessel series on several grids."""
580     t_air, c_air = 50.0, 0.05
581     d_const = float(APPENDIX2.diffusivity(np.array(C_INIT), np.array(T_INIT)))
582     props = replace(APPENDIX2, d_const=d_const)
583     chamber = Chamber.constant(t_air, c_air)
584     spec = ProblemSpec(props, chamber, Radius.constant(R0))
585     if field == "temp":
586         t_out = np.concatenate([[0.0], Q1_TABLE_T])
587         diffusivity, transfer, coeff, u0, u_inf = props.k0 / (props.rho0 * props.cp0), props.h, props.k0, T_INIT, t_air
588     else:
589         t_out = np.array([0.0, 300.0, 600.0, 1800.0, 3600.0, 7200.0, 14400.0, 28800.0, 43200.0, 86400.0])
590         diffusivity, transfer, coeff, u0, u_inf = d_const, props.hm, d_const, C_INIT, c_air

```

```

591 rows = []
592 for k in range(levels):
593     n = 20 * 2**k
594     sol = solve(spec, n, t_out, **opts)
595     num = getattr(sol, field)[1:, :: n // 20]
596     exact = verify.analytic_field(np.linspace(0, 1, 21), t_out[1:], diffusivity, transfer, coeff, u0, u_inf)
597     rows.append({"k": k, "dr_cm": 0.1 / 2**k, "max_abs_error": float(np.max(np.abs(num - exact)))})
598 for i in range(len(rows) - 1):
599     rows[i]["observed_order"] = float(np.log2(rows[i]["max_abs_error"] / rows[i + 1]["max_abs_error"]))
600 return {
601     "field": field,
602     "biot": transfer * R0 / coeff,
603     "diffusivity": diffusivity,
604     "times_s": t_out[1:].tolist(),
605     "levels": rows,
606 }
607
608
609 @stage(
610     "verification",
611     deps=("ingest", "q3"),
612     description="Analytic Bessel-series comparison and independent 2-D axisymmetric cross-check / end effects",
613 )
614 def verification(ctx: StageContext) -> dict[str, Any]:
615     opts = time_options(ctx)
616     refine = int(ctx.cfg("a.grid.refine", 3))
617     levels = min(int(ctx.cfg("a.grid.convergence_levels", 5)), refine + 2)
618     analytic = {f: _analytic_case(ctx, f, levels, opts) for f in ("temp", "moist")}
619     ctx.log.info("verification.analytic", **{f: a["levels"][refine]["max_abs_error"] for f, a in analytic.items()})
620     ctx.write_json("analytic.json", analytic)
621
622     # independent 2-D axisymmetric model, coarse grid; insulated ends must reproduce the 1-D scheme exactly
623     chamber = load_chamber(ctx)
624     props = PROPS["q3"]
625     nz = int(ctx.cfg("a.verification.axisym_nz", 25))
626     k2 = int(ctx.cfg("a.verification.axisym_refine", 0))
627     nr = 20 * 2**k2
628     t3 = float(ctx.dep_data("q3", "summary.json")["t_dry_s"])
629     t_out = np.arange(0.0, 12.0 * HOUR + 0.5, 600.0)
630     spec = ProblemSpec(props, chamber, Radius.constant(R0))
631     one_d = solve(spec, nr, t_out, **opts)
632     ins = verify.solve_axisymmetric(props, chamber, t_out, nr=nr, nz=nz, end_bc="insulated", **opts)
633     cross = {
634         "max_abs_diff_moist": float(np.max(np.abs(ins["moist"][:, :, 0] - one_d.moist))),
635         "max_abs_diff_temp": float(np.max(np.abs(ins["temp"][:, :, 0] - one_d.temp))),
636         "z_uniformity_moist": float(np.max(np.abs(ins["moist"] - ins["moist"][:, :, 1]))),
637     }
638     cross["passed"] = bool(cross["max_abs_diff_moist"] < 1e-6 and cross["max_abs_diff_temp"] < 1e-4)
639     ctx.log.info("verification.cross_check", **cross)
640
641     # end effects: Robin conditions on the end faces, whole drying period, coarse grid
642     t_end = np.arange(0.0, math.ceil(1.05 * t3 / HOUR) * HOUR + 0.5, 1800.0)
643     one_d_long = solve(spec, nr, t_end, detect_dry=True, **opts)
644     two_d = verify.solve_axisymmetric(props, chamber, t_end, nr=nr, nz=nz, end_bc="robin", detect_dry=True, **opts)
645     mid = two_d["moist"][:, :, 0]
646     end_face = two_d["moist"][:, :, -1]
647     diff_mid = np.abs(mid - one_d_long.moist)
648     end_effect = {
649         "nz": nz,
650         "dz_cm": 12.5 / nz,
651         "nr": nr,
652         "t_dry_1d_h": None if one_d_long.t_dry is None else one_d_long.t_dry / HOUR,
653         "t_dry_2d_h": two_d.get("t_dry", np.nan) / HOUR,

```

```

654     "max_abs_diff_midplane_moist": float(diff_mid.max()),
655     "max_abs_diff_midplane_temp": float(np.max(np.abs(two_d["temp"][:, :, 0] - one_d_long.temp))),
656     "centre_moist_diff_at_1d_dry": float(
657         abs(
658             np.interp(one_d_long.t_dry or t3, t_end, mid[:, 0])
659             - np.interp(one_d_long.t_dry or t3, t_end, one_d_long.moist[:, 0])
660         )
661     ),
662     "end_face_centre_moist_at_1d_dry": float(np.interp(one_d_long.t_dry or t3, t_end, end_face[:, 0])),
663     "axial_profile_at_1d_dry": {
664         "z_cm": (two_d["z"] * 100).tolist(),
665         "centre_line_moist": [
666             float(np.interp(one_d_long.t_dry or t3, t_end, two_d["moist"][:, 0, j])) for j in range(nz + 1)
667         ],
668     },
669 }
670 if one_d_long.t_dry is not None and "t_dry" in two_d:
671     end_effect["t_dry_rel_diff_pct"] = 100.0 * (two_d["t_dry"] / one_d_long.t_dry - 1.0)
672 ctx.log.info("verification.end_effect", **{k: v for k, v in end_effect.items() if not isinstance(v, dict)})
673 np.savez_compressed(
674     ctx.out("axisym_snapshots.npz"),
675     t=t_end,
676     r=two_d["r"],
677     z=two_d["z"],
678     moist_mid=mid,
679     moist_end=end_face,
680     moist_1d=one_d_long.moist,
681     centre_line=two_d["moist"][:, 0, :],
682 )
683 report = {"analytic": analytic, "cross_check_2d": cross, "end_effect_2d": end_effect}
684 report["passed"] = bool(
685     cross["passed"] and all(a["levels"][refine]["max_abs_error"] < 1e-4 for a in analytic.values())
686 )
687 ctx.write_json("verification_report.json", report)
688 ctx.number("AnalyticErrTemp", analytic["temp"]["levels"][refine]["max_abs_error"], ".1e")
689 ctx.number("AnalyticErrMoist", analytic["moist"]["levels"][refine]["max_abs_error"], ".1e")
690 ctx.number("AnalyticOrderTemp", analytic["temp"]["levels"][refine - 1]["observed_order"], ".2f")
691 ctx.number("AnalyticOrderMoist", analytic["moist"]["levels"][refine - 1]["observed_order"], ".2f")
692 ctx.number("AnalyticBiotHeat", analytic["temp"]["biot"], ".3f")
693 ctx.number("AnalyticBiotMass", analytic["moist"]["biot"], ".3f")
694 ctx.number("CrossCheckDiffMoist", cross["max_abs_diff_moist"], ".1e")
695 ctx.number("CrossCheckDiffTemp", cross["max_abs_diff_temp"], ".1e")
696 ctx.number("EndEffectMidplaneDiff", end_effect["max_abs_diff_midplane_moist"], ".1e")
697 ctx.number("EndEffectCentreDiffAtDry", end_effect["centre_moist_diff_at_1d_dry"], ".1e")
698 if "t_dry_rel_diff_pct" in end_effect:
699     ctx.number("EndEffectDryDiffPct", end_effect["t_dry_rel_diff_pct"], ".3f")
700     ctx.number("EndEffectDryHoursTwoD", end_effect["t_dry_2d_h"], ".2f")
701     ctx.number("EndEffectDryHoursOneD", end_effect["t_dry_1d_h"], ".2f")
702 ctx.number("EndEffectDzCm", end_effect["dz_cm"], ".2f")
703 return {
704     "passed": report["passed"],
705     "cross_check": cross["passed"],
706     "end_effect_pct": end_effect.get("t_dry_rel_diff_pct"),
707 }
708
709
710 # -----
711 # sensitivity: parameter elasticities of the drying time and alternative interpretations
712 # -----
713
714
715 @stage(
716     "sensitivity",

```

```

717     deps=("ingest", "q3", "q4"),
718     description="Drying-time elasticities (h, h_m, D0, C_air, T plateau, shrinkage) and alternative interpretations",
719 )
720 def sensitivity(ctx: StageContext) -> dict[str, Any]:
721     factors = [float(f) for f in ctx.cfg("a.sensitivity.factors", [0.8, 0.9, 1.1, 1.2])]
722     n = production_n(ctx)
723     opts = time_options(ctx)
724     horizon = float(ctx.cfg("a.drying.horizon_s", 432000))
725     base3 = build_spec(ctx, "q3")
726     base4 = build_spec(ctx, "q4")
727     t3 = float(ctx.dep_data("q3", "summary.json")["t_dry_s"])
728     t4 = float(ctx.dep_data("q4", "summary.json")["main"]["t_dry_s"])
729
730     def job(label: str, spec: ProblemSpec) -> dict[str, Any]:
731         return {"label": label, "recipe": spec_to_recipe(spec), "n": n, "opts": opts, "mode": "dry", "horizon": horizon
732             ↪ }
733
734     jobs: list[dict[str, Any]] = []
735     params = {
736         "h": lambda f: replace(base3, props=base3.props.scaled(h=f)),
737         "hm": lambda f: replace(base3, props=base3.props.scaled(hm=f)),
738         "D0": lambda f: replace(base3, props=base3.props.scaled(d0=f)),
739         "Cair": lambda f: replace(base3, chamber=replace(base3.chamber, hum_scale=f)),
740         "Tplateau": lambda f: replace(base3, chamber=replace(base3.chamber, temp_scale=f)),
741         "shrink": lambda f: replace(base4, radius=load_radius(ctx, f)),
742         "D0q4": lambda f: replace(base4, props=base4.props.scaled(d0=f)),
743     }
744     for name, make in params.items():
745         for f in factors:
746             jobs.append(job(f"{name}:{f}", make(f)))
747     alternatives = {
748         "plateau_last_sample": replace(base3, chamber=load_chamber(ctx, "last_sample")),
749         "plateau_nominal": replace(base3, chamber=load_chamber(ctx, "nominal")),
750         "face_arithmetic": replace(base3, face_scheme="arithmetic"),
751         "face_harmonic": replace(base3, face_scheme="harmonic"),
752     }
753     for name, spec in alternatives.items():
754         jobs.append(job(f"alt:{name}", spec))
755     results = {r["label"]: r for r in _parallel(ctx, jobs)}
756
757     table: list[dict[str, Any]] = []
758     elasticities: dict[str, float] = {}
759     for name in params:
760         base_t = t4 if name in ("shrink", "D0q4") else t3
761         row: dict[str, Any] = {"parameter": name, "base_h": base_t / HOUR}
762         for f in factors:
763             row[f"x{f}"] = results[f"{name}:{f}"]["t_dry"] / HOUR
764         if 0.9 in factors and 1.1 in factors:
765             e = ((results[f"{name}:1.1"]["t_dry"] - results[f"{name}:0.9"]["t_dry"]) / base_t) / 0.2
766             row["elasticity"] = e
767             elasticities[name] = e
768         table.append(row)
769     alt_rows = [
770         {
771             "case": name,
772             "t_dry_h": results[f"alt:{name}"]["t_dry"] / HOUR,
773             "diff_pct": 100.0 * (results[f"alt:{name}"]["t_dry"] / t3 - 1.0),
774         }
775         for name in alternatives
776     ]
777     q4s = ctx.dep_data("q4", "summary.json")
778     q3s = ctx.dep_data("q3", "summary.json")
779     alt_rows.append(

```

```

779         {
780             "case": "q4_" + q4s["alternative"]["formulation"],
781             "t_dry_h": q4s["alternative"]["t_dry_h"],
782             "diff_pct": 100.0 * (q4s["alternative"]["t_dry_h"] / q4s["main"]["t_dry_h"] - 1.0),
783         }
784     )
785     alt_rows.append(
786         {
787             "case": "q4_fixed_radius",
788             "t_dry_h": q4s["fixed_radius"]["t_dry_h"],
789             "diff_pct": 100.0 * (q4s["fixed_radius"]["t_dry_h"] / q4s["main"]["t_dry_h"] - 1.0),
790         }
791     )
792     if q3s.get("t_mean_below_h") is not None:
793         alt_rows.append(
794             {
795                 "case": "q3_mean_criterion",
796                 "t_dry_h": q3s["t_mean_below_h"],
797                 "diff_pct": 100.0 * (q3s["t_mean_below_h"] / q3s["t_dry_h"] - 1.0),
798             }
799         )
800     ctx.write_json(
801         "sensitivity.json", {"factors": factors, "table": table, "elasticities": elasticities, "alternatives": alt_rows
802         ↵ }
803     )
804     for name, e in elasticities.items():
805         ctx.number("Elas" + _tag(name), e, ".3f")
806     for row in alt_rows:
807         key = "Alt" + _tag(row["case"])
808         ctx.number(key + "Hours", row["t_dry_h"], ".2f")
809         ctx.number(key + "Pct", row["diff_pct"], ".2f")
810     return {"jobs": len(jobs), "elasticities": elasticities}

```

pipelines/a/solver.py

```

1  """Radial finite-volume solver for the coupled heat/moisture equations on a (possibly shrinking) cylinder.
2
3  Discretisation (MDR-0004): vertex-centred finite volumes on the Landau coordinate  $\xi = r/R(t)$  in  $[0, 1]$ ,
4  nodes  $\xi_i = i/n$ , control volumes  $[\xi_{i-1/2}, \xi_{i+1/2}]$  with the radial measure  $(\xi^2 \text{ differences})/2$ ,
5  Robin conditions applied as face fluxes on the outer half-cell, symmetry at the axis. Time integration
6  by scipy's adaptive BDF (or Radau) with a block-tridiagonal Jacobian sparsity pattern; every requested
7  output instant is sampled from the integrator's dense interpolant.
8
9  Two formulations for a moving boundary (MDR-0006):
10 * ``lagrangian`` - material coordinate, dry matter conserved, no convective term,  $1/R(t)^2$  scaling;
11 * ``eulerian`` - laboratory-frame PDE on  $0 < r < R(t)$ ; the transformation adds  $+\xi (R\dot{r}/R) d/d\xi$ .
12 For a constant radius both coincide with the fixed-domain scheme.
13 """
14
15 from __future__ import annotations
16
17 import math
18 from dataclasses import dataclass, field
19 from typing import Any
20
21 import numpy as np
22 from scipy import sparse
23 from scipy.integrate import solve_ivp
24
25 from pipelines.a.physics import C_INIT, C_TARGET, T_INIT, Chamber, FluxCap, Properties, Radius
26

```

```

27 FACE_SCHEMES = ("midpoint", "arithmetic", "harmonic")
28 FORMULATIONS = ("lagrangian", "eulerian")
29
30
31 @dataclass(frozen=True)
32 class Grid:
33     n: int
34     xi: np.ndarray # nodes, n+1
35     dxi: float
36     xi_face: np.ndarray # interior faces xi_{i+1/2}, n values
37     vol: np.ndarray # control-volume measure (xi_{i+1/2}^2 - xi_{i-1/2}^2)/2, n+1 values
38
39
40 def make_grid(n: int) -> Grid:
41     xi = np.linspace(0.0, 1.0, n + 1)
42     faces = 0.5 * (xi[:-1] + xi[1:])
43     lower = np.concatenate([[0.0], faces])
44     upper = np.concatenate([faces, [1.0]])
45     return Grid(n, xi, 1.0 / n, faces, 0.5 * (upper**2 - lower**2))
46
47
48 @dataclass(frozen=True)
49 class ProblemSpec:
50     """Everything that defines one initial-boundary-value problem."""
51
52     props: Properties
53     chamber: Chamber
54     radius: Radius
55     formulation: str = "lagrangian"
56     face_scheme: str = "midpoint"
57     t_init: float = T_INIT
58     c_init: float = C_INIT
59     flux_cap: FluxCap | None = None # energy-limited evaporation (model-evaluation extension, MDR-0008)
60
61     def describe(self) -> dict[str, Any]:
62         return {
63             "properties": self.props.describe(),
64             "chamber": self.chamber.describe(),
65             "radius": self.radius.describe(),
66             "formulation": self.formulation,
67             "face_scheme": self.face_scheme,
68             "t_init": self.t_init,
69             "c_init": self.c_init,
70             "flux_cap": None if self.flux_cap is None else self.flux_cap.describe(),
71         }
72
73     def surface_moisture_loss(self, t: Any, c_surface: Any, c_air: Any) -> Any:
74         """Boundary loss law hm (C_s - C_air) in (kg/kg) m/s, bounded by the energy cap when one is set."""
75         loss = self.props.hm * (np.asarray(c_surface, dtype=float) - np.asarray(c_air, dtype=float))
76         if self.flux_cap is None:
77             return loss
78         return np.minimum(loss, np.asarray(self.flux_cap.value(t), dtype=float) / self.flux_cap.rho_s)
79
80
81 @dataclass
82 class Solution:
83     t: np.ndarray # output instants, s
84     xi: np.ndarray # nodes
85     temp: np.ndarray # (len(t), n+1) degC
86     moist: np.ndarray # (len(t), n+1) kg/kg
87     radius: np.ndarray # R(t) at output instants, m
88     air_temp: np.ndarray
89     air_hum: np.ndarray

```

```

90 stats: dict[str, Any] = field(default_factory=dict)
91 cum_moist_loss: np.ndarray | None = None # integral of hm (C_s - C_air)/R dt, integrated as an ODE state
92 cum_heat_loss: np.ndarray | None = None # integral of h (T_s - T_air)/R dt
93 t_dry: float | None = None # exact instant at which max_i C_i first drops below C_TARGET
94 dry_profile: np.ndarray | None = None # moisture profile at t_dry
95 dry_temp: np.ndarray | None = None
96
97 @property
98 def r(self) -> np.ndarray:
99     """Physical node positions (len(t), n+1) in m."""
100     return np.outer(self.radius, self.xi)
101
102 def surface_moisture_flux(self, props: Properties) -> np.ndarray:
103     """h_m (C_s - C_air) at the output instants, (kg/kg) m/s."""
104     return props.hm * (self.moist[:, -1] - self.air_hum)
105
106 def surface_heat_flux(self, props: Properties) -> np.ndarray:
107     return props.h * (self.temp[:, -1] - self.air_temp)
108
109
110 class RadialSystem:
111     """Right-hand side f(t, y) of the semi-discrete system, y = [T_0..T_n, C_0..C_n, Q_m, Q_h].
112
113     The two trailing states accumulate the boundary losses hm (C_s - C_air)/R and h (T_s - T_air)/R so that
114     the discrete balances sum_i vol_i C_i + Q_m = const (and its thermal analogue for constant properties)
115     are linear invariants of the ODE system, which BDF/Radau preserve to integrator accuracy.
116     """
117
118     def __init__(self, spec: ProblemSpec, grid: Grid) -> None:
119         if spec.face_scheme not in FACE_SCHEMES:
120             raise ValueError(f"face_scheme must be one of {FACE_SCHEMES}")
121         if spec.formulation not in FORMULATIONS:
122             raise ValueError(f"formulation must be one of {FORMULATIONS}")
123         self.spec = spec
124         self.grid = grid
125         self.n1 = grid.n + 1
126         self.convective = spec.formulation == "eulerian" and not spec.radius.is_constant
127
128     def face_coefficients(self, temp: np.ndarray, moist: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
129         props = self.spec.props
130         scheme = self.spec.face_scheme
131         if scheme == "midpoint":
132             cf = 0.5 * (moist[:-1] + moist[1:])
133             tf = 0.5 * (temp[:-1] + temp[1:])
134             return props.diffusivity(cf, tf), props.k(cf)
135         dn = props.diffusivity(moist, temp)
136         kn = props.k(moist)
137         if scheme == "arithmetic":
138             return 0.5 * (dn[:-1] + dn[1:]), 0.5 * (kn[:-1] + kn[1:])
139         return 2.0 * dn[:-1] * dn[1:] / (dn[:-1] + dn[1:]), 2.0 * kn[:-1] * kn[1:] / (kn[:-1] + kn[1:])
140
141     def __call__(self, t: float, y: np.ndarray) -> np.ndarray:
142         g = self.grid
143         n1 = self.n1
144         temp = y[:n1]
145         moist = y[n1 : 2 * n1]
146         props = self.spec.props
147         radius = float(self.spec.radius.value(t))
148         r2 = radius * radius
149         t_air = float(self.spec.chamber.air_temperature(t))
150         c_air = float(self.spec.chamber.air_humidity(t))
151         d_face, k_face = self.face_coefficients(temp, moist)
152         flux_c = g.xi_face * d_face * (moist[1:] - moist[:-1]) / g.dxi # xi D dC/dxi at interior faces

```

```

153     flux_t = g.xi_face * k_face * (temp[1:] - temp[:-1]) / g.dxi
154     div_c = np.empty(n1)
155     div_t = np.empty(n1)
156     div_c[:-1] = flux_c
157     div_t[:-1] = flux_t
158     loss_c = float(self.spec.surface_moisture_loss(t, moist[-1], c_air)) # hm (C_s - C_air), capped if set
159     div_c[-1] = -radius * loss_c # outer face: xi D dC/dxi = R (-hm (C_s - C_air))
160     div_t[-1] = -radius * props.h * (temp[-1] - t_air)
161     div_c[1:] -= flux_c
162     div_t[1:] -= flux_t
163     dc = div_c / (r2 * g.vol)
164     dt = div_t / (r2 * g.vol * props.rho(moist) * props.cp(moist))
165     if self.convective:
166         rate = float(self.spec.radius.rate(t)) / radius
167         grad_c = np.empty(n1)
168         grad_t = np.empty(n1)
169         grad_c[0] = 0.0
170         grad_t[0] = 0.0
171         grad_c[1:-1] = (moist[2:] - moist[:-2]) / (2.0 * g.dxi)
172         grad_t[1:-1] = (temp[2:] - temp[:-2]) / (2.0 * g.dxi)
173         grad_c[-1] = (3.0 * moist[-1] - 4.0 * moist[-2] + moist[-3]) / (2.0 * g.dxi)
174         grad_t[-1] = (3.0 * temp[-1] - 4.0 * temp[-2] + temp[-3]) / (2.0 * g.dxi)
175         dc = dc + g.xi * rate * grad_c
176         dt = dt + g.xi * rate * grad_t
177     losses = [loss_c / radius, props.h * (temp[-1] - t_air) / radius]
178     return np.concatenate([dt, dc, losses])
179
180 def jacobian_sparsity(self) -> sparse.csr_matrix:
181     n1 = self.n1
182     rows: list[int] = []
183     cols: list[int] = []
184     for i in range(n1):
185         neighbours = [j for j in (i - 1, i, i + 1) if 0 <= j < n1]
186         if self.convective and i == n1 - 1:
187             neighbours.append(n1 - 3)
188         for j in neighbours:
189             for row_off in (0, n1):
190                 for col_off in (0, n1):
191                     rows.append(i + row_off)
192                     cols.append(j + col_off)
193     rows += [2 * n1, 2 * n1 + 1]
194     cols += [2 * n1 - 1, n1 - 1] # Q_m depends on C_n, Q_h on T_n
195     data = np.ones(len(rows))
196     return sparse.csr_matrix((data, (rows, cols)), shape=(2 * n1 + 2, 2 * n1 + 2))
197
198
199 def _dry_event(n1: int, target: float) -> Any:
200     def event(t: float, y: np.ndarray) -> float:
201         return float(np.max(y[n1 : 2 * n1]) - target)
202
203     event.terminal = False # type: ignore[attr-defined]
204     event.direction = -1 # type: ignore[attr-defined]
205     return event
206
207
208 def integrate_pieewise(
209     fun: Any,
210     y0: np.ndarray,
211     t_out: np.ndarray,
212     breakpoints: np.ndarray,
213     *,
214     method: str,
215     rtol: float,

```

```

216     atol: float,
217     jac_sparsity: Any,
218     event: Any = None,
219     max_step: float = np.inf,
220 ) -> tuple[np.ndarray, dict[str, Any], float | None, np.ndarray | None]:
221     """Integrate y' = fun(t, y) from t = 0 to t_out[-1], restarting the integrator at every breakpoint.
222
223     The chamber history is piecewise linear (slope jumps every 60 s) and the radius is a PCHIP spline
224     (curvature jumps every 1800 s); a multistep method that steps across such kinks loses accuracy in a
225     way that is not controlled by the tolerance. Restarting at the kinks keeps the forcing smooth inside
226     every segment. Returns the states at ``t_out`` (which must start at 0), integrator statistics, and
227     the first root of ``event`` with the state there (if an event is given and fires).
228     """
229     t_out = np.asarray(t_out, dtype=float)
230     t_end = float(t_out[-1])
231     stops = np.unique(np.concatenate([np.asarray(breakpoints, dtype=float), [t_end]]))
232     stops = stops[(stops > 0.0) & (stops <= t_end)]
233     y = np.asarray(y0, dtype=float)
234     states = np.empty((len(t_out), len(y)))
235     states[0] = y
236     filled = 1
237     t_prev = 0.0
238     stats: dict[str, Any] = {"nfev": 0, "njev": 0, "nlu": 0, "segments": len(stops)}
239     event_t: float | None = None
240     event_y: np.ndarray | None = None
241     for t_next in stops:
242         pts = t_out[(t_out > t_prev) & (t_out <= t_next)]
243         t_eval = pts if len(pts) and pts[-1] == t_next else np.concatenate([pts, [t_next]])
244         watching = event is not None and event_t is None
245         result = solve_ivp(
246             fun,
247             (t_prev, float(t_next)),
248             y,
249             method=method,
250             t_eval=t_eval,
251             rtol=rtol,
252             atol=atol,
253             jac_sparsity=jac_sparsity,
254             dense_output=watching,
255             events=[event] if watching else None,
256             max_step=max_step,
257         )
258         if not result.success:
259             raise RuntimeError(f"solve_ivp failed on [{t_prev}, {t_next}]: {result.message}")
260         k = len(pts)
261         states[filled : filled + k] = result.y[:, :k].T
262         filled += k
263         y = result.y[:, -1]
264         for key in ("nfev", "njev", "nlu"):
265             stats[key] += int(getattr(result, key))
266         if watching and result.t_events and len(result.t_events[0]):
267             event_t = float(result.t_events[0][0])
268             event_y = np.asarray(result.sol(event_t), dtype=float)
269         t_prev = float(t_next)
270     if filled != len(t_out):
271         raise RuntimeError("output instants were not all visited (t_out must be strictly increasing from 0)")
272     return states, stats, event_t, event_y
273
274
275 def solve(
276     spec: ProblemSpec,
277     n: int,
278     t_out: np.ndarray,

```

```

279     *,
280     method: str = "BDF",
281     rtol: float = 1e-7,
282     atol: float = 1e-9,
283     detect_dry: bool = False,
284     target: float = C_TARGET,
285     max_step: float = np.inf,
286 ) -> Solution:
287     """Integrate the semi-discrete system and sample the dense solution at ``t_out`` (must start at 0)."""
288     grid = make_grid(n)
289     system = RadialSystem(spec, grid)
290     t_out = np.asarray(t_out, dtype=float)
291     if t_out[0] != 0.0 or np.any(np.diff(t_out) <= 0):
292         raise ValueError("t_out must start at 0 and be strictly increasing")
293     y0 = np.concatenate([np.full(grid.n + 1, spec.t_init), np.full(grid.n + 1, spec.c_init), [0.0, 0.0]])
294     breakpoints = np.concatenate([spec.chamber.knots(), spec.radius.knots()])
295     states, stats, t_dry, state = integrate_piecewise(
296         system,
297         y0,
298         t_out,
299         breakpoints,
300         method=method,
301         rtol=rtol,
302         atol=atol,
303         jac_sparsity=system.jacobian_sparsity(),
304         event=_dry_event(grid.n + 1, target) if detect_dry else None,
305         max_step=max_step,
306     )
307     n1 = grid.n + 1
308     radius = np.asarray(spec.radius.value(t_out), dtype=float)
309     sol = Solution(
310         t=t_out,
311         xi=grid.xi,
312         temp=states[:, :n1].copy(),
313         moist=states[:, n1 : 2 * n1].copy(),
314         radius=radius,
315         air_temp=np.asarray(spec.chamber.air_temperature(t_out), dtype=float),
316         air_hum=np.asarray(spec.chamber.air_humidity(t_out), dtype=float),
317         stats={"method": method, "rtol": rtol, "atol": atol, "n_intervals": n, **stats},
318         cum_moist_loss=states[:, 2 * n1].copy(),
319         cum_heat_loss=states[:, 2 * n1 + 1].copy(),
320     )
321     if t_dry is not None and state is not None:
322         sol.t_dry = t_dry
323         sol.dry_temp = state[:n1].copy()
324         sol.dry_profile = state[n1 : 2 * n1].copy()
325     return sol
326
327
328 def solve_until_dry(
329     spec: ProblemSpec,
330     n: int,
331     dt_out: float = 60.0,
332     *,
333     horizon: float = 5 * 86400.0,
334     max_horizon: float = 30 * 86400.0,
335     **kwargs: Any,
336 ) -> Solution:
337     """Integrate until max_i C_i < C_TARGET, keep output rows up to the first ``dt_out`` multiple after it."""
338     while True:
339         t_out = np.arange(0.0, horizon + 0.5 * dt_out, dt_out)
340         sol = solve(spec, n, t_out, detect_dry=True, **kwargs)
341         if sol.t_dry is not None:

```

```

342         break
343     horizon *= 2.0
344     if horizon > max_horizon:
345         raise RuntimeError("drying criterion not met within the maximum horizon")
346     last = math.ceil(sol.t_dry / dt_out - 1e-9)
347     keep = slice(0, last + 1)
348     sol.t = sol.t[keep]
349     sol.temp = sol.temp[keep]
350     sol.moist = sol.moist[keep]
351     sol.radius = sol.radius[keep]
352     sol.air_temp = sol.air_temp[keep]
353     sol.air_hum = sol.air_hum[keep]
354     if sol.cum_moist_loss is not None and sol.cum_heat_loss is not None:
355         sol.cum_moist_loss = sol.cum_moist_loss[keep]
356         sol.cum_heat_loss = sol.cum_heat_loss[keep]
357     sol.stats["t_dry_grid"] = float(last * dt_out)
358     return sol
359
360
361 def subgrid_indices(n: int, n_out: int = 20) -> np.ndarray:
362     """Indices of the nodes coinciding with the coarse output grid (n must be an integer multiple of n_out)."""
363     if n % n_out:
364         raise ValueError(f"n={n} is not a multiple of the output grid {n_out}")
365     return np.arange(0, n + 1, n // n_out)
366
367
368 def sample_physical(profile: np.ndarray, xi: np.ndarray, radius: float, r_targets: np.ndarray) -> np.ndarray:
369     """Cubic-spline sample of a nodal profile at physical radii; NaN where r > R(t)."""
370     from scipy.interpolate import CubicSpline
371
372     spline = CubicSpline(xi, profile, bc_type=((1, 0.0), "not-a-knot"))
373     xi_t = np.asarray(r_targets, dtype=float) / radius
374     out = np.full(xi_t.shape, np.nan)
375     inside = xi_t <= 1.0 + 1e-12
376     out[inside] = spline(np.minimum(xi_t[inside], 1.0))
377     return out

```

pipelines/a/stages.py

```

1  """Problem A stages: validate (here) plus the science and deliverable stages of the submodules.
2
3  Conventions (see docs/ENGINEERING_STANDARD.md):
4  * every stage is ``@stage(name, deps=(...))`` and returns a JSON-serialisable metrics dict;
5  * outputs go only to ``ctx.out(...)``; paper numbers via ``ctx.number("Key", value)``;
6  * ``results`` writes result*.xlsx with ``forge.xlsx.write_result`` from the organisers' templates;
7  * ``figures`` writes figures/*.pdf (+png preview) and ``tables`` writes tables/*.tex.
8
9  Stage map: q1, q2, q3, q4, convergence, verification, sensitivity (pipelines/a/science.py); derived (derived.py);
10 results, tables (pipelines/a/outputs.py); figures (pipelines/a/figures.py).
11 """
12
13 from __future__ import annotations
14
15 from typing import Any
16
17 from forge.context import StageContext
18 from forge.runner import stage
19 from pipelines.a import derived, extension, figures, outputs, science # noqa: F401 - registers the stages on import
20 from pipelines.a.contracts import INPUT_CONTRACTS
21 from pipelines.common.validation import run_validation
22

```

```

23
24 @stage(
25     "validate", deps=("ingest",), description="Validate the drying-chamber and radius series against their contracts"
26 )
27 def validate(ctx: StageContext) -> dict[str, Any]:
28     return run_validation(ctx, INPUT_CONTRACTS)

```

pipelines/a/verify.py

```

1  """Independent checks of the drying solutions (MDR-0007). Nothing here calls the radial solver's internals.
2
3  * analytic Bessel-series solution of the constant-property Robin cylinder;
4  * discrete balances (moisture, energy) recomputed from the solution arrays only;
5  * extremum-principle, monotonicity and drying-criterion checks;
6  * grid-convergence bookkeeping (errors against the finest grid, observed orders);
7  * an independently written two-dimensional axisymmetric finite-volume model (physical coordinates)
8    used both as a code cross-check (insulated ends) and to quantify end effects (Robin ends).
9  """
10
11 from __future__ import annotations
12
13 from itertools import pairwise
14 from typing import Any
15
16 import numpy as np
17 from scipy import sparse
18 from scipy.optimize import brentq
19 from scipy.special import j0, j1, jn_zeros
20
21 from pipelines.a.physics import C_TARGET, LENGTH, R0, Chamber, Properties
22
23 # -----
24 # 1. Analytic solution: infinite cylinder, uniform initial value, Robin boundary, constant ambient
25 # -----
26
27
28 def robin_eigenvalues(bi: float, count: int) -> np.ndarray:
29     """Roots of  $\lambda J_1(\lambda) = Bi J_0(\lambda)$ ; the  $n$ -th root lies between consecutive zeros of  $J_0$ ."""
30     zeros = np.concatenate([[0.0], jn_zeros(0, count)])
31     roots = []
32     for lo, hi in pairwise(zeros):
33         f = lambda x: x * j1(x) - bi * j0(x) # noqa: E731
34         roots.append(brentq(f, lo + 1e-12, hi - 1e-12, xtol=1e-14, rtol=1e-14, maxiter=500))
35     return np.asarray(roots)
36
37
38 def analytic_robin_cylinder(xi: np.ndarray, fourier: np.ndarray, bi: float, n_terms: int = 400) -> np.ndarray:
39     """Dimensionless excess  $\theta(x_i, Fo) = (u - u_{\infty}) / (u_0 - u_{\infty})$ ; returns shape  $(\text{len}(\text{fourier}), \text{len}(xi))$ ."""
40     lam = robin_eigenvalues(bi, n_terms)
41     coeff = 2.0 * bi / ((lam**2 + bi**2) * j0(lam))
42     basis = j0(np.outer(lam, xi)) # (n_terms, nxi)
43     decay = np.exp(-np.outer(np.asarray(fourier, dtype=float), lam**2)) # (nt, n_terms)
44     return (decay * coeff) @ basis
45
46
47 def analytic_field(
48     xi: np.ndarray, t: np.ndarray, diffusivity: float, transfer: float, coeff: float, u0: float, u_inf: float
49 ) -> np.ndarray:
50     """Physical field  $u(x_i, t)$  for a constant-property cylinder of radius  $R_0$  with Robin coefficient ``transfer``."""
51     bi = transfer * R0 / coeff
52     fo = diffusivity * np.asarray(t, dtype=float) / R0**2

```

```

53     theta = analytic_robin_cylinder(xi, fo, bi)
54     theta[np.asarray(t) == 0.0] = 1.0 # the series converges slowly at Fo = 0; the initial value is exact
55     return u_inf + (u0 - u_inf) * theta
56
57
58 # -----
59 # 2. Balances and qualitative properties computed from output arrays
60 # -----
61
62
63 def control_volumes(xi: np.ndarray) -> np.ndarray:
64     faces = 0.5 * (xi[:-1] + xi[1:])
65     lower = np.concatenate([[0.0], faces])
66     upper = np.concatenate([faces, [1.0]])
67     return 0.5 * (upper**2 - lower**2)
68
69
70 def _balance(lhs: np.ndarray, cumulative: np.ndarray) -> dict[str, float]:
71     residual = lhs - cumulative
72     scale = max(abs(lhs[-1]), 1e-300)
73     return {
74         "max_abs_residual": float(np.max(np.abs(residual))),
75         "final_relative_residual": float(abs(residual[-1]) / scale),
76         "max_relative_residual": float(np.max(np.abs(residual)) / scale),
77     }
78
79
80 def moisture_balance(
81     t: np.ndarray,
82     xi: np.ndarray,
83     moist: np.ndarray,
84     radius: np.ndarray,
85     air_hum: np.ndarray,
86     hm: float,
87     cum_loss: np.ndarray | None = None,
88     flux: np.ndarray | None = None,
89 ) -> dict[str, Any]:
90     """Discrete water inventory W = sum_i vol_i C_i versus the time-integrated surface flux.
91
92     Fixed radius: d/dt (R^2 W) = -R hm (C_s - C_air); shrinking (Lagrangian): d/dt W = -hm (C_s - C_air)/R(t).
93     ``sampled``: flux from the output samples (trapezoid rule, limited by the output spacing);
94     ``exact``: flux accumulated inside the integrator (``cum_loss`` = int hm (C_s - C_air)/R dt), which turns the
95     balance into a linear invariant of the ODE system and therefore tests the scheme itself.
96     ``flux`` overrides the boundary loss law hm (C_s - C_air) (e.g. the energy-capped law of MDR-0008).
97
98     """
99     vol = control_volumes(xi)
100     inventory = moist @ vol
101     flux = hm * (moist[:, -1] - air_hum) if flux is None else np.asarray(flux, dtype=float)
102     if np.allclose(radius, radius[0]):
103         lhs = radius[0] ** 2 * (inventory - inventory[0])
104         integrand = -radius[0] * flux
105         exact_scale = radius[0] ** 2
106     else:
107         lhs = inventory - inventory[0]
108         integrand = -flux / radius
109         exact_scale = 1.0
110     cumulative = np.concatenate([[0.0], np.cumsum(0.5 * (integrand[1:] + integrand[:-1]) * np.diff(t))])
111     report: dict[str, Any] = {
112         "inventory_initial": float(inventory[0]),
113         "inventory_final": float(inventory[-1]),
114         "loss_fraction": float(1.0 - inventory[-1] / inventory[0]),
115         "sampled": _balance(lhs, cumulative),
116     }

```

```

116     if cum_loss is not None:
117         report["exact"] = _balance(lhs, -exact_scale * cum_loss)
118     best = report.get("exact", report["sampled"])
119     report["max_relative_residual"] = best["max_relative_residual"]
120     report["final_relative_residual"] = best["final_relative_residual"]
121     return report
122
123
124 def energy_balance(
125     t: np.ndarray,
126     xi: np.ndarray,
127     temp: np.ndarray,
128     radius: float,
129     air_temp: np.ndarray,
130     props: Properties,
131     cum_loss: np.ndarray | None = None,
132 ) -> dict[str, Any]:
133     """Constant-property energy balance:  $\rho \text{ cp } R^2 \frac{d}{dt} \sum \text{vol}_i T_i = -R h (T_s - T_{\text{air}})$ ."""
134     vol = control_volumes(xi)
135     rho_cp = float(props.rho(np.array(0.0)) * props.cp(np.array(0.0)))
136     inventory = rho_cp * radius**2 * (temp @ vol)
137     integrand = -radius * props.h * (temp[:, -1] - air_temp)
138     cumulative = np.concatenate([[0.0], np.cumsum(0.5 * (integrand[1:] + integrand[:-1]) * np.diff(t))])
139     report: dict[str, Any] = {"sampled": _balance(inventory - inventory[0], cumulative)}
140     if cum_loss is not None: #  $\rho \text{ cp } R^2 \frac{dW}{dt} = -R h (T_s - T_{\text{air}})$  and  $Q_h = \int h (T_s - T_{\text{air}}) / R dt$ 
141         report["exact"] = _balance(inventory - inventory[0], -(radius**2) * cum_loss)
142     best = report.get("exact", report["sampled"])
143     report["max_relative_residual"] = best["max_relative_residual"]
144     report["final_relative_residual"] = best["final_relative_residual"]
145     return report
146
147
148 def extremum_checks(
149     temp: np.ndarray, moist: np.ndarray, air_temp: np.ndarray, air_hum: np.ndarray, t0: float, c0: float
150 ) -> dict[str, Any]:
151     t_lo, t_hi = min(t0, float(air_temp.min())), max(t0, float(air_temp.max()))
152     c_lo, c_hi = min(c0, float(air_hum.min())), max(c0, float(air_hum.max()))
153     tol = 1e-9
154     dc = np.diff(moist, axis=1) # along r: should be  $\leq 0$  (drying from the surface)
155     return {
156         "temp_min": float(temp.min()),
157         "temp_max": float(temp.max()),
158         "temp_bounds": [t_lo, t_hi],
159         "temp_within_bounds": bool(temp.min()  $\geq$  t_lo - tol and temp.max()  $\leq$  t_hi + tol),
160         "moist_min": float(moist.min()),
161         "moist_max": float(moist.max()),
162         "moist_bounds": [c_lo, c_hi],
163         "moist_within_bounds": bool(moist.min()  $\geq$  c_lo - tol and moist.max()  $\leq$  c_hi + tol),
164         "moist_monotone_in_r": bool(np.all(dc  $\leq$  1e-8)),
165         "moist_max_positive_slope": float(max(dc.max(), 0.0)),
166         "moist_argmax_is_centre": bool(np.all(moist.max(axis=1) - moist[:, 0]  $\leq$  1e-9)), # ties up to roundoff
167     }
168
169
170 def drying_checks(
171     t: np.ndarray, moist: np.ndarray, t_dry: float, dry_profile: np.ndarray, target: float = C_TARGET
172 ) -> dict[str, Any]:
173     """The exact drying instant must separate 'not yet dry' rows from 'dry' rows on the output grid."""
174     max_c = moist.max(axis=1)
175     before = t < t_dry
176     return {
177         "t_dry": float(t_dry),
178         "t_dry_hours": float(t_dry / 3600.0),

```

```

179     "profile_max_at_t_dry": float(dry_profile.max()),
180     "profile_max_error_at_t_dry": float(abs(dry_profile.max() - target)),
181     "rows_before_all_wet": bool(np.all(max_c[before] >= target)),
182     "rows_after_all_dry": bool(np.all(max_c[-before] < target)),
183     "last_row_time": float(t[-1]),
184     "last_row_max": float(max_c[-1]),
185 }
186
187
188 # -----
189 # 3. Convergence bookkeeping
190 # -----
191
192
193 def convergence_table(levels: list[dict[str, Any]], key: str) -> list[dict[str, Any]]:
194     """levels: [{"k": k, "dr_cm": ..., key: array}] with arrays sampled on common points; finest is last."""
195     reference = levels[-1][key]
196     rows: list[dict[str, Any]] = []
197     errors: list[float] = []
198     for level in levels[:-1]:
199         err = float(np.max(np.abs(level[key] - reference)))
200         errors.append(err)
201         rows.append({"k": level["k"], "dr_cm": level["dr_cm"], "max_abs_error": err})
202     for i, row in enumerate(rows):
203         if i + 1 < len(rows) and errors[i + 1] > 0:
204             row["observed_order"] = float(np.log2(errors[i] / errors[i + 1]))
205         else:
206             row["observed_order"] = None
207     return rows
208
209
210 def richardson_estimate(coarse_err: float, fine_err: float, order: float = 2.0) -> float:
211     """Error estimate of the fine level from two successive errors against a common finer reference."""
212     return float(abs(fine_err) / (2**order - 1))
213
214
215 # -----
216 # 4. Independent two-dimensional axisymmetric finite-volume model (fixed radius, physical coordinates)
217 # -----
218
219
220 class AxisymmetricSystem:
221     """Vertex-centred FV on (r, z) in [0, R0] x [0, L/2]; z = 0 is the mid-plane (symmetry)."""
222
223     def __init__(self, props: Properties, chamber: Chamber, nr: int, nz: int, end_bc: str = "robin") -> None:
224         self.props = props
225         self.chamber = chamber
226         self.nr, self.nz = nr, nz
227         self.end_bc = end_bc
228         self.dr = R0 / nr
229         self.dz = (LENGTH / 2.0) / nz
230         r = np.linspace(0.0, R0, nr + 1)
231         faces = 0.5 * (r[:-1] + r[1:])
232         self.r_face = faces
233         lower = np.concatenate([[0.0], faces])
234         upper = np.concatenate([faces, [R0]])
235         self.m_r = 0.5 * (upper**2 - lower**2) # radial measure per node
236         self.w_z = np.full(nz + 1, self.dz)
237         self.w_z[0] = self.w_z[-1] = 0.5 * self.dz # half cells at the mid-plane and the end face
238         self.shape = (nr + 1, nz + 1)
239         self.m = (nr + 1) * (nz + 1)
240
241     def __call__(self, t: float, y: np.ndarray) -> np.ndarray:

```

```

242     p = self.props
243     temp = y[: self.m].reshape(self.shape)
244     moist = y[self.m :].reshape(self.shape)
245     t_air = float(self.chamber.air_temperature(t))
246     c_air = float(self.chamber.air_humidity(t))
247     # radial faces (between i and i+1), shape (nr, nz+1)
248     cf = 0.5 * (moist[:-1, :] + moist[1:, :])
249     tf = 0.5 * (temp[:-1, :] + temp[1:, :])
250     fc_r = self.r_face[:, None] * p.diffusivity(cf, tf) * (moist[1:, :] - moist[:-1, :]) / self.dr
251     ft_r = self.r_face[:, None] * p.k(cf) * (temp[1:, :] - temp[:-1, :]) / self.dr
252     # axial faces (between j and j+1), shape (nr+1, nz)
253     cz = 0.5 * (moist[:, :-1] + moist[:, 1:])
254     tz = 0.5 * (temp[:, :-1] + temp[:, 1:])
255     fc_z = p.diffusivity(cz, tz) * (moist[:, 1:] - moist[:, :-1]) / self.dz
256     ft_z = p.k(cz) * (temp[:, 1:] - temp[:, :-1]) / self.dz
257     div_c = np.zeros(self.shape)
258     div_t = np.zeros(self.shape)
259     div_c[:-1, :] += fc_r
260     div_c[1:, :] -= fc_r
261     div_t[:-1, :] += ft_r
262     div_t[1:, :] -= ft_r
263     div_c[-1, :] += -R0 * p.hm * (moist[-1, :] - c_air)
264     div_t[-1, :] += -R0 * p.h * (temp[-1, :] - t_air)
265     div_c *= self.w_z[None, :]
266     div_t *= self.w_z[None, :]
267     ax_c = np.zeros(self.shape)
268     ax_t = np.zeros(self.shape)
269     ax_c[:, :-1] += fc_z
270     ax_c[:, 1:] -= fc_z
271     ax_t[:, :-1] += ft_z
272     ax_t[:, 1:] -= ft_z
273     if self.end_bc == "robin":
274         ax_c[:, -1] += -p.hm * (moist[:, -1] - c_air)
275         ax_t[:, -1] += -p.h * (temp[:, -1] - t_air)
276     div_c += ax_c * self.m_r[:, None]
277     div_t += ax_t * self.m_r[:, None]
278     measure = self.m_r[:, None] * self.w_z[None, :]
279     dc = div_c / measure
280     dt = div_t / (measure * p.rho(moist) * p.cp(moist))
281     return np.concatenate([dt.ravel(), dc.ravel()])
282
283 def jacobian_sparsity(self) -> sparse.csr_matrix:
284     nr1, nz1 = self.shape
285     idx = np.arange(self.m).reshape(self.shape)
286     pairs = [(idx, idx)]
287     pairs.append((idx[:-1, :], idx[1:, :]))
288     pairs.append((idx[1:, :], idx[:-1, :]))
289     pairs.append((idx[:, :-1], idx[:, 1:]))
290     pairs.append((idx[:, 1:], idx[:, :-1]))
291     rows = np.concatenate([a.ravel() for a, _ in pairs])
292     cols = np.concatenate([b.ravel() for _, b in pairs])
293     blocks = []
294     for ro in (0, self.m):
295         for co in (0, self.m):
296             blocks.append((rows + ro, cols + co))
297     r_all = np.concatenate([b[0] for b in blocks])
298     c_all = np.concatenate([b[1] for b in blocks])
299     return sparse.csr_matrix((np.ones(len(r_all)), (r_all, c_all)), shape=(2 * self.m, 2 * self.m))
300
301
302 def solve_axisymmetric(
303     props: Properties,
304     chamber: Chamber,

```

```

305     t_out: np.ndarray,
306     nr: int = 20,
307     nz: int = 25,
308     end_bc: str = "robin",
309     t_init: float = 28.0,
310     c_init: float = 2.55,
311     rtol: float = 1e-7,
312     atol: float = 1e-9,
313     detect_dry: bool = False,
314     method: str = "BDF",
315 ) -> dict[str, Any]:
316     system = AxisymmetricSystem(props, chamber, nr, nz, end_bc)
317     y0 = np.concatenate([np.full(system.m, t_init), np.full(system.m, c_init)])
318
319     def event(t: float, y: np.ndarray) -> float:
320         return float(np.max(y[system.m :]) - C_TARGET)
321
322     event.terminal = False # type: ignore[attr-defined]
323     event.direction = -1 # type: ignore[attr-defined]
324     from pipelines.a.solver import integrate_piecewise # generic ODE driver only (not the radial scheme)
325
326     states, stats, t_dry, _ = integrate_piecewise(
327         system,
328         y0,
329         np.asarray(t_out, dtype=float),
330         chamber.knots(),
331         method=method,
332         rtol=rtol,
333         atol=atol,
334         jac_sparsity=system.jacobian_sparsity(),
335         event=event if detect_dry else None,
336     )
337     temp = states[:, : system.m].reshape(len(t_out), *system.shape)
338     moist = states[:, system.m :].reshape(len(t_out), *system.shape)
339     out: dict[str, Any] = {
340         "t": np.asarray(t_out, dtype=float),
341         "r": np.linspace(0.0, R0, nr + 1),
342         "z": np.linspace(0.0, LENGTH / 2.0, nz + 1),
343         "temp": temp,
344         "moist": moist,
345         "nfev": stats["nfev"],
346         "nlu": stats["nlu"],
347     }
348     if t_dry is not None:
349         out["t_dry"] = t_dry
350     return out

```

pipelines/common/__init__.py

pipelines/common/stages.py

```
1 """Problem-agnostic stages: ingest, lint, test, paper, qa, package, release."""
2
3 from __future__ import annotations
4
5 import importlib
6 import json
7 import os
8 import re
9 import shlex
10 import shutil
11 import subprocess
12 import xml.etree.ElementTree as ET
13 from datetime import UTC, datetime
14 from pathlib import Path
15 from typing import Any
16
17 from forge import packaging, pdfqa, xlsx
18 from forge.context import StageContext
19 from forge.hashing import hash_tree, sha256_file
20 from forge.runner import stage
21 from forge.tex import aux_page, compile_tex, tex_escape, tex_path
22
23 PLANNED_STAGE_FILES = {
24     "lint": ["ruff_check.txt", "ruff_format.txt", "mypy.txt", "lint_report.json", "manifest.json", "events.jsonl"],
25     "test": ["junit.xml", "coverage.json", "pytest.txt", "test_report.json", "manifest.json", "events.jsonl"],
26     "qa": ["qa_report.json", "manifest.json", "events.jsonl"],
27 }
28
29 MEMBER_DESCRIPTIONS = (
30     (r"^forge/", "云端运行时（阶段执行、清单、契约、排版、打包）"),
31     (r"^pipelines/common/", "通用阶段：数据接入、门禁、论文、质量检查、打包发布"),
32     (r"^pipelines/", "本题建模与求解程序"),
33     (r"^tools/", "本地控制面：提交云端任务、下载、校验散列"),
34     (r"^tests/", "单元 / 性质 / 集成测试"),
35     (r"^configs/", "运行配置与项目元数据"),
36     (r"^manuscript/", "论文 LaTeX 源码"),
37     (r"^docs/", "工程规范、建模标准、决策记录"),
38     (r"/ingest/", "原始数据接入结果（散列、题目文本、规范化数据）"),
39     (r"/validate/", "数据契约校验报告与数据画像"),
40     (r"/results/", "结果工作簿与求解输出"),
41     (r"/figures/", "论文图（矢量 PDF 与预览 PNG）"),
42     (r"/tables/", "论文表（LaTeX 片段与 CSV）"),
43     (r"/lint/", "静态检查报告"),
44     (r"/test/", "测试报告与覆盖率"),
45     (r"/qa/", "论文与结果文件质量门禁报告"),
46     (r"manifest\.json$", "阶段清单（输入输出散列、环境、参数）"),
47     (r"events\.jsonl$", "结构化运行日志"),
48     (r"^AI工具使用详情\.pdf$", "AI 工具使用详情"),
49     (r"^AI_USAGE\.md$", "AI 工具使用记录（源文件）"),
50     (r"^Makefile$", "本地快捷命令（只提交云端任务）"),
51     (r"^pyproject\.toml$", "项目元数据与静态检查配置"),
52     (r"^README\.md$", "仓库说明与复现入口"),
53     (r"^CHANGELOG\.md$", "变更记录"),
54     (r"^UNLICENSE$", "公有领域声明（Unlicense）"),
55     (r"^REPRODUCE\.md$", "复现说明"),
56     (r"^MANIFEST\.json$", "支撑材料成员清单与 SHA-256"),
57 )
58
59 def _describe(name: str) -> str:
60     for pattern, text in MEMBER_DESCRIPTIONS:
61         if re.search(pattern, name):
```

```

62         return text
63     return ""
64
65
66 def _to_parquet(df: Any, target: Path) -> None:
67     frame = df.copy()
68     frame.columns = [str(c).strip() for c in frame.columns]
69     try:
70         frame.to_parquet(target, index=False)
71     except Exception:
72         for col in frame.columns:
73             if frame[col].dtype == object:
74                 frame[col] = frame[col].map(
75                     lambda v: None if v is None or (isinstance(v, float) and v != v) else str(v)
76                 )
77         frame.to_parquet(target, index=False)
78
79
80 @stage("ingest", description="Hash raw inputs, extract the problem text, convert every attachment sheet to parquet")
81 def ingest(ctx: StageContext) -> dict[str, Any]:
82     import fitz
83
84     document = fitz.open(ctx.problem_pdf)
85     text = "\n\n".join(page.get_text("text", sort=True) for page in document)
86     ctx.out("problem_text.txt").write_text(text, encoding="utf-8")
87     inventory: dict[str, Any] = {
88         "problem_pdf": {
89             "path": ctx.project["problem_pdf"],
90             "sha256": sha256_file(ctx.problem_pdf),
91             "pages": len(document),
92         },
93         "files": [],
94     }
95     for path in sorted(ctx.attachments.rglob("*.xlsx")):
96         if path.name.startswith("-$"):
97             continue
98         rel = path.relative_to(ctx.attachments).as_posix()
99         is_template = path.parent == ctx.templates or ctx.templates in path.parents
100         entry: dict[str, Any] = {
101             "path": rel,
102             "sha256": sha256_file(path),
103             "bytes": path.stat().st_size,
104             "template": is_template,
105             "sheets": [],
106         }
107         for name, df in xlsx.read_sheets(path).items():
108             sheet: dict[str, Any] = {"name": name, "rows": len(df), "columns": [str(c) for c in df.columns]}
109             if not is_template:
110                 safe = re.sub(r"[^\w.-]+", "_", f"{path.stem}_{name}")
111                 target = ctx.out("data", f"{safe}.parquet")
112                 _to_parquet(df, target)
113                 sheet["parquet"] = target.relative_to(ctx.stage_dir).as_posix()
114                 sheet["dtypes"] = {str(c): str(t) for c, t in df.dtypes.items()}
115                 sheet["head"] = df.head(5).astype(str).values.tolist()
116                 entry["sheets"].append(sheet)
117             inventory["files"].append(entry)
118             ctx.log.info("ingest.file", path=rel, template=is_template, sheets=[s["name"] for s in entry["sheets"]])
119     ctx.write_json("inventory.json", inventory)
120     return {"files": len(inventory["files"]), "problem_chars": len(text)}
121
122
123 @stage("lint", description="Static gates: ruff check, ruff format --check, mypy")
124 def lint(ctx: StageContext) -> dict[str, Any]:

```

```

125     returncodes: dict[str, int] = {}
126
127     def run(name: str, cmd: list[str]) -> None:
128         proc = subprocess.run(cmd, cwd=str(ctx.repo), capture_output=True, text=True, check=False)
129         ctx.out(f"{name}.txt").write_text(proc.stdout + ("\n" + proc.stderr if proc.stderr else ""), encoding="utf-8")
130         returncodes[name] = proc.returncode
131         ctx.log.info("lint.tool", tool=name, rc=proc.returncode, tail=(proc.stdout + proc.stderr)[-400:])
132
133     run("ruff_check", ["ruff", "check", "--no-cache", "--output-format", "concise", "."])
134     run("ruff_format", ["ruff", "format", "--check", "--no-cache", "."])
135     run("mypy", ["mypy", "--cache-dir", "/tmp/mypy", *ctx.cfg("gates.mypy_paths", ["forge"])]])
136     required = list(ctx.cfg("gates.lint_required", ["ruff_check", "ruff_format", "mypy"]))
137     passed = all(returncodes.get(name, 1) == 0 for name in required)
138     ctx.write_json("lint_report.json", {"passed": passed, "required": required, "returncodes": returncodes})
139     if not passed:
140         raise RuntimeError(f"lint gate failed: {returncodes}")
141     return {"passed": passed, **returncodes}
142
143
144 @stage("test", description="pytest (unit, property, integration) with JUnit and coverage reports")
145 def test(ctx: StageContext) -> dict[str, Any]:
146     env = {
147         **os.environ,
148         "COVERAGE_FILE": "/tmp/.coverage",
149         "HYPOTHESIS_STORAGE_DIRECTORY": "/tmp/hypothesis",
150         "FORGE_RUN_DIR": str(ctx.run_dir),
151     }
152     cmd = [
153         "python",
154         "-m",
155         "pytest",
156         "tests",
157         "-q",
158         "-p",
159         "no:cacheprovider",
160         "--rootdir",
161         str(ctx.repo),
162         f"--junitxml={ctx.out('junit.xml')}",
163         "--cov=forge",
164         "--cov=pipelines",
165         f"--cov-report=json:{ctx.out('coverage.json')}",
166         "--cov-report=term",
167     ]
168     extra = ctx.param("pytest_args")
169     if extra:
170         cmd += shlex.split(str(extra))
171     proc = subprocess.run(cmd, cwd=str(ctx.repo), capture_output=True, text=True, env=env, check=False)
172     ctx.out("pytest.txt").write_text(proc.stdout + "\n" + proc.stderr, encoding="utf-8")
173     print(proc.stdout[-5000:], flush=True)
174     summary: dict[str, Any] = {"returncode": proc.returncode}
175     junit = ctx.stage_dir / "junit.xml"
176     if junit.exists():
177         root = ET.parse(junit).getroot()
178         suite = root if root.tag == "testsuite" else root.find("testsuite")
179         if suite is not None:
180             summary.update({k: int(suite.get(k, 0)) for k in ("tests", "failures", "errors", "skipped")})
181     coverage = ctx.stage_dir / "coverage.json"
182     if coverage.exists():
183         summary["coverage_percent"] = round(json.loads(coverage.read_text())["totals"]["percent_covered"], 2)
184     ctx.write_json("test_report.json", summary)
185     if proc.returncode != 0:
186         raise RuntimeError(f"tests failed: {summary}\n{proc.stdout[-2500:]}")
187     return summary

```

```

188
189
190 def _support_members(ctx: StageContext, *, planned: bool) -> list[tuple[str, Path]]:
191     members = packaging.code_members(ctx.repo, ctx.cfg("package.code_globs", []))
192     for name in ctx.cfg("package.stages", []):
193         directory = ctx.run_dir / name
194         if (directory / "manifest.json").exists():
195             members += packaging.run_members(ctx.run_dir, [name])
196         elif planned and name in PLANNED_STAGE_FILES:
197             members += [(f"runs/{ctx.run_id}/{name}/{f}", directory / f) for f in PLANNED_STAGE_FILES[name]]
198     members.append(("AI工具使用详情.pdf", ctx.run_dir / "paper" / "AI工具使用详情.pdf"))
199     members.append(("REPRODUCE.md", ctx.run_dir / "package" / "REPRODUCE.md"))
200     members.append(("MANIFEST.json", ctx.run_dir / "package" / "MANIFEST.json"))
201     return sorted(dict(members).items())
202
203
204 def _load_numbers(run_dir: Path) -> tuple[dict[str, str], list[str]]:
205     numbers: dict[str, str] = {}
206     conflicts: list[str] = []
207     for path in sorted(run_dir.glob("*/numbers.json")):
208         for key, value in json.loads(path.read_text(encoding="utf-8")).items():
209             if key in numbers and numbers[key] != value:
210                 conflicts.append(f"{key}: {numbers[key]!r} vs {value!r} ({path.parent.name})")
211             numbers[key] = value
212     return numbers, conflicts
213
214
215 def _stage_rows(ctx: StageContext) -> list[dict[str, Any]]:
216     rows = []
217     for manifest in sorted(ctx.run_dir.glob("*/manifest.json")):
218         data = json.loads(manifest.read_text(encoding="utf-8"))
219         rows.append(
220             {
221                 "stage": data["stage"],
222                 "status": data["status"],
223                 "duration_s": data.get("duration_s"),
224                 "outputs_digest": data.get("outputs_digest", ""),
225                 "modal_task_id": data.get("runtime", {}).get("modal_task_id"),
226             }
227         )
228     return rows
229
230
231 @stage(
232     "paper",
233     description="Assemble generated inputs (numbers, tables, figures, code, support list) and compile with XeLaTeX",
234 )
235 def paper(ctx: StageContext) -> dict[str, Any]:
236     sources = list(ctx.cfg("paper.sources", []))
237     for source in sources:
238         ctx.dep(source)
239     build = Path("/tmp/build")
240     shutil.rmtree(build, ignore_errors=True)
241     shutil.copytree(
242         ctx.repo / "manuscript", build, ignore=shutil.ignore_patterns("generated", "*.aux", "*.log", "*.pdf")
243     )
244     generated = build / "generated"
245     (generated / "tables").mkdir(parents=True, exist_ok=True)
246     (build / "figures").mkdir(exist_ok=True)
247
248     numbers, conflicts = _load_numbers(ctx.run_dir)
249     for conflict in conflicts:
250         ctx.log.warn("paper.number_conflict", detail=conflict)

```

```

251 lines = ["% generated by forge — do not edit"] + [
252     f"\newcommand{{\val{key}}}{\tex_escape(value)}}" for key, value in sorted(numbers.items())
253 ]
254 (generated / "numbers.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
255
256 n_tables = n_figures = 0
257 for source in sources:
258     directory = ctx.run_dir / source
259     for table in sorted((directory / "tables").glob("*.tex")) if (directory / "tables").is_dir() else []:
260         shutil.copy2(table, generated / "tables" / table.name)
261         n_tables += 1
262     for figure in sorted((directory / "figures").glob("*")) if (directory / "figures").is_dir() else []:
263         if figure.suffix.lower() in {".pdf", ".png"}:
264             shutil.copy2(figure, build / "figures" / figure.name)
265             n_figures += 1
266
267 code_ref = ctx.params.get("code_ref") or {}
268 meta = [
269     f"\newcommand{{\RunId}}{\tex_escape(ctx.run_id)}",
270     f"\newcommand{{\CodeSha}}{\tex_escape(str(code_ref.get('git_sha', '')))}",
271     f"\newcommand{{\BuildDate}}{\datetime.now(UTC).strftime('%Y-%m-%d')}",
272 ]
273 (generated / "meta.tex").write_text("\n".join(meta) + "\n", encoding="utf-8")
274
275 files = packaging.code_members(ctx.repo, ctx.cfg("paper.code_globs", []))
276 listing = ["% generated by forge — complete source listing"]
277 for rel, src in files:
278     target = build / "code" / rel
279     target.parent.mkdir(parents=True, exist_ok=True)
280     shutil.copy2(src, target)
281     language = {"py": "Python", ".tex": "[LaTeX]TeX"}.get(src.suffix, "")
282     listing.append(f"\needspace{{6\baselineskip}}\subsection*{{\tex_path(rel)}}

```

```

314     if result["aux"].exists():
315         shutil.copy2(result["aux"], ctx.out("main.aux"))
316     body_end = aux_page(result["aux"], "page:body-end")
317     abstract_end = aux_page(result["aux"], "page:abstract-end")
318     ai_summary = None
319     if (build / "ai_usage.tex").exists():
320         ai = compile_tex(build, "ai_usage.tex", "ai_usage", passes=2, bibtex=False)
321         shutil.copy2(ai["pdf"], ctx.out("AI工具使用详情.pdf"))
322         ai_summary = ai["summary"]
323     info = pdfqa.inspect_pdf(ctx.out("main.pdf"), with_text=False)
324     body_pages = (body_end - abstract_end) if body_end and abstract_end else None
325     report = {
326         "pages": info["pages"],
327         "all_a4": info["all_a4"],
328         "abstract_end_page": abstract_end,
329         "body_end_page": body_end,
330         "body_pages": body_pages,
331         "log": result["summary"],
332         "unembedded_fonts": info["unembedded_fonts"],
333         "fonts": info["fonts"],
334         "numbers": len(numbers),
335         "number_conflicts": conflicts,
336         "tables": n_tables,
337         "figures": n_figures,
338         "code_files": len(files),
339         "support_members": len(members),
340         "ai_usage_log": ai_summary,
341         "stages": stage_rows,
342     }
343     ctx.write_json("paper_report.json", report)
344     summary = result["summary"]
345     return {
346         "pages": info["pages"],
347         "body_pages": body_pages,
348         "undefined_refs": len(summary["undefined_references"]) + len(summary["undefined_citations"]),
349         "overfull_max_pt": summary["overfull_max_pt"],
350         "missing_chars": summary["missing_characters"],
351         "figures": n_figures,
352         "tables": n_tables,
353     }
354
355
356 @stage("qa", deps=("paper",), description="Quality gates over the compiled paper and the result workbooks")
357 def qa(ctx: StageContext) -> dict[str, Any]:
358     for required in ctx.cfg("qa.required_stages", ["lint", "test"]):
359         ctx.dep(required)
360     paper_dir = ctx.dep("paper")
361     report = json.loads((paper_dir / "paper_report.json").read_text(encoding="utf-8"))
362     checks: list[dict[str, Any]] = []
363
364     def check(name: str, ok: bool, detail: Any = "") -> None:
365         checks.append({"name": name, "ok": bool(ok), "detail": detail})
366         ctx.log.info("qa.check", name=name, ok=bool(ok), detail=detail)
367
368     max_body = int(ctx.cfg("paper.max_body_pages", 30))
369     check("abstract_single_page", report["abstract_end_page"] == 1, report["abstract_end_page"])
370     check(
371         "body_pages_within_limit",
372         report["body_pages"] is not None and report["body_pages"] <= max_body,
373         f"{report['body_pages']} <= {max_body}",
374     )
375     log = report["log"]
376     check(

```

```

377         "no_undefined_references",
378         not log["undefined_references"] and not log["undefined_citations"],
379         {"refs": log["undefined_references"][:10], "cites": log["undefined_citations"][:10]},
380     )
381     check("no_tex_errors", not log["errors"], log["errors"][:5])
382     check("fonts_embedded", not report["unembedded_fonts"], report["unembedded_fonts"])
383     check("a4_pages", report["all_a4"])
384     check(
385         "overfull_within_limit",
386         log["overfull_max_pt"] <= float(ctx.cfg("paper.max_overfull_pt", 20.0)),
387         f"max {log['overfull_max_pt']} pt, count {log['overfull_count']}",
388     )
389     check("no_missing_characters", log["missing_characters"] == 0, log["missing_character_samples"])
390     check("no_number_conflicts", not report["number_conflicts"], report["number_conflicts"][:5])
391     check("ai_usage_pdf_present", (paper_dir / "AI工具使用详情.pdf").exists())
392
393     info = pdfqa.inspect_pdf(paper_dir / "main.pdf", with_text=True)
394     deny_hits = pdfqa.scan_strings(info["texts"], ctx.cfg("qa.identity_denylist", []))
395     watch_hits = pdfqa.scan_strings(info["texts"], ctx.cfg("qa.identity_watchlist", []))
396     check("identity_denylist_clean", not deny_hits, deny_hits[:5])
397     check("paper_size_under_20mb", (paper_dir / "main.pdf").stat().st_size <= 20 * 1024 * 1024)
398
399     results_reports: list[dict[str, Any]] = []
400     results_stage = str(ctx.cfg("qa.results_stage", "results"))
401     require_results = bool(ctx.param("require_results", ctx.cfg("qa.require_results", True)))
402     if ctx.has_stage(results_stage):
403         contracts = importlib.import_module(f"pipelines.{ctx.project['package']}.contracts")
404         results_dir = ctx.run_dir / results_stage
405         for contract in contracts.RESULT_CONTRACTS:
406             rep = xlsx.check_workbook(results_dir / contract.file, contract, ctx.templates / contract.template)
407             results_reports.append(rep)
408             check(f"result:{contract.file}", rep["ok"], rep["errors"][:5])
409     else:
410         check("results_stage_present", not require_results, f"stage '{results_stage}' not completed in this run")
411
412     passed = all(c["ok"] for c in checks)
413     ctx.write_json(
414         "qa_report.json",
415         {
416             "passed": passed,
417             "checks": checks,
418             "identity_watchlist_hits": watch_hits,
419             "results": results_reports,
420             "paper": {k: v for k, v in report.items() if k != "fonts"},
421         },
422     )
423     if not passed:
424         raise RuntimeError("QA gate failed: " + ", ".join(c["name"] for c in checks if not c["ok"]))
425     return {"passed": passed, "checks": len(checks), "watchlist_hits": len(watch_hits)}
426
427
428 def _reproduce_md(ctx: StageContext) -> str:
429     rows = _stage_rows(ctx)
430     code_ref = ctx.params.get("code_ref") or {}
431     manifest = json.loads((ctx.run_dir / "ingest" / "manifest.json").read_text(encoding="utf-8"))
432     packages = manifest.get("runtime", {}).get("packages", {})
433     lines = [
434         f"# 复现说明 (run `{ctx.run_id}`) ",
435         "",
436         "本目录内所有结果均由 Modal 云端按下列阶段生成; 本地只提交任务、下载和核对 SHA-256。",
437         "",
438         f"- 代码版本: git `{code_ref.get('git_sha', '')}` (分支 `{code_ref.get('branch', '')}`)",
439         f"- 运行环境: Python {manifest.get('runtime', {}).get('python', '')}, "

```

```

440         f"{manifest.get('runtime', {}).get('platform', '')}",
441         "- 依赖版本: " + ", ".join(f"{k}=={v}" for k, v in sorted(packages.items())),
442         "",
443         "## 阶段",
444         "",
445         "| 阶段 | 状态 | 用时 (s) | 输出摘要 | Modal 任务 |",
446         "|---|---|---:|---|---|",
447     ]
448     lines += [
449         f"| {r['stage']} | {r['status']} | {r['duration_s'] or 0:.1f} | `{r['outputs_digest'][:16]}` | "
450         f"`{r['modal_task_id'] or ''}` |"
451         for r in rows
452     ]
453     lines += [
454         "",
455         "## 复现命令",
456         "",
457         "```bash",
458         "python3 tools/cli.py provision",
459         "python3 tools/cli.py run ingest,validate --new-run",
460         "python3 tools/cli.py run <科学阶段...> --size medium",
461         "python3 tools/cli.py run lint,test,paper,qa,package,release",
462         "python3 tools/cli.py release --version <tag>",
463         "```",
464         "",
465         "每个阶段目录下的 `manifest.json` 记录输入、输出散列、参数与环境; `events.jsonl` 为结构化日志。",
466         "",
467     ]
468     return "\n".join(lines)
469
470
471 @stage("package", deps=("qa",), description="Build the deterministic support-materials zip and its member manifest")
472 def package(ctx: StageContext) -> dict[str, Any]:
473     planned = set(json.loads((ctx.dep("paper") / "support_members.json").read_text(encoding="utf-8")))
474     ctx.out("REPRODUCE.md").write_text(_reproduce_md(ctx), encoding="utf-8")
475     members = [(name, path) for name, path in _support_members(ctx, planned=False) if name != "MANIFEST.json"]
476     missing = [name for name, path in members if not path.exists()]
477     if missing:
478         raise RuntimeError(f"support members missing on disk: {missing[:10]}")
479     actual = {name for name, _ in members} | {"MANIFEST.json"}
480     drift = sorted(actual ^ planned)
481     if drift and not ctx.param("allow_member_drift"):
482         raise RuntimeError(f"support material members differ from the list printed in the paper: {drift[:20]}")
483     manifest = {
484         "run_id": ctx.run_id,
485         "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
486         "members": [
487             {"path": name, "sha256": sha256_file(path), "bytes": path.stat().st_size} for name, path in members
488         ],
489     }
490     manifest_path = ctx.out("MANIFEST.json")
491     manifest_path.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
492     members.append(("MANIFEST.json", manifest_path))
493     zip_info = packaging.build_zip(members, ctx.out("支撑材料.zip"), comment=f"forge run {ctx.run_id}")
494     limit = int(ctx.cfg("package.max_bytes", 20 * 1024 * 1024))
495     ctx.write_json("package_report.json", {**zip_info, "limit_bytes": limit, "drift": drift})
496     if zip_info["bytes"] > limit:
497         raise RuntimeError(f"support zip {zip_info['bytes']} bytes exceeds limit {limit}")
498     return {"members": zip_info["members"], "bytes": zip_info["bytes"], "drift": len(drift)}
499
500
501 @stage(
502     "release",

```

```

503     deps=("package",),
504     description="Assemble deliverables (paper, result workbooks, support zip) with SHA-256 manifest",
505 )
506 def release(ctx: StageContext) -> dict[str, Any]:
507     qa_report = json.loads((ctx.dep("qa") / "qa_report.json").read_text(encoding="utf-8"))
508     if not qa_report["passed"] and not ctx.param("allow_failed_qa"):
509         raise RuntimeError("release refused: QA gate did not pass")
510     paper_dir = ctx.dep("paper")
511     files: dict[str, Path] = {str(ctx.cfg("release.paper_name", "论文.pdf")): paper_dir / "main.pdf"}
512     results_stage = str(ctx.cfg("qa.results_stage", "results"))
513     if ctx.has_stage(results_stage):
514         for name in ctx.project.get("result_files", []):
515             files[name] = ctx.run_dir / results_stage / name
516     elif not ctx.param("allow_missing_results"):
517         raise RuntimeError("release refused: results stage missing")
518     files["支撑材料.zip"] = ctx.dep("package") / "支撑材料.zip"
519     files["AI工具使用详情.pdf"] = paper_dir / "AI工具使用详情.pdf"
520     manifest: dict[str, Any] = {
521         "run_id": ctx.run_id,
522         "code_ref": ctx.params.get("code_ref"),
523         "files": {},
524         "qa_passed": qa_report["passed"],
525         "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
526     }
527     for name, source in files.items():
528         if not source.exists():
529             raise RuntimeError(f"release file missing: {name} ({source})")
530         target = ctx.out(name)
531         shutil.copy2(source, target)
532         manifest["files"][name] = {"sha256": sha256_file(target), "bytes": target.stat().st_size}
533     notes = [
534         f"# 发布说明 — run {ctx.run_id}",
535         "",
536         f"- 代码: `{(ctx.params.get('code_ref') or {}).get('git_sha', '')}`",
537         f"- QA: {'通过' if qa_report['passed'] else '未通过 (强制发布)'}",
538         "",
539         "| 文件 | 字节 | SHA-256 |",
540         "|---|---|---|",
541     ]
542     notes += [f"| {n} | {m['bytes']} | `{m['sha256']}`" for n, m in manifest["files"].items()]
543     ctx.out("RELEASE_NOTES.md").write_text("\n".join(notes) + "\n", encoding="utf-8")
544     ctx.write_json("release_manifest.json", manifest)
545     return {
546         "files": len(files),
547         "paper_bytes": manifest["files"][str(ctx.cfg("release.paper_name", "论文.pdf"))]["bytes"],
548     }
549
550
551 FMT_IGNORE = (".git", "artifacts", "releases", "__pycache__", ".forge", ".DS_Store", ".mypy_cache", ".ruff_cache")
552
553
554 @stage("fmt", description="ruff --fix and ruff format in the cloud; changed files are exported for a local copy-back")
555 def fmt(ctx: StageContext) -> dict[str, Any]:
556     work = Path("/tmp/fmt")
557     shutil.rmtree(work, ignore_errors=True)
558     shutil.copytree(ctx.repo, work, ignore=shutil.ignore_patterns(*FMT_IGNORE, "*.pyc"))
559     before = hash_tree(work, ignore=FMT_IGNORE)
560     fix = subprocess.run(
561         ["ruff", "check", "--fix", "--no-cache", "--output-format", "concise", "."],
562         cwd=work,
563         capture_output=True,
564         text=True,
565         check=False,

```

```

566 )
567 form = subprocess.run(["ruff", "format", "--no-cache", "."], cwd=work, capture_output=True, text=True, check=False)
568 after = hash_tree(work, ignore=FMT_IGNORE)
569 changed = sorted(rel for rel in after if rel not in before or after[rel]["sha256"] != before[rel]["sha256"])
570 for rel in changed:
571     shutil.copy2(work / rel, ctx.out("files", rel))
572 ctx.out("ruff_fix.txt").write_text(fix.stdout + fix.stderr, encoding="utf-8")
573 ctx.out("ruff_format.txt").write_text(form.stdout + form.stderr, encoding="utf-8")
574 ctx.write_json("fmt_report.json", {"changed": changed, "remaining": fix.stdout.strip().splitlines()[-1:]})
575 ctx.log.info("fmt.done", changed=len(changed), remaining=fix.stdout.strip().splitlines()[-1:])
576 return {"changed": len(changed)}

```

pipelines/common/validation.py

```

1  """Shared implementation of the ``validate`` stage (input data contracts + profiling)."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import pandas as pd
8
9  from forge.context import StageContext
10 from forge.contracts import FrameContract, normalise_columns, validate_frame
11
12
13 def run_validation(ctx: StageContext, contracts: dict[str, FrameContract]) -> dict[str, Any]:
14     """Validate every ingested parquet named in ``contracts`` (key = parquet stem)."""
15     data_dir = ctx.dep("ingest") / "data"
16     reports: list[dict[str, Any]] = []
17     profile: dict[str, Any] = {}
18     for stem, contract in contracts.items():
19         path = data_dir / f"{stem}.parquet"
20         if not path.exists():
21             reports.append({"contract": contract.name, "ok": False, "errors": [f"parquet missing: {stem}"], "rows": 0})
22             continue
23         df = normalise_columns(pd.read_parquet(path))
24         report = validate_frame(df, contract)
25         report["parquet"] = path.name
26         reports.append(report)
27         desc = df.describe(include="all").astype(str).to_dict()
28         profile[stem] = {
29             "rows": len(df),
30             "columns": list(df.columns),
31             "describe": desc,
32             "nulls": {c: int(df[c].isna().sum()) for c in df.columns},
33         }
34         ctx.log.info(
35             "validate.frame", contract=contract.name, ok=report["ok"], rows=report["rows"], errors=report["errors"][:5]
36         )
37     ok = all(r["ok"] for r in reports)
38     ctx.write_json("validation_report.json", {"ok": ok, "frames": reports})
39     ctx.write_json("data_profile.json", profile)
40     if not ok:
41         failed = [r["contract"] for r in reports if not r["ok"]]
42         raise RuntimeError(f"input contracts violated: {failed}")
43     return {"ok": ok, "frames": len(reports)}

```

pyproject.toml

```
1 [project]
2 name = "cumcm2026-a"
3 version = "0.1.0"
4 description = "药材的烘干问题 — cloud-only reproducible modelling study (CUMCM 2026 Problem A)"
5 requires-python = ">=3.11"
6 license = "Unlicense"
7
8 [tool.ruff]
9 line-length = 120
10 target-version = "py311"
11 extend-exclude = ["artifacts", "releases", "manuscript"]
12
13 [tool.ruff.lint]
14 select = ["E", "F", "W", "I", "B", "UP", "NPY", "RUF", "S102"]
15 ignore = ["RUF001", "RUF002", "RUF003", "E741", "B905"]
16
17 [tool.ruff.lint.per-file-ignores]
18 "tests/**" = ["S101", "B011"]
19
20 [tool.mypy]
21 python_version = "3.11"
22 ignore_missing_imports = true
23 check_untyped_defs = true
24 warn_unused_ignores = false
25 no_implicit_optional = true
26 exclude = "(artifacts|releases|manuscript)/"
27
28 [tool.pytest.ini_options]
29 testpaths = ["tests"]
30 addopts = "-p no:cacheprovider -q"
31 markers = ["integration: needs the mounted repository inputs (runs in Modal)"]
32
33 [tool.coverage.run]
34 source = ["forge", "pipelines"]
```

tests/__init__.py

tests/conftest.py

```
1 """Shared test configuration: deterministic hypothesis profile, no on-disk example database."""
2
3 from __future__ import annotations
4
5 import os
6 from pathlib import Path
7
8 import pytest
9 from hypothesis import HealthCheck, settings
10
11 settings.register_profile(
12     "forge", database=None, deadline=None, max_examples=60, suppress_health_check=[HealthCheck.too_slow]
13 )
14 settings.load_profile("forge")
15
```

```

16 REPO = Path(os.environ.get("FORGE_REPO", Path(__file__).resolve().parents[1]))
17
18
19 @pytest.fixture(scope="session")
20 def repo() -> Path:
21     return REPO
22
23
24 @pytest.fixture
25 def tmp_repo(tmp_path: Path) -> Path:
26     """A minimal repository layout (configs only) for runtime tests that must not touch real data."""
27     (tmp_path / "configs").mkdir()
28     (tmp_path / "configs" / "default.toml").write_text("[run]\nseed = 7\n[paper]\nnsources = []\n", encoding="utf-8")
29     (tmp_path / "configs" / "project.json").write_text(
30         '{"letter": "T", "package": "t", "problem_pdf": "T.pdf", "attachments_dir": "att", '
31         '"templates_dir": "att/tpl", "result_files": []}',
32         encoding="utf-8",
33     )
34     (tmp_path / "att" / "tpl").mkdir(parents=True)
35     return tmp_path

```

tests/integration/__init__.py

tests/integration/test_ingest_validate.py

```

1 """End-to-end ingest + validate against the real attachments (runs inside Modal where /repo exists)."""
2
3 from __future__ import annotations
4
5 import json
6 import os
7 from pathlib import Path
8
9 import pytest
10
11 from forge import runner
12
13 REPO = Path(os.environ.get("FORGE_REPO", "/repo"))
14 pytestmark = pytest.mark.integration
15
16
17 @pytest.mark.skipif(not (REPO / "configs" / "project.json").exists(), reason="repository inputs not mounted")
18 def test_ingest_then_validate_on_real_inputs(tmp_path: Path) -> None:
19     vol = tmp_path / "vol"
20     ingest = runner.execute("ingest", "itest", {"code_ref": {"git_sha": "test"}}, False, REPO, vol)
21     assert ingest["status"] == "completed" and ingest["metrics"]["files"] >= 1
22     inventory = json.loads((vol / "runs" / "itest" / "ingest" / "inventory.json").read_text(encoding="utf-8"))
23     assert any(not f["template"] for f in inventory["files"])
24     validate = runner.execute("validate", "itest", {}, False, REPO, vol)
25     assert validate["status"] == "completed" and validate["metrics"]["ok"]

```

tests/unit/__init__.py

tests/unit/test_config.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import pytest
6
7 from forge.config import Config, deep_merge, load_config
8
9
10 def test_deep_merge_is_recursive_and_non_destructive() -> None:
11     base = {"a": {"x": 1, "y": 2}, "b": 1}
12     merged = deep_merge(base, {"a": {"y": 3, "c": 4}})
13     assert merged == {"a": {"x": 1, "y": 3}, "b": 1, "c": 4}
14     assert base == {"a": {"x": 1, "y": 2}, "b": 1}
15
16
17 def test_config_dotted_access_and_digest() -> None:
18     cfg = Config({"paper": {"sources": ["q1"], "max_body_pages": 30}})
19     assert cfg.get("paper.max_body_pages") == 30
20     assert cfg.get("paper.missing", "d") == "d"
21     assert cfg["paper.sources"] == ["q1"]
22     with pytest.raises(KeyError):
23         cfg["nope.nope"]
24     assert cfg.digest == Config({"paper": {"max_body_pages": 30, "sources": ["q1"]}}).digest
25
26
27 def test_load_config_with_profile_and_overrides(tmp_repo: Path) -> None:
28     (tmp_repo / "configs" / "fast.toml").write_text("[run]\nseed = 99\n", encoding="utf-8")
29     cfg = load_config(tmp_repo, profile="fast", overrides={"paper": {"sources": ["x"]}})
30     assert cfg.get("run.seed") == 99
31     assert cfg.get("paper.sources") == ["x"]
```

tests/unit/test_contracts.py

```
1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from forge.contracts import Column, FrameContract, validate_frame
9
10 CONTRACT = FrameContract(
11     name="t",
12     columns=(
13         Column("t", "int", min=0, unique=True, monotonic="increasing", step=60),
14         Column("v", "float", min=0, max=1),
15     ),
16     min_rows=2,
17 )
18
```

```

19
20 def test_valid_frame_passes() -> None:
21     df = pd.DataFrame({"t": [0, 60, 120], "v": [0.1, 0.2, 0.3]})
22     report = validate_frame(df, CONTRACT)
23     assert report["ok"], report["errors"]
24
25
26 def test_violations_are_reported_precisely() -> None:
27     df = pd.DataFrame({"t": [0, 60, 60], "v": [0.1, 1.5, np.nan], "extra": [1, 2, 3]})
28     report = validate_frame(df, CONTRACT)
29     joined = " | ".join(report["errors"])
30     assert not report["ok"]
31     assert "unexpected columns" in joined
32     assert "duplicated" in joined
33     assert "max 1.5 > 1" in joined
34     assert "null" in joined
35     assert "not increasing" in joined
36
37
38 def test_column_names_are_stripped() -> None:
39     df = pd.DataFrame({"t ": [0, 60], " v": [0.5, 0.6]})
40     assert validate_frame(df, CONTRACT)["ok"]
41
42
43 @given(st.lists(st.floats(min_value=0, max_value=1, allow_nan=False), min_size=2, max_size=40))
44 def test_property_ranges_within_bounds_always_pass(values: list[float]) -> None:
45     df = pd.DataFrame({"t": np.arange(len(values)) * 60, "v": values})
46     assert validate_frame(df, CONTRACT)["ok"]
47
48
49 def test_regex_and_custom_checks() -> None:
50     contract = FrameContract(
51         "ids", (Column("id", "str", regex=r"[ABC]\d{3}"),), checks=(lambda df: ["custom"] if len(df) > 1 else [],)
52     )
53     assert validate_frame(pd.DataFrame({"id": ["A001"]}), contract)["ok"]
54     report = validate_frame(pd.DataFrame({"id": ["A001", "Z9"]}), contract)
55     assert any("violate" in e for e in report["errors"]) and "custom" in report["errors"]

```

tests/unit/test_derived.py

```

1 """Unit tests of the helpers behind the derived paper quantities (MDR-0009)."""
2
3 from __future__ import annotations
4
5 import numpy as np
6
7 from pipelines.a.derived import first_time_within, fourier_number, richardson_error
8
9
10 def test_fourier_number() -> None:
11     assert fourier_number(2.0, 3.0, 2.0) == 1.5
12
13
14 def test_richardson_error_recovers_the_limit_of_a_second_order_sequence() -> None:
15     limit = 10.0
16     values = [limit + 4.0 / 4**k for k in range(4)] # error /4 per level: grids halved, order 2
17     est, err = richardson_error(values, order=2.0)
18     assert abs(est - limit) < 1e-12
19     assert abs(err - 4.0 / 4**2) < 1e-12
20
21

```

```

22 def test_first_time_within_requires_the_gap_to_stay_small() -> None:
23     t = np.arange(6.0)
24     a = np.array([0.0, 0.9, 0.2, 0.7, 0.1, 0.05])
25     b = np.zeros(6)
26     assert first_time_within(t, a, b, 0.5) == 4.0
27     assert first_time_within(t, a + 10.0, b, 0.5) is None
28     assert first_time_within(t, b, b, 0.5) == 0.0

```

tests/unit/test_drying.py

```

1  """Tests of the Problem-A physics, solver and independent verifiers (small grids, short horizons)."""
2
3  from __future__ import annotations
4
5  import math
6  from dataclasses import replace
7
8  import numpy as np
9  import pytest
10 from hypothesis import given
11 from hypothesis import strategies as st
12 from scipy.special import j0, j1
13
14 from pipelines.a import verify
15 from pipelines.a.physics import (
16     APPENDIX2,
17     APPENDIX3,
18     APPENDIX4,
19     C_INIT,
20     R0,
21     T_INIT,
22     Chamber,
23     Radius,
24 )
25 from pipelines.a.solver import (
26     ProblemSpec,
27     RadialSystem,
28     make_grid,
29     sample_physical,
30     solve,
31     solve_until_dry,
32     subgrid_indices,
33 )
34
35 # ---- physics -----
36
37
38 def test_property_laws_match_the_appendices() -> None:
39     c = np.array(2.55)
40     assert APPENDIX2.diffusivity(c, np.array(28.0)) == pytest.approx(7e-9 * math.exp(-0.89 / 2.55))
41     assert APPENDIX3.rho(c) == pytest.approx(650 + 128 * 2.55)
42     assert APPENDIX3.cp(c) == pytest.approx(1450 + 2736 * 2.55 / 3.55)
43     assert APPENDIX3.k(c) == pytest.approx(0.21 + 0.38 * 2.55 / 3.55)
44     d3 = 2.4e-3 * math.exp(-0.45 / 2.55) * math.exp(-3850 / (28 + 273.15))
45     assert APPENDIX3.diffusivity(c, np.array(28.0)) == pytest.approx(d3, rel=1e-12)
46     d4 = 4.2e-4 * math.exp(-0.30 / 0.2) * math.exp(-3850 / (50 + 273.15))
47     assert APPENDIX4.diffusivity(np.array(0.2), np.array(50.0)) == pytest.approx(d4, rel=1e-12)
48     assert APPENDIX4.h == 25.0 and APPENDIX4.hm == 8e-7
49     assert APPENDIX2.diffusivity(np.array(0.0), np.array(28.0)) == 0.0 # floor: no division by zero
50
51

```

```

52 def test_chamber_interpolates_linearly_and_holds_the_plateau() -> None:
53     t = np.arange(0, 14401, 60, dtype=float)
54     temp = 28 + 22 * np.minimum(t / 7200, 1.0)
55     hum = np.full_like(t, 0.05)
56     ch = Chamber.from_series(t, temp, hum, "mean_last_hour")
57     assert ch.air_temperature(30.0) == pytest.approx(28 + 22 * 30 / 7200)
58     assert ch.air_temperature(1e6) == pytest.approx(50.0)
59     assert Chamber.from_series(t, temp, hum, "nominal").hum_plateau == 0.05
60     assert Chamber.from_series(t, temp, hum, "last_sample").temp_plateau == pytest.approx(temp[-1])
61
62
63 def test_radius_is_monotone_clamped_and_scaled() -> None:
64     t = np.arange(0, 7201, 1800, dtype=float)
65     r_cm = np.array([2.0, 1.8, 1.7, 1.7, 1.65])
66     rad = Radius.from_series(t, r_cm)
67     fine = np.linspace(0, 9000, 500)
68     values = rad.value(fine)
69     assert np.all(np.diff(values) <= 1e-15)
70     assert rad.value(0.0) == pytest.approx(0.02) and rad.value(1e5) == pytest.approx(0.0165)
71     assert rad.rate(1e5) == 0.0 and rad.rate(900.0) < 0
72     half = Radius.from_series(t, r_cm, shrink_scale=0.5)
73     assert half.value(7200.0) == pytest.approx(0.02 - 0.5 * (0.02 - 0.0165))
74     assert Radius.constant().is_constant and not rad.is_constant
75
76
77 # ---- solver -----
78
79
80 def test_grid_measures_sum_to_half() -> None:
81     g = make_grid(20)
82     assert g.vol.sum() == pytest.approx(0.5)
83     assert g.vol[0] == pytest.approx(g.dxi**2 / 8)
84     np.testing.assert_allclose(verbose.control_volumes(g.xi), g.vol)
85
86
87 def test_equilibrium_is_a_steady_state() -> None:
88     spec = ProblemSpec(APPENDIX2, Chamber.constant(T_INIT, C_INIT), Radius.constant())
89     system = RadialSystem(spec, make_grid(10))
90     y = np.concatenate([np.full(11, T_INIT), np.full(11, C_INIT)])
91     np.testing.assert_allclose(system(0.0, y), 0.0, atol=1e-14)
92
93
94 def test_solver_matches_the_bessel_series_for_constant_properties() -> None:
95     d_const = 5e-9
96     props = replace(APPENDIX2, d_const=d_const)
97     spec = ProblemSpec(props, Chamber.constant(50.0, 0.05), Radius.constant())
98     t_out = np.array([0.0, 300.0, 900.0, 1800.0])
99     sol = solve(spec, 40, t_out, rtol=1e-8, atol=1e-10)
100    alpha = props.k0 / (props.rho0 * props.cp0)
101    exact_t = verify.analytic_field(sol.xi, t_out, alpha, props.h, props.k0, T_INIT, 50.0)
102    exact_c = verify.analytic_field(sol.xi, t_out, d_const, props.hm, d_const, C_INIT, 0.05)
103    assert np.max(np.abs(sol.temp - exact_t)) < 2e-3
104    assert np.max(np.abs(sol.moist - exact_c)) < 5e-3 # thin early boundary layer on the coarse test grid
105
106
107 def test_discrete_moisture_and_energy_balances_hold() -> None:
108     spec = ProblemSpec(APPENDIX2, Chamber.constant(50.0, 0.05), Radius.constant())
109     sol = solve(spec, 20, np.arange(0.0, 1801.0, 2.0))
110     mb = verify.moisture_balance(
111         sol.t, sol.xi, sol.moist, sol.radius, sol.air_hum, spec.props.hm, cum_loss=sol.cum_moist_loss
112     )
113     eb = verify.energy_balance(sol.t, sol.xi, sol.temp, R0, sol.air_temp, spec.props, cum_loss=sol.cum_heat_loss)
114     assert mb["sampled"]["max_relative_residual"] < 1e-4 and eb["sampled"]["max_relative_residual"] < 1e-4

```

```

115     assert mb["exact"]["max_relative_residual"] < 1e-8 and eb["exact"]["max_relative_residual"] < 1e-8
116     ex = verify.extremum_checks(sol.temp, sol.moist, sol.air_temp, sol.air_hum, T_INIT, C_INIT)
117     assert ex["temp_within_bounds"] and ex["moist_within_bounds"] and ex["moist_monotone_in_r"]
118
119
120 def test_moving_boundary_reduces_to_the_fixed_domain_for_constant_radius() -> None:
121     chamber = Chamber.constant(50.0, 0.05)
122     t_out = np.arange(0.0, 3601.0, 600.0)
123     ref = solve(ProblemSpec(APPENDIX4, chamber, Radius.constant()), 20, t_out, rtol=1e-10, atol=1e-12)
124     pseudo = Radius(np.array([0.0, 1.0, 2.0]), np.array([R0, R0, R0]), force_interpolant=True)
125     for form in ("lagrangian", "eulerian"):
126         test = solve(ProblemSpec(APPENDIX4, chamber, pseudo, formulation=form), 20, t_out, rtol=1e-10, atol=1e-12)
127         assert np.max(np.abs(test.moist - ref.moist)) < 1e-8
128         assert np.max(np.abs(test.temp - ref.temp)) < 1e-6
129
130
131 def test_lagrangian_shrinkage_conserves_dry_basis_water() -> None:
132     t = np.array([0.0, 1800.0, 3600.0, 7200.0])
133     rad = Radius.from_series(t, np.array([2.0, 1.8, 1.7, 1.6]))
134     spec = ProblemSpec(APPENDIX4, Chamber.constant(50.0, 0.05), rad, formulation="lagrangian")
135     sol = solve(spec, 20, np.arange(0.0, 7201.0, 5.0))
136     mb = verify.moisture_balance(
137         sol.t, sol.xi, sol.moist, sol.radius, sol.air_hum, spec.props.hm, cum_loss=sol.cum_moist_loss
138     )
139     assert mb["sampled"]["max_relative_residual"] < 2e-4
140     assert mb["exact"]["max_relative_residual"] < 1e-8
141     # the Eulerian alternative loses water through the receding boundary: its balance residual is not small
142     alt = solve(replace(spec, formulation="eulerian"), 20, np.arange(0.0, 7201.0, 5.0))
143     mb_alt = verify.moisture_balance(
144         alt.t, alt.xi, alt.moist, alt.radius, alt.air_hum, spec.props.hm, cum_loss=alt.cum_moist_loss
145     )
146     assert mb_alt["exact"]["max_relative_residual"] > 1e-3
147
148
149 def test_solve_until_dry_detects_the_crossing_consistently() -> None:
150     props = replace(APPENDIX2, d0=7e-7) # fast drying for the test
151     spec = ProblemSpec(props, Chamber.constant(50.0, 0.05), Radius.constant())
152     sol = solve_until_dry(spec, 20, 60.0, horizon=3600.0)
153     assert sol.t_dry is not None and sol.dry_profile is not None
154     checks = verify.drying_checks(sol.t, sol.moist, sol.t_dry, sol.dry_profile)
155     assert checks["rows_before_all_wet"] and checks["rows_after_all_dry"]
156     assert checks["profile_max_error_at_t_dry"] < 1e-6
157     assert sol.t[-1] == pytest.approx(60.0 * math.ceil(sol.t_dry / 60.0))
158
159
160 def test_subgrid_and_physical_sampling() -> None:
161     idx = subgrid_indices(160)
162     assert len(idx) == 21 and idx[1] == 8
163     xi = np.linspace(0, 1, 41)
164     profile = 1.0 - xi**2
165     out = sample_physical(profile, xi, 0.015, np.array([0.0, 0.0075, 0.015, 0.016]))
166     assert out[0] == pytest.approx(1.0) and out[1] == pytest.approx(0.75, abs=1e-9)
167     assert out[2] == pytest.approx(0.0, abs=1e-9) and np.isnan(out[3])
168
169
170 def test_piecewise_integration_is_exact_for_piecewise_linear_forcing() -> None:
171     """y' = -y + f(t) with f piecewise linear (kinks every 60 s): restarting at the kinks keeps BDF accurate."""
172     from pipelines.a.solver import integrate_piecewise
173
174     knots = np.arange(0.0, 601.0, 60.0)
175     values = np.array([0, 1, 0, 2, 0, 3, 0, 4, 0, 5, 0], dtype=float)
176
177     def fun(t: float, y: np.ndarray) -> np.ndarray:

```

```

178         return -y + np.interp(t, knots, values)
179
180     t_out = np.arange(0.0, 601.0, 30.0)
181     states, stats, event_t, _ = integrate_pieewise(
182         fun, np.array([0.0]), t_out, knots[1:], method="BDF", rtol=1e-10, atol=1e-12, jac_sparsity=None
183     )
184     # exact solution:  $y(t) = \int_0^t e^{-(t-s)} f(s) ds$ , evaluated with a fine trapezoid rule on each segment
185     fine = np.linspace(0.0, 600.0, 600001)
186     f_fine = np.interp(fine, knots, values)
187     exact = [np.trapezoid(np.exp(-(t - fine[fine <= t])) * f_fine[fine <= t], fine[fine <= t]) for t in t_out]
188     assert stats["segments"] == 10 and event_t is None
189     np.testing.assert_allclose(states[:, 0], exact, atol=1e-7)
190
191
192 # ---- verifiers -----
193
194
195 @given(st.floats(min_value=0.05, max_value=50.0))
196 def test_robin_eigenvalues_satisfy_the_characteristic_equation(bi: float) -> None:
197     lam = verify.robin_eigenvalues(bi, 6)
198     assert np.all(np.diff(lam) > 0)
199     np.testing.assert_allclose(lam * j1(lam) - bi * j0(lam), 0.0, atol=1e-9)
200
201
202 def test_analytic_series_has_the_right_limits() -> None:
203     xi = np.linspace(0, 1, 11)
204     theta = verify.analytic_robin_cylinder(xi, np.array([1e-4, 5.0]), 1.0)
205     assert np.all(theta[0, :-2] > 0.99) # essentially unchanged interior at tiny Fourier number
206     assert np.all(np.abs(theta[1]) < 1e-3) # fully equilibrated
207
208
209 def test_convergence_table_reports_second_order() -> None:
210     levels = [{"k": k, "dr_cm": 0.1 / 2**k, "v": np.array([1.0 + (0.1 / 2**k) ** 2])}] for k in range(4)]
211     rows = verify.convergence_table(levels, "v")
212     assert rows[0]["observed_order"] == pytest.approx(2.0, abs=0.2)
213
214
215 def test_axisymmetric_model_with_insulated_ends_reproduces_the_radial_scheme() -> None:
216     chamber = Chamber.constant(50.0, 0.05)
217     t_out = np.array([0.0, 300.0, 900.0])
218     one_d = solve(ProblemSpec(APPENDIX3, chamber, Radius.constant()), 20, t_out)
219     two_d = verify.solve_axisymmetric(APPENDIX3, chamber, t_out, nr=20, nz=3, end_bc="insulated")
220     assert np.max(np.abs(two_d["moist"][:, :, 0] - one_d.moist)) < 1e-7
221     assert np.max(np.abs(two_d["temp"][:, :, 0] - one_d.temp)) < 1e-5
222     assert np.max(np.abs(two_d["moist"] - two_d["moist"][:, :, :1])) < 1e-9

```

tests/unit/test_extension.py

```

1 """Tests of the model-evaluation extension (MDR-0008): psychrometrics, the energy flux cap, the C_eq driving force."""
2
3 from __future__ import annotations
4
5 from dataclasses import replace
6
7 import numpy as np
8 import pytest
9
10 from pipelines.a import verify
11 from pipelines.a.common import spec_from_recipe, spec_to_recipe
12 from pipelines.a.extension import latent_diagnostic, slug
13 from pipelines.a.physics import (

```

```

14     APPENDIX2,
15     APPENDIX3,
16     C_INIT,
17     Chamber,
18     FluxCap,
19     Radius,
20     dry_solid_density,
21     humidity_ratio,
22     latent_heat,
23     relative_humidity,
24     saturation_pressure,
25     wet_bulb_temperature,
26 )
27 from pipelines.a.solver import ProblemSpec, solve, solve_until_dry
28
29
30 def test_psychrometrics_reproduce_reference_values() -> None:
31     assert saturation_pressure(50.0) == pytest.approx(12352.0, rel=0.01) # steam tables: 12.35 kPa
32     assert saturation_pressure(20.0) == pytest.approx(2339.0, rel=0.01)
33     assert relative_humidity(50.0, 0.05) == pytest.approx(0.61, abs=0.02)
34     tw = wet_bulb_temperature(50.0, 0.05)
35     assert 40.5 < tw < 43.0
36     w_sat = float(humidity_ratio(saturation_pressure(30.0)))
37     assert wet_bulb_temperature(30.0, w_sat) == pytest.approx(30.0) # saturated air: wet bulb = dry bulb
38     assert wet_bulb_temperature(50.0, 0.02) < wet_bulb_temperature(50.0, 0.05) < 50.0
39     assert latent_heat(50.0) == pytest.approx(2.383e6, rel=0.005) # steam tables: 2382 kJ/kg
40     assert dry_solid_density(APPENDIX3, C_INIT) == pytest.approx((650 + 128 * 2.55) / 3.55)
41
42
43 def test_flux_cap_limits_early_loss_and_keeps_the_balance_exact() -> None:
44     chamber = Chamber.constant(50.0, 0.05)
45     rho_s = float(dry_solid_density(APPENDIX3, C_INIT))
46     cap = FluxCap.wet_bulb(chamber, APPENDIX3.h, rho_s)
47     assert 5e-5 < cap.plateau < 1.5e-4 # ~ 25 W/(m^2 K) x 8 K / 2.4 MJ/kg
48     assert cap.value(1e9) == pytest.approx(cap.plateau)
49     base = ProblemSpec(APPENDIX3, chamber, Radius.constant())
50     capped = replace(base, flux_cap=cap)
51     t_out = np.arange(0.0, 3601.0, 60.0)
52     sol_b = solve(base, 20, t_out)
53     sol_c = solve(capped, 20, t_out)
54     vol = verify.control_volumes(sol_b.xi)
55     assert sol_c.moist[-1] @ vol > sol_b.moist[-1] @ vol # less water leaves when the flux is capped
56     flux = capped.surface_moisture_loss(sol_c.t, sol_c.moist[:, -1], sol_c.air_hum)
57     assert np.all(flux <= cap.plateau / rho_s * (1 + 1e-12))
58     mb = verify.moisture_balance(
59         sol_c.t,
60         sol_c.xi,
61         sol_c.moist,
62         sol_c.radius,
63         sol_c.air_hum,
64         APPENDIX3.hm,
65         cum_loss=sol_c.cum_moist_loss,
66         flux=flux,
67     )
68     assert mb["exact"]["max_relative_residual"] < 1e-8
69     # below the release moisture the loss law is the problem's own
70     c_release = 0.05 + cap.plateau / (rho_s * APPENDIX3.hm)
71     low = np.array([0.5 * c_release])
72     assert capped.surface_moisture_loss(0.0, low, 0.05) == pytest.approx(APPENDIX3.hm * (low - 0.05))
73     # the cap survives the worker recipe round trip
74     again = spec_from_recipe(spec_to_recipe(capped))
75     assert again.flux_cap is not None and again.flux_cap.value(30.0) == pytest.approx(cap.value(30.0))
76     assert slug("cap:1.0") == "cap_1p0"

```

```

77
78
79 def test_equilibrium_moisture_driving_force_bounds_the_surface_and_is_monotone_for_constant_d() -> None:
80     # constant D: the problem is linear, max C < 0.15 <=> theta < (0.15 - C_eq)/(2.55 - C_eq), so a larger C_eq
81     # must take longer; with the exponential D(C) of the appendices a drier surface forms a low-D crust and the
82     # ordering is not guaranteed (case hardening), which is exactly what the extension stage quantifies.
83     props = replace(APPENDIX2, d_const=2e-8)
84     base = ProblemSpec(props, Chamber.constant(50.0, 0.05), Radius.constant())
85     slow = replace(base, chamber=Chamber.constant(50.0, 0.10))
86     sol_base = solve_until_dry(base, 20, 60.0, horizon=3600.0)
87     sol_slow = solve_until_dry(slow, 20, 60.0, horizon=3600.0)
88     assert sol_base.t_dry is not None and sol_slow.t_dry is not None
89     assert sol_slow.t_dry > sol_base.t_dry
90     assert sol_slow.moist[:, -1].min() >= 0.10 - 1e-9 # the surface never drops below the driving-force floor
91     assert sol_slow.dry_profile is not None and sol_slow.dry_profile.max() == pytest.approx(0.15, abs=1e-6)
92
93
94 def test_latent_diagnostic_scales_with_the_evaporation_flux() -> None:
95     spec = ProblemSpec(APPENDIX3, Chamber.constant(50.0, 0.05), Radius.constant())
96     sol = solve(spec, 20, np.arange(0.0, 3601.0, 60.0))
97     rho_s = float(dry_solid_density(APPENDIX3, C_INIT))
98     hist = {
99         "t": sol.t,
100         "surface_temp": sol.temp[:, -1],
101         "air_temp": sol.air_temp,
102         "surface_moist": sol.moist[:, -1],
103         "air_hum": sol.air_hum,
104         "mean_moist": 2.0 * (sol.moist @ verify.control_volumes(sol.xi)),
105     }
106     diag = latent_diagnostic(hist, spec, rho_s)
107     j0 = rho_s * APPENDIX3.hm * (C_INIT - 0.05)
108     assert diag["summary"]["j_w_initial"] == pytest.approx(j0)
109     assert diag["summary"]["dt_lat_peak_K"] == pytest.approx(float(latent_heat(28.0)) * j0 / APPENDIX3.h)
110     assert diag["arrays"]["e_lat"][-1] > 0 and np.all(np.diff(diag["arrays"]["e_lat"]) >= 0)
111     assert diag["summary"]["water_removed_kg_per_m"] > 0

```

tests/unit/test_hashing.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from forge.hashing import hash_tree, sha256_bytes, sha256_json, tree_digest
9
10
11 @given(st.binary())
12 def test_sha256_bytes_is_deterministic(data: bytes) -> None:
13     assert sha256_bytes(data) == sha256_bytes(data)
14     assert len(sha256_bytes(data)) == 64
15
16
17 @given(
18     st.dictionaries(
19         st.text(min_size=1, max_size=8), st.integers() | st.floats(allow_nan=False) | st.text(max_size=5), max_size=6
20     )
21 )
22 def test_sha256_json_ignores_key_order(mapping: dict) -> None:
23     reversed_mapping = dict(reversed(list(mapping.items())))

```

```

24     assert sha256_json(mapping) == sha256_json(reversed_mapping)
25
26
27 def test_hash_tree_ignores_records_and_is_sorted(tmp_path: Path) -> None:
28     (tmp_path / "b.txt").write_text("b")
29     (tmp_path / "a.txt").write_text("a")
30     (tmp_path / "manifest.json").write_text("{}")
31     (tmp_path / "__pycache__").mkdir()
32     (tmp_path / "__pycache__" / "x.pyc").write_bytes(b"\x00")
33     tree = hash_tree(tmp_path, ignore=("manifest.json", "__pycache__"))
34     assert list(tree) == ["a.txt", "b.txt"]
35     digest = tree_digest(tree)
36     (tmp_path / "manifest.json").write_text('{"changed": true}')
37     assert tree_digest(hash_tree(tmp_path, ignore=("manifest.json", "__pycache__"))) == digest

```

tests/unit/test_manifest.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from forge.manifest import Manifest
6
7
8 def test_manifest_lifecycle_and_verification(tmp_path: Path) -> None:
9     stage_dir = tmp_path / "stage"
10    stage_dir.mkdir()
11    (stage_dir / "out.json").write_text("{}")
12    (stage_dir / "events.jsonl").write_text("{}\n")
13    manifest = Manifest.start("run1", "stage", {"p": 1}, {"git_sha": "abc"}, {"cfg", {"ingest": "d"}}, {})
14    assert manifest.status == "running" and manifest.runtime["python"]
15    manifest.complete(stage_dir, {"m": 1.5})
16    assert manifest.status == "completed"
17    assert list(manifest.outputs) == ["out.json"]
18    manifest.write(stage_dir / "manifest.json")
19    loaded = Manifest.read(stage_dir / "manifest.json")
20    assert loaded.outputs_digest == manifest.outputs_digest
21    assert loaded.verify_outputs(stage_dir) == []
22    (stage_dir / "out.json").write_text('{"x": 1}')
23    assert loaded.verify_outputs(stage_dir) == ["sha256 mismatch: out.json"]
24
25
26 def test_manifest_failure_records_traceback() -> None:
27     manifest = Manifest.start("run1", "stage", {}, {}, "cfg", {}, {})
28     try:
29         raise ValueError("boom")
30     except ValueError as exc:
31         manifest.fail(exc)
32     assert manifest.status == "failed"
33     assert manifest.error and manifest.error["type"] == "ValueError"
34     assert manifest.duration_s is not None

```

tests/unit/test_packaging_tex.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from forge.packaging import build_zip, code_members, run_members
6 from forge.tex import aux_page, summarize_log, tex_escape
7
8
9 def test_zip_is_deterministic(tmp_path: Path) -> None:
10     src = tmp_path / "src"
11     src.mkdir()
12     (src / "b.py").write_text("print(2)")
13     (src / "a.py").write_text("print(1)")
14     (src / "__pycache__").mkdir()
15     (src / "__pycache__" / "a.pyc").write_bytes(b"x")
16     members = code_members(src, ["**/*.py"])
17     assert [m[0] for m in members] == ["a.py", "b.py"]
18     one = build_zip(members, tmp_path / "one.zip")
19     two = build_zip(list(reversed(members)), tmp_path / "two.zip")
20     assert one["sha256"] == two["sha256"] and one["members"] == 2
21
22
23 def test_run_members_prefix(tmp_path: Path) -> None:
24     run = tmp_path / "runs" / "r1"
25     (run / "q1").mkdir(parents=True)
26     (run / "q1" / "out.json").write_text("{}")
27     assert run_members(run, ["q1", "missing"]) == [(("runs/r1/q1/out.json", run / "q1" / "out.json"))]
28
29
30 def test_log_summary_and_aux_page(tmp_path: Path) -> None:
31     log = (
32         "LaTeX Warning: Reference `fig:x' on page 3 undefined on input line 10.\n"
33         "Overfull \hbox (12.5pt too wide) in paragraph\n"
34         "Missing character: There is no  in font Latin Modern!\n"
35         "! Undefined control sequence.\n"
36         "Output written on main.pdf (7 pages).\n"
37     )
38     summary = summarize_log(log)
39     assert summary["undefined_references"] == ["fig:x"]
40     assert summary["overfull_max_pt"] == 12.5 and summary["missing_characters"] == 1
41     assert summary["errors"] == ["Undefined control sequence."] and summary["pages_from_log"] == 7
42     aux = tmp_path / "main.aux"
43     aux.write_text("\n\\newlabel{page:body-end}{{}{27}}{{page:body-end}}\n\\newlabel{sec:a}{{2.1}{5}}{{}}\n")
44     assert aux_page(aux, "page:body-end") == 27 and aux_page(aux, "sec:a") == 5 and aux_page(aux, "nope") is None
45
46
47 def test_tex_escape() -> None:
48     assert tex_escape("a_b & 100% #1") == r"a\_b \& 100\% \#1"
```

tests/unit/test_runner.py

```
1 from __future__ import annotations
2
3 import json
4 from pathlib import Path
5
6 import pytest
7
```

```

8 from forge import runner
9 from forge.context import StageContext
10
11
12 @pytest.fixture(autouse=True)
13 def isolated_registry(monkeypatch: pytest.MonkeyPatch) -> None:
14     monkeypatch.setattr(runner, "STAGES", {})
15     monkeypatch.setattr(runner, "discover", lambda repo: None)
16
17
18 def test_execute_records_manifest_skips_reruns_and_enforces_deps(tmp_repo: Path, tmp_path: Path) -> None:
19     vol = tmp_path / "vol"
20
21     @runner.stage("first", description="first stage")
22     def first(ctx: StageContext) -> dict:
23         ctx.out("a.txt").write_text("hello")
24         ctx.number("Answer", 42)
25         return {"n": 1}
26
27     @runner.stage("second", deps=("first",))
28     def second(ctx: StageContext) -> dict:
29         assert (ctx.dep("first") / "a.txt").read_text() == "hello"
30         return {"ok": True}
31
32     with pytest.raises(RuntimeError):
33         runner.execute("second", "r1", {}, False, tmp_repo, vol)
34     result = runner.execute("first", "r1", {"code_ref": {"git_sha": "x"}}, False, tmp_repo, vol)
35     assert result["status"] == "completed" and result["metrics"] == {"n": 1}
36     manifest = json.loads((vol / "runs" / "r1" / "first" / "manifest.json").read_text())
37     assert (
38         manifest["status"] == "completed" and "a.txt" in manifest["outputs"] and "numbers.json" in manifest["outputs"]
39     )
40     assert json.loads((vol / "runs" / "r1" / "first" / "numbers.json").read_text()) == {"Answer": "42"}
41     assert runner.execute("first", "r1", {}, False, tmp_repo, vol)["status"] == "skipped"
42     assert runner.execute("second", "r1", {}, False, tmp_repo, vol)["status"] == "completed"
43     combined = runner.execute("all", "r1", {"stages": ["first", "second"]}, False, tmp_repo, vol)
44     assert combined["status"] == "completed" and [r["status"] for r in combined["results"]] == ["skipped", "skipped"]
45
46
47 def test_failed_stage_writes_failed_manifest(tmp_repo: Path, tmp_path: Path) -> None:
48     vol = tmp_path / "vol"
49
50     @runner.stage("bad")
51     def bad(ctx: StageContext) -> dict:
52         raise ValueError("boom")
53
54     with pytest.raises(ValueError):
55         runner.execute("bad", "r1", {}, False, tmp_repo, vol)
56     manifest = json.loads((vol / "runs" / "r1" / "bad" / "manifest.json").read_text())
57     assert manifest["status"] == "failed" and manifest["error"]["message"] == "boom"
58
59
60 def test_from_run_copies_dependency_outputs(tmp_repo: Path, tmp_path: Path) -> None:
61     vol = tmp_path / "vol"
62
63     @runner.stage("base")
64     def base(ctx: StageContext) -> dict:
65         ctx.out("x.txt").write_text("x")
66         return {}
67
68     @runner.stage("child", deps=("base",))
69     def child(ctx: StageContext) -> dict:
70         return {"seen": (ctx.dep("base") / "x.txt").read_text()}

```

```

71
72 runner.execute("base", "old", {}, False, tmp_repo, vol)
73 result = runner.execute("child", "new", {"from_run": "old"}, False, tmp_repo, vol)
74 assert result["metrics"] == {"seen": "x"}

```

tests/unit/test_xlsx.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import openpyxl
6 import pytest
7
8 from forge.xlsx import SheetContract, WorkbookContract, check_workbook, read_sheets, sheet_headers, write_result
9
10
11 def _template(path: Path) -> Path:
12     wb = openpyxl.Workbook()
13     ws = wb.active
14     ws.title = "温度"
15     ws.append(["时间\\到药材中心的距离", 0, 0.1, "...", 2])
16     ws.append([1, None, None, None, None])
17     ws2 = wb.create_sheet("水分浓度")
18     ws2.append(["时间\\到药材中心的距离", 0, 0.1, "...", 2])
19     wb.save(path)
20     return path
21
22
23 def test_write_result_keeps_all_rounds_and_passes_contract(tmp_path: Path) -> None:
24     template = _template(tmp_path / "tpl.xlsx")
25     header = ["ignored", 0.0, 0.1, 0.2]
26     rows = [[t, 28.123456, 28.2, 28.3] for t in range(1, 6)]
27     info = write_result(template, tmp_path / "out.xlsx", {"温度": (header, rows), "水分浓度": (header, rows)})
28     assert info["sheets"]["温度"]["rows"] == 5
29     headers = sheet_headers(tmp_path / "out.xlsx")
30     assert headers["温度"][0] == "时间\\到药材中心的距离"
31     assert headers["温度"][1:] == [0.0, 0.1, 0.2]
32     frames = read_sheets(tmp_path / "out.xlsx")
33     assert float(frames["温度"].iloc[0, 1]) == pytest.approx(28.1235)
34     contract = WorkbookContract(
35         "out.xlsx",
36         "tpl.xlsx",
37         (
38             SheetContract("温度", min_rows=5, max_rows=5, header_len=4, numeric_from_col=0),
39             SheetContract("水分浓度", min_rows=5, header_len=4, numeric_from_col=0),
40         ),
41     )
42     report = check_workbook(tmp_path / "out.xlsx", contract, template)
43     assert report["ok"], report["errors"]
44
45
46 def test_check_workbook_flags_blank_and_text_cells(tmp_path: Path) -> None:
47     template = _template(tmp_path / "tpl.xlsx")
48     rows = [[1, 1.0, None, "x"], [2, 2.0, 2.0, 2.0]]
49     write_result(template, tmp_path / "out.xlsx", {"温度": (["h", 0, 0.1, 0.2], rows)})
50     contract = WorkbookContract("out.xlsx", "tpl.xlsx", (SheetContract("温度", min_rows=2, numeric_from_col=0),))
51     report = check_workbook(tmp_path / "out.xlsx", contract, template)
52     joined = " ".join(report["errors"])
53     assert not report["ok"] and "blank" in joined and "non-numeric" in joined
54

```

```

55
56 def test_write_result_rejects_unknown_sheet(tmp_path: Path) -> None:
57     template = _template(tmp_path / "tpl.xlsx")
58     with pytest.raises(KeyError):
59         write_result(template, tmp_path / "out.xlsx", {"nope": (["h"], [])})
60
61
62 def test_write_only_path_for_large_sheets(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> None:
63     import forge.xlsx as mod
64
65     monkeypatch.setattr(mod, "BIG_SHEET_CELLS", 10)
66     template = _template(tmp_path / "tpl.xlsx")
67     rows = [[t, 1.0, 2.0] for t in range(1, 8)]
68     info = write_result(template, tmp_path / "big.xlsx", {"温度": (["h", 0, 0.1], rows)})
69     assert info["write_only"]
70     headers = sheet_headers(tmp_path / "big.xlsx")
71     assert list(headers) == ["温度", "水分浓度"]
72     assert headers["温度"][0] == "时间\\到药材中心的距离"

```

tools/__init__.py

tools/cli.py

```

1 #!/usr/bin/env python3
2 """Forge control plane (local). Never computes: submits cloud jobs, downloads, verifies hashes.
3
4 Examples
5     python3 tools/cli.py run ingest                                # new run id, small container
6     python3 tools/cli.py run validate --run-id <id>              # continue the same run
7     python3 tools/cli.py run q1,q2 --size medium                 # sequential stages in one container
8     python3 tools/cli.py run q3 --size large --spawn             # detached; poll with `wait`/`status`
9     python3 tools/cli.py exec --code 'print(sorted(p.name for p in RUN_DIR.iterdir()))'
10    python3 tools/cli.py status                                    # manifests of the last run
11    python3 tools/cli.py download --stage results                 # into artifacts/, verified against manifests
12    python3 tools/cli.py release --version v1.0.0                 # deliverables into releases/v1.0.0/
13 """
14
15 from __future__ import annotations
16
17 import argparse
18 import hashlib
19 import json
20 import os
21 import shutil
22 import subprocess
23 import sys
24 from datetime import datetime
25 from pathlib import Path
26
27 REPO = Path(__file__).resolve().parents[1]
28 PROJECT = json.loads((REPO / "configs" / "project.json").read_text(encoding="utf-8"))
29 APP_FILE = REPO / "forge" / "modal_app.py"
30 STATE_DIR = REPO / ".forge"
31 LAST_RUN = STATE_DIR / "last_run"
32 MODAL = shutil.which("modal") or str(Path.home() / ".local" / "bin" / "modal")
33
34

```

```

35 def _sh(cmd: list[str], **kw) -> subprocess.CompletedProcess:
36     print("$ " + " ".join(cmd), flush=True)
37     return subprocess.run(cmd, cwd=REPO, check=False, **kw)
38
39
40 def git_ref() -> dict:
41     def out(*args: str) -> str:
42         try:
43             return subprocess.run(["git", *args], cwd=REPO, capture_output=True, text=True, check=False).stdout.strip()
44         except OSError:
45             return ""
46
47     sha = out("rev-parse", "HEAD") or "nogit"
48     return {
49         "git_sha": sha,
50         "branch": out("rev-parse", "--abbrev-ref", "HEAD"),
51         "dirty": bool(out("status", "--porcelain")),
52     }
53
54
55 def new_run_id() -> str:
56     return datetime.now().strftime("%Y%m%d-%H%M%S") + "-" + git_ref()["git_sha"][:7]
57
58
59 def remember(run_id: str) -> None:
60     STATE_DIR.mkdir(exist_ok=True)
61     LAST_RUN.write_text(run_id, encoding="utf-8")
62
63
64 def resolve_run_id(explicit: str | None, create: bool) -> str:
65     if explicit:
66         remember(explicit)
67         return explicit
68     if create or not LAST_RUN.exists():
69         run_id = new_run_id()
70         remember(run_id)
71         return run_id
72     return LAST_RUN.read_text(encoding="utf-8").strip()
73
74
75 def parse_params(items: list[str] | None) -> dict:
76     params: dict = {}
77     for item in items or []:
78         key, _, raw = item.partition("=")
79         try:
80             params[key] = json.loads(raw)
81         except json.JSONDecodeError:
82             params[key] = raw
83     return params
84
85
86 def modal_run(args: list[str], detach: bool = False) -> int:
87     cmd = [MODAL, "run"] + (["--detach"] if detach else []) + [str(APP_FILE)] + args
88     return _sh(cmd).returncode
89
90
91 def cmd_run(ns: argparse.Namespace) -> int:
92     stages = [s for s in ns.stage.split(",") if s]
93     run_id = resolve_run_id(ns.run_id, create=ns.new_run)
94     params = parse_params(ns.param)
95     params.update(
96         {"code_ref": git_ref(), "size": ns.size, "submitted_at": datetime.now().isoformat(timespec="seconds")}
97     )

```

```

98     if ns.from_run:
99         params["from_run"] = ns.from_run
100     if ns.profile:
101         params["profile"] = ns.profile
102     stage = stages[0] if len(stages) == 1 else "all"
103     if stage == "all":
104         params["stages"] = stages
105     args = ["--stage", stage, "--run-id", run_id, "--params", json.dumps(params, ensure_ascii=False), "--size", ns.size
106         ↪ ]
107     if ns.force:
108         args.append("--force")
109     if ns.spawn:
110         args.append("--spawn")
111     print(f"run_id={run_id} stages={stages} size={ns.size} spawn={ns.spawn}")
112     return modal_run(args, detach=ns.spawn)
113
114 def cmd_exec(ns: argparse.Namespace) -> int:
115     code = Path(ns.file).read_text(encoding="utf-8") if ns.file else ns.code
116     if not code:
117         print("provide --code or --file", file=sys.stderr)
118         return 2
119     run_id = resolve_run_id(ns.run_id, create=False)
120     return modal_run(
121         ["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": code}, ensure_ascii=False)]
122     )
123
124
125 def cmd_provision(_: argparse.Namespace) -> int:
126     return modal_run(["--stage", "provision"])
127
128
129 STATUS_SNIPPET = r"""
130 import json
131 from pathlib import Path
132 run = RUN_DIR
133 if not run.exists():
134     print("run directory missing:", run)
135 else:
136     rows = []
137     for m in sorted(run.glob("*/manifest.json")):
138         d = json.loads(m.read_text())
139         err = (d.get("error") or {}).get("message", "")[:80]
140         digest = d.get("outputs_digest", "")[:12]
141         rows.append((d["stage"], d["status"], d.get("duration_s"), digest, len(d.get("outputs", {})), err))
142     print(f"{'stage':14s} {'status':10s} {'seconds':>9s} {'digest':12s} {'files':>5s} error")
143     for r in rows:
144         print(f"{r[0]:14s} {r[1]:10s} {str(r[2]):>9s} {r[3]:12s} {r[4]:5d} {r[5]}")
145     print("run_dir:", run)
146 """
147
148
149 def cmd_status(ns: argparse.Namespace) -> int:
150     run_id = resolve_run_id(ns.run_id, create=False)
151     return modal_run(["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": STATUS_SNIPPET})])
152
153
154 def cmd_runs(_: argparse.Namespace) -> int:
155     return _sh([MODAL, "volume", "ls", PROJECT["volume"], "runs"]).returncode
156
157
158 def cmd_logs(ns: argparse.Namespace) -> int:
159     run_id = resolve_run_id(ns.run_id, create=False)

```

```

160     code = (
161         "from pathlib import Path\n"
162         f"p = RUN_DIR / {ns.stage!r} / 'events.jsonl'\n"
163         "print(p.read_text()[~%d:] if p.exists() else 'no events for stage')\n" % ns.tail
164     )
165     return modal_run(["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": code})])
166
167
168 def sha256(path: Path) -> str:
169     h = hashlib.sha256()
170     with path.open("rb") as fh:
171         for chunk in iter(lambda: fh.read(1 << 20), b""):
172             h.update(chunk)
173     return h.hexdigest()
174
175
176 def verify_dir(root: Path) -> int:
177     """Verify every downloaded stage directory against its manifest (transport check only)."""
178     problems = 0
179     manifests = sorted(root.rglob("manifest.json"))
180     if not manifests:
181         print(f"no manifests under {root}")
182         return 1
183     for manifest in manifests:
184         data = json.loads(manifest.read_text(encoding="utf-8"))
185         stage_dir = manifest.parent
186         bad = []
187         for rel, meta in data.get("outputs", {}).items():
188             file = stage_dir / rel
189             if not file.exists():
190                 bad.append(f"missing {rel}")
191             elif sha256(file) != meta["sha256"]:
192                 bad.append(f"sha256 mismatch {rel}")
193         status = "OK " if not bad else "BAD"
194         print(
195             f"[{status}] {data.get('stage'):14s} {data.get('status'):10s} "
196             f"files={len(data.get('outputs', {}))} {stage_dir}"
197         )
198         for b in bad:
199             print("    " + b)
200         problems += len(bad)
201     return 0 if problems == 0 else 1
202
203
204 def volume_get(remote: str, local_parent: Path) -> int:
205     """Download a remote directory into ``local_parent/<basename>`` (replacing any previous copy)."""
206     target = local_parent / Path(remote).name
207     shutil.rmtree(target, ignore_errors=True)
208     local_parent.mkdir(parents=True, exist_ok=True)
209     return _sh([MODAL, "volume", "get", "--force", PROJECT["volume"], remote, str(local_parent) + "/"]).returncode
210
211
212 def cmd_download(ns: argparse.Namespace) -> int:
213     run_id = resolve_run_id(ns.run_id, create=False)
214     if ns.stage:
215         remote, parent = f"runs/{run_id}/{ns.stage}", REPO / ns.dest / "runs" / run_id
216     else:
217         remote, parent = f"runs/{run_id}", REPO / ns.dest / "runs"
218     rc = volume_get(remote, parent)
219     if rc != 0:
220         return rc
221     return verify_dir(parent / Path(remote).name)
222

```

```

223
224 def cmd_verify(ns: argparse.Namespace) -> int:
225     return verify_dir(Path(ns.path))
226
227
228 def cmd_release(ns: argparse.Namespace) -> int:
229     run_id = resolve_run_id(ns.run_id, create=False)
230     dest = REPO / "releases" / ns.version
231     if dest.exists() and not ns.force:
232         print(f"{dest} exists; pass --force to overwrite", file=sys.stderr)
233         return 2
234     staging = REPO / "artifacts" / "release_staging" / run_id
235     rc = volume_get(f"runs/{run_id}/release", staging)
236     if rc != 0:
237         return rc
238     shutil.rmtree(dest, ignore_errors=True)
239     shutil.copytree(staging / "release", dest)
240     manifest = json.loads((dest / "release_manifest.json").read_text(encoding="utf-8"))
241     bad = 0
242     for name, meta in manifest["files"].items():
243         file = dest / name
244         ok = file.exists() and sha256(file) == meta["sha256"]
245         bad += 0 if ok else 1
246         print(f"[{'OK' if ok else 'BAD'}] {name} {meta['bytes']} bytes sha256={meta['sha256'][:16]}")
247     print(f"release {ns.version} from run {run_id}: {'verified' if not bad else f'{bad} problems'} -> {dest}")
248     return 0 if not bad else 1
249
250
251 def cmd_wait(ns: argparse.Namespace) -> int:
252     interpreter = (
253         Path(MODAL).read_text(encoding="utf-8", errors="ignore").splitlines()[0].rstrip("#!").strip()
254         if Path(MODAL).exists()
255         else sys.executable
256     )
257     code = (
258         "import json, modal, sys\n"
259         f"call = modal.FunctionCall.from_id({ns.call_id!r})\n"
260         "print('STAGE_RESULT ' + json.dumps(call.get(), ensure_ascii=False, default=str))\n"
261     )
262     return _sh([interpreter, "-c", code]).returncode
263
264
265 def cmd_fmt(ns: argparse.Namespace) -> int:
266     """Format in the cloud, then copy the exported files back into the working tree."""
267     run_id = resolve_run_id(ns.run_id, create=False)
268     params = {"code_ref": git_ref(), "size": "small"}
269     rc = modal_run(["--stage", "fmt", "--run-id", run_id, "--params", json.dumps(params), "--force"])
270     if rc != 0:
271         return rc
272     dest = REPO / "artifacts" / "runs" / run_id / "fmt"
273     rc = volume_get(f"runs/{run_id}/fmt", dest.parent)
274     if rc != 0:
275         return rc
276     files = dest / "files"
277     changed = sorted(p for p in files.rglob("*") if p.is_file()) if files.exists() else []
278     for src in changed:
279         rel = src.relative_to(files)
280         target = REPO / rel
281         target.parent.mkdir(parents=True, exist_ok=True)
282         shutil.copy2(src, target)
283         print(f"updated {rel}")
284     report = json.loads((dest / "fmt_report.json").read_text(encoding="utf-8"))
285     print(f"{len(changed)} files formatted; remaining ruff findings: {report.get('remaining')}")

```

```

286     return 0
287
288
289 def cmd_new_run(_: argparse.Namespace) -> int:
290     run_id = new_run_id()
291     remember(run_id)
292     print(run_id)
293     return 0
294
295
296 def build_parser() -> argparse.ArgumentParser:
297     p = argparse.ArgumentParser(prog="forge", description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter
        ↵ )
298     sub = p.add_subparsers(dest="command", required=True)
299
300     r = sub.add_parser("run", help="run one stage or a comma-separated sequence in one container")
301     r.add_argument("stage")
302     r.add_argument("--run-id")
303     r.add_argument("--new-run", action="store_true", help="start a fresh run id")
304     r.add_argument("--size", default="small", choices=["small", "medium", "large"])
305     r.add_argument("--force", action="store_true", help="re-run even if the stage already completed")
306     r.add_argument("--spawn", action="store_true", help="detach: return immediately, poll with status/wait")
307     r.add_argument("--from-run", help="copy missing dependency outputs from this earlier run")
308     r.add_argument("--profile", help="overlay configs/<profile>.toml")
309     r.add_argument("--param", action="append", help="key=value (JSON-parsed) forwarded to the stage")
310     r.set_defaults(fn=cmd_run)
311
312     e = sub.add_parser("exec", help="run an ad-hoc Python snippet in the cloud against the run directory")
313     e.add_argument("--code")
314     e.add_argument("--file")
315     e.add_argument("--run-id")
316     e.set_defaults(fn=cmd_exec)
317
318     sub.add_parser("provision", help="verify the cloud toolchain").set_defaults(fn=cmd_provision)
319
320     s = sub.add_parser("status", help="print manifests of a run")
321     s.add_argument("--run-id")
322     s.set_defaults(fn=cmd_status)
323
324     lg = sub.add_parser("logs", help="tail the event log of a stage")
325     lg.add_argument("stage")
326     lg.add_argument("--run-id")
327     lg.add_argument("--tail", type=int, default=6000)
328     lg.set_defaults(fn=cmd_logs)
329
330     sub.add_parser("runs", help="list runs on the volume").set_defaults(fn=cmd_runs)
331
332     d = sub.add_parser("download", help="download a run (or one stage) into artifacts/ and verify hashes")
333     d.add_argument("--run-id")
334     d.add_argument("--stage")
335     d.add_argument("--dest", default="artifacts")
336     d.set_defaults(fn=cmd_download)
337
338     v = sub.add_parser("verify", help="verify a downloaded directory against its manifests")
339     v.add_argument("path")
340     v.set_defaults(fn=cmd_verify)
341
342     rel = sub.add_parser("release", help="download the release stage into releases/<version>/ and verify")
343     rel.add_argument("--version", required=True)
344     rel.add_argument("--run-id")
345     rel.add_argument("--force", action="store_true")
346     rel.set_defaults(fn=cmd_release)
347

```

```

348     w = sub.add_parser("wait", help="block on a spawned function call id and print its result")
349     w.add_argument("call_id")
350     w.set_defaults(fn=cmd_wait)
351
352     f = sub.add_parser("fmt", help="ruff --fix + format in the cloud and copy the results back")
353     f.add_argument("--run-id")
354     f.set_defaults(fn=cmd_fmt)
355
356     sub.add_parser("new-run", help="mint a run id").set_defaults(fn=cmd_new_run)
357     return p
358
359
360 def main(argv: list[str] | None = None) -> int:
361     os.chdir(REPO)
362     ns = build_parser().parse_args(argv)
363     return int(ns.fn(ns))
364
365
366 if __name__ == "__main__":
367     sys.exit(main())

```